

Discovering the Value of IBM App Connect Professional Cloud Integration

Lab Exercises



Catalog Number

Contents

LAB INFORMATION.....	1
INTRODUCTION.....	1
ICONS USED IN LABS	1
IBM APP CONNECT PROFESSIONAL STUDIO OVERVIEW.....	2
WEB MANAGEMENT CONSOLE – DASHBOARD VIEW.....	3
LAB 1 AN APP CONNECT PROFESSIONAL HELLO.....	5
1.1 OVERVIEW	5
1.2 CONCEPTS	5
1.3 ACTIVITIES USED	6
1.4 DOCUMENT KEYS.....	6
1.5 APP CONNECT PROFESSIONAL HELLO	7
1.6 ENHANCED APP CONNECT PROFESSIONAL HELLO	21
LAB 2 LOAD ACCOUNTS FROM A FLAT FILE TO SALESFORCE.COM	35
2.1 OVERVIEW	35
2.2 CONCEPTS	35
2.3 ACTIVITIES USED	35
2.4 DOCUMENT KEYS.....	35
2.5 FLAT FILE TO SALESFORCE.COM	36
2.6 CREATING A FLAT FILE SCHEMA.....	39
2.6.1 CREATE A FLAT FILE SCHEMA USING THE FLAT FILE WIZARD.....	39
2.7 CREATING A NEW ORCHESTRATION	46
2.7.1 BUILD THE ORCHESTRATION –	46
2.7.2 CONFIGURE THE FOR EACH ACTIVITY	57
2.7.3 CONFIGURE THE SALESFORCE CREATE OBJECTS ACTIVITY BY SELECTING THE CHECKLIST	
ITEMS:	62
2.7.4 NOW WE WILL ADD SOME BUSINESS LOGIC TO THE ORCHESTRATION.....	75
2.7.5 NEXT YOU WILL CREATE AN ERROR MESSAGE FOR THOSE RECORDS THAT DO NOT MEET THE	
BUSINESS LOGIC VALIDATION.	80
2.7.6 YOU ARE NOW READY TO VALIDATE YOUR ORCHESTRATION TO MAKE SURE ALL OF YOUR	
CHECKLIST ITEMS HAVE BEEN ANSWERED CORRECTLY.	84
2.8 WEB MANAGEMENT CONSOLE	87
2.8.1 NOW IT IS TIME TO DEPLOY THE PROJECT ON THE IBM APP CONNECT PROFESSIONAL	
WEB MANAGEMENT CONSOLE (WMC).	87
LAB 3 SINGLE DATABASE TABLE OPERATION INTO SALESFORCE.COM	97
3.1 OVERVIEW	97
3.2 CONCEPTS	97
3.3 ACTIVITIES USED	97
3.4 DOCUMENT KEYS.....	97
3.5 LOAD DATABASE RECORDS INTO SALESFORCE	98
LAB 4 LOAD SALESFORCE.COM ACCOUNTS TO ESB VIA WMQ	117
4.1 OVERVIEW	117
4.2 CONCEPTS	117
4.3 ACTIVITIES USED	117
4.4 DOCUMENT KEYS.....	117
4.5 LOAD SALESFORCE.COM ACCOUNTS TO ESB VIA WMQ	118
4.6 RUN THE LOAD SALESFORCE.COM ACCOUNTS TO ESB VIA WMQ LAB	136
LAB 5 INVOKE WEB SERVICE.....	139
5.1 OVERVIEW	139
5.2 CONCEPTS	139
5.3 ACTIVITIES USED	139
5.4 DOCUMENT KEYS.....	139
5.5 INVOKE WEBSERVICE WITH FIXED SYMBOL	ERROR! BOOKMARK NOT DEFINED.
5.6 INVOKE WEBSERVICE WITH INPUT	ERROR! BOOKMARK NOT DEFINED.
LAB 6 INTERACTING WITH RESTFUL SERVICES AND SALESFORCE.....	153
6.1 OVERVIEW	153
6.2 CONCEPTS	153
6.3 ACTIVITIES USED	153

6.4 DOCUMENT KEYS..... 153

6.5 EXPOSE AN ORCHESTRATION AS A RESTFUL SERVICE THAT INVOKES A SECOND REST SERVICE AND
SYNCHRONIZES DATA INTO SALESFORCE.COM..... **ERROR! BOOKMARK NOT DEFINED.**

APPENDIX A - CREATING A SALESFORCE.COM DEVELOPER ACCOUNT..... 173

APPENDIX B - UPDATING THE SALESFORCE ACCOUNT OBJECT 178

APPENDIX C - NOTICES 181

THIS PAGE INTENTIONALLY LEFT BLANK

Lab Information

This Proof of Technology is designed to provide a fairly comprehensive overview of the common capabilities of the IBM App Connect Professional cloud integration product and to provide an opportunity for participants to gain the fundamental skills necessary to begin to build integration solutions.

Introduction

IBM App Connect Professional enables companies to rapidly connect their hybrid world of public clouds, private clouds, and on-premise applications.

Rapidly deliver cloud integration projects in days and achieve higher return on investment in software as a service (SaaS) and cloud models. Leverage reusable process templates and use a “configuration, not coding” approach to solve the entire lifecycle of your cloud integration needs, including:

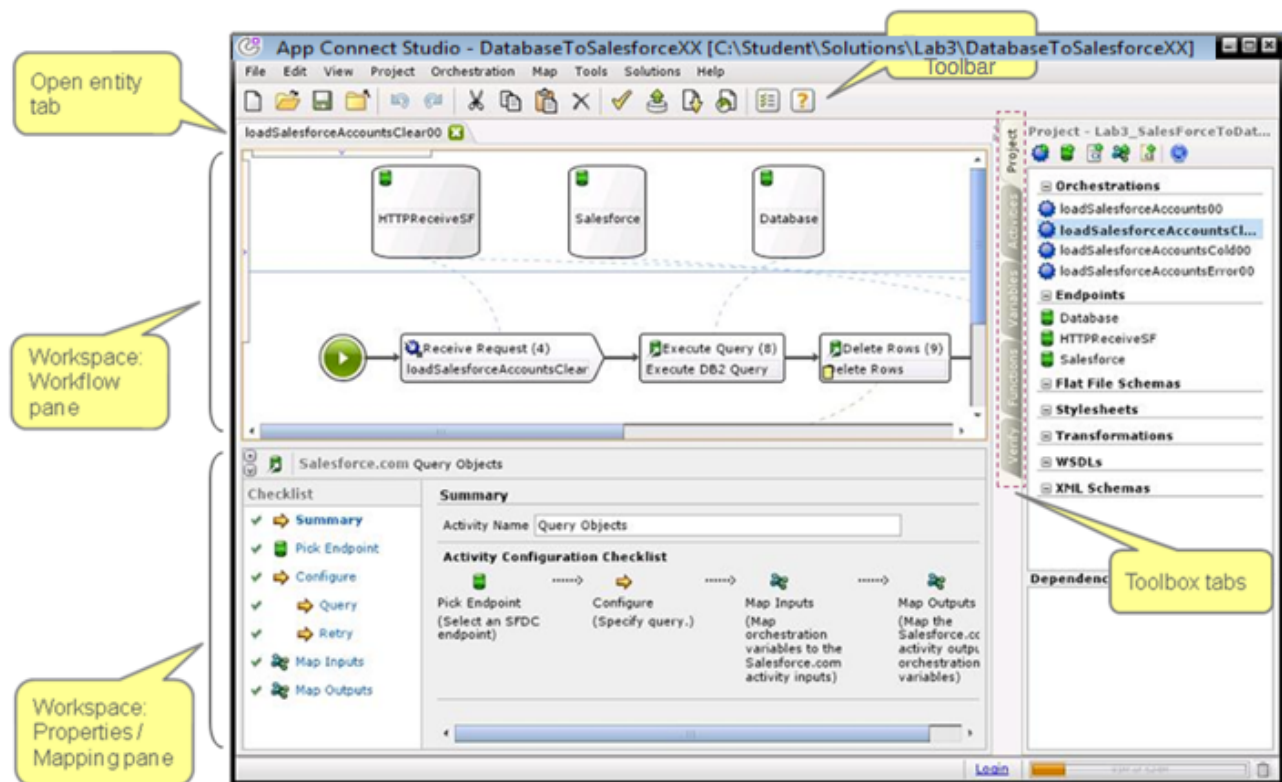
- **Data Migration** — You can access, cleanse and migrate data from legacy systems to your new SaaS and cloud applications and accelerate your adoption of these new applications. For example, you can migrate customer data from legacy CRM systems to a cloud CRM service.
- **Data Synchronization** — You can orchestrate integration processes to synchronize business data across cloud and on-premise applications in real-time. For example, you can synchronize order information between a cloud CRM service and an on-premise ERP application in real-time.
- **UI Mashups** — When you need to gather information from disparate systems and display this within the native user interface of a single application. You can mashup this data to present a single unified view, without leaving the current application. For example, you can mashup data from multiple applications and then provide the output back to the calling application.

Icons Used in Labs

The following symbols appear in this document at places where additional guidance is available.

Icon	Purpose	Explanation
	Important!	This symbol calls attention to a particular step or command. For example, it might alert you to type a command carefully because it is case sensitive.
	Information	This symbol indicates information that might not be necessary to complete a step, but is helpful or good to know.
	Trouble-shooting	This symbol indicates that you can fix a specific problem by completing the associated troubleshooting information.

IBM App Connect Professional Studio Overview

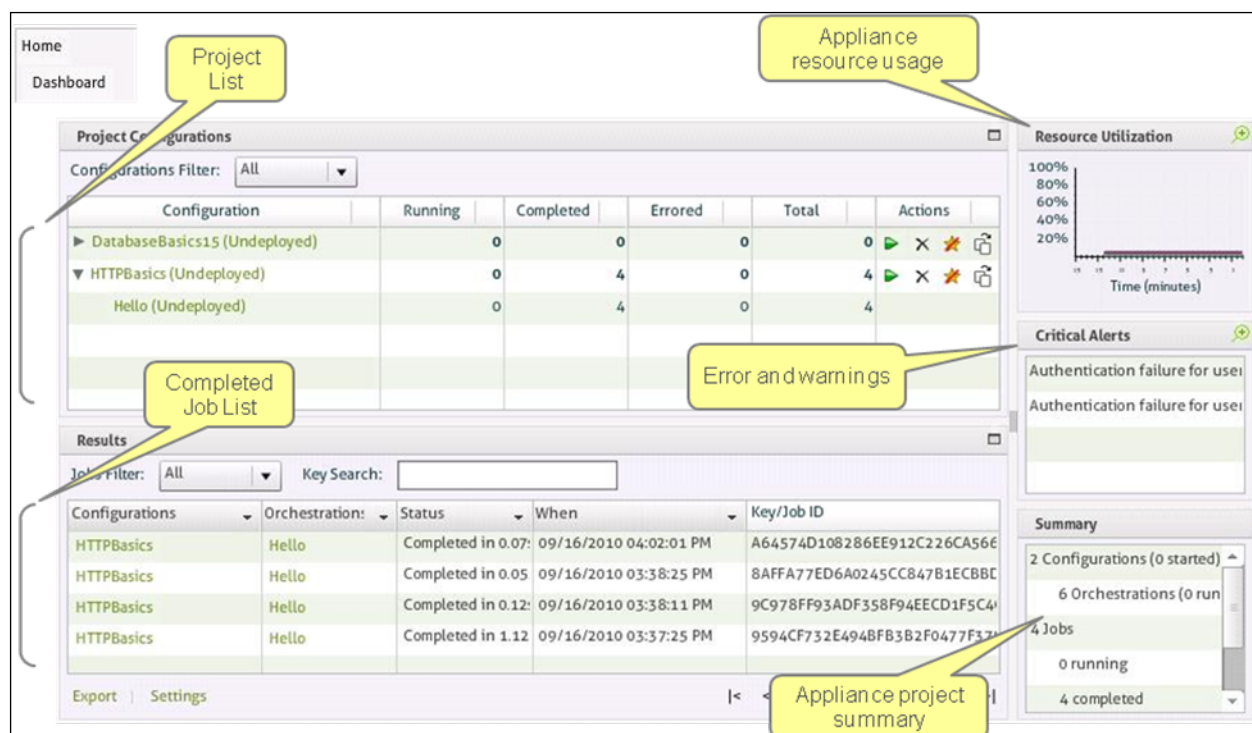


Studio provides a Workspace in which you design and configure your integration projects. The Toolbox is the area next to the Workspace where multiple tabs contain components that you need to build an integration project. Components such as endpoints, schemas, orchestrations, style sheets, transformation, WSDLs, activities, variables, and functions. Key areas of the Studio include:

- **Menu:** The main menu supports the following options: File, Edit, View, Project, Orchestration, Map, Tools, Solutions and Help
- **Toolbar:** The toolbar buttons support project level operations (create, open, save, and close); redo and undo; and cut, copy, paste, and delete. Additionally, these icons offer project validation, project publishing, project export, generation of project documentation. Finally, buttons allow setting of preferences and accessing Help.
- **Toolbox Tabs:** To the right of the Workspace is the Toolbox, which is comprised of five sideways-oriented tabs: Project, Activities, Variables, Functions, and Verify. (You can make the Toolbox larger or smaller by moving the invisible slider between the workspace and the Toolbox.)
- **Entity Tabs:** In the top half of the Workspace, the entities selected from Project tab display as tabs. You can manage these tabs by right-clicking on a tab and selecting the appropriate action from the drop-down menu: Close; Close Other Tabs; Close All Tabs; Close All But Current Tab.
- **Workspace:** When you select an orchestration, the Workspace displays. The top half of the Workspace displays a tab view of the entity selected from the Project tab. An orchestration or workflow contains the logical flow of your integration. Drag-and-drop activities from the Activities tab

onto the orchestration to visually define the logical flow of your integration. The bottom half of the Workspace is where you define or configure the activities you added to the orchestration. It is referred to by different names, depending on what you are doing at the time. If you are creating a map, this area is called the *Mapping Editor*. If you are configuring endpoints or configuring an activity, it is referred to as the *Properties pane*.

Web Management Console – Dashboard View



The Web Management Console (WMC) is a browser-based application hosted by the Integration Appliance. From any Web browser, the WMC provides the ability to configure and monitor hardware, orchestrations, and network status of an Integration Appliance.

System Administrators can use the WMC for the following monitoring tasks:

- Obtain Integration Appliance status information, such as memory-usage statistics, disk-space statistics, CPU status, fan status, and power-supply status.
- Obtain overall processing information at a glance, and then drill-down on specific orchestrations and obtain comprehensive information about specific messages, activities, errors, and other details.
- View logs and error notifications.
- Deployment of projects. A deployment of a project contains all runtime components necessary to execute the processing defined in the orchestrations of the project. With respect to deployment, there are two important concepts to keep in mind as you develop a project:
 - Publishing and Deploying are two distinct steps. *Publishing* is the act of promoting a project from Studio to an integration environment (cloud or Integration Appliance) or to another

intermediate location. A project does not run in the integration environment until you *deploy* it.

- Projects can have multiple configurations that support completely different environments (development, test, production). Using Studio to create configuration properties for projects gives you greater flexibility at deployment time, because you can bind the properties to different environments.
- View the Project List. The dashboard view provides a listing of all the Studio published projects. A project containing multiple orchestrations will show the status of each orchestration. Projects that are published are represented by the green arrow icon in the 'Action' column and are listed as 'Undeployed' in the 'Configuration' column. Once a project is deployed, the status in the 'Action' column changes to a red bar icon and the project will be listed as 'Running' in the Configuration column.
- View the Completed Project List. As you run each test on your completed orchestration it is listed in the lower half of the dashboard view.

Lab 1 An App Connect Professional Hello

1.1 Overview

In this lab, you will create a new project, [HTTPBasicsXX](#), with an orchestration that receives an HTTP request and sends back a hard-coded response.

When the orchestration is complete, you will publish and deploy the project. The orchestration will then be tested using both HTTP GET and POST methods. With the orchestration Logging Level set to “All” you will be able to examine both the headers and body of the HTTP request.

1.2 Concepts

A **Project** is a set of configuration documents that describe the data and components, such as Endpoints and Orchestrations, involved in an integration solution.

Endpoints define the configuration information needed to communicate with external systems or to provide access to the integration appliance’s running orchestrations.

An **Orchestration** is the sequence of processing activities (such as data mapping and control logic activities), endpoints, and data types defined and configured using Studio.

An inbound **HTTP Endpoint** allows the appliance to receive HTTP requests from an external system.

An **HTTP Receive Request** activity may be used as a “Starter” activity. In actual use cases, this type of starter is independent of polling intervals. As a lightweight, testing tool, it may be used to kick off a process or to perform queries that allow you to test the effect of your orchestrations.

Later, the **HTTP Request-Response model** will be used in conjunction with database operations to 1) return contents of a database table and 2) to insert a new row into the same table.

The **Web Management Console** (WMC) is a browser-accessible tool that allows you to monitor the status of orchestrations, the health of the Appliance and to drill down to detailed information for specific messages, orchestrations or their activities, and any errors that occur.

1.3 Activities Used

- HTTP Receive Request
- HTTP Send Response

1.4 Document Keys

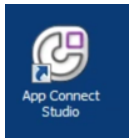
- Menu bar: Edit → Preferences
- Studio Artifact: **HTTP Receive Request**
- Entered Text: Hello Receive Request
- Project Object Name: SampleProject1
- Context Menu Item: Define Default Value
- Activity Checklist Step: Pick Endpoint
- Orchestration Variable: Account

1.5 App Connect Professional Hello

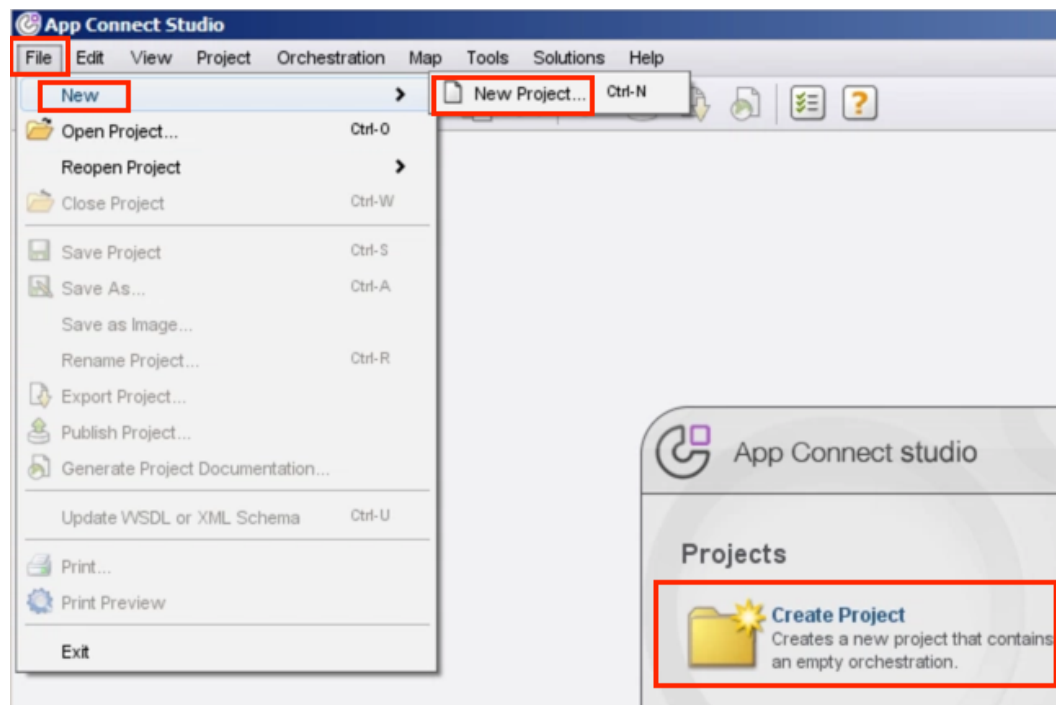
Your student suffix is: _____

During the lab work, project names and artifacts will contain a 'XX'. Replace the 'XX' with your student suffix.

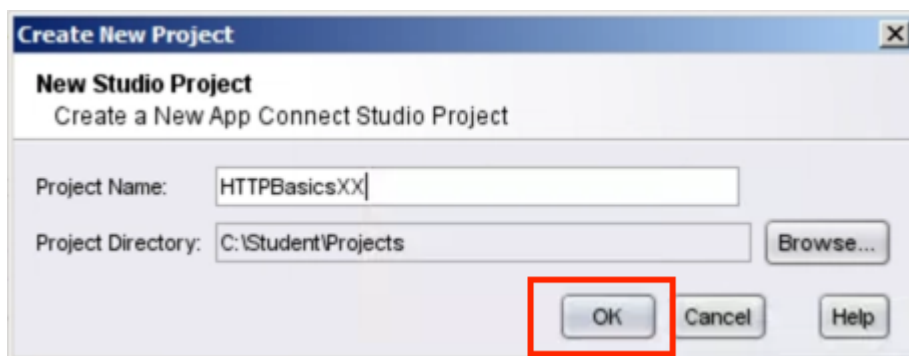
Start the IBM App Connect Studio. The icon for the IBM App Connect Studio is located on the desktop.



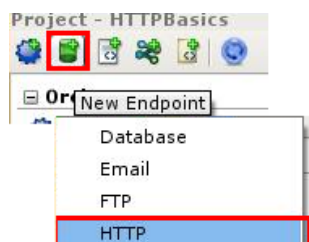
- __1. Create a new project.
- __2. Click *File* → *New Project* or click Create Project on the splash screen.



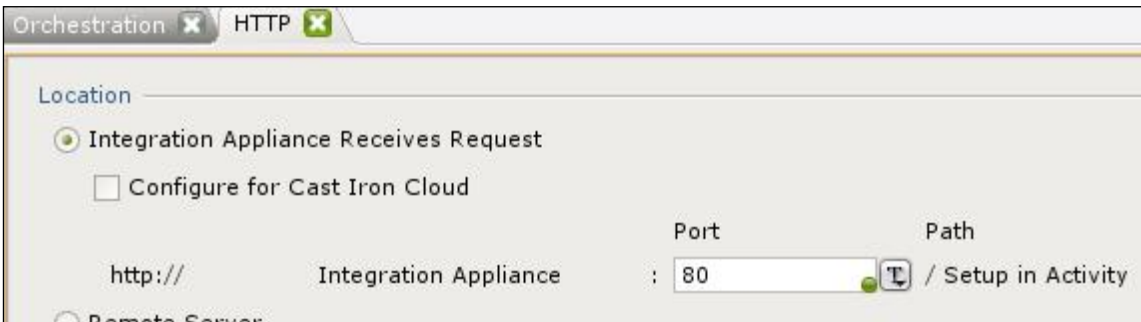
- __3. Enter "HTTPBasicsXX", where XX is your student suffix.
- __4. Click OK.



- __5. Create a new HTTP Endpoint.
 - __a. Use the New Endpoint button on the Project tab.



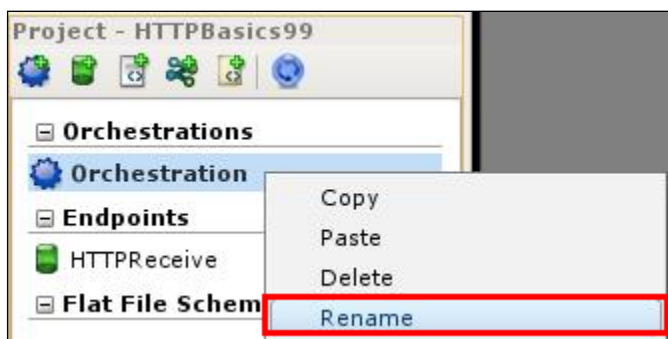
- __6. For this **HTTP Receive Request** endpoint the defaults will suffice.



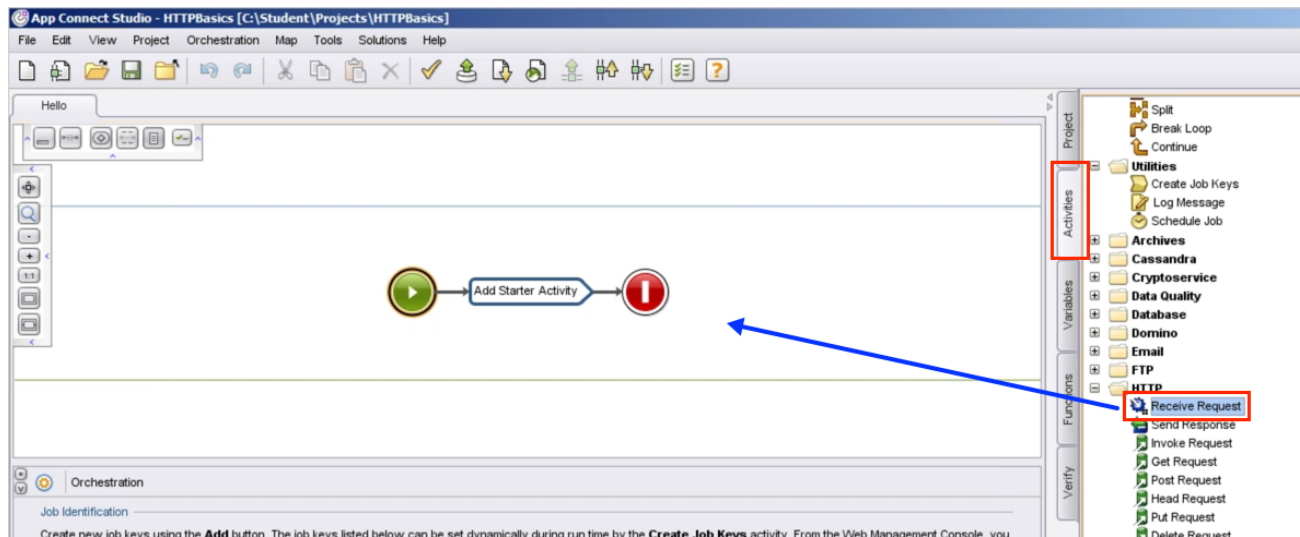
- __7. Close the HTTP tab, and then rename the endpoint to **HTTPReceive** by clicking on the name in the Toolbox.



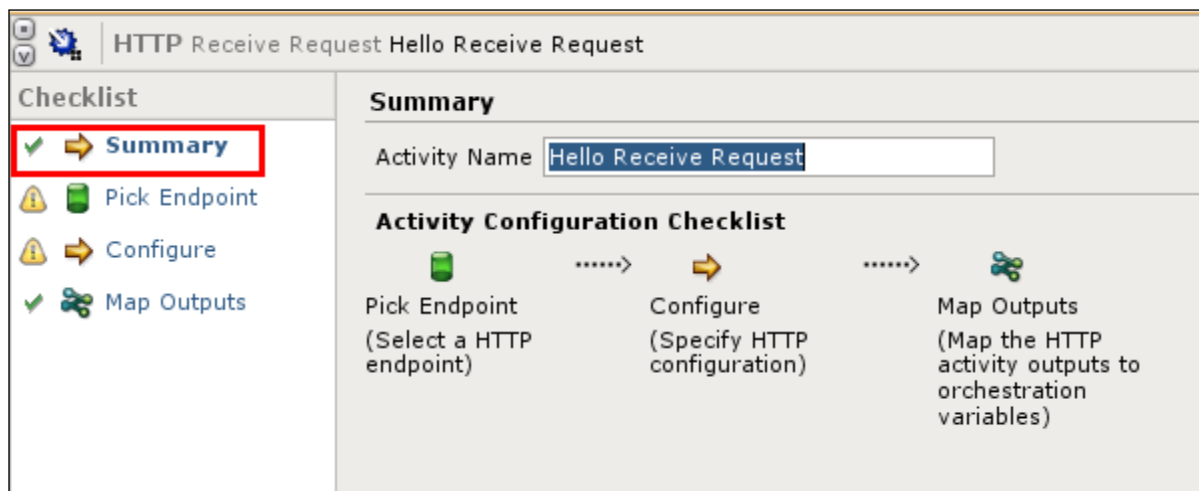
- __8. Rename the default orchestration to **HelloXX**, this time by right-clicking the existing name (as an alternate method).



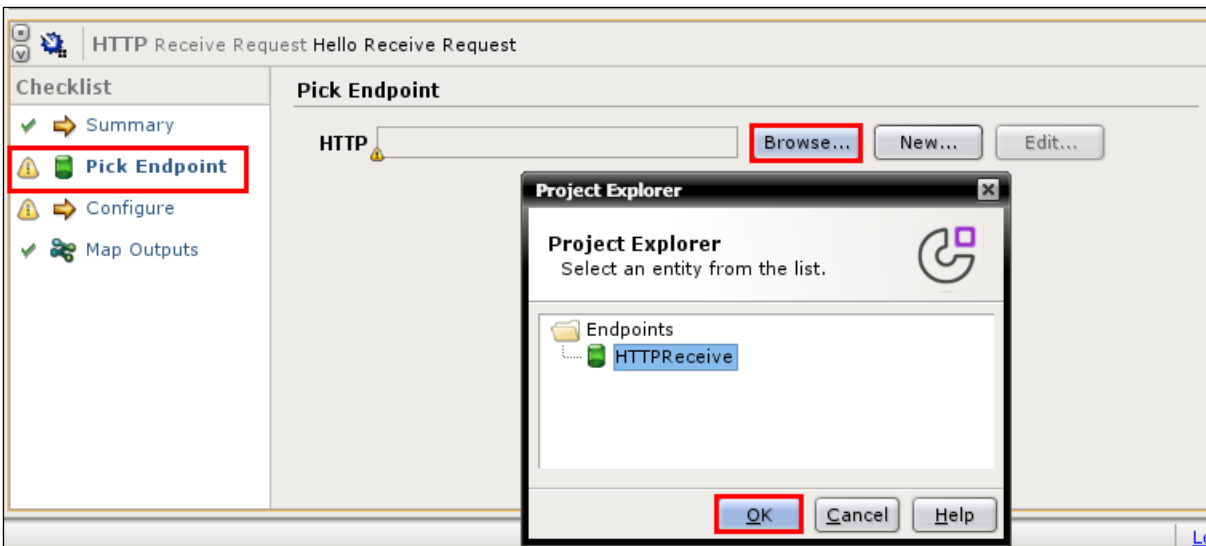
- ___9. Start the orchestration by opening the Activities tab in the Toolbox and dragging an **HTTP Receive Request** activity on to the workspace. For new activities in sequence you don't have to be precise about where you drop the object. Studio will place it in the orchestration correctly.



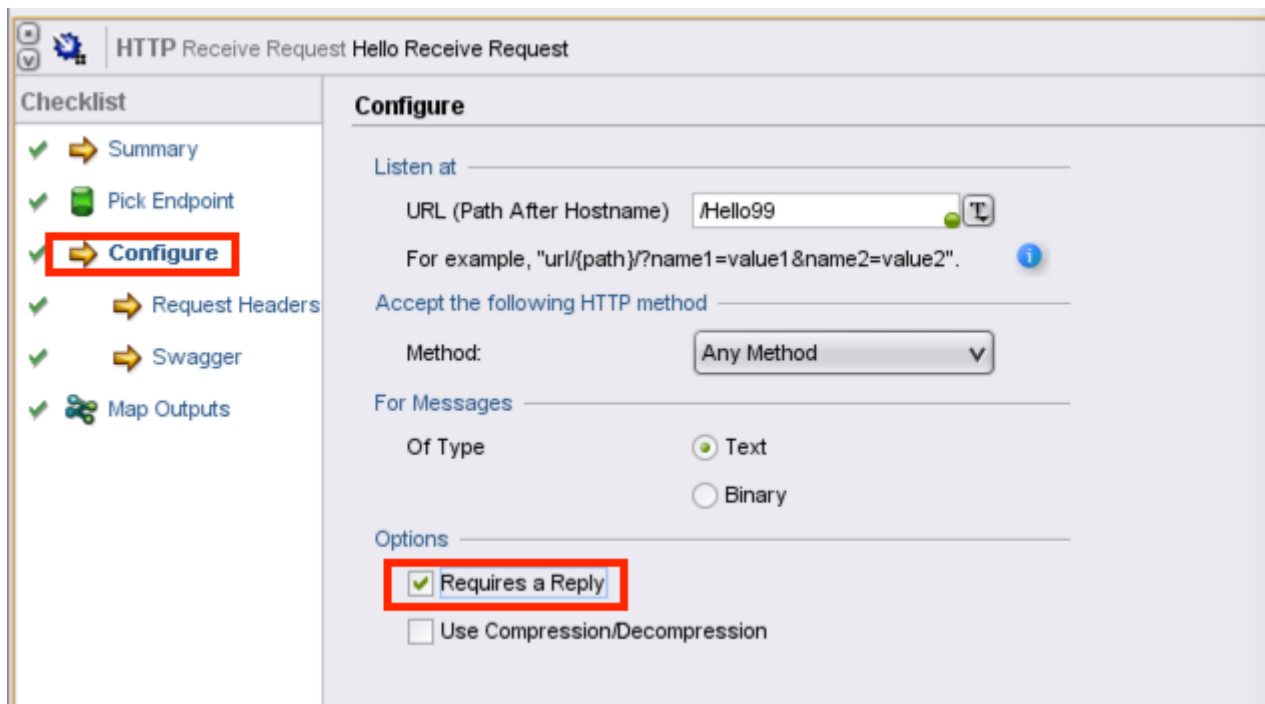
- ___10. Keep in mind that the first activity in an orchestration must be a “Starter” activity, such as this one. When it's added to the flow, the Properties pane will display the summary information for this activity. Start by renaming this activity to **Hello Receive Request**.



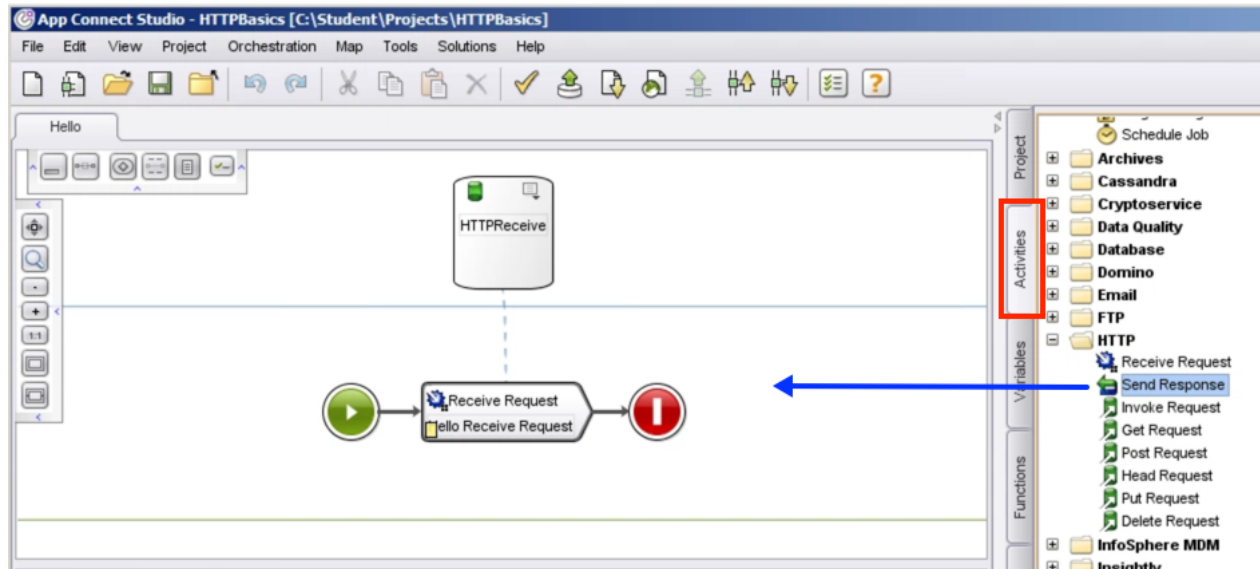
- ___11. The checklist graphically shows the configuration steps that need to be completed. Select the Pick Endpoint step and browse for the HTTPReceive endpoint you defined earlier.



- ___12. Select the Configure step and enter 'HelloXX' for the listening URL, and also check the option 'Requires a Reply'. For example:



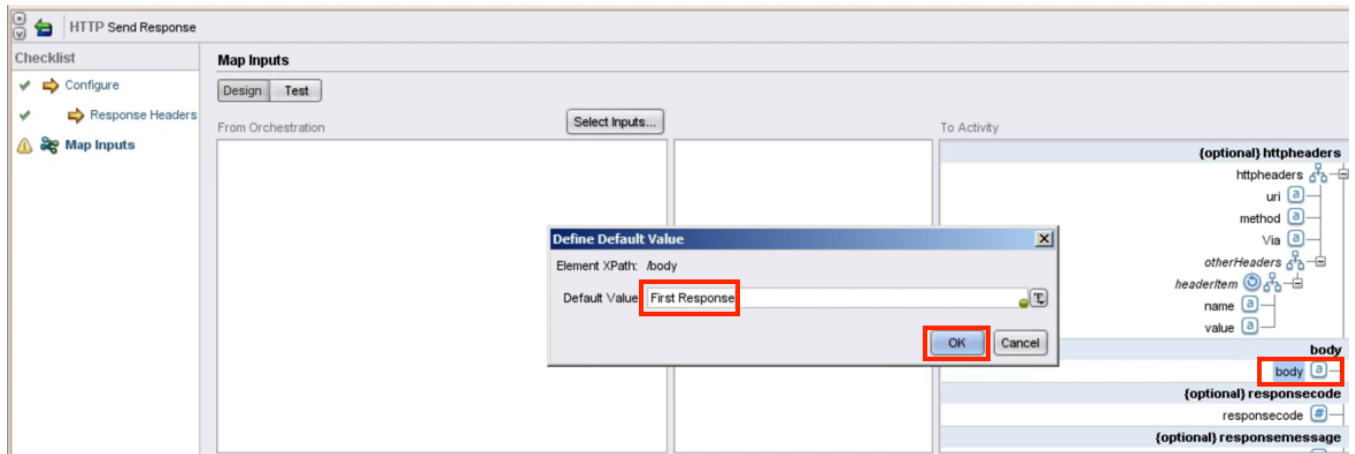
- ___13. Add an **HTTP Send Response** activity to the orchestration, by dragging an HTTP Send Response activity on to the workspace:



- ___14. The Configure step should already show that this activity is replying to Hello Receive Request.

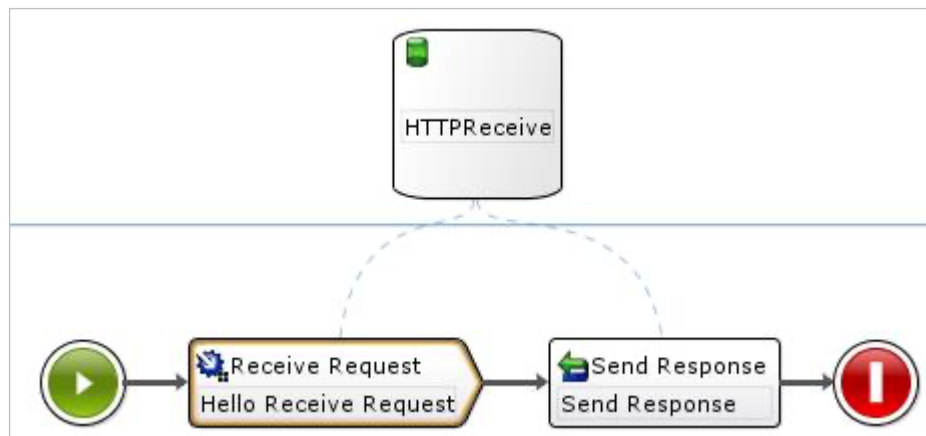


- ___15. Configure the Map Inputs step to simply return a fixed value. Right-click the “body” object to bring up its context menu and select the Define Default Value option and type in ‘First Response’.

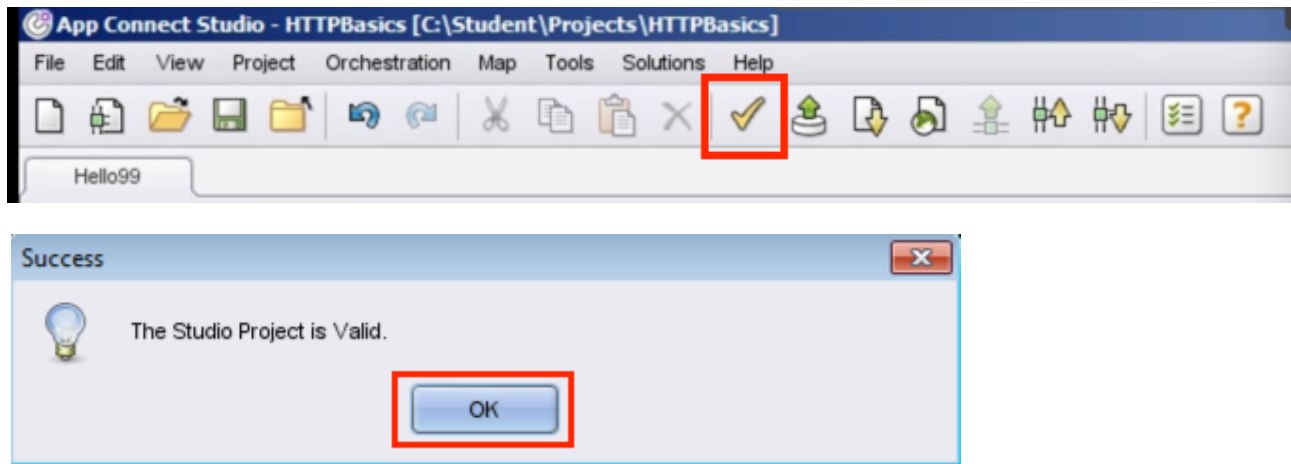


Notice that the body object has a new icon showing it has a Default value.

- ___16. The completed orchestration will look like this:



- __17. Validate the project via *Orchestration* → *Validate*, or via the Toolbar icon:



- __18. Save your project from the Toolbar, using the Toolbar icon:



- __19. Publish the project, saving first if prompted, via menu item *File* → *Publish Project*, or via the Toolbar icon:

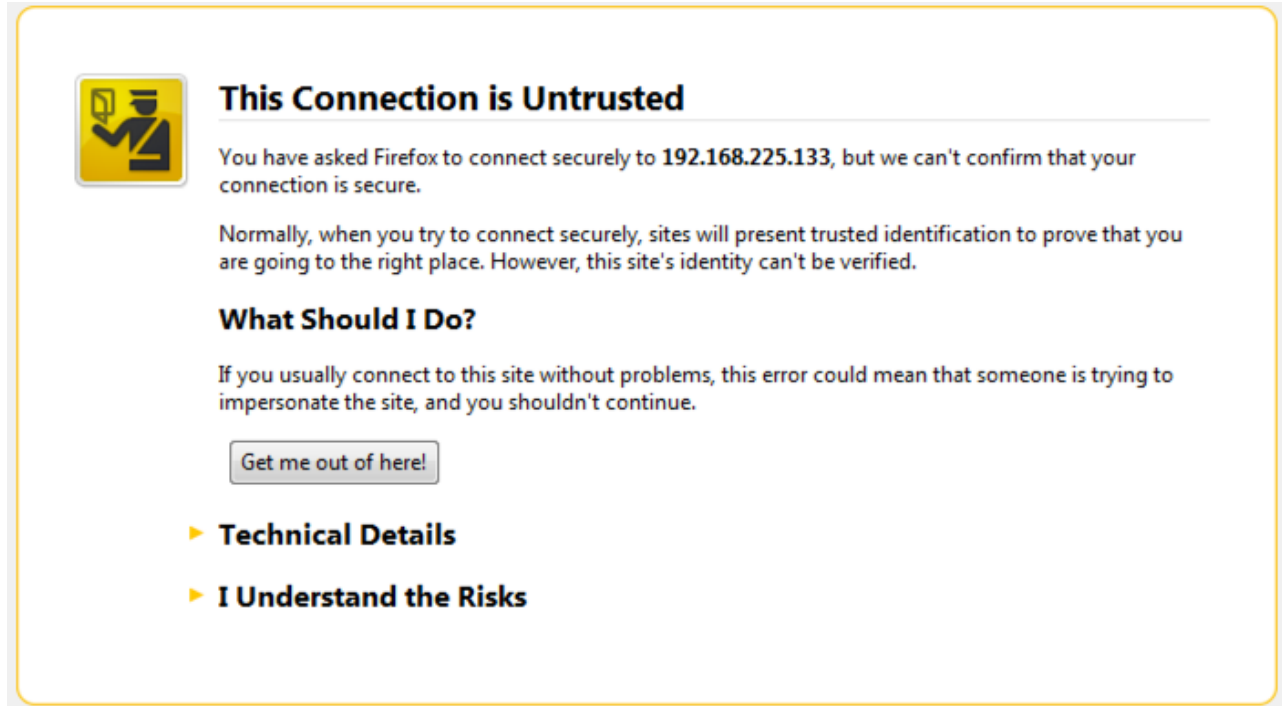


- __20. Enter the connection details the first time and click OK. A successful publish will return a confirmation message (Refer to Connection Parameters Spreadsheet).

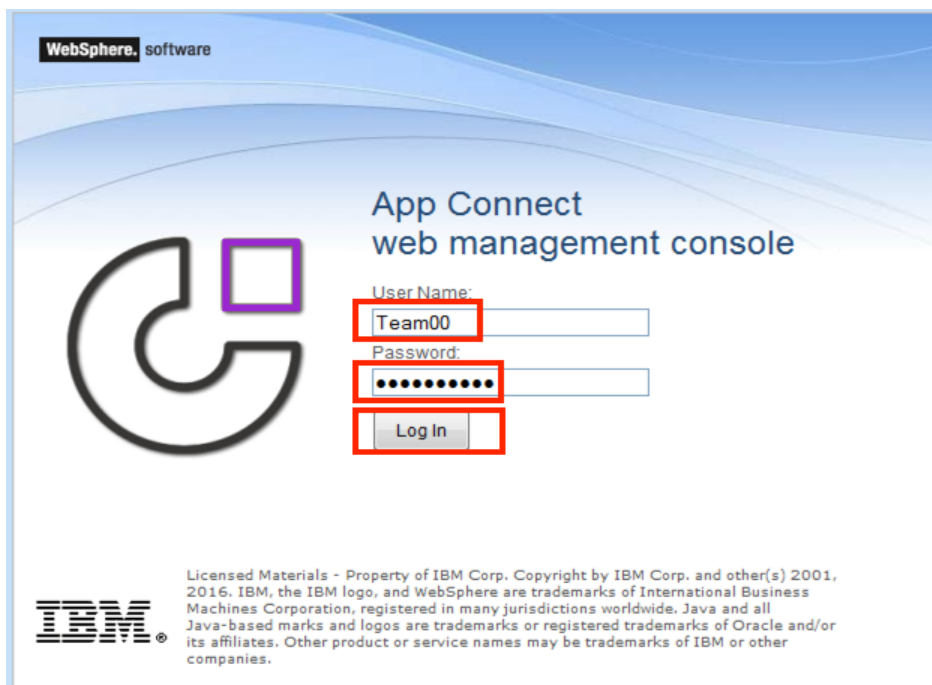
The 'Publish Project' dialog box is shown. It has a title bar 'Publish Project' and a subtitle 'Publishes a WebSphere Cast Iron Studio Project to an Integration Appliance'. The dialog contains three text input fields: 'Host name:' with the placeholder '<Appliance EMgmt Address (WMC)>', 'User Name:' with the placeholder '< WMC Id >', and 'Password:' with the placeholder '< WMC Password >'. Below the password field is a checked checkbox labeled 'Secure'. At the bottom right, there are 'OK' and 'Cancel' buttons, with the 'OK' button highlighted by a red rectangle.

- ___21. From the Firefox browser, launch the Web Management Console using the URL, Appliance EMgmt Address (WMC) provided on the Connection Parameters Spreadsheet.

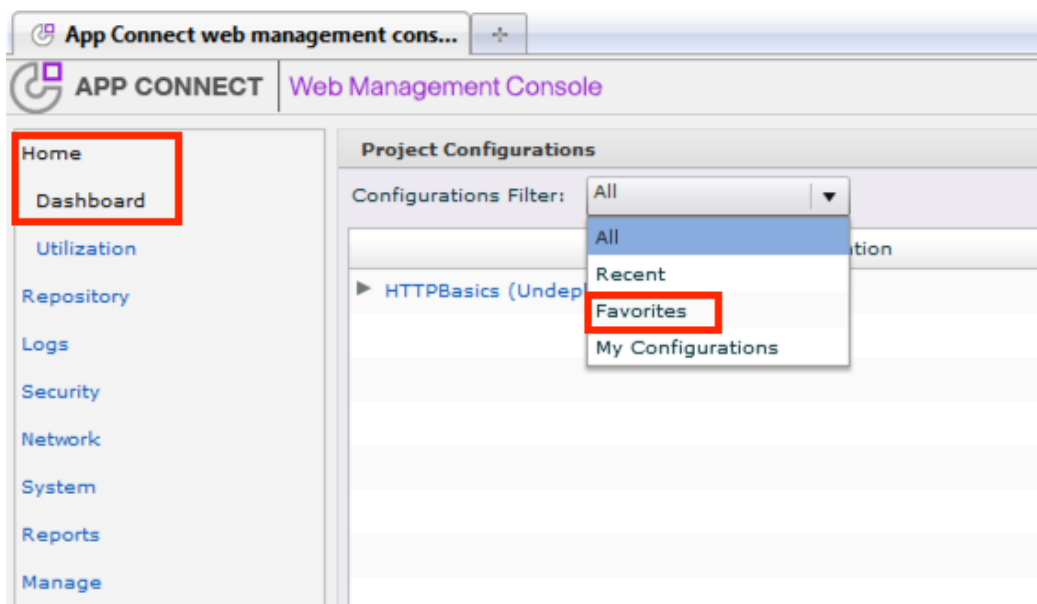
If you are prompted with a security notice, click “I Understand the Risks” and “Add Exception” then click “Confirm Security Exception”.



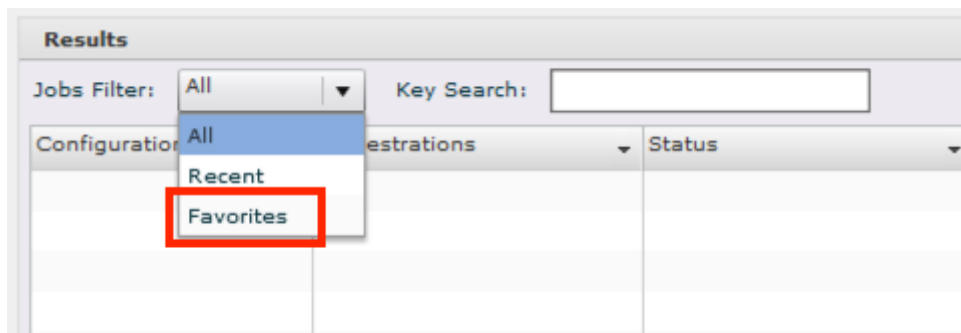
Log in using the, <WMC Id> and <WMC Password> from the Connection Parameters spreadsheet.



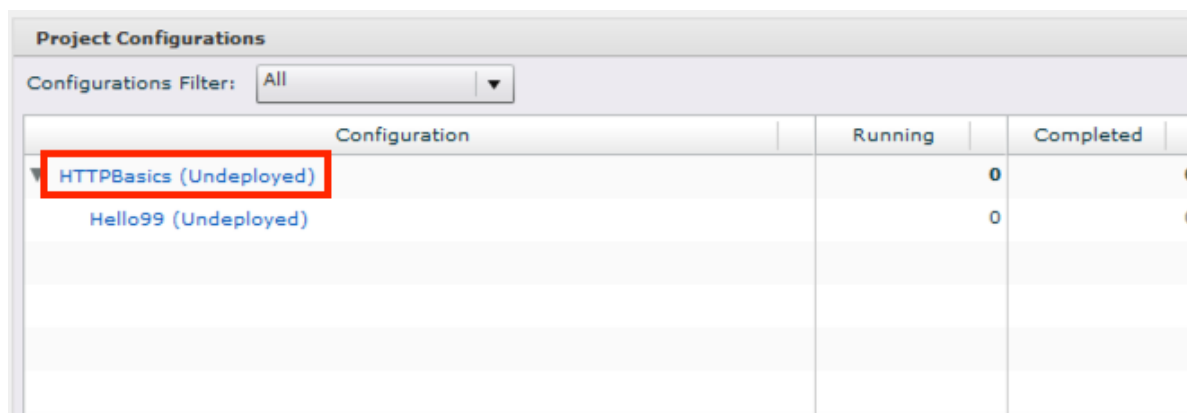
- __22. Once in the Web Management Console, from the *Home* → *Dashboard*, select the 'ALL' drop-down in Project Configurations and select 'Favorites'. This will limit the view to only your projects.



- __23. Repeat this step in Results, to limit the view to only your projects.



- __24. From the *Home* → *Dashboard* page click the project/configuration name link.



- __25. Clicking the configuration's link opens the edit page with the list of orchestrations displayed.

__26. Click the Edit button in the Orchestrations section.

Configuration Details

Summary

Configuration: HTTPBasics # Orchestrations: 1
Status: Undeployed # Properties: 0
Last Published: 10/26/2016 07:58:34 AM # Assets: 0
User: Team00 [admin] # Downtimes: 0

[Download](#)

Orchestrations (1)

Name	Status	Logging Level
Hello99	Enabled	Error Values

[Edit](#)

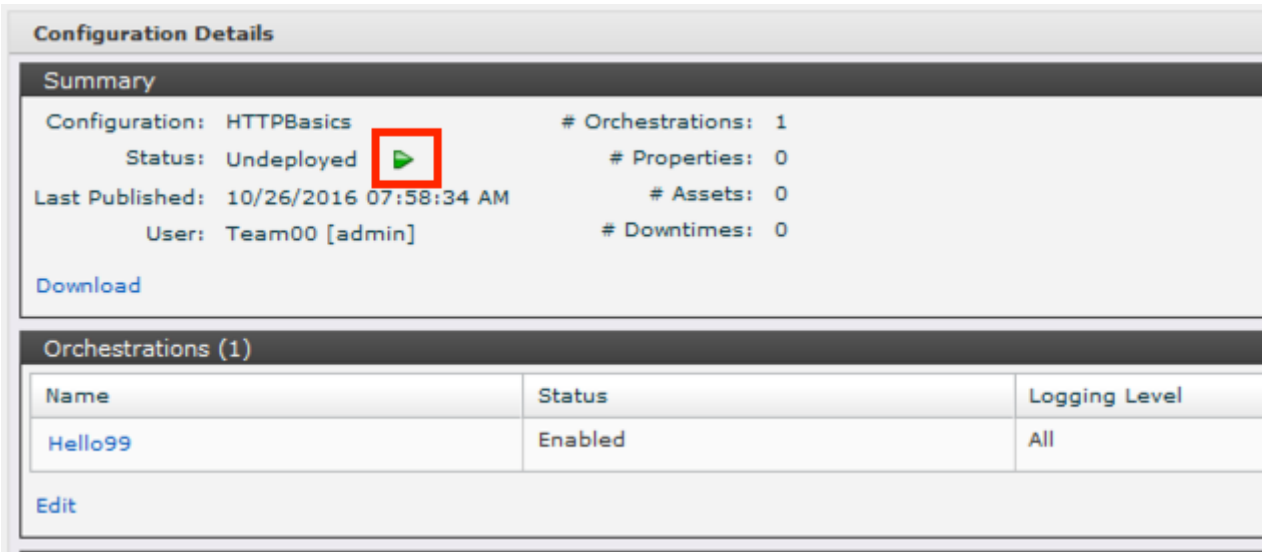
__27. Apply changes to all the orchestrations in the project with the unnamed top row, or select the orchestrations individually. Change the Logging Level to 'All' and click the Save link.

Edit Orchestration Settings ment Console

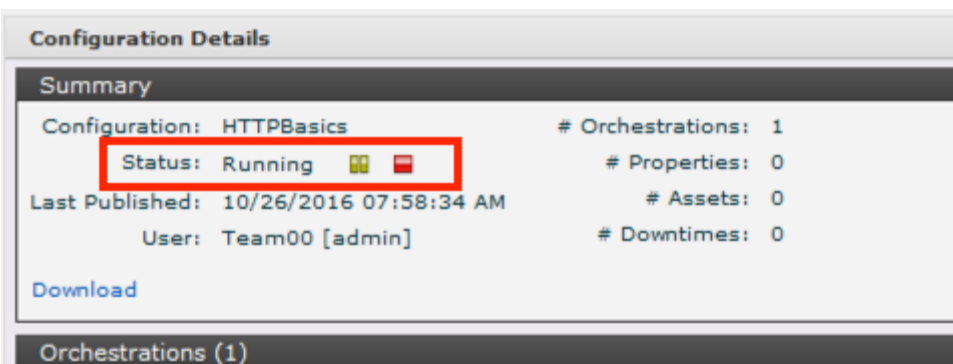
Name	Enabled	Logging Level	Log Synchronously	Max Simultaneous Jobs
	☐	None	☐	☒ Unlimited
Hello99	☒	All	☐	☐ Unlimited 10

[Save](#) [Cancel](#) | [Help](#)

- __28. Back on the Configurations page, deploy and run the project using the Run Configuration icon:



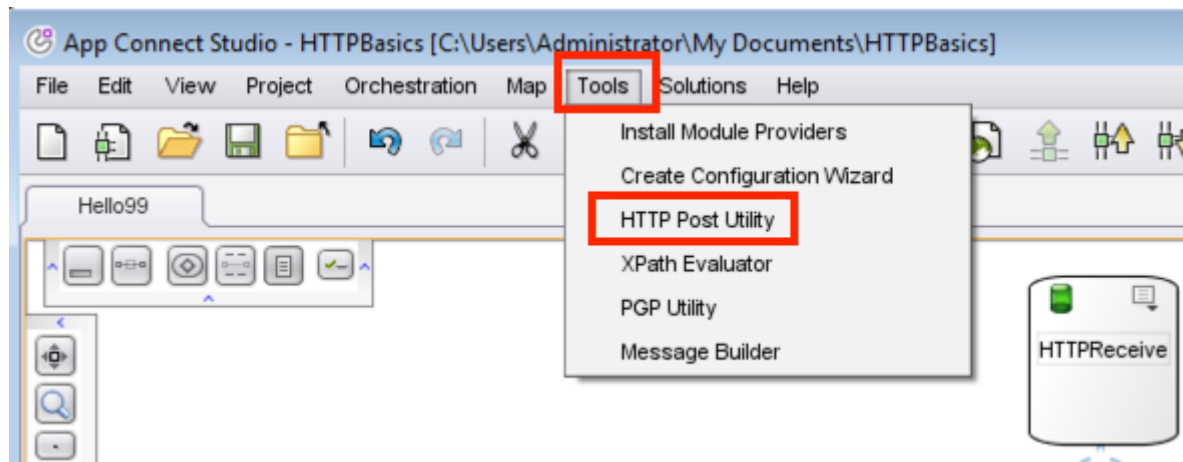
- __29. After a few moments, the icons change to show the status of the configuration to running. If the WMC does not refresh right away, hit the F5 key to force the refresh.



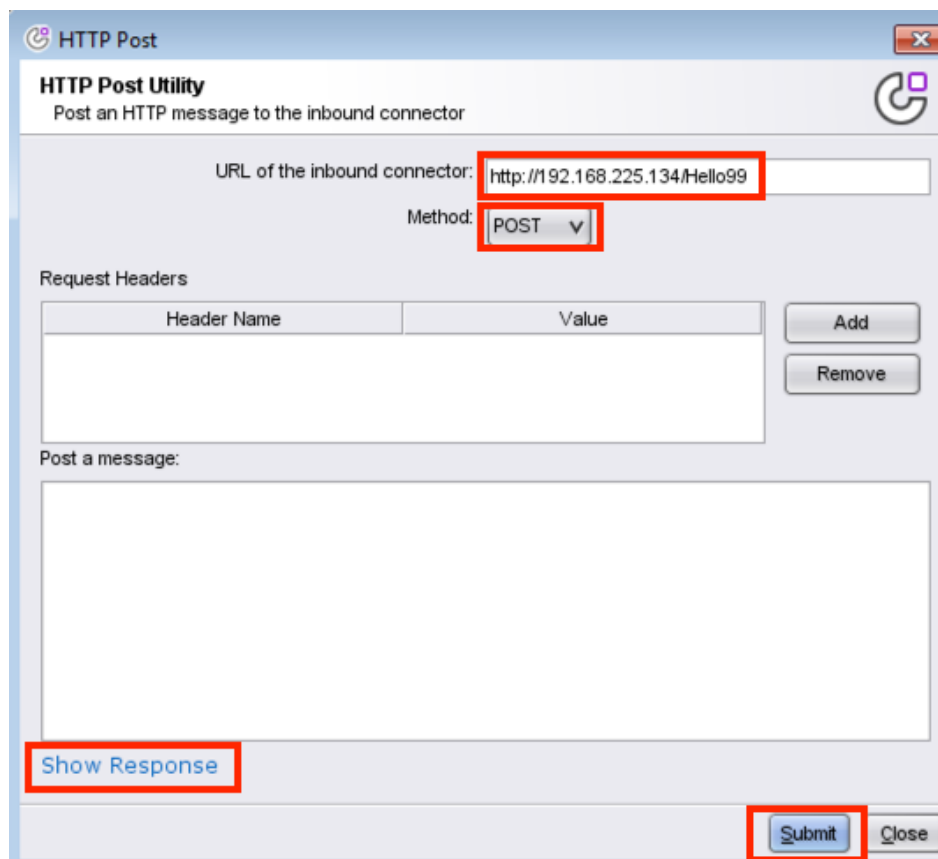
- __30. In the App Connect Studio, trigger the orchestration twice, once for POST and once for GET. To issue a POST, use the HTTP Post Utility that is included with the Studio installation. To issue a GET, simply enter the URL into your browser. In either case, use the IP address of the appliance's Edata interface. See the Connectivity Parameter spreadsheet.

POST: Launch the HTTP Post Utility with the appropriate URL. Then browse for the input file in the C:\Student\Lab Resources\Lab 1 folder. After submission you should see the reply text that was entered into the body object of the HTTP Send Response activity.

HTTP Post Utility from the Toolbar



Click the Show Response to open up the window that will allow you to see the output results. Click Submit to execute the orchestration.



- ___31. The WMC will show the results of each run. From the Dashboard, expand the configuration and click the orchestration being tested. This will display the Orchestration Details page that shows the results of each run of the orchestration.

Project Configurations

Configuration	Running	Completed	Errored	Total	Actions
HTTPBasics (Running)	0	2	0	2	[Icons]
Hello99 (Running)	0	2	0	2	

Orchestration Details

Configuration: HTTPBasics Orchestration: Hello99

Job ID: Key: Status: Any Start Time: 12:00 AM End Time: 12:00 AM Search Reset

Key/Job ID	Status	Start Time	End Time
ABDDA0CDECC882B83D2B4ADA164C487F	Completed in 0.09s	10/26/2016 08:28:17 AM	10/26/2016 08:28:17 AM
86808E3E3933B834098D9180CA084B7A	Completed in 1.20s	10/26/2016 08:27:36 AM	10/26/2016 08:27:37 AM

32. Select the Job ID link for the POST run in the top pane. Click the 'Details' link in the bottom pane and then the HTTP Send Response section. Click the 'body (input)' link to dynamically generate a file with the variable's contents.

This will open a file containing the response that was displayed in the browser.

Subsequent labs will present better opportunities for viewing input and output values.

Job Details - ABDDA0CDECC882B83D2B4ADA164C487F

Summary Job Keys **Details**

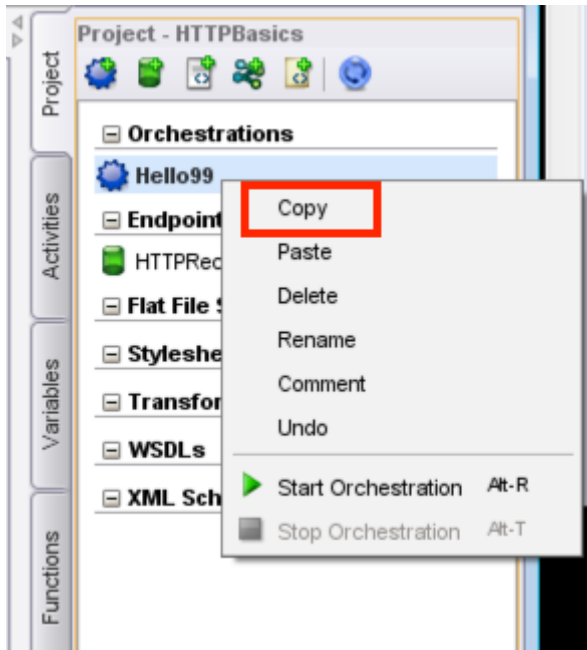
Displaying variables and events for activity: HTTP - Send Response

Elapsed	Activity	Variable
0.019s	HTTP (4) Hello Receive Request	body (input)
0.030s	HTTP (7) Send Response	httpheaders (input)
		responsemessage (input)
		responsecode (input)
		header (input)

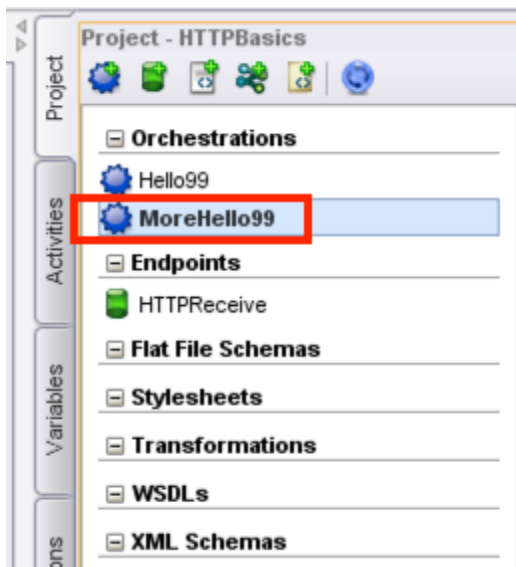
Your content is ready. [Download Now](#) [X]

1.6 Enhanced App Connect Professional Hello

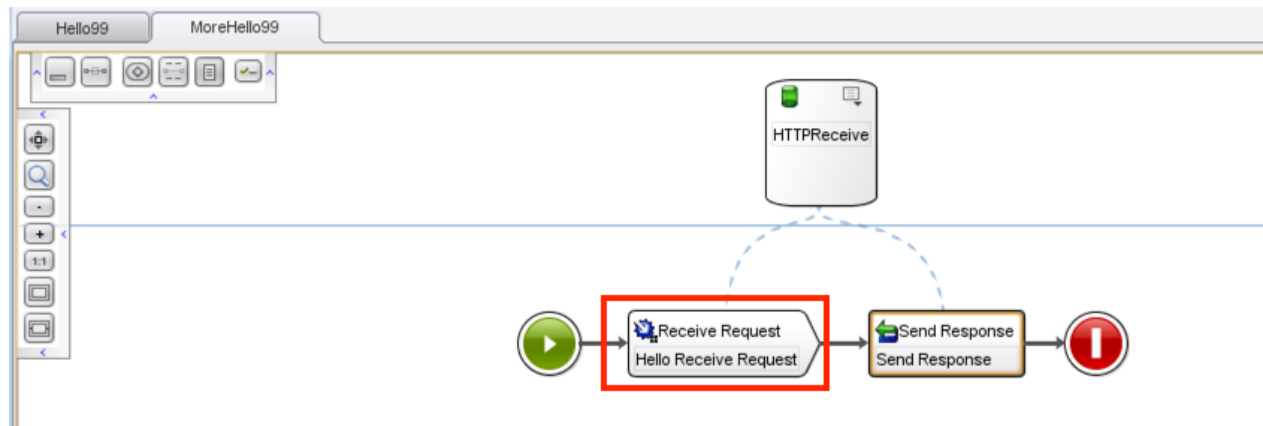
- __1. In the App Connect Studio right-click the orchestration to Copy and Paste a new orchestration from your existing orchestration.



- __2. Rename the copied orchestration to [MoreHelloXX](#).



- __3. Highlight the Receive Request activity on the workspace.



- __4. Select Configure from the Checklist and change the URL to MoreHelloXX. Ensure “Requires a Reply” is selected with the checkmark as indicated in the screenshot below.

The screenshot shows the 'Configure' dialog for the 'HTTP Receive Request Hello Receive Request' activity. The dialog is divided into two main sections: 'Checklist' and 'Configure'.

Checklist:

- Summary
- Pick Endpoint
- Configure** (highlighted with a red box)
- Request Headers
- Swagger
- Map Outputs

Configure:

Listen at _____

URL (Path After Hostname) **MoreHello99** (highlighted with a red box)

For example, "url/{path}?name1=value1&name2=value2".

Accept the following HTTP method _____

Method: Any Method

For Messages _____

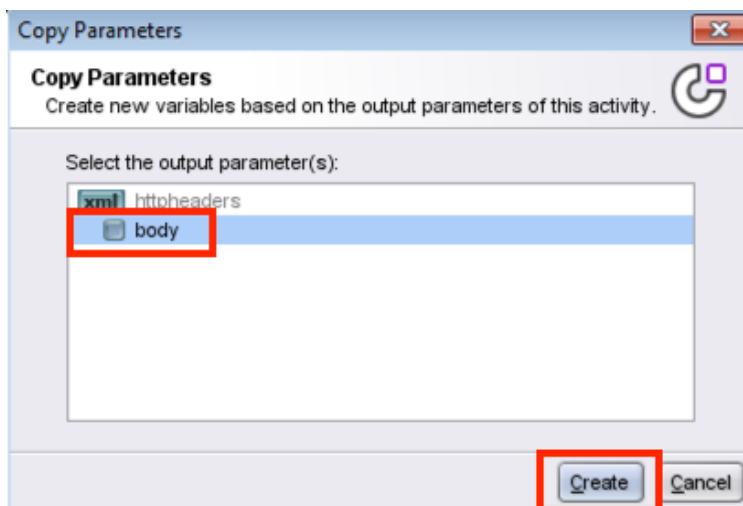
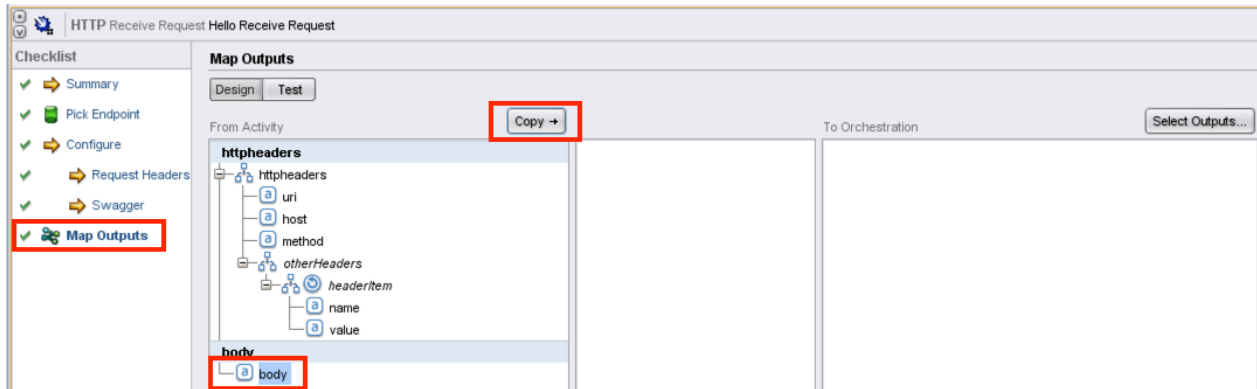
Of Type ☒ Text ☐ Binary

Options _____

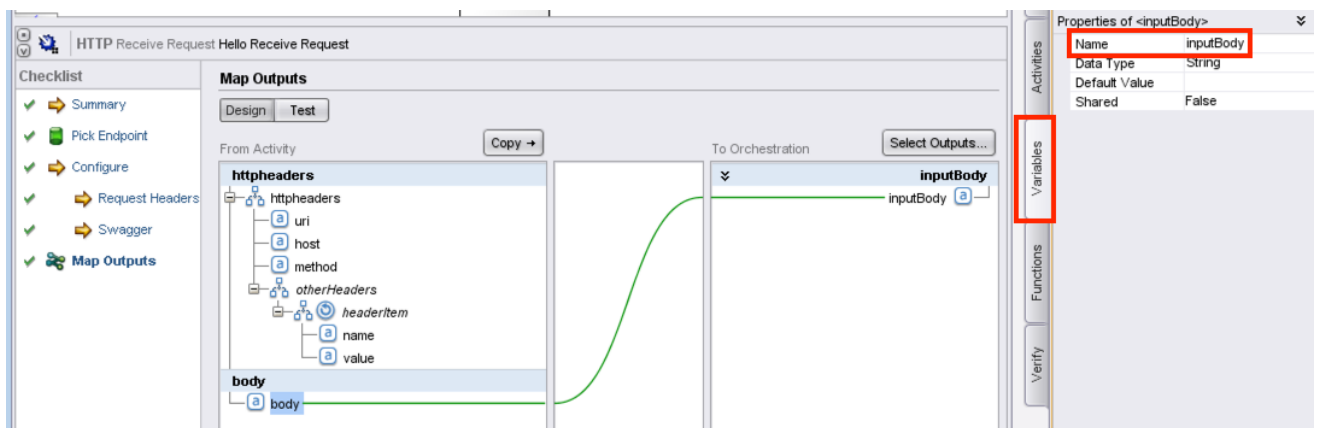
☒ Requires a Reply

☐ Use Compression/Decompression

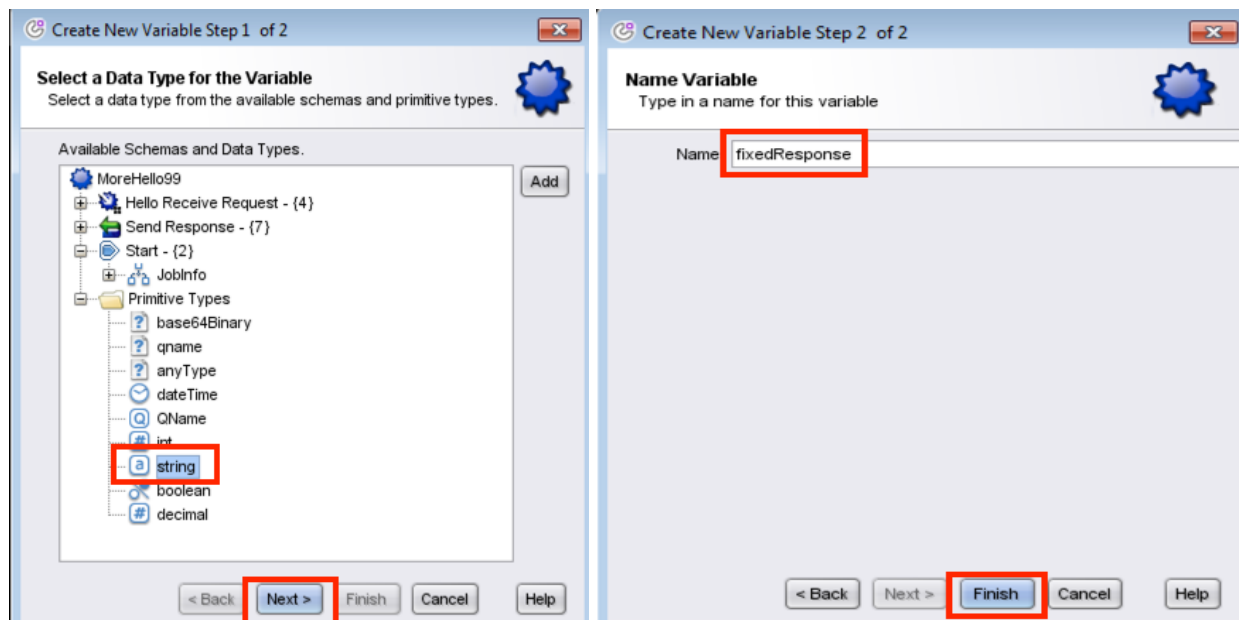
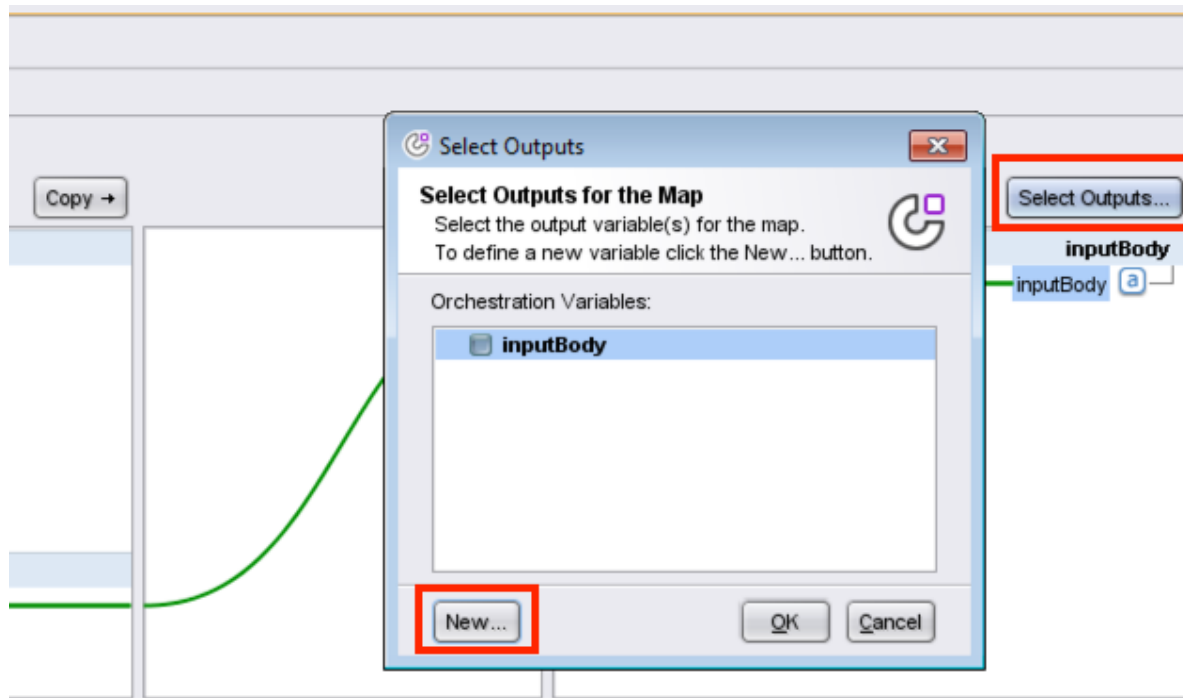
__5. Map Outputs: Copy input body to orchestration, click the body and select Copy.



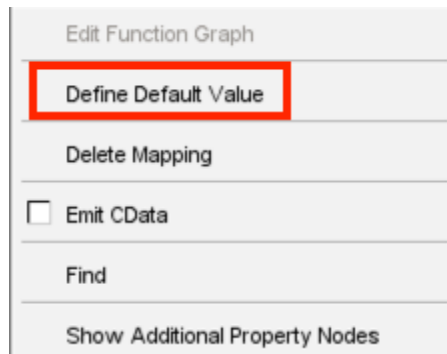
__6. Select the 'Variables' Tab and rename the orchestration variable to inputBody.



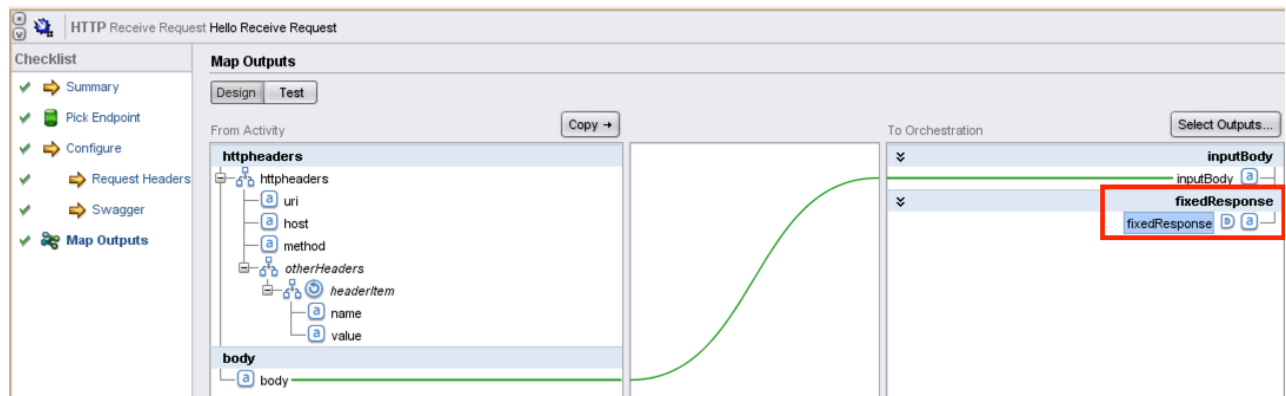
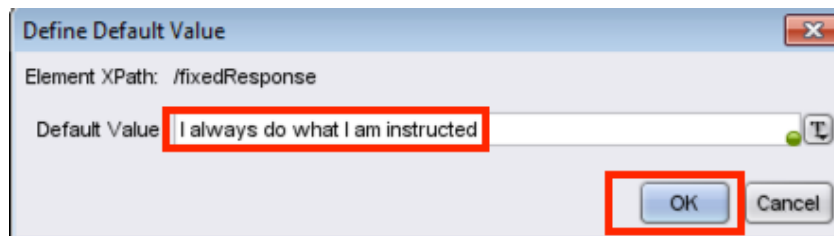
- ___7. Manually create a new string variable with a default value of your choosing using the Select Outputs and New buttons.



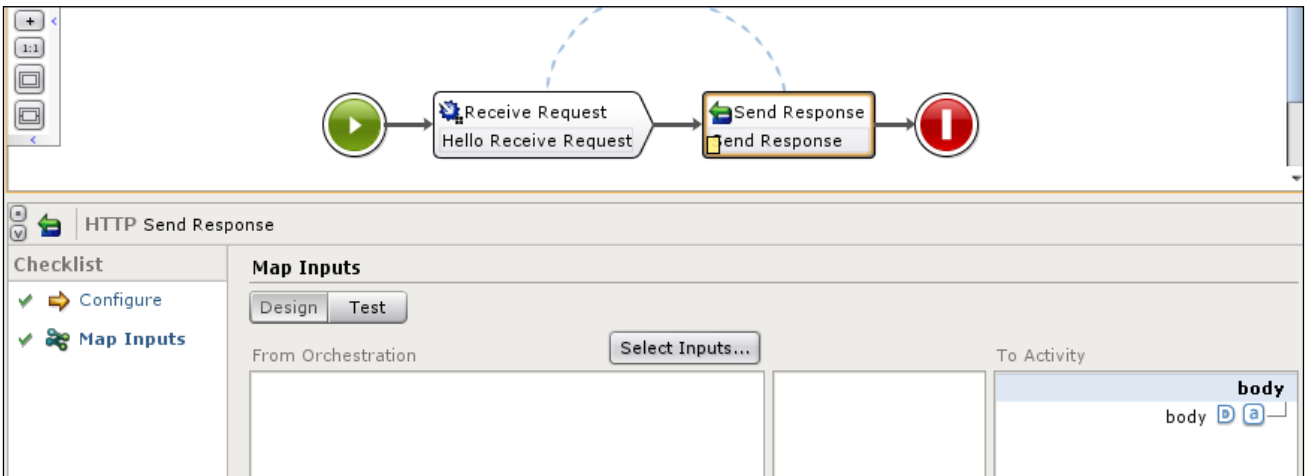
- __8. Right Click, on 'fixedResponse' and define a default value.



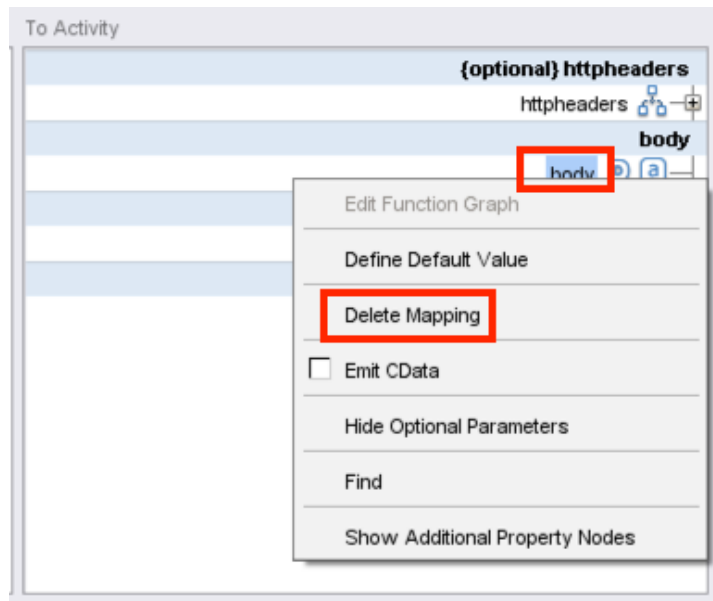
- __9. Add the default text: ex: 'fixedResponse' = "I always do as I am instructed".



__10. Highlight the Send Response activity in the workspace.

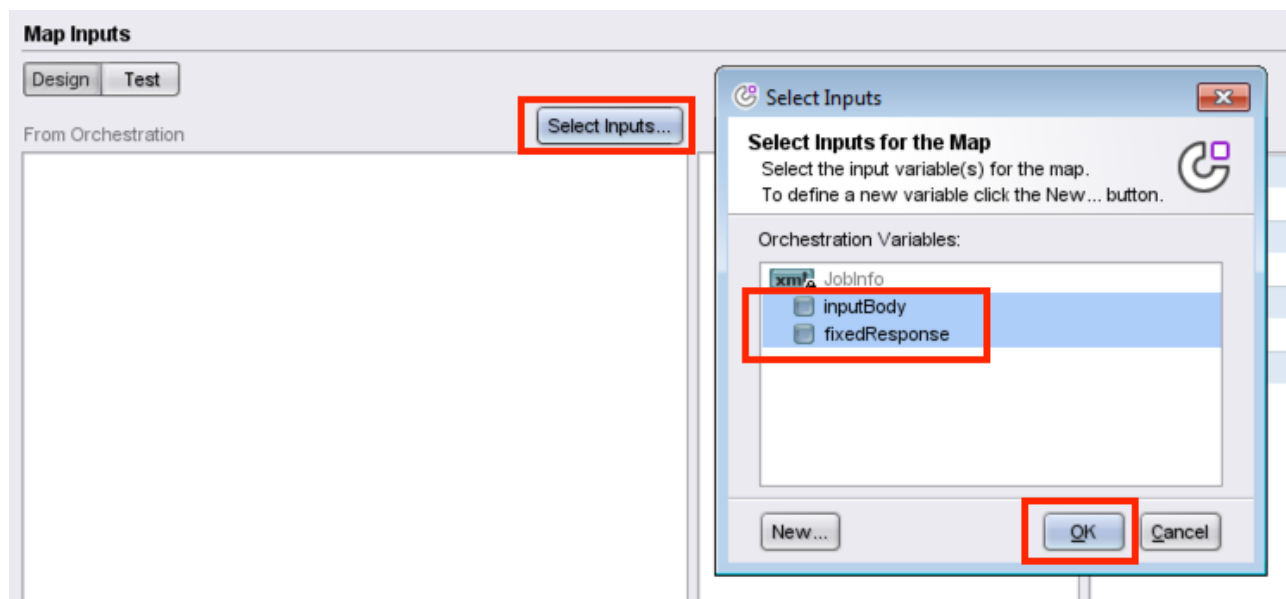


- __11. Select the Map Inputs, right click body and delete the default mapping for body (context menu of object).

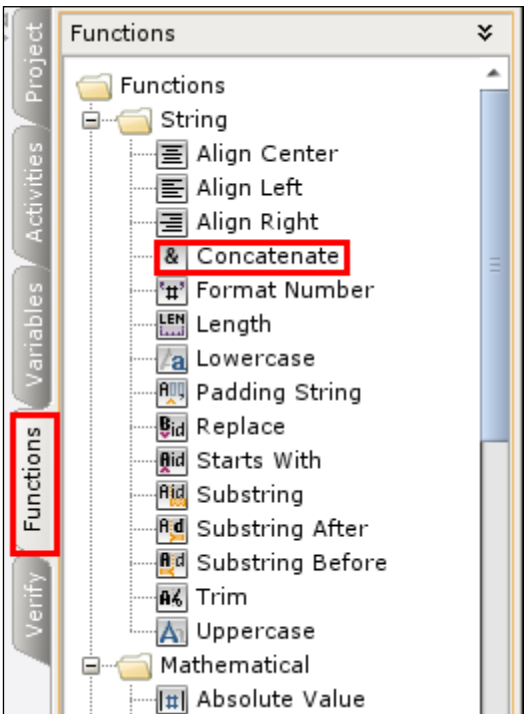


- __12. Click 'Select Inputs' button.

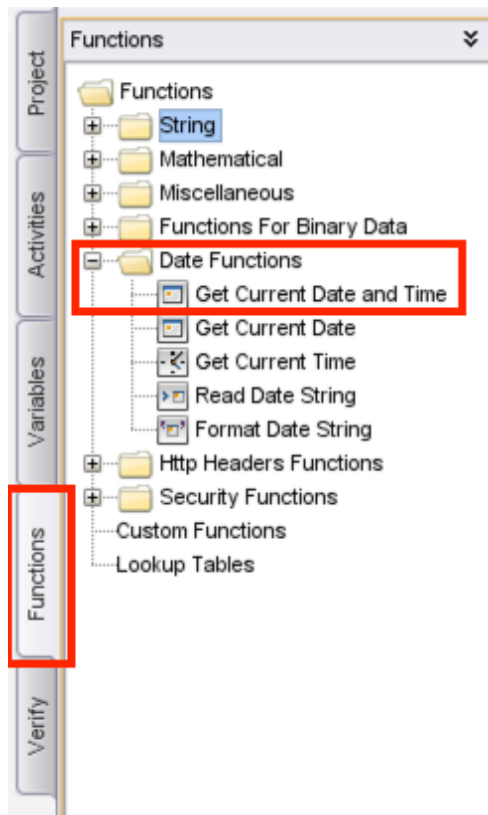
Select both variables inputBody and fixedResponse from steps 4 and 5.



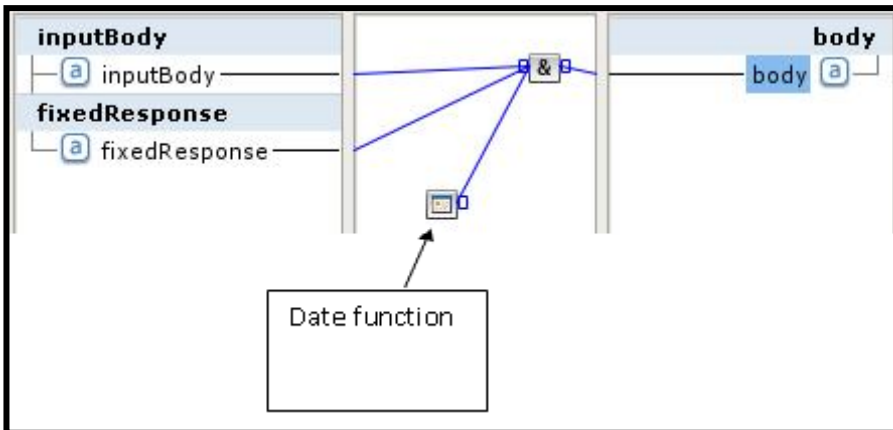
__13. Select the Functions Tab and drag the 'Concatenate' function to the function graph (center pane).



__14. Drag Get Current Date and Time function to the function graph pane.



__15. Connect the input variables and the Date function to the Concatenate function.



__16. Double-click Concatenate function to open its parameters, then add strings with the Add button.

Function: Concatenate
Joins up to 1000 parameters into a single string

Return Type: string

Parameters

Name	Required Type	Value
input	string	/inputBody
input	string	/fixedResponse
input	string	Get Current Date and Time(2)

Buttons: Add, Delete, Up, Down, Help, OK, Cancel

__17. Add the following text strings and reorder the parameters to form a coherent output string:

: The Body text was:

. The variable text is

and that is all.

Function: Concatenate
Joins up to 1000 parameters into a single string

Return Type: string

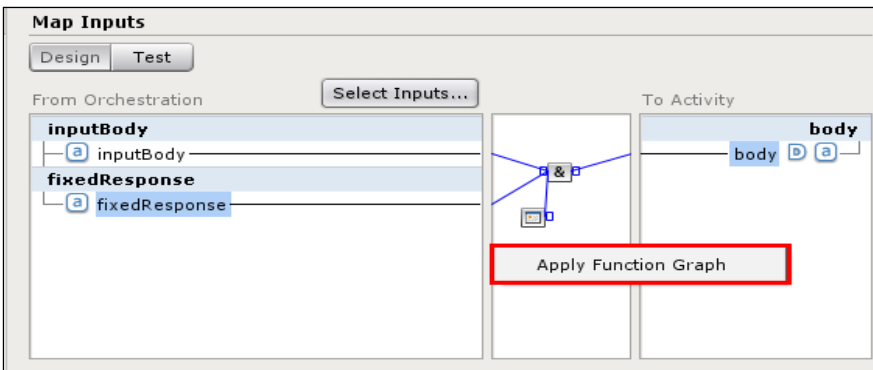
Parameters

Name	Required Type	Value
input	string	Get Current Date and Time(2)
input	string	: The body text was:
input	string	/inputBody
input	string	. The variable text is
input	string	/fixedResponse
input	string	and that's all.

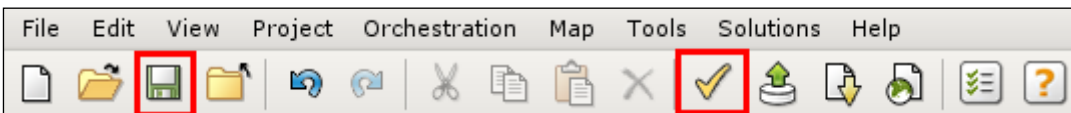
Buttons: Add, Delete, Up, Down, Help, OK, Cancel

__18. Map Concatenate function to output body.

Right-click in function graph pane to **Apply Function Graph**.



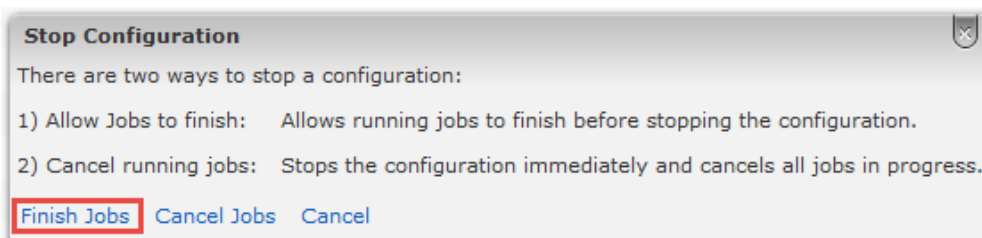
__19. Save and Verify the project.



__20. Before you can deploy and test the updated project, you must go out to the WMC to Stop and Undeploy the current project.

__a. Stop Configuration

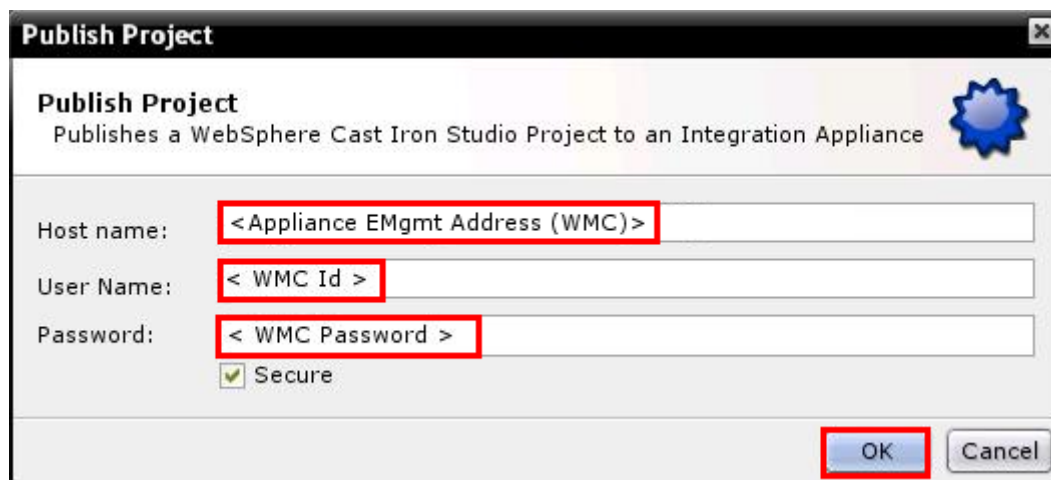
Project Configurations							
Configurations Filter: All							
Configuration	Last Published	Running	Completed	Errored	Total	Actions	
▶ HTTPBasics (Running)	10/26/2016 07:58:34	0	2	0	2	  	



__b. Undeploy Configuration

Project Configurations							
Configurations Filter: All							
Configuration	Last Published	Running	Completed	Errored	Total	Actions	
▼ HTTPBasics (Stopped)	10/26/2016 07:58:34	0	2	0	2	   	
Hello99 (Stopped)		0	2	0	2		

__21. Deploy and run the updated project using the Run Configuration icon:



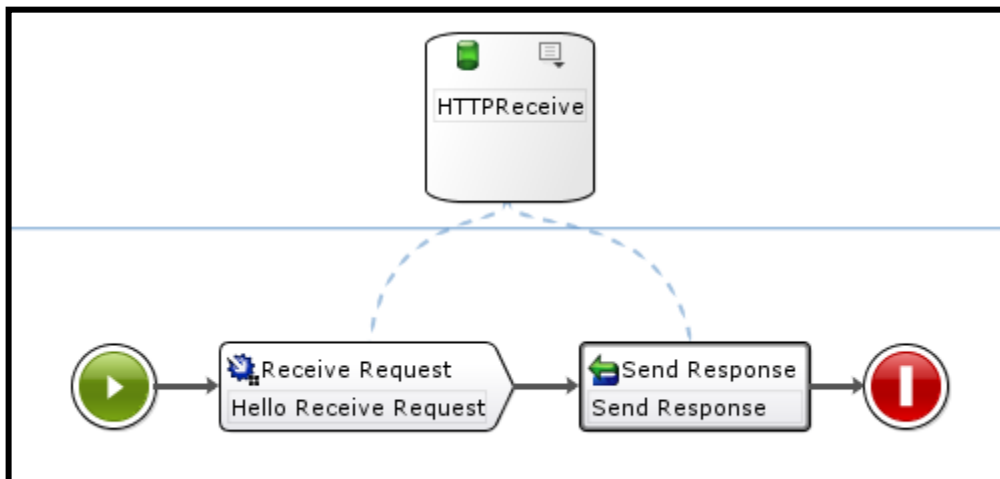
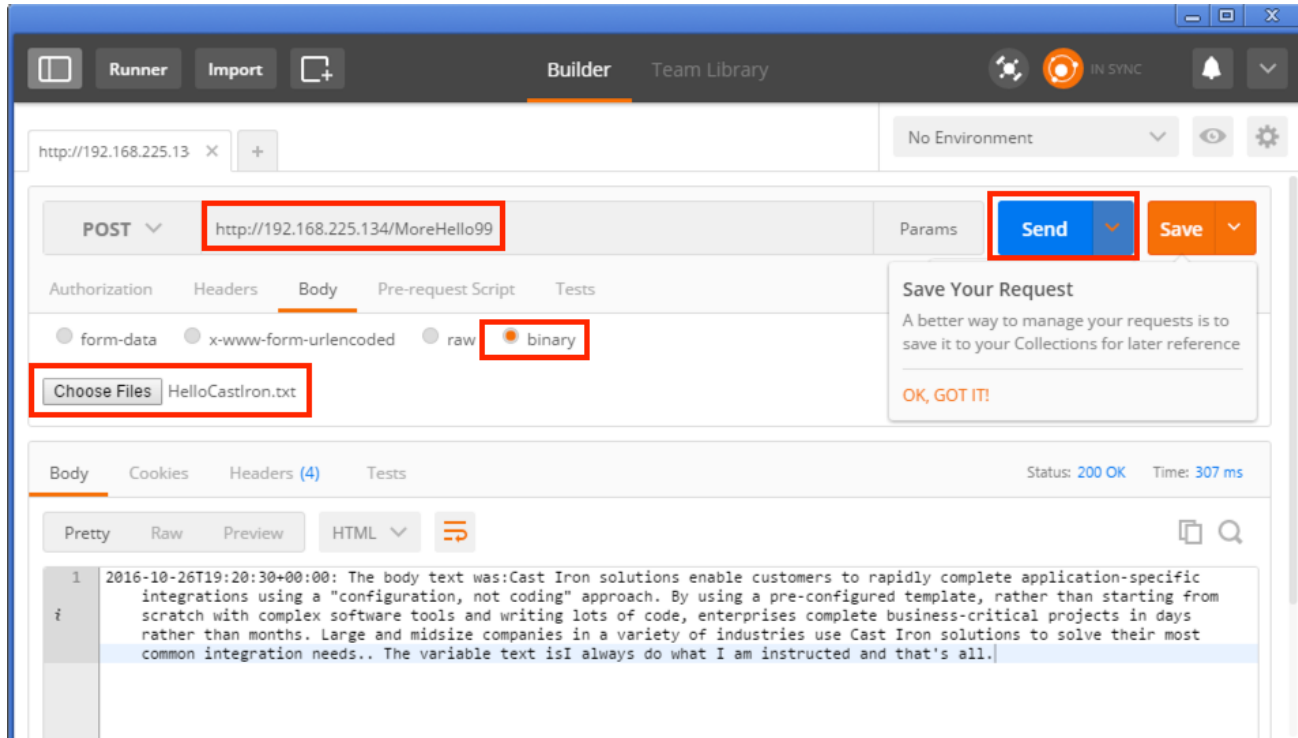
__22. Start Configuration

Project Configurations								
Configurations Filter: All								
Configuration	Last Published	Running	Completed	Errored	Total	Actions		
▼ HTTPBasics (Undeployed)	10/26/2016 07:58:34	0	2	0	2	   		
Hello99 (Undeployed)		0	2	0	2			

__23. Configuration Now Running

Project Configurations							
Configurations Filter: All ▼							
Configuration	Last Published	Running	Completed	Errored	Total	Actions	
▼ HTTPBasics (Running)	10/26/2016 03:08:57	0	2	0	2	   	
Hello99 (Running)		0	2	0	2		
MoreHello99 (Running)		0	0	0	0		

24. Test the project. Use the Postman utility inside of Google Chrome. This can be launched by going to the desktop and clicking on the Chrome App Launcher on the Desktop. Use the edata port of your appliance as the hostname, and then the URL of your Orchestration as the URI. Ensure also the binary encoding type is selected, and you choose the requisite file which is c:\student\Lab Resources\Lab 1\HelloCastIron.txt. When run properly, you will see the output as below.



This completes Lab #1

Lab 2 Load Accounts from a Flat File to Salesforce.com

2.1 Overview

In this lab, you will create a new project, FlatFileToSFDCXX (where xx is your team number) with an orchestration that adds Accounts to Salesforce.com.

2.2 Concepts

In many cases, there are legacy applications that need to be accommodated. Many of the applications use flat files to exchange data with other applications. Most ERP applications also support flat file imports/exports. To use flat files in App Connect Professional integrations the file must be described by a Flat File Schema. The **Transform Read Flat File** activity parses the schema to transform data from string to a structured representation. The **Transform Write Flat File** activity transforms data from structured data to string.

The **Transform Read/Write Flat File** activities are similar to the **Transform Read/Write XML** activities. One significant difference is that a flat file schema defines one document type, whereas an XML schema may define a collection of document types.

2.3 Activities Used

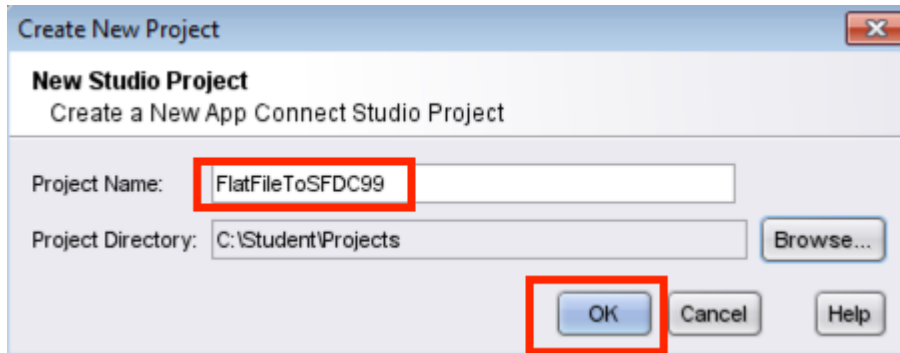
- HTTP Receive Request
- Transform Read Flat File
- For Each (Used to loop through multiple records coming from a source of data)
- If Then...Else (Used to add Business Rules or Business Logic)
- Salesforce – Create Objects
- Log Message (Used to create internal log messages)

2.4 Document Keys

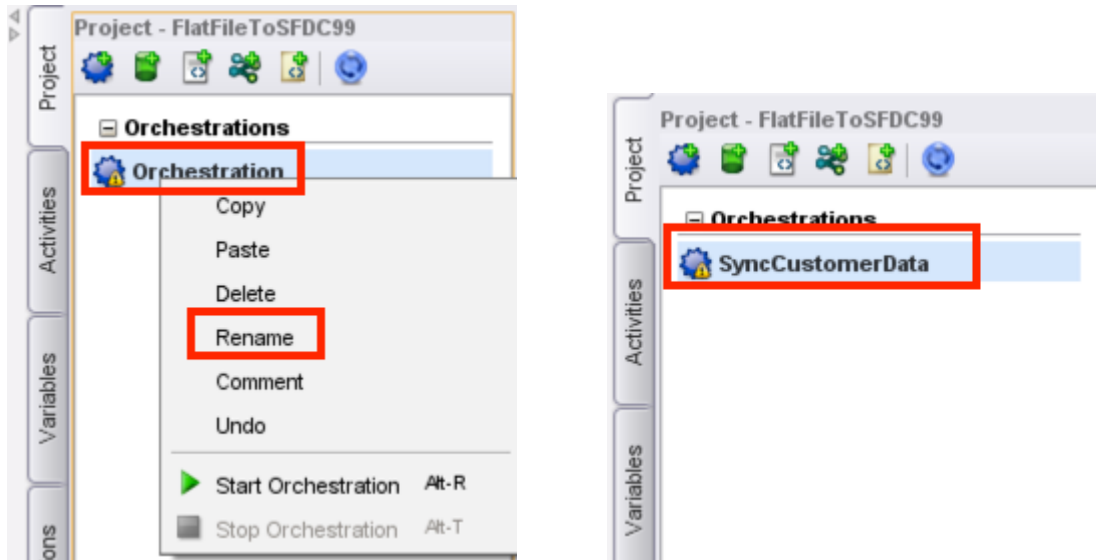
- Menu bar: Edit → Preferences
- Studio Artifact: **HTTP Receive Request**
- Entered Text: 'Hello Receive Request'
- Project Object Name: SampleProject1
- Context Menu Item: Define Default Value
- Activity Checklist Step: Pick Endpoint
- Orchestration Variable: Account
- Function Name: Mathematical / Sum

2.5 Flat File to Salesforce.com

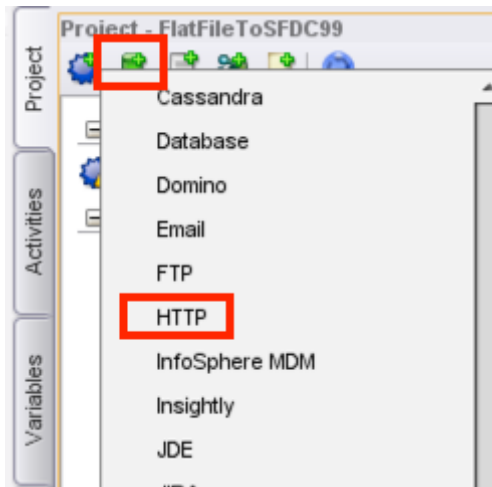
1. In App Connect Studio, create a new project FlatFileToSFDCXX where XX represents you student number. Click OK.



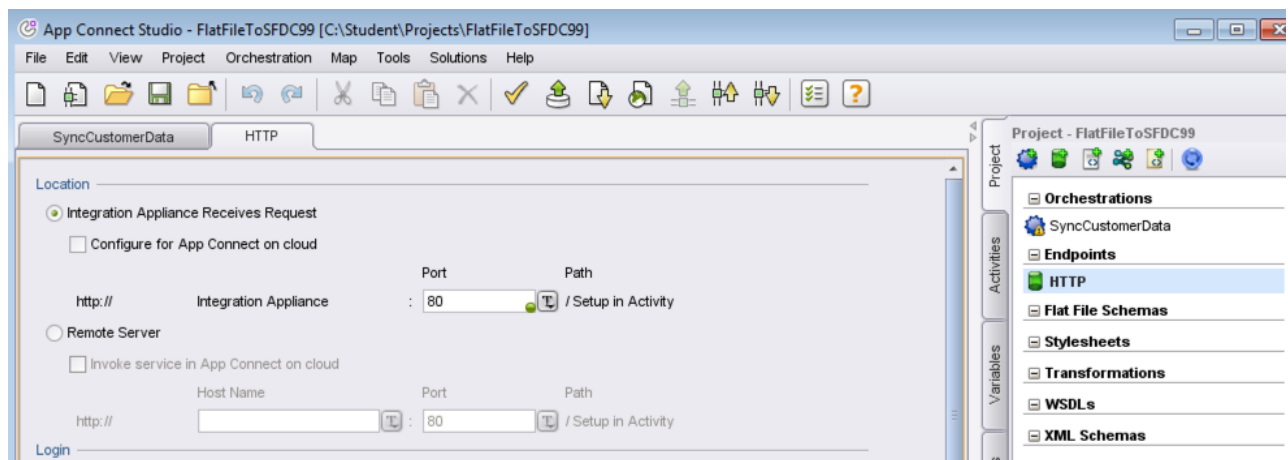
2. Rename default Orchestration to SyncCustomerData by right clicking Orchestration and selecting Rename from list:



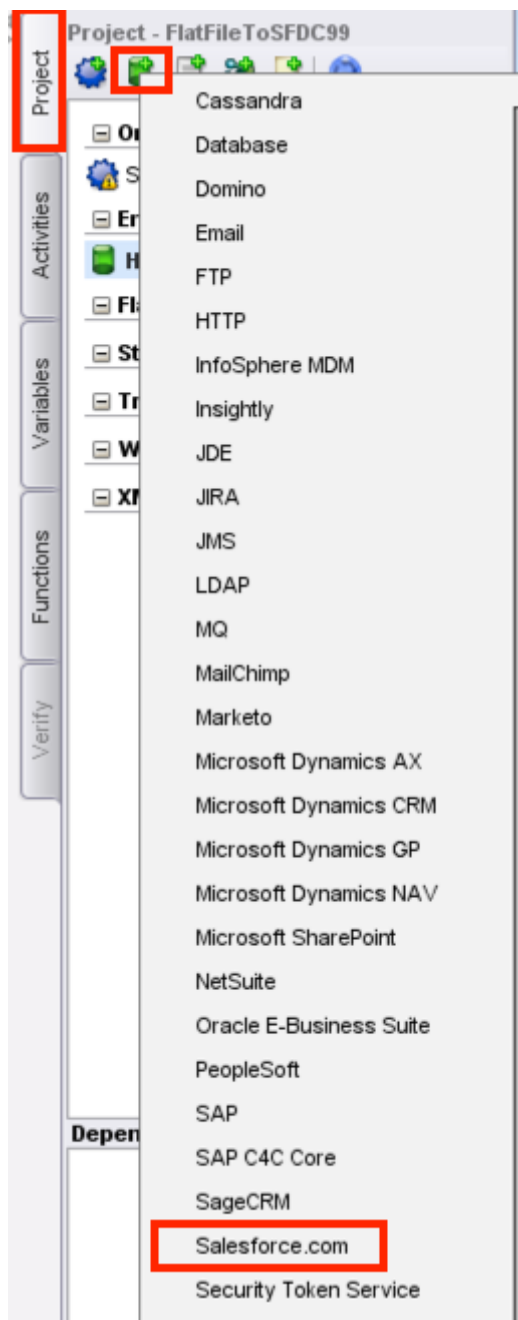
- __3. Create an HTTP Endpoint by clicking on the New Endpoint icon and selecting HTTP from the list:



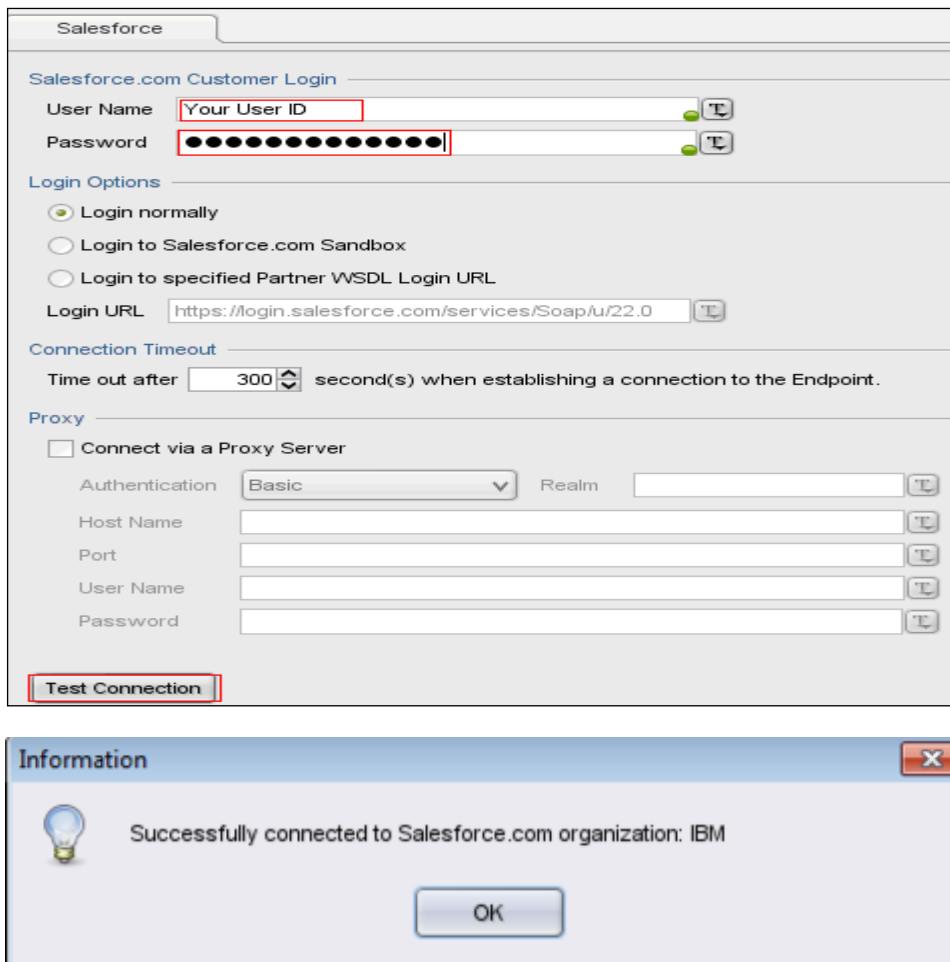
- __4. This will generate an HTTP Endpoint with default settings which you DO NOT need to CHANGE.



- __5. Repeat the process to create a Salesforce.com endpoint.



6. Enter your Salesforce.com UserID/Password Login credentials. Then click the Test Connection button to validate access to your Salesforce.com instance:



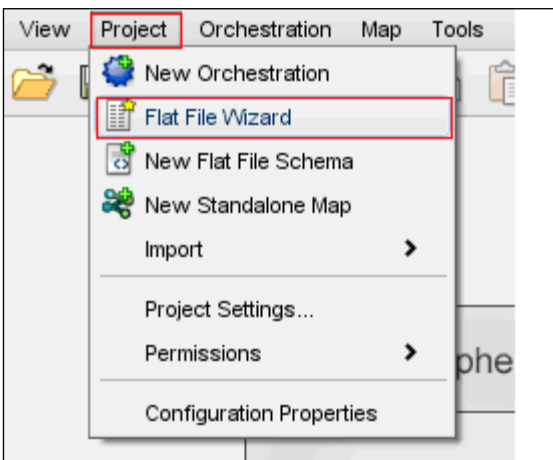
The image shows two screenshots from a software application. The top screenshot is the 'Salesforce.com Customer Login' dialog box. It has a 'User Name' field containing 'Your User ID' and a 'Password' field with masked characters. Below these are 'Login Options' with radio buttons for 'Login normally' (selected), 'Login to Salesforce.com Sandbox', and 'Login to specified Partner WSDL Login URL'. There is a 'Login URL' field with the text 'https://login.salesforce.com/services/Soap/u/22.0'. A 'Connection Timeout' section shows 'Time out after 300 second(s)'. A 'Proxy' section has a checkbox for 'Connect via a Proxy Server' which is unchecked, and fields for 'Authentication' (Basic), 'Host Name', 'Port', 'User Name', and 'Password'. At the bottom is a 'Test Connection' button. The bottom screenshot is an 'Information' message box with a lightbulb icon, stating 'Successfully connected to Salesforce.com organization: IBM' and an 'OK' button.

2.6 Creating a Flat File Schema

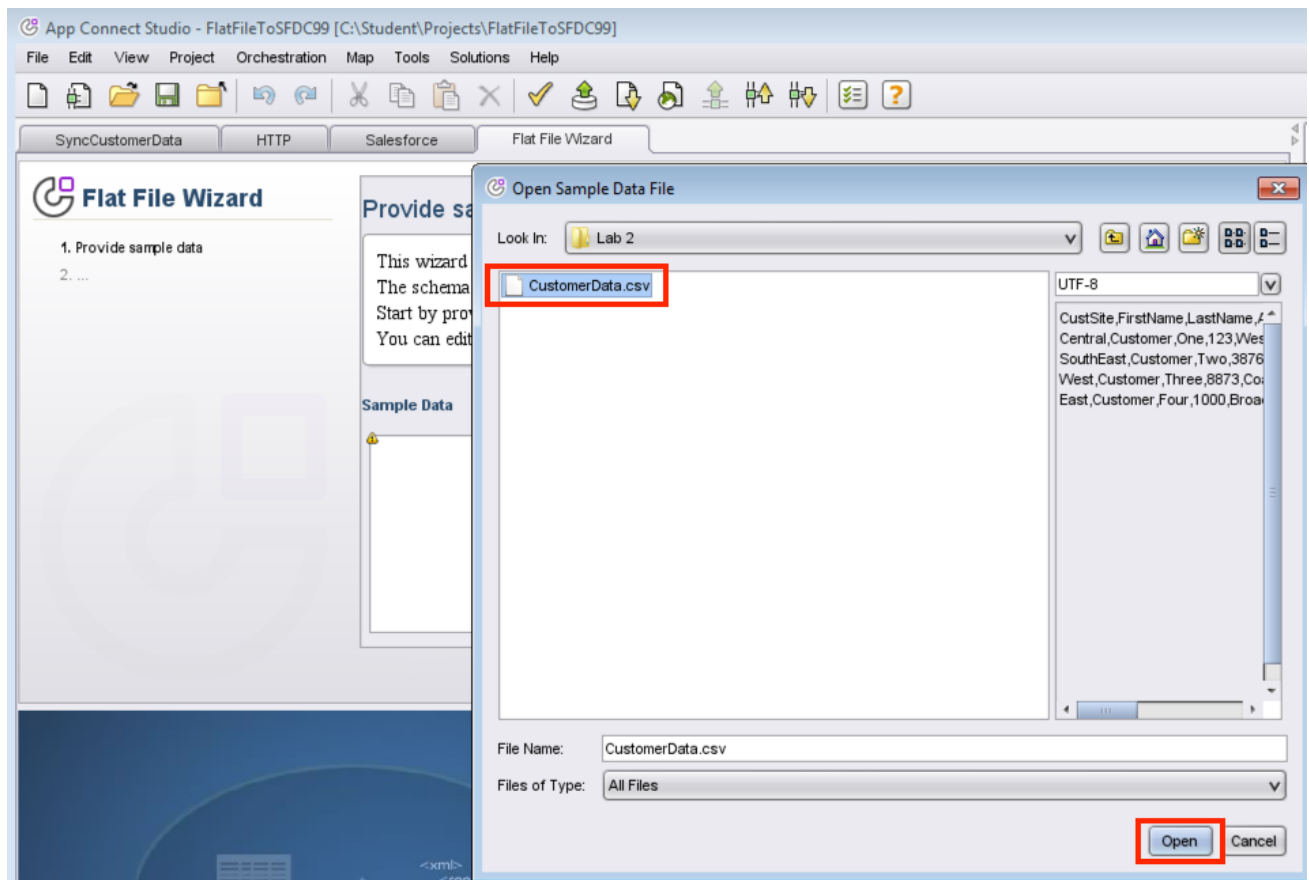
2.6.1 Create a Flat File Schema using the Flat File Wizard.

The schema can be generated by importing a sample data file. The file that will be imported contains two types of records: Header and Data. The Header indicates the field names. The header and data rows have the same structure of nine (9) fields.

- __16. Open the Flat File Wizard using the Project dropdown menu: Project → Flat File Wizard.



- __17. Provide sample data by browsing for the input file **CustomerData.csv** in folder **C:\Student\Lab Resources\Lab 2**. Click **Open**. Click **Next**.



__18. **Specify File Format** - The Character-delimited format (default) applies to our data. Click **Next**.

App Connect Studio - FlatFileToSFDC99 [C:\Student\Projects\FlatFileToSFDC99]

File Edit View Project Orchestration Map Tools Solutions Help

SyncCustomerData HTTP Salesforce Flat File Wizard

Flat File Wizard

- 1. Provide sample data
- 2. File Format
- 3. ...

File Format

Specify the file format.

☒ Character-delimited format (including Comma Separated Values format)
A file where each line contains one record, with fields separated by a single character such as a comma or tab. May contain a header or footer.

☐ This file is in another format

< Back **Next >** Finish Cancel Help

CustSite,FirstName,LastName,Address1,Address2,City,State,Zipcode,Telephone
Central,Customer,One,123,West Avenue,New Orleans,LA,70058,5045551212
SouthEast,Customer,Two,3876,Norfolk Drive,Atlanta,GA,30328,7705551212
West,Customer,Three,8873,Coast Highway,Mountain View,CA,90645,6505551212
East,Customer,Four,1000,Broadway,New York,NY,10093,2125551212

__19. **Specify Field delimiters** - Accept the defaults. Click **Next**.

App Connect Studio - FlatFileToSFDC99 [C:\Student\Projects\FlatFileToSFDC99]

File Edit View Project Orchestration Map Tools Solutions Help

SyncCustomerData HTTP Salesforce Flat File Wizard

Flat File Wizard

- 1. Provide sample data
- 2. File Format
- 3. Field Delimiters
- 4. Header & Trailer
- 5. Edit Schema Name

Field Delimiters

Specify the field separator character. You may also specify an optional quoting ("encapsulation") character that encloses the field values.

Fields separated by: Comma (,)

Fields enclosed by: Double Quote ("')

< Back **Next >** Finish Cancel Help

CustSite	FirstName	LastName	Address1	Address2	City	State	Zipcode	Telephone
Central	Customer	One	123	West Avenue	New Orleans	LA	70058	5045551212
SouthEast	Customer	Two	3876	Norfolk Drive	Atlanta	GA	30328	7705551212
West	Customer	Three	8873	Coast Highway	Mountain View	CA	90645	6505551212
East	Customer	Four	1000	Broadway	New York	NY	10093	2125551212

- __20. **Specify Header and Trailer** - The sample data is constructed to assist the definition of the schema, with a header record, so select just the **Header** option. Click **Next**.

App Connect Studio - FlatFileToSFDC99 [C:\Student\Projects\FlatFileToSFDC99]

File Edit View Project Orchestration Map Tools Solutions Help

SyncCustomerData HTTP Salesforce Flat File Wizard

Flat File Wizard

1. Provide sample data
2. File Format
3. Field Delimiters
4. Header & Trailer
5. Edit Schema Name

Header & Trailer

Specify whether header and trailer rows are present.

☒ Treat first row as header for field names

☐ Treat last 0 row(s) as trailer

< Back **Next >** Finish Cancel Help

custsite	firstname	lastname	address1	address2	city	state	zipcode	telephone
Central	Customer	One	123	West Avenue	New Orleans	LA	70058	5045551212
SouthEast	Customer	Two	3876	Norfolk Drive	Atlanta	GA	30328	7705551212
West	Customer	Three	8873	Coast Highway	Mountain View	CA	90645	6505551212
East	Customer	Four	1000	Broadway	New York	NY	10093	2125551212

- __21. **Edit Schema Name** - Taken from the input file name, take default name of **CustomerData**. Click **Finish**.

App Connect Studio - FlatFileToSFDC99 [C:\Student\Projects\FlatFileToSFDC99]

File Edit View Project Orchestration Map Tools Solutions Help

SyncCustomerData HTTP Salesforce Flat File Wizard

Flat File Wizard

- 1. Provide sample data
- 2. File Format
- 3. Field Delimiters
- 4. Header & Trailer
- 5. Edit Schema Name

Edit Schema Name

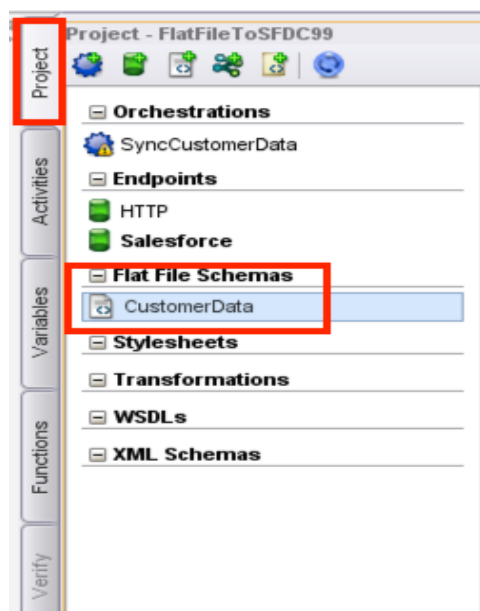
Edit the schema name so it describes the data and file format.
When the wizard completes, the Flat File Schema will be added to your project with this name.
To read a flat file and convert it to XML, add a "Read Flat File" activity to the orchestration, and specify this schema.
To convert XML data to a flat file in this format, use a "Write Flat File" activity.

Schema name:

< Back Next > **Finish** Cancel Help

custsite	firstname	lastname	address1	address2	city	state	zipcode	telephone
Central	Customer	One	123	West Avenue	New Orleans	LA	70058	5045551212
SouthEast	Customer	Two	3876	Norfolk Drive	Atlanta	GA	30328	7705551212
West	Customer	Three	8873	Coast Highway	Mountain View	CA	90645	6505551212
East	Customer	Four	1000	Broadway	New York	NY	10093	2125551212

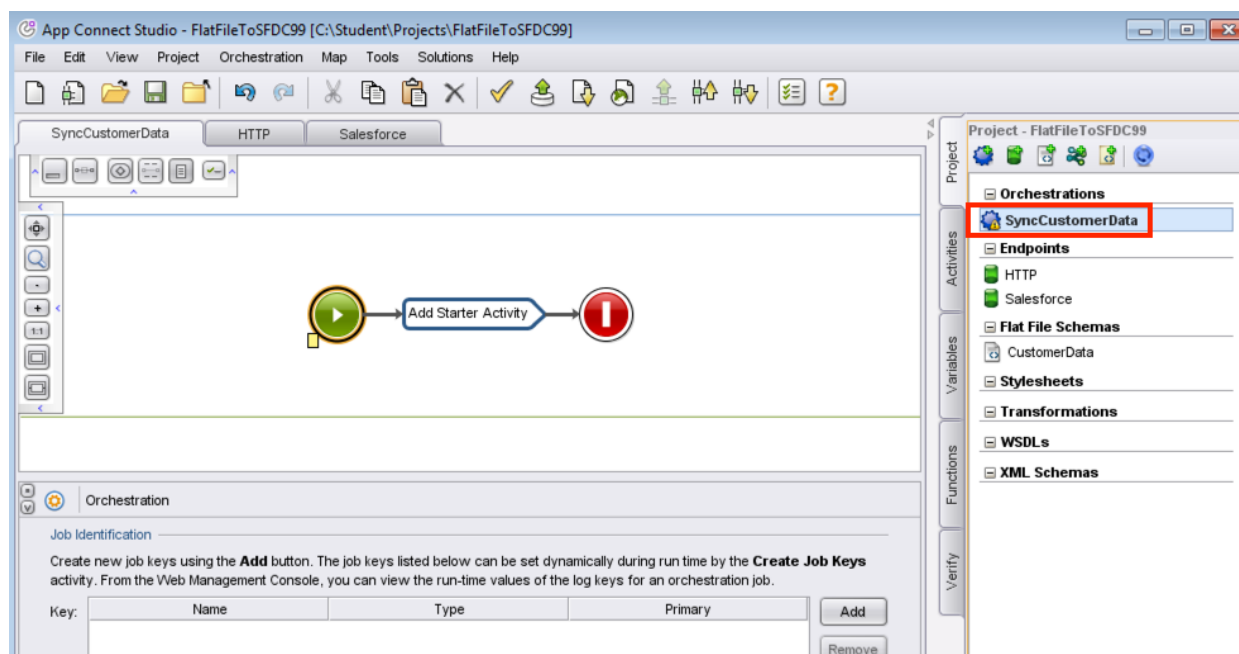
CustomerData Schema is added to the **Flat File Schemas** folder.



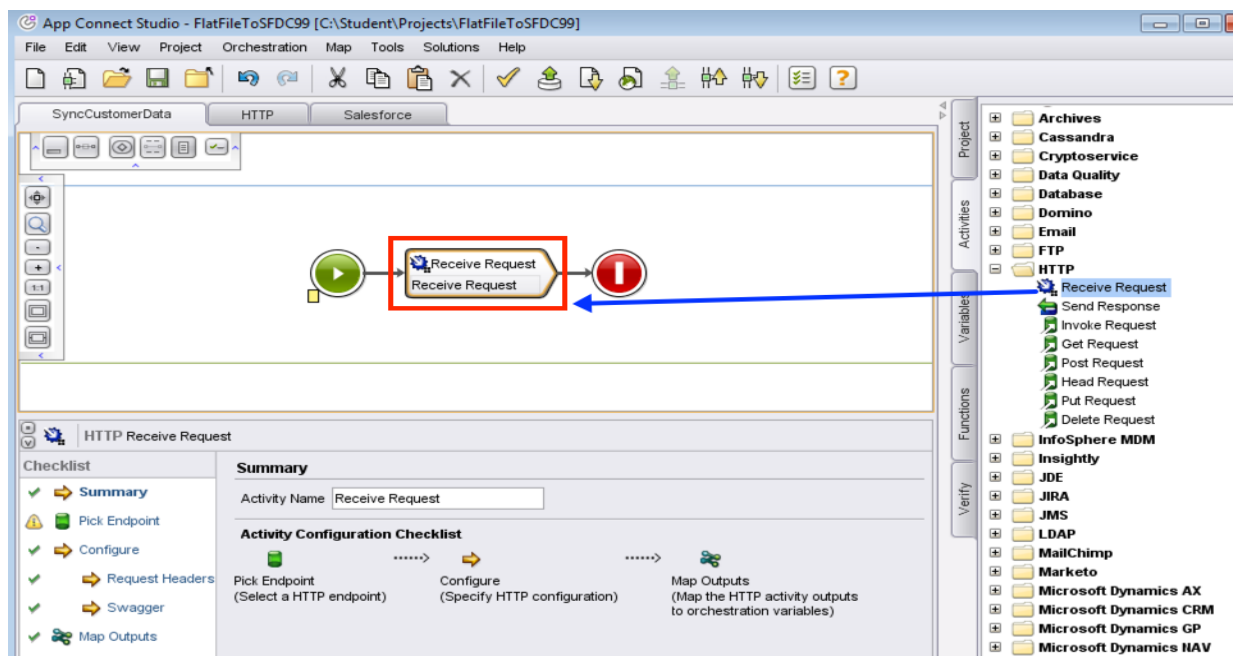
2.7 Creating a New Orchestration

2.7.1 Build the Orchestration

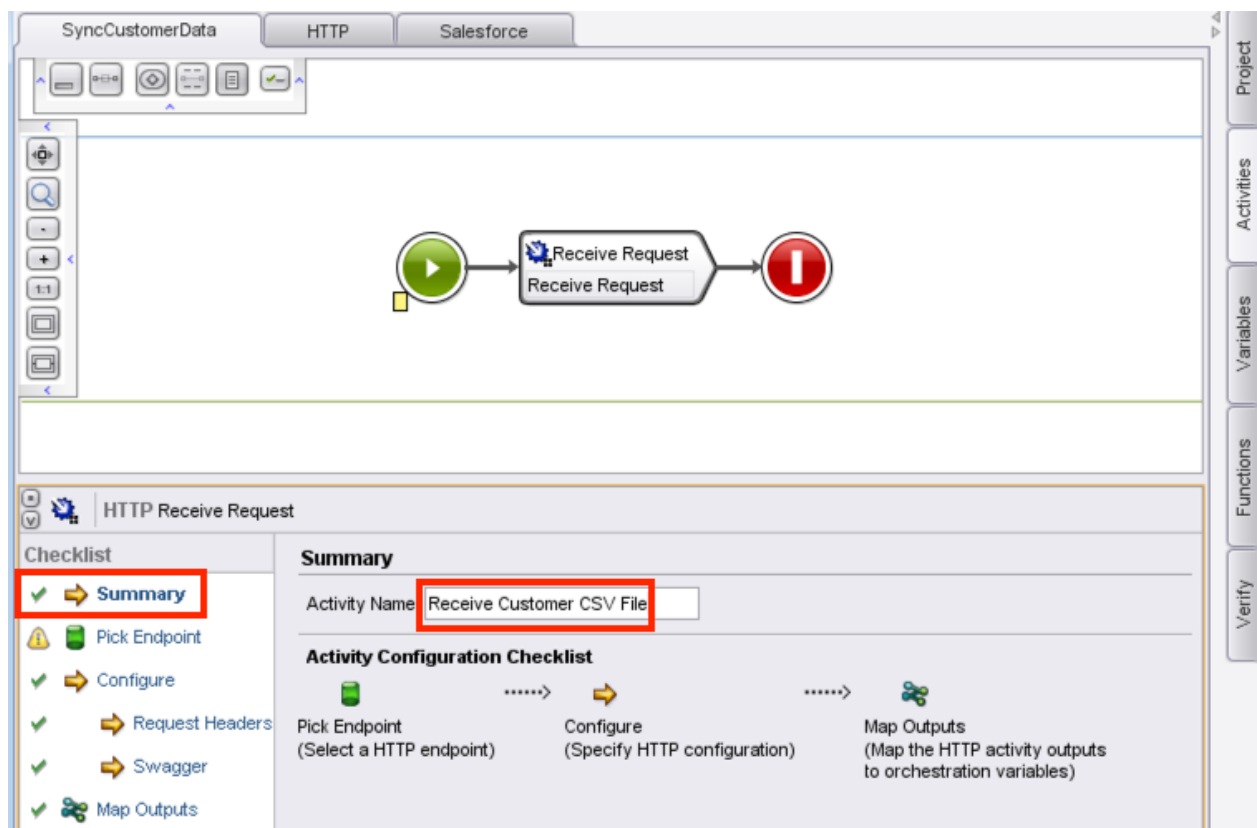
__22. Double click the Orchestration Name – **SyncCustomerData**.



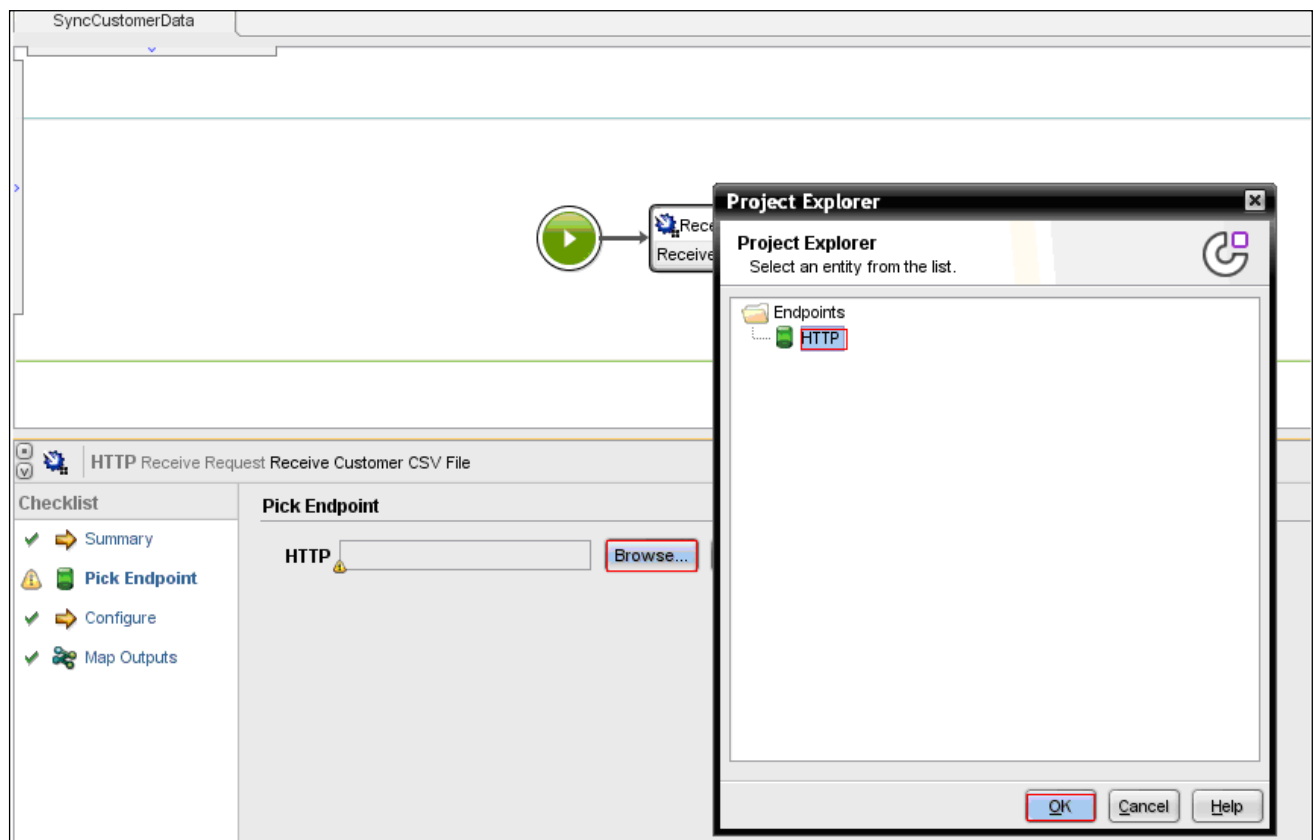
__23. Add an **HTTP Receive Request** activity to the orchestration by left clicking and holding on **Receive Request** from the **HTTP** folder on the **Activities Tab** and dragging it to the workspace between the green and red icons.



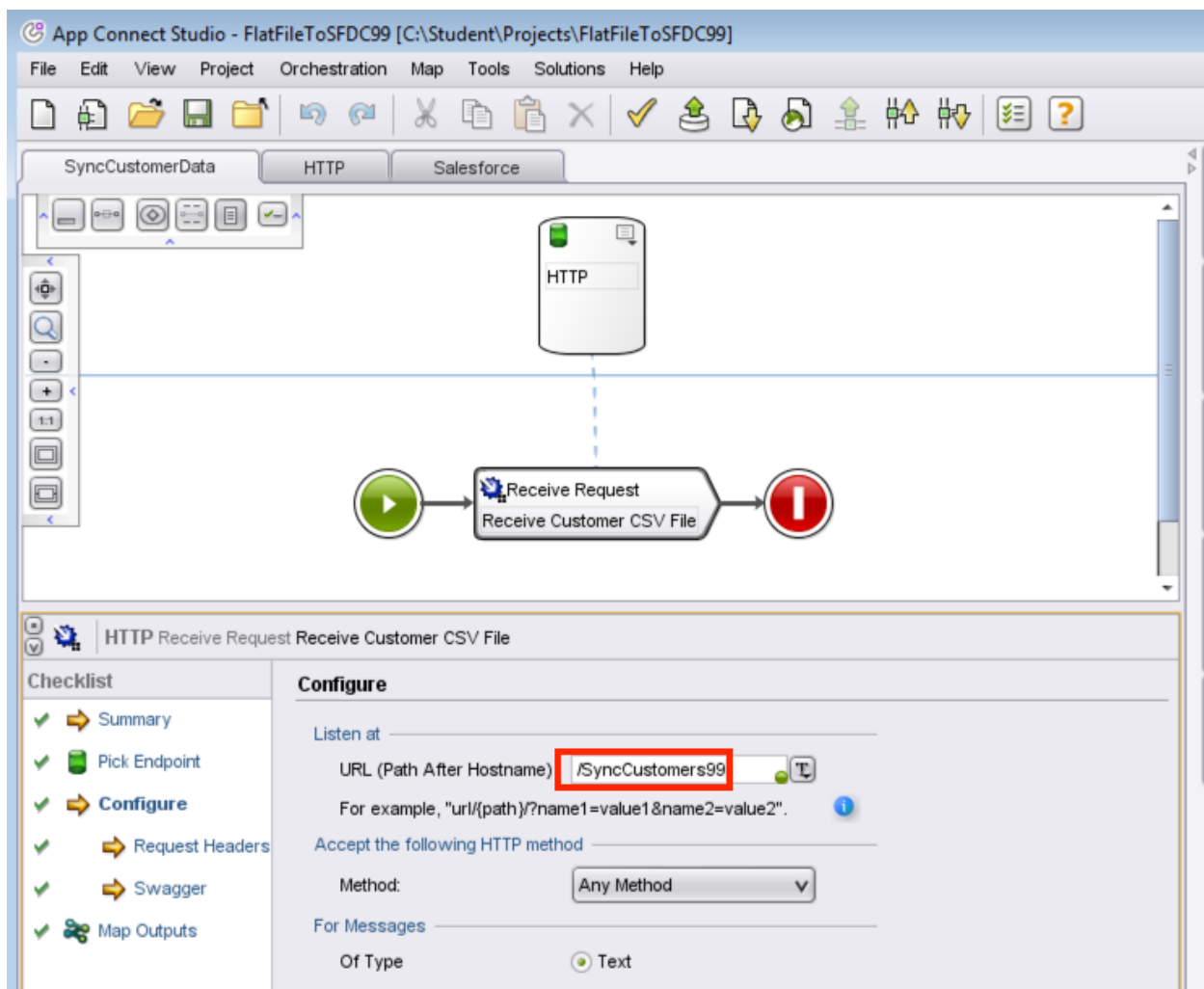
__24. Click **Summary** - Change the Activity Name to: **Receive Customer CSV File**.



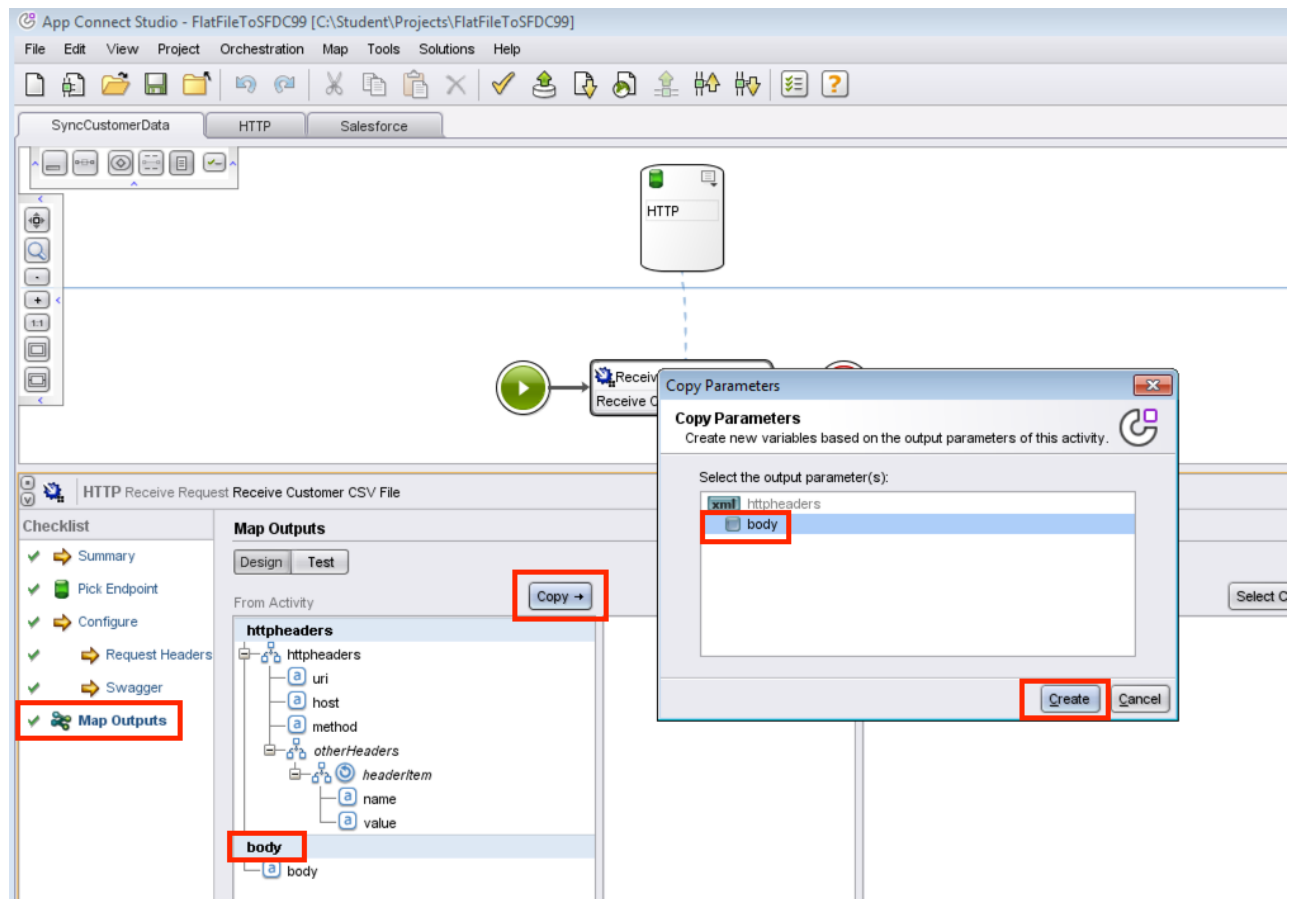
__25. Click **Pick Endpoint** – Click **Browse** and select **HTTP** then **OK**:



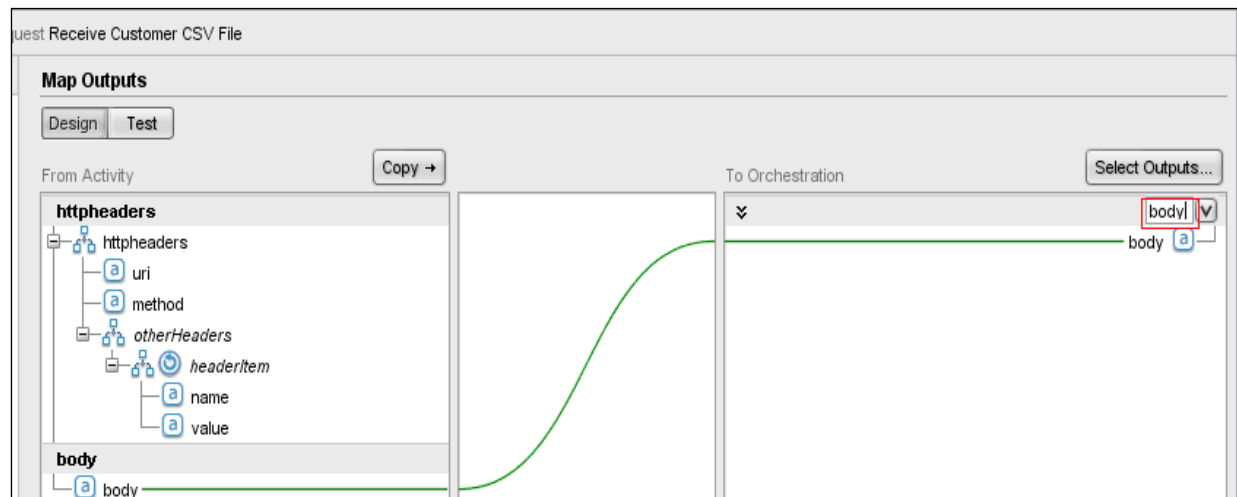
__26. Click **Configure** – URL set to: SyncCustomersXX.

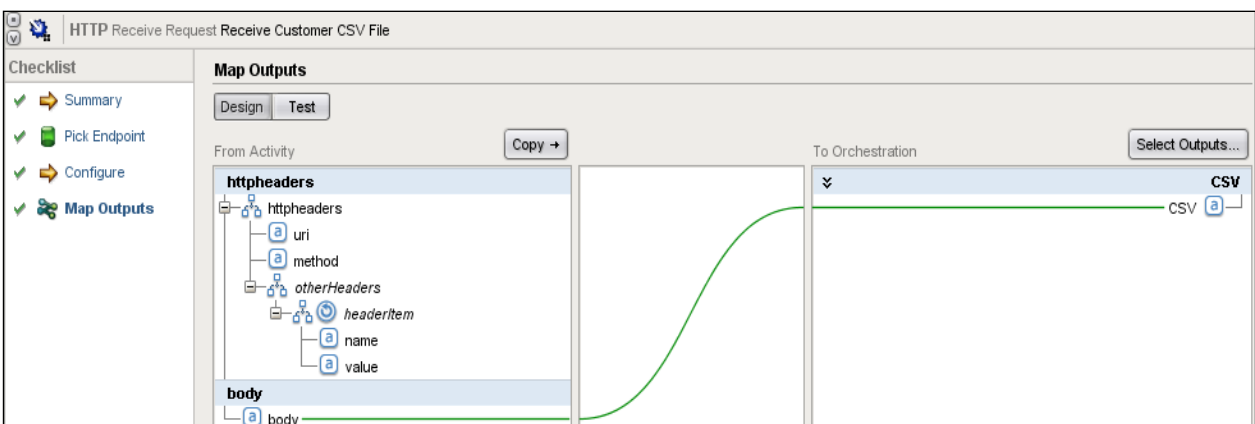


__27. Click **Map Outputs** - Click **Copy** then select **body** and click **Create**:



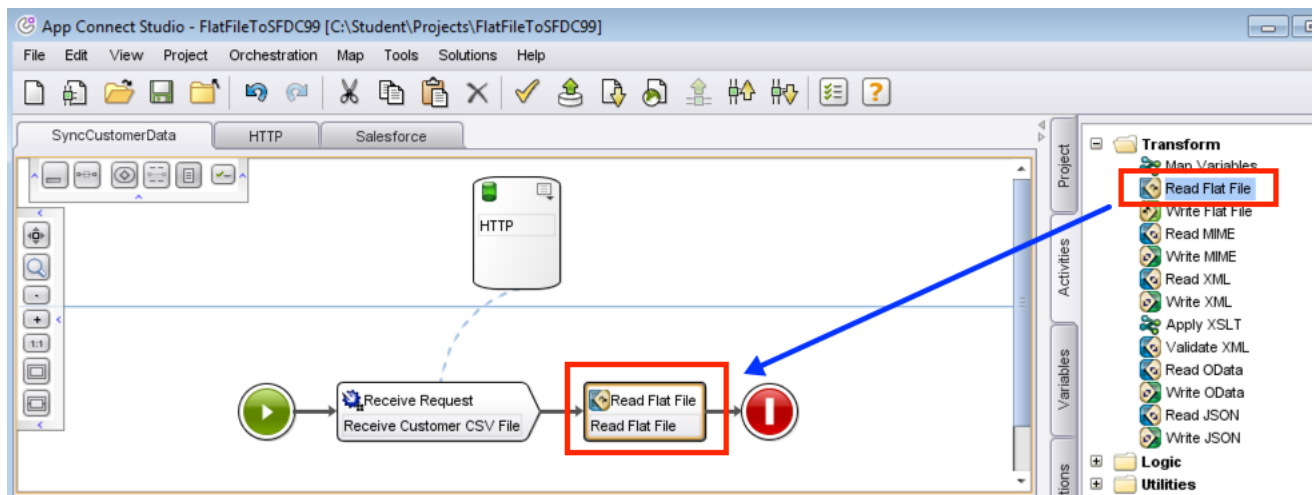
__28. In order to identify the Customer CSV data in the next Activity....Rename the **body** variable to **CSV** by double clicking on the **body** column name, type **CSV** and then hit **Enter**:



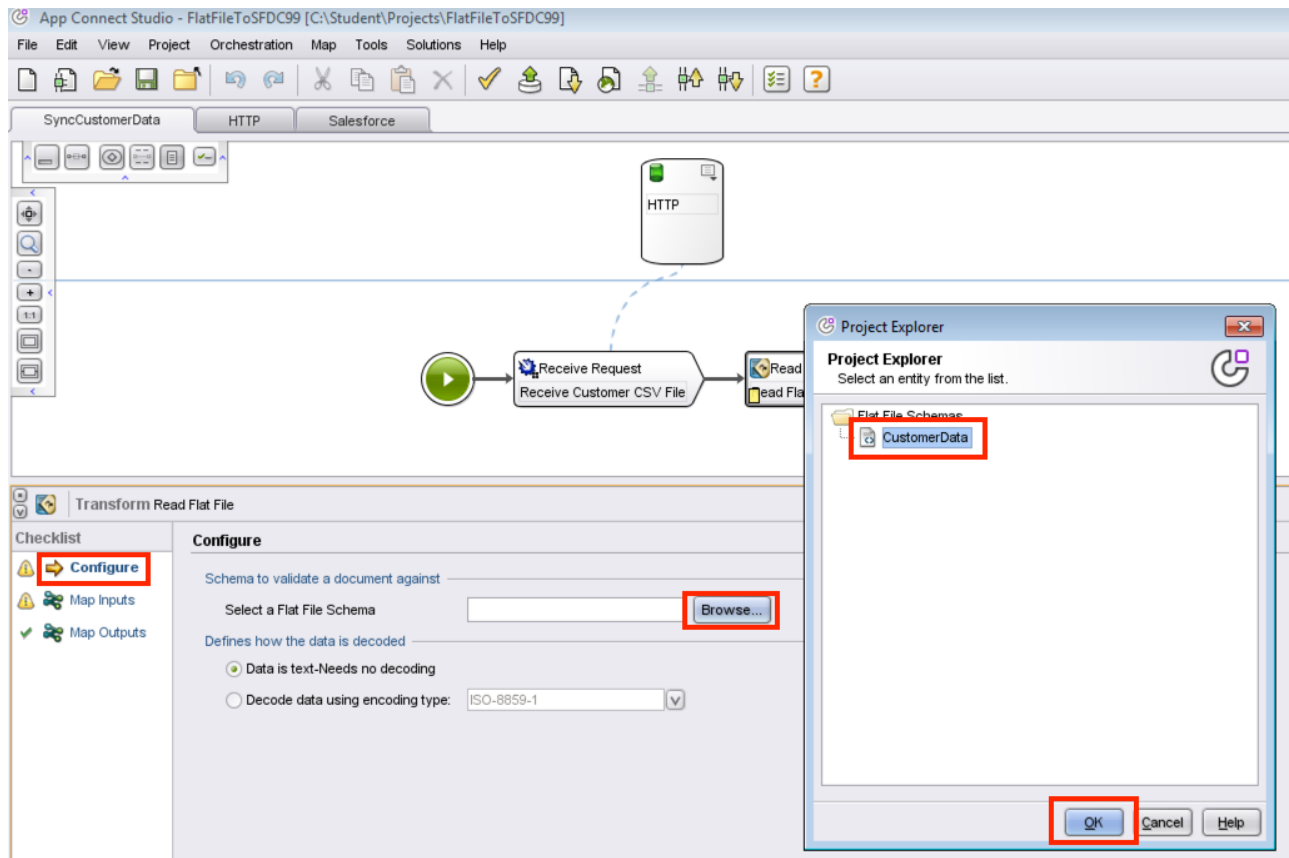


The configuration for the HTTP Receive Request Activity is complete.

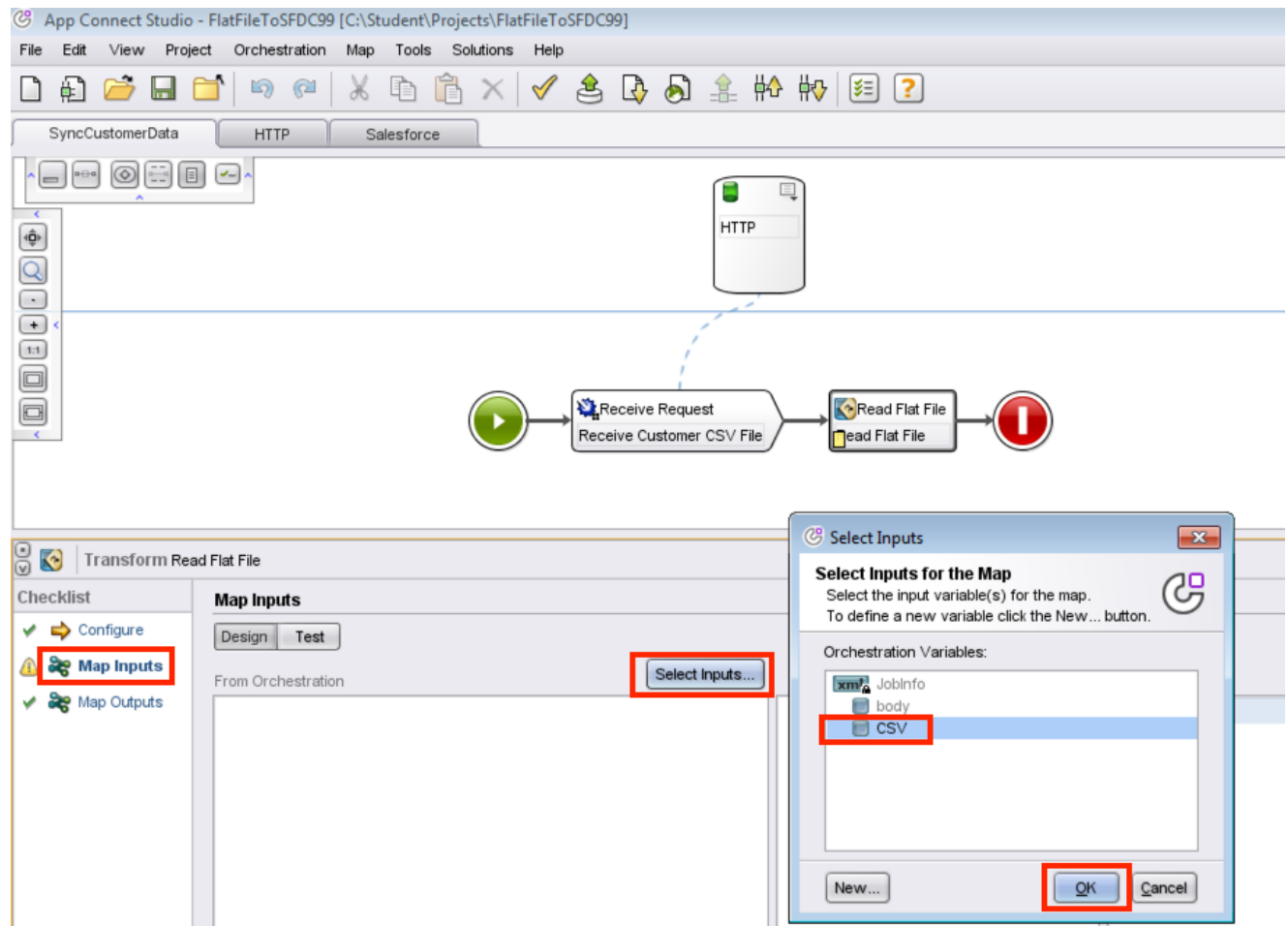
- ___29. Next add a **Read Flat File** activity to the orchestration from the **Transform** folder on the **Activities Tab** and drag it to the right of the **Receive Request** activity.



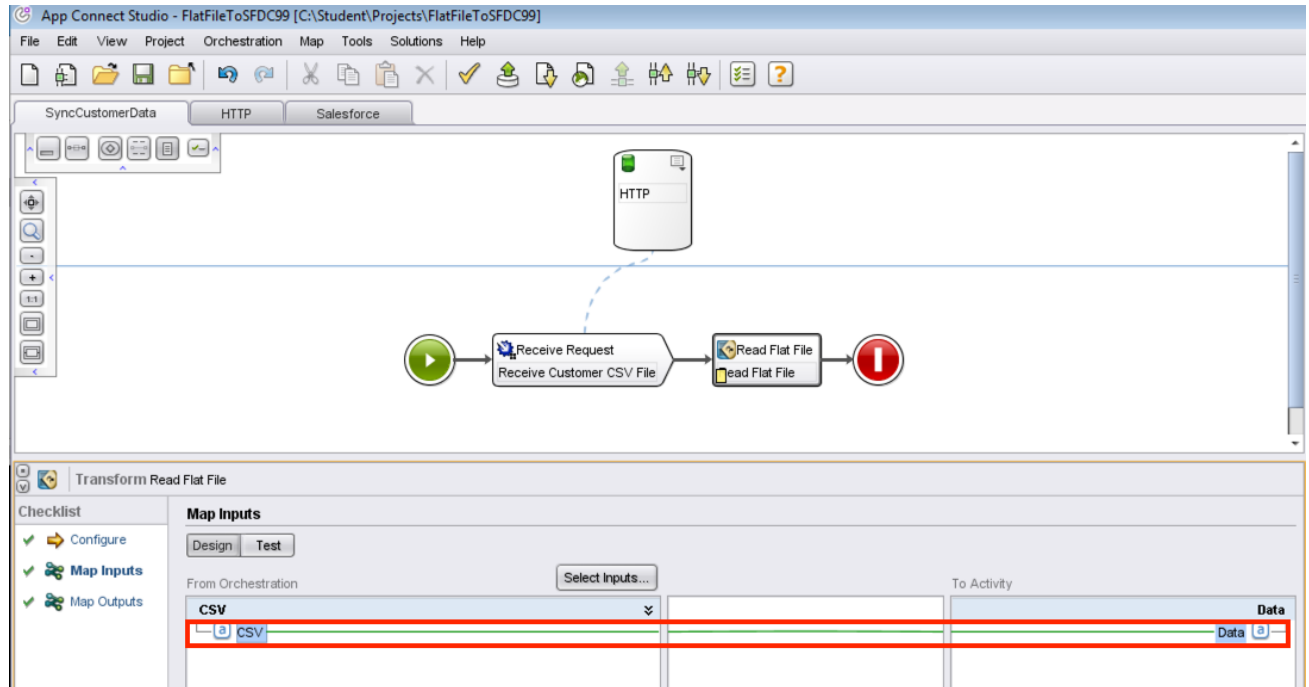
- ___30. Click **Configure** – Click **Browse** to locate the flat file schema **CustomerData** created previously. Click **OK**.



- __31. Click **Map Inputs**: The input to the **Read Flat File Activity** is the output from the previous **Receive Request Activity**...Click **Select Inputs** then choose **CSV** and then **OK**.



- ___32. Next position your cursor on the **CSV** field under the From Orchestration column, left click and hold, then drag your cursor on top of the **Data** field under the **To Activity** column and release...This will map the **CSV** field to the **Data** field.



- ___33. Click **Map Outputs** - The output is an XML representation of the input flat file. As described by the schema, the first record of the input is header information and non-data.

Note: the output structure shows that the data section is a recurring node, with zero or more entries.

__34. Click **Copy** then select **XML** then **Create**:

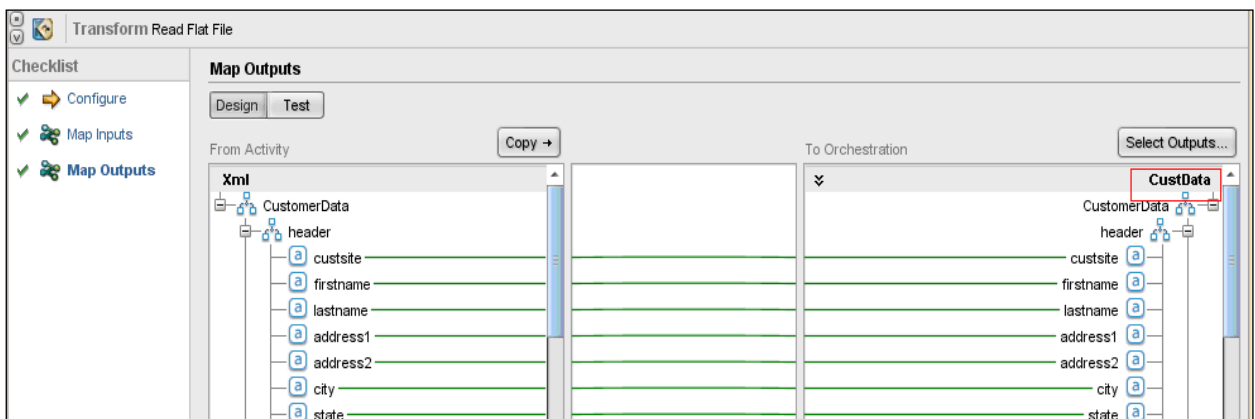
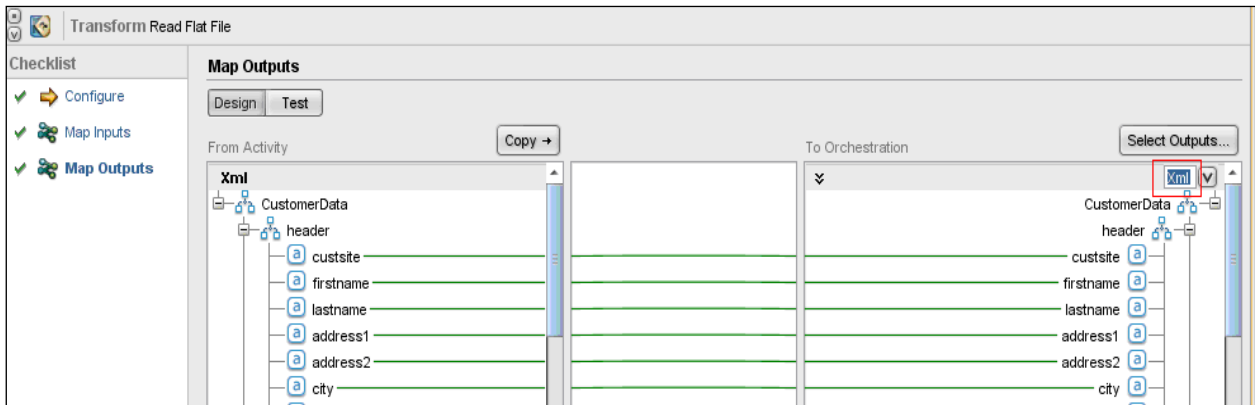
The image consists of three screenshots from the App Connect Studio interface, illustrating the steps to copy XML output parameters.

Top Screenshot: Shows the main workflow canvas. An HTTP connector is linked to a 'Receive Request' activity (labeled 'Receive Customer CSV File'). This is followed by a 'Read Flat File' activity (labeled 'Read Flat File'). The workflow ends with a red stop icon.

Middle Screenshot: The 'Transform Read Flat File' window is open, showing the 'Map Outputs' tab. In the 'From Activity' section, the 'Xml' output is expanded, showing a tree structure with 'header' and 'data' nodes. The 'data' node contains fields: 'custsite', 'firstname', 'lastname', 'address1', 'address2', 'city', 'state', 'zipcode', and 'telephone'. A red box highlights the 'Copy' button. To the right, the 'Copy Parameters' dialog is open, showing 'Select the output parameter(s):' with '[xml] Xml' selected. A red box highlights the 'Create' button.

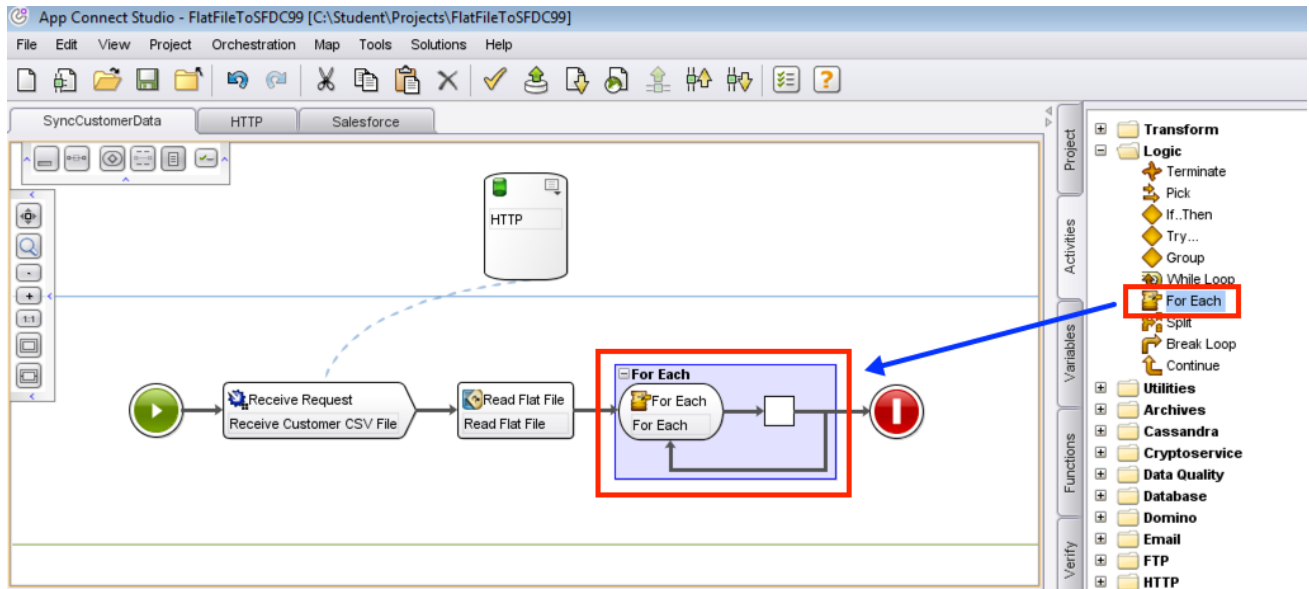
Bottom Screenshot: The 'Map Outputs' window is shown with the 'Copy' button highlighted. The 'From Activity' section shows the 'Xml' output tree. The 'To Orchestration' section shows the 'Select Outputs...' button. The 'Xml' output is expanded, showing the same tree structure as in the middle screenshot. The 'data' node contains fields: 'custsite', 'firstname', 'lastname', 'address1', 'address2', 'city', 'state', 'zipcode', and 'telephone'. Green lines connect the fields from the 'From Activity' section to the 'To Orchestration' section, indicating the mapping of output parameters.

- ___35. Rename **XML** under the **To Orchestration** column to **CustData**, in order to identify it in the next step:



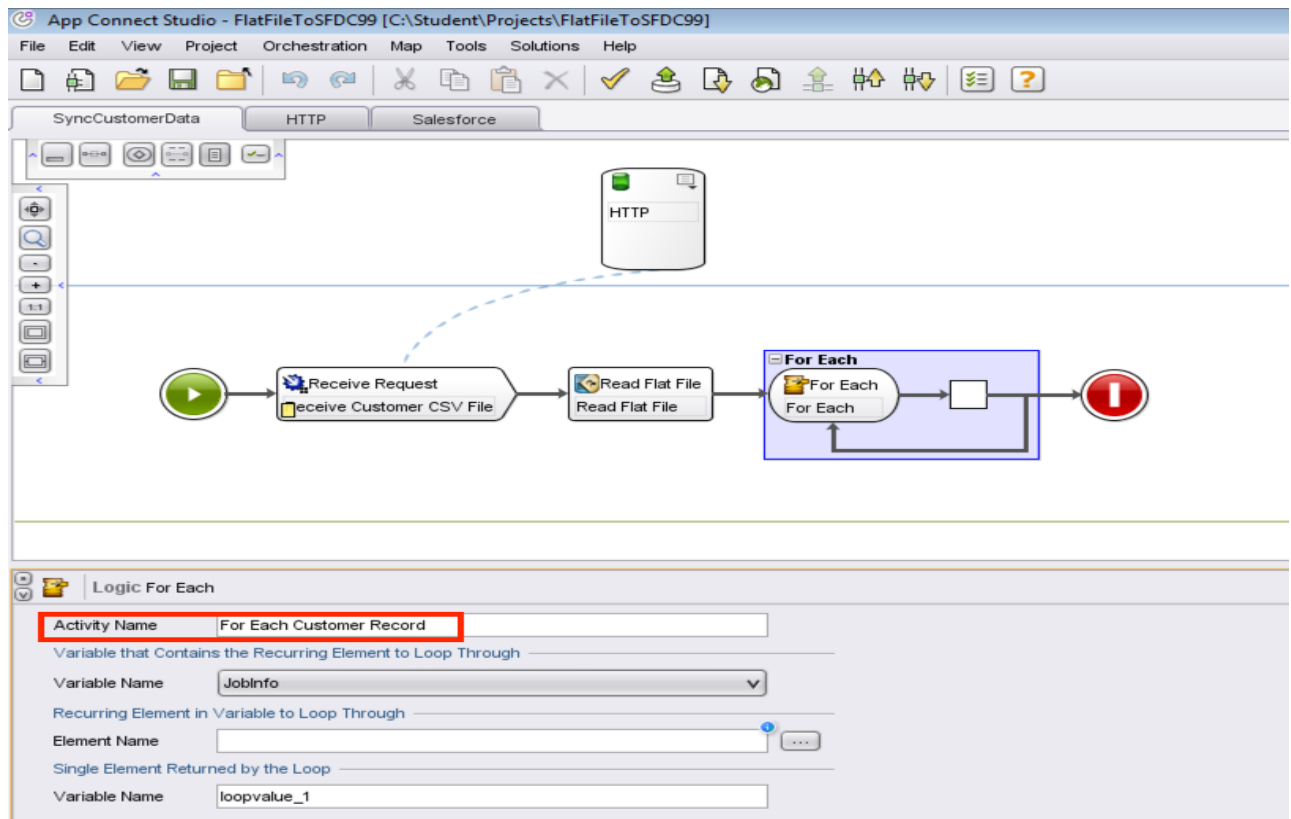
The configuration for the Read Flat File Activity is complete.

- __36. Next add a **For Each Activity** to the right of the **Read Flat File** activity from the **Logic** folder on the **Activities Tab**. This is used to accommodate recurring records coming from the CSV file.

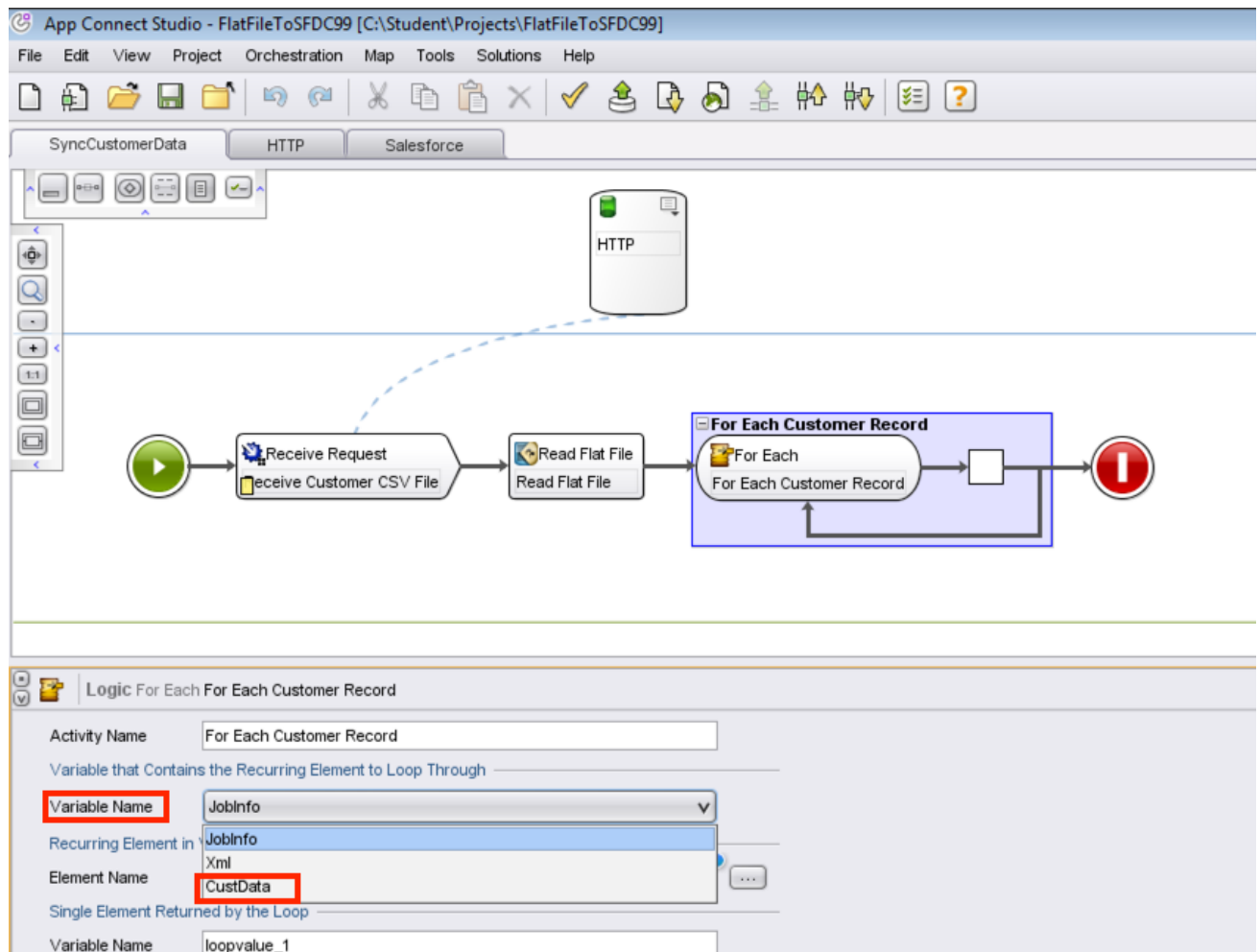


2.7.2 Configure the For Each Activity

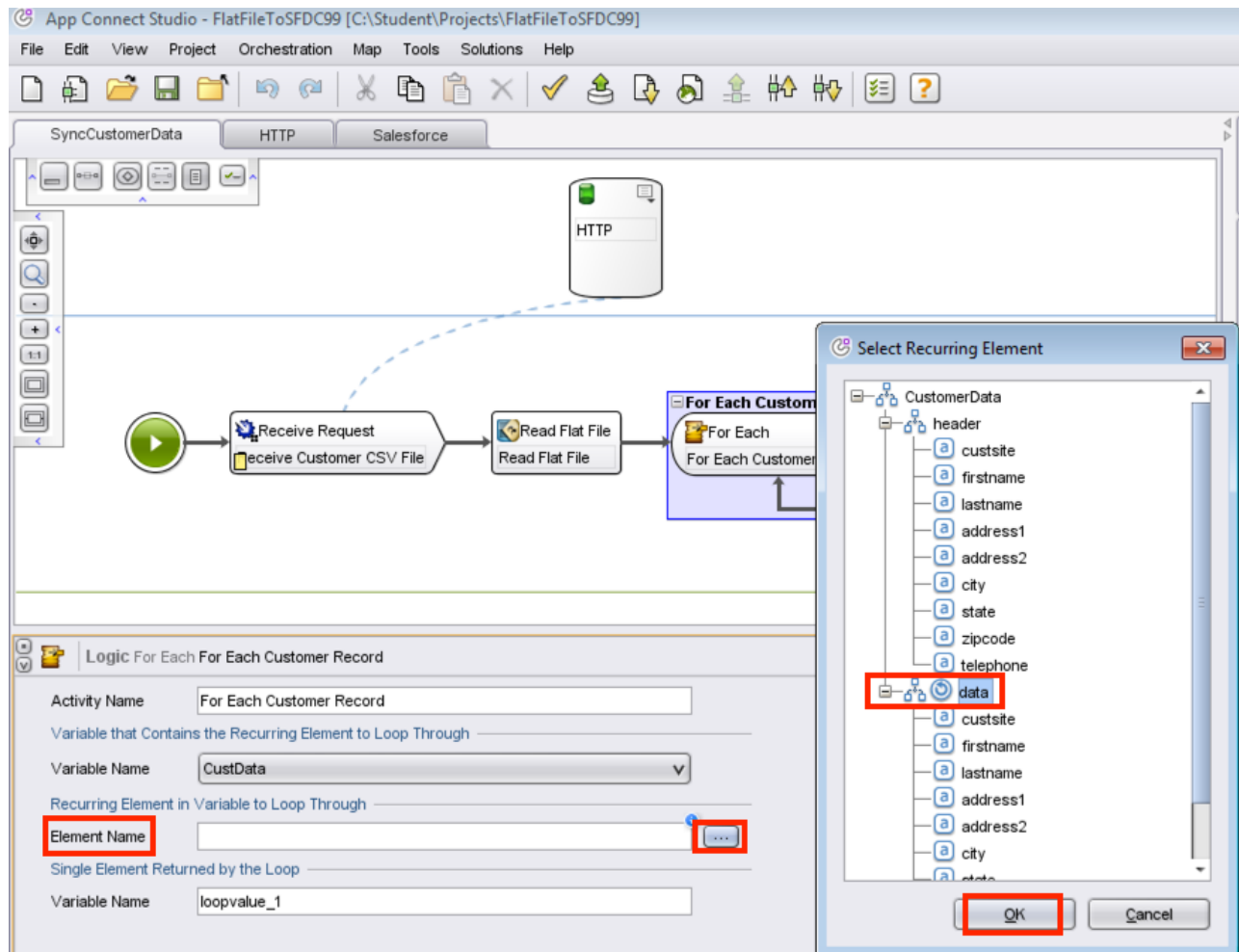
- __37. Activity Name: Change to “For Each Customer Record”.



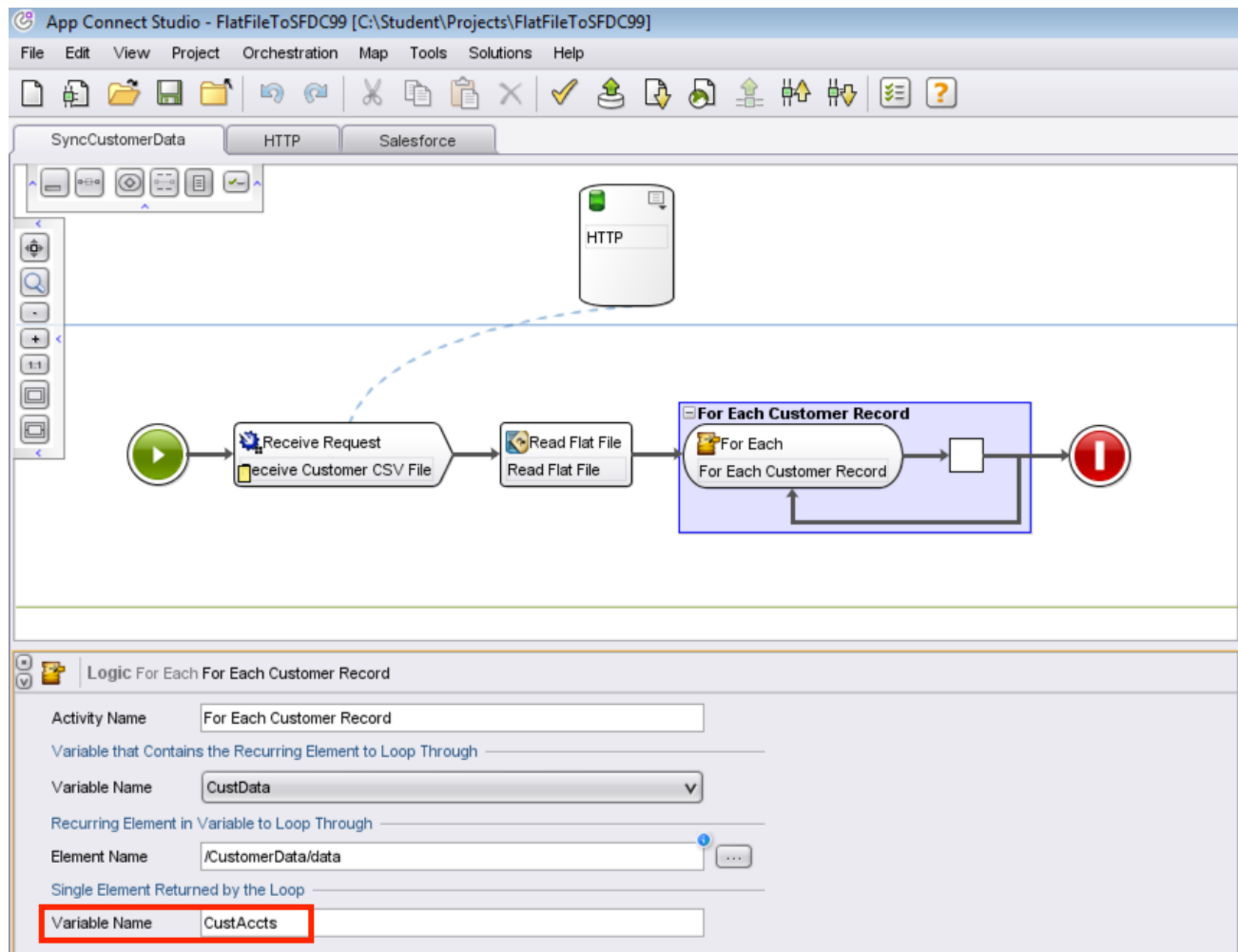
__38. **Variable Name:** Select **CustData** from the drop down list.



- ___39. **Element Name:** Click the ... box next to the **Element Name** field and then select the **data** recurring field from the **CustData** list then **OK**.

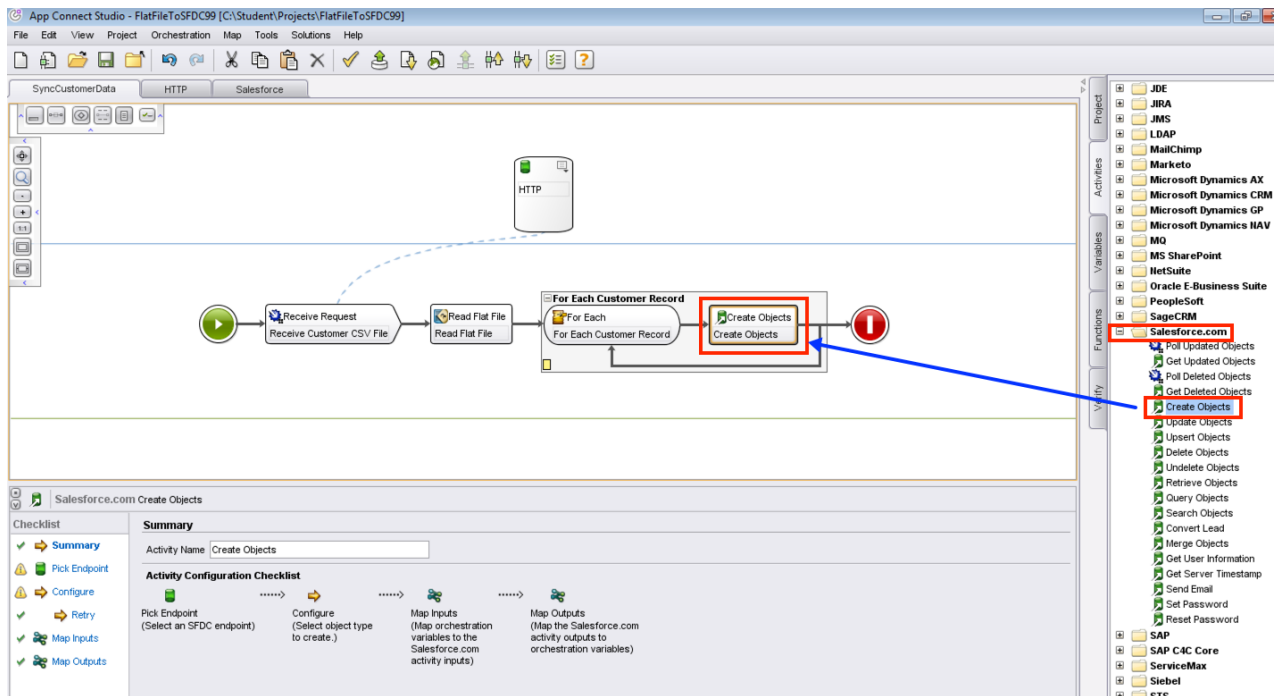


- __40. **Variable Name:** Change **Variable Name** parameter to **CustAccts**.



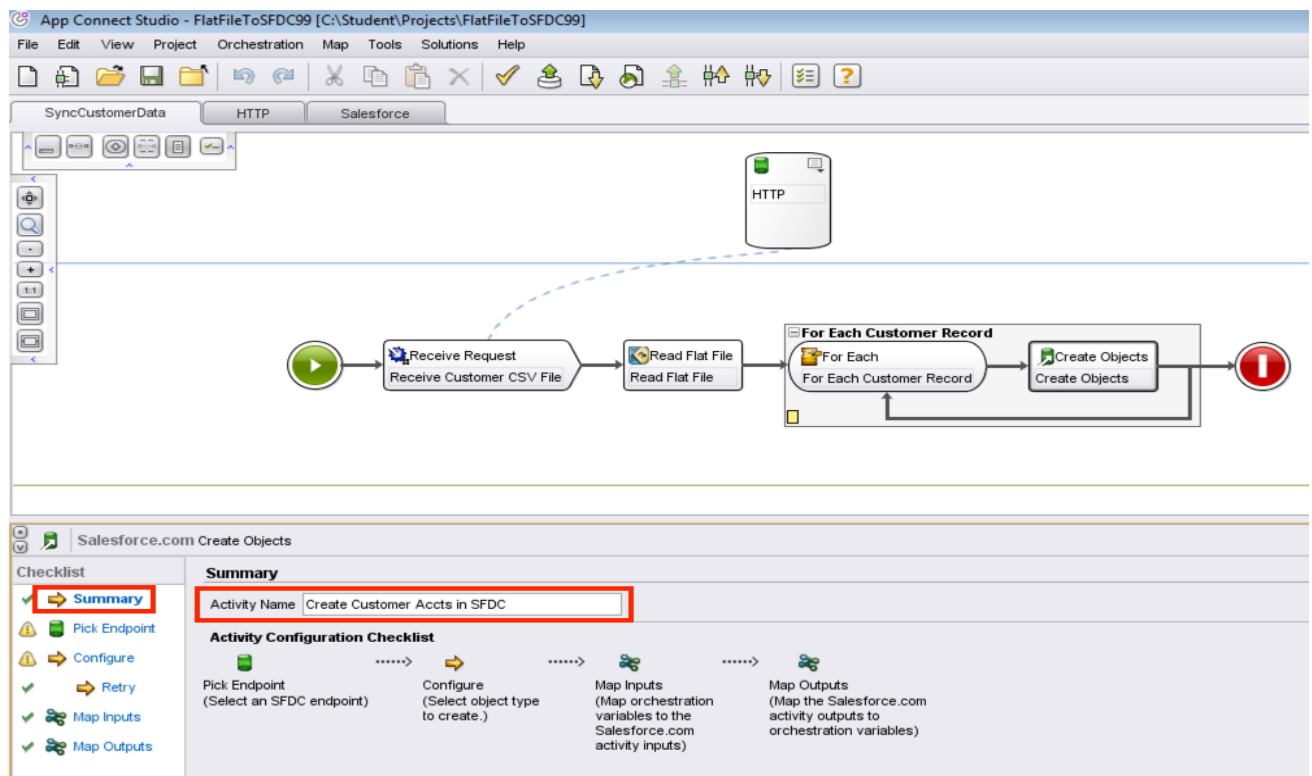
The configuration for the For Each Activity is complete.

- __41. Add **Create Objects** activity from the **Salesforce.com** folder located on the **Activities Tab** next to the **For Each** activity.

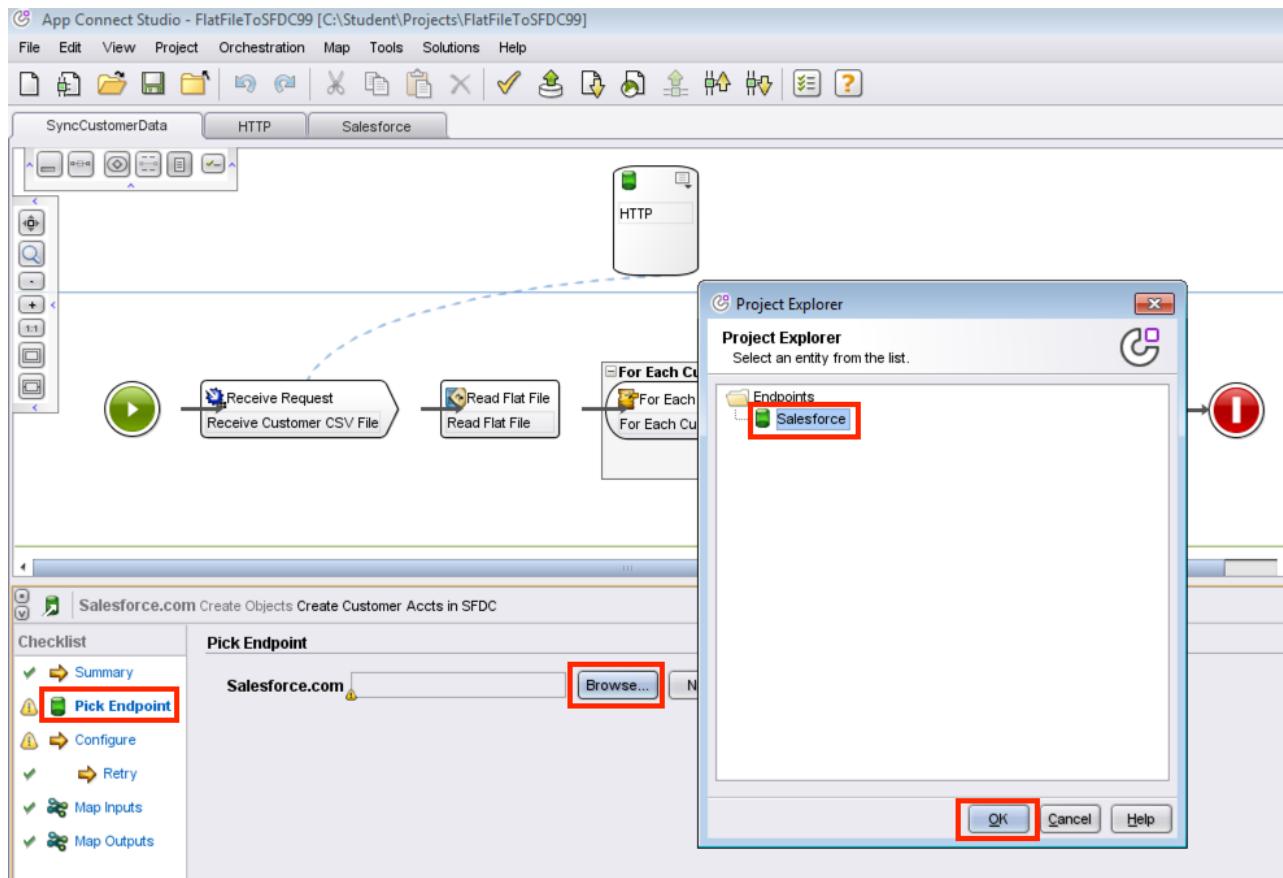


2.7.3 Configure the Salesforce Create Objects Activity by selecting the Checklist items:

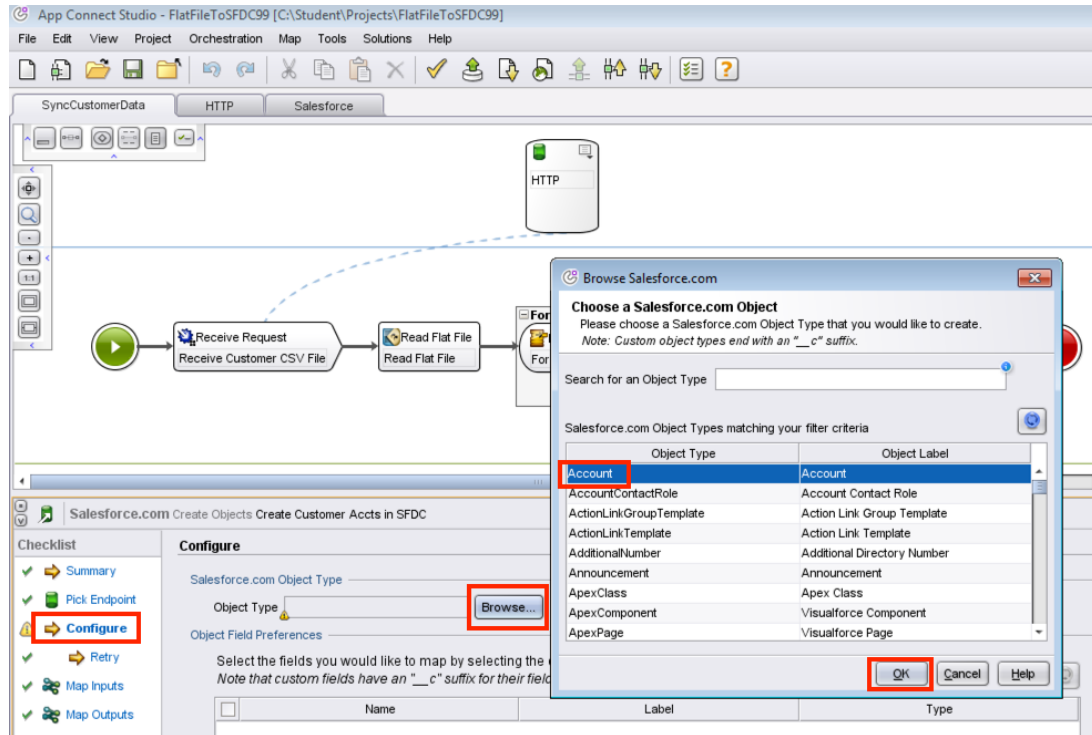
__42. **Summary:** Change the **Activity Name** to “Create Customer Accts in SFDC”.

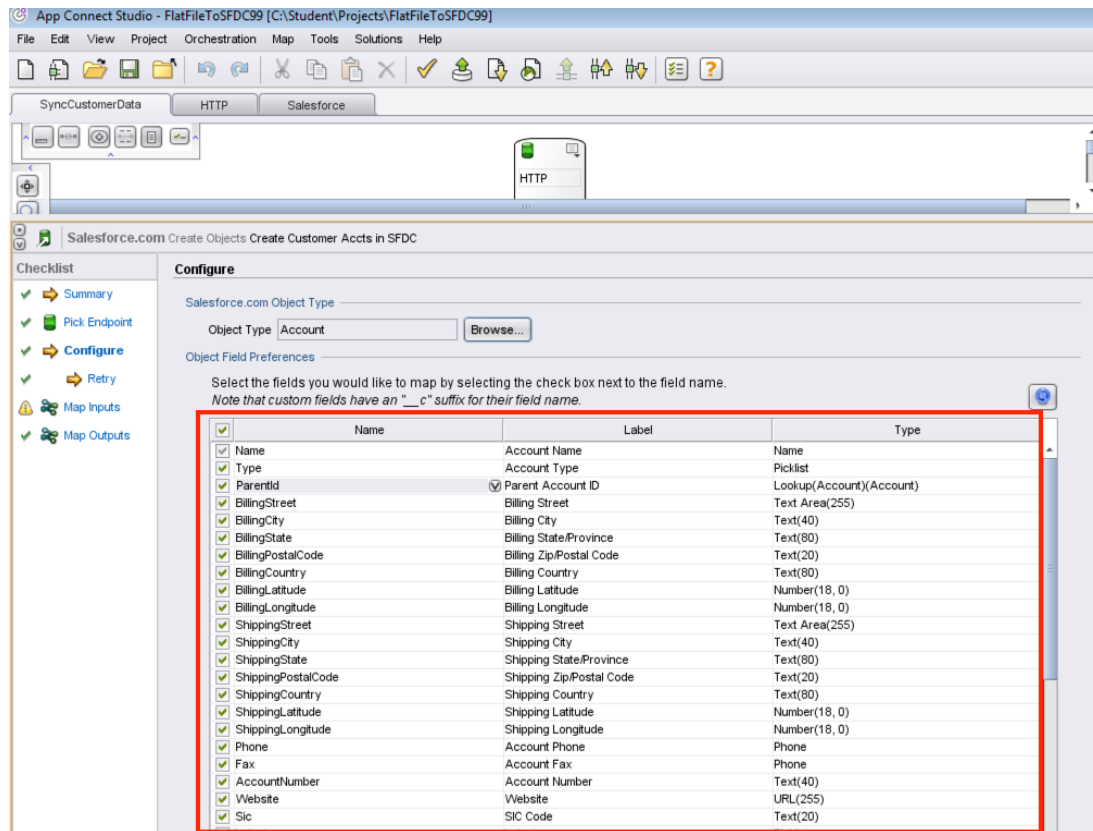


__43. **Pick Endpoint:** Click **Browse** and select **Salesforce** as the endpoint then **OK**.

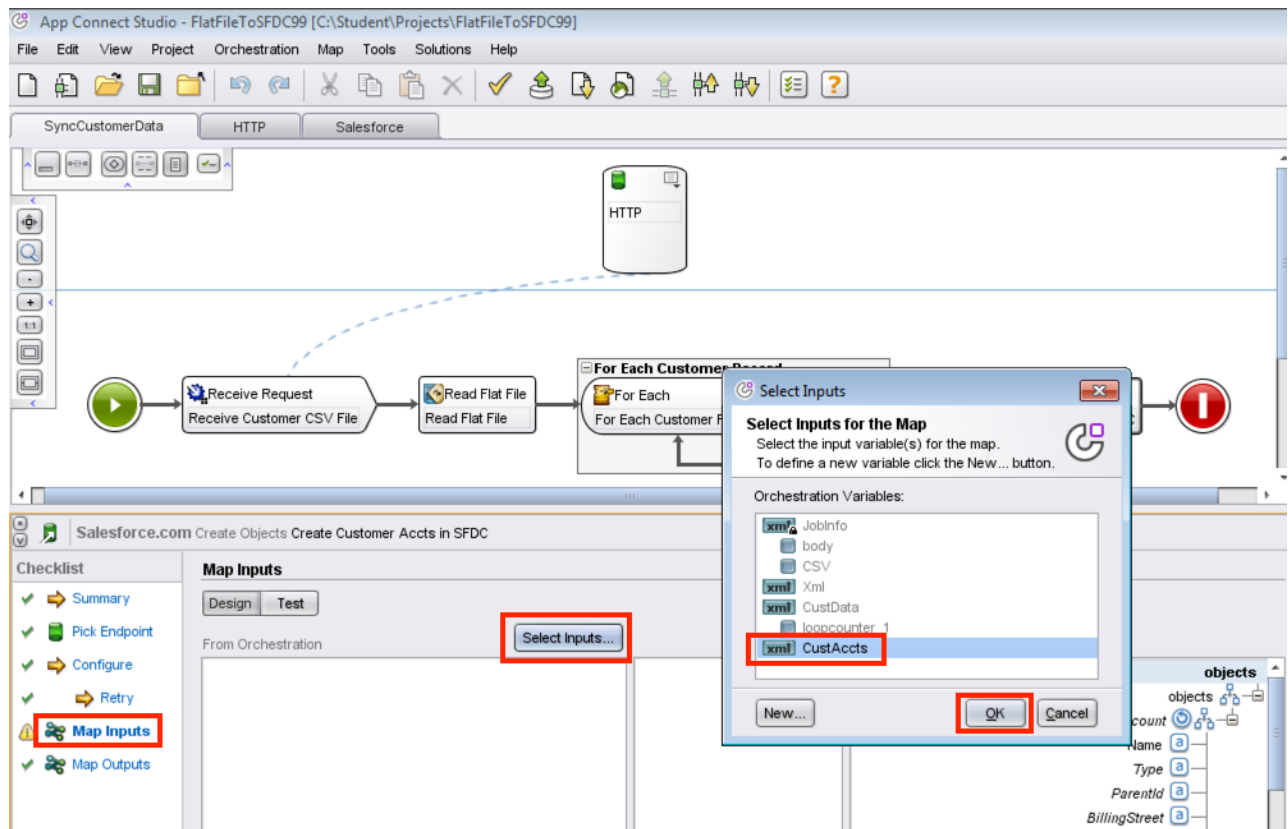


- __44. **Configure:** Click **Browse** and select the **Account Object** from **Salesforce** list then **OK**. This will then display all of the **Account Object** metadata fields available including any customized fields that may have been added to the **Salesforce Account Object**.

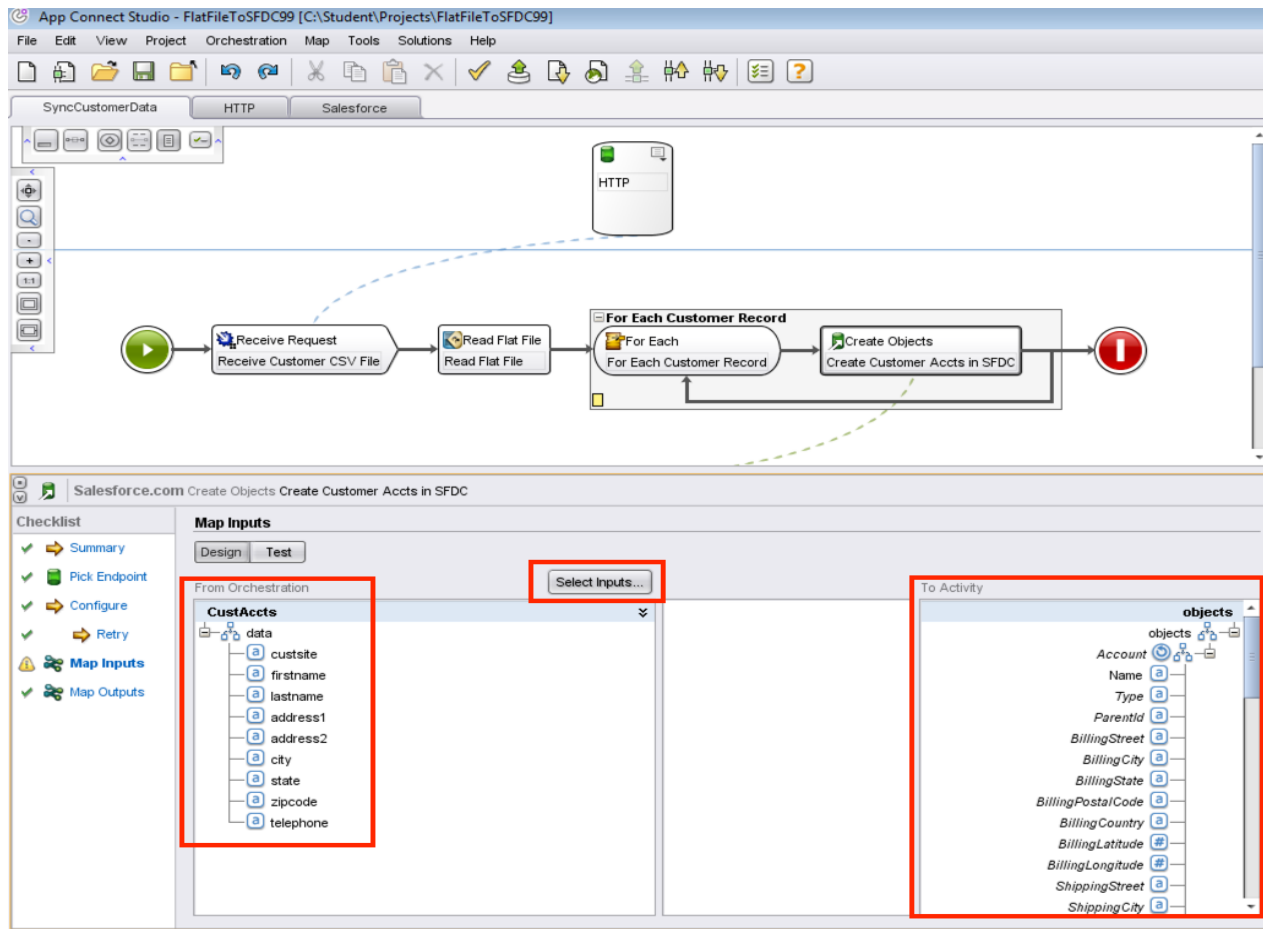




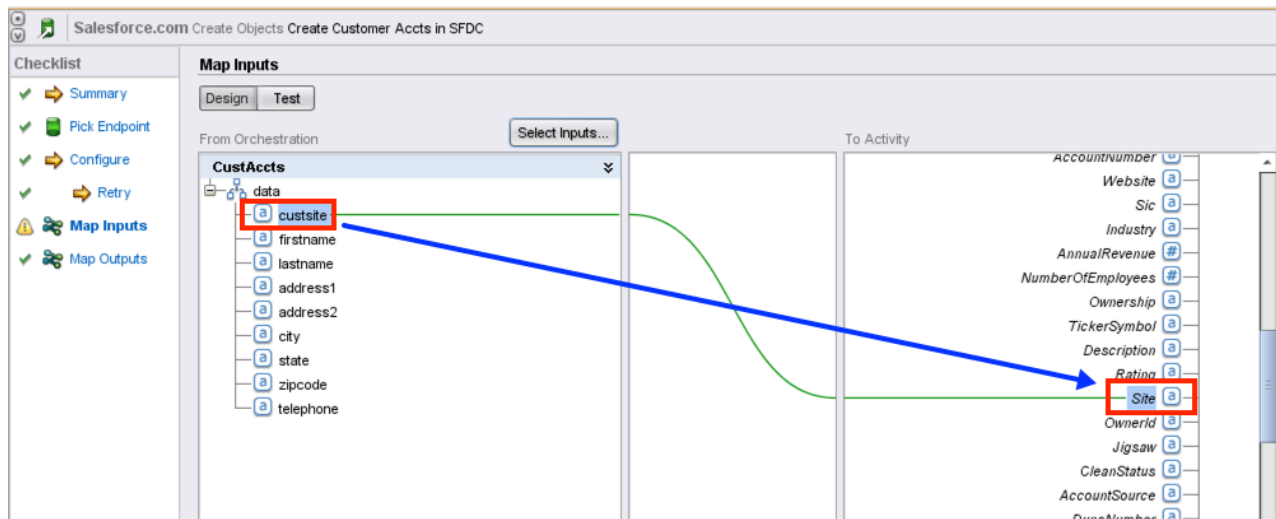
- ___45. **Map Inputs:** The input to this activity is the output from our previous activity, the **For Each** activity. Click **Select Inputs** and choose **CustAccts** then **OK**.



The representation will then be the source of data, located under the **From Orchestration** column on the left and the target of the data located on the right under the **To Activity** column.



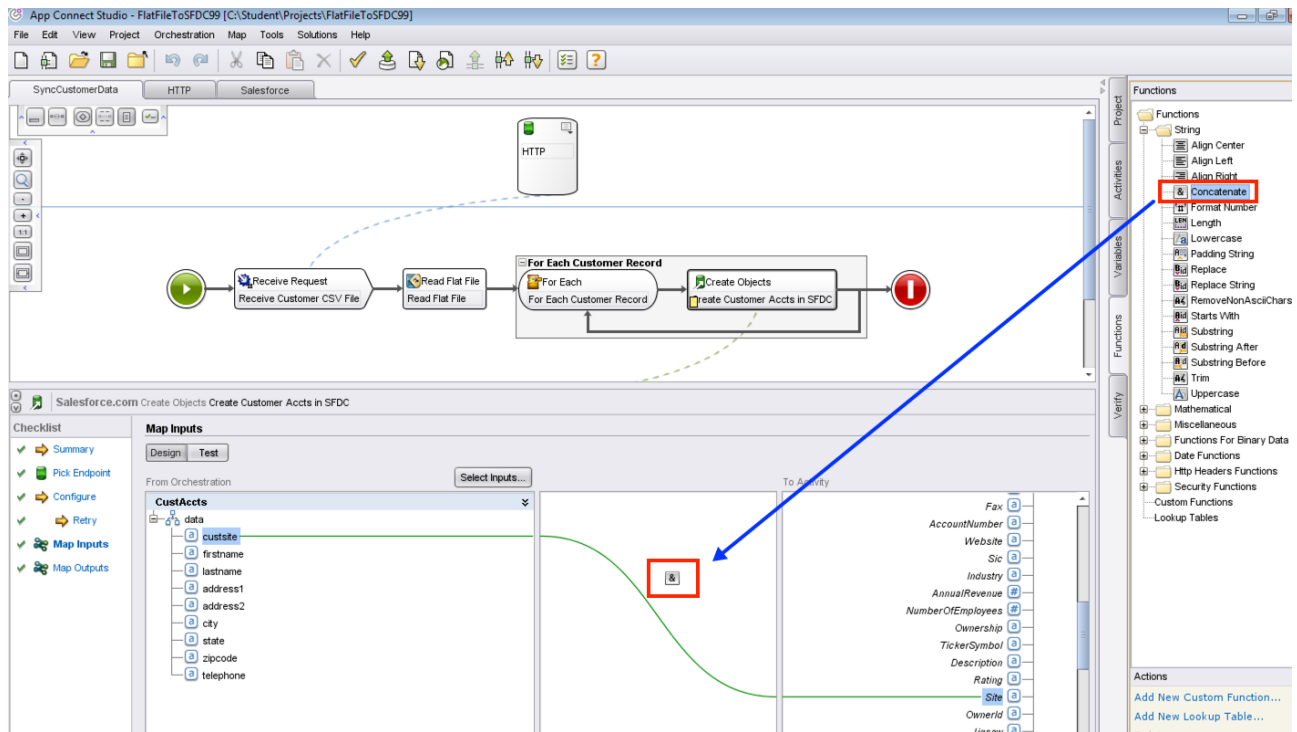
- __46. Place your cursor on the **custsite** field located under the source of data column **From Orchestration**, left click and hold down, then drag that field to the **site** field located under the target of data column **To Activity**.



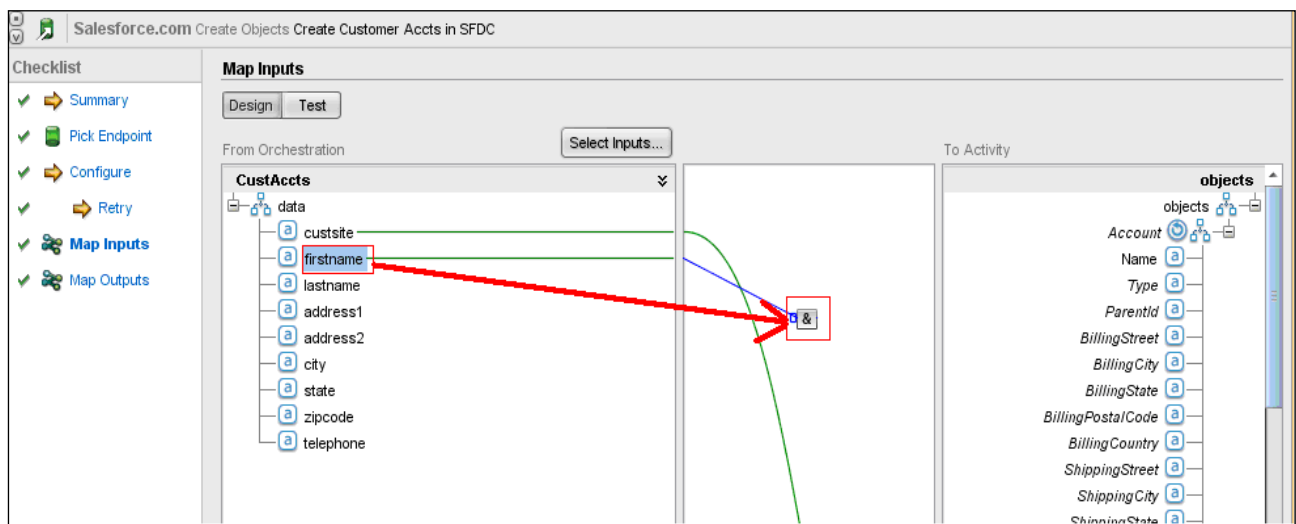
You will notice that the source of data contains a **firstname** and **lastname** field. However, the target of data only has a **Name** field. This will require a manipulation of the data between the source and target systems.



47. Next select the **Functions** tab on the right hand side of App Connect Studio to reveal the available functions used to transform data being sent from a source of data to a target of data. Select the **Concatenate** function from the **String** folder and drag it to the area between the **From Orchestration** column and the **To Activity** column.



- ___48. Select the **firstname** field from the **From Orchestration** column and drag it to the **Concatenate** icon.



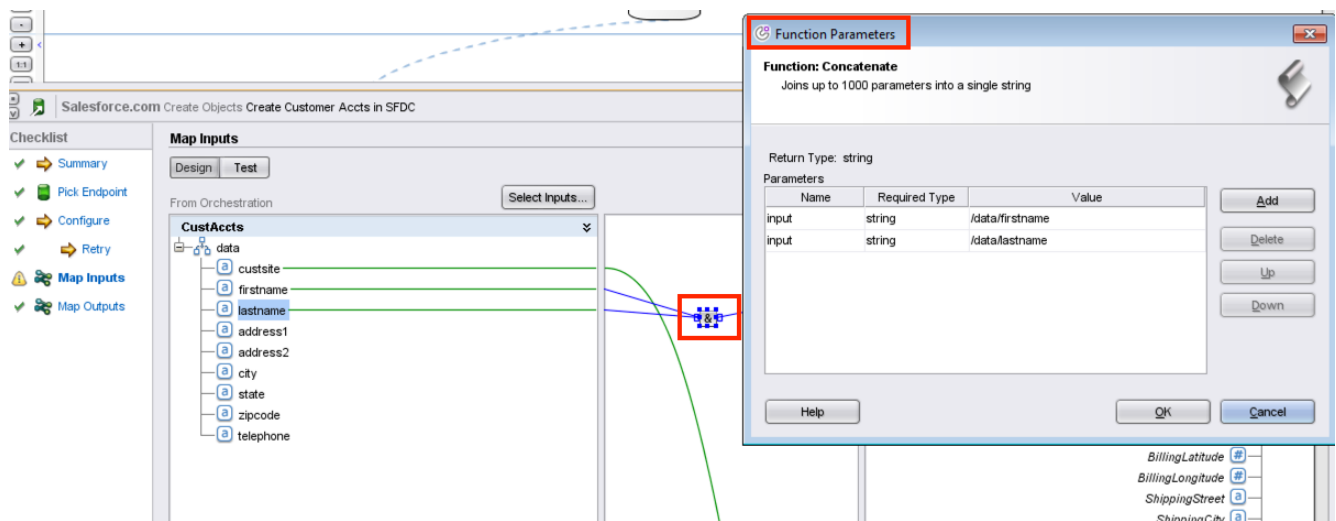
- ___49. Select the **lastname** field from the **From Orchestration** column and drag it to the **Concatenate** icon.



__50. Left click and hold the **Concatenate** icon and drag it to the **Name** field located under the **To Activity** column.

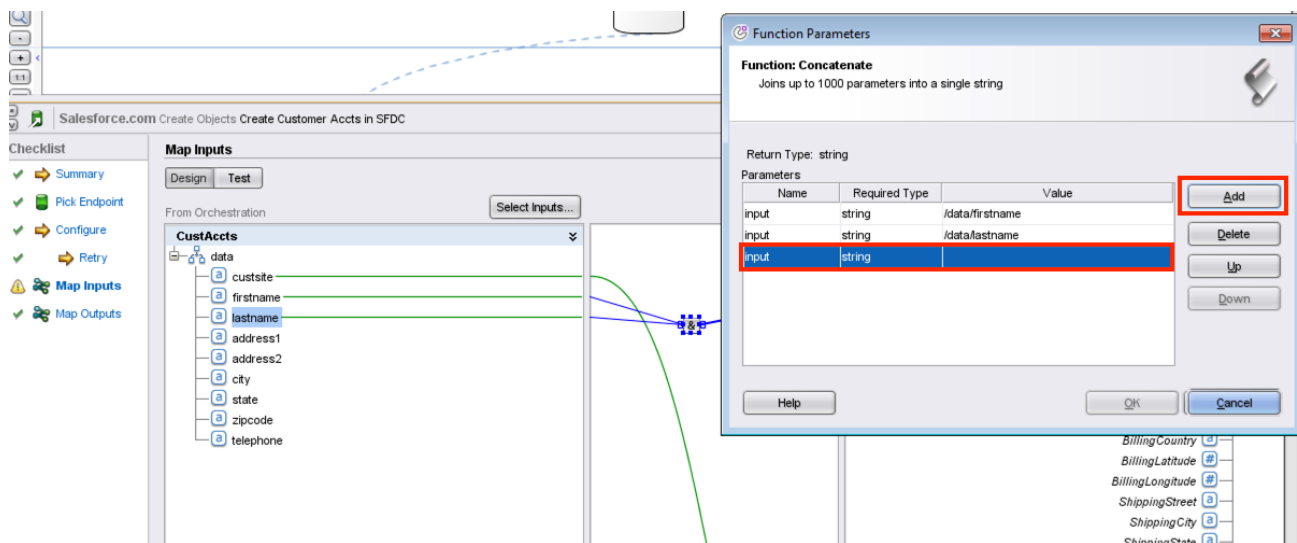


- __51. Double click the **Concatenate** icon to open the **Function Parameters** dialog box.

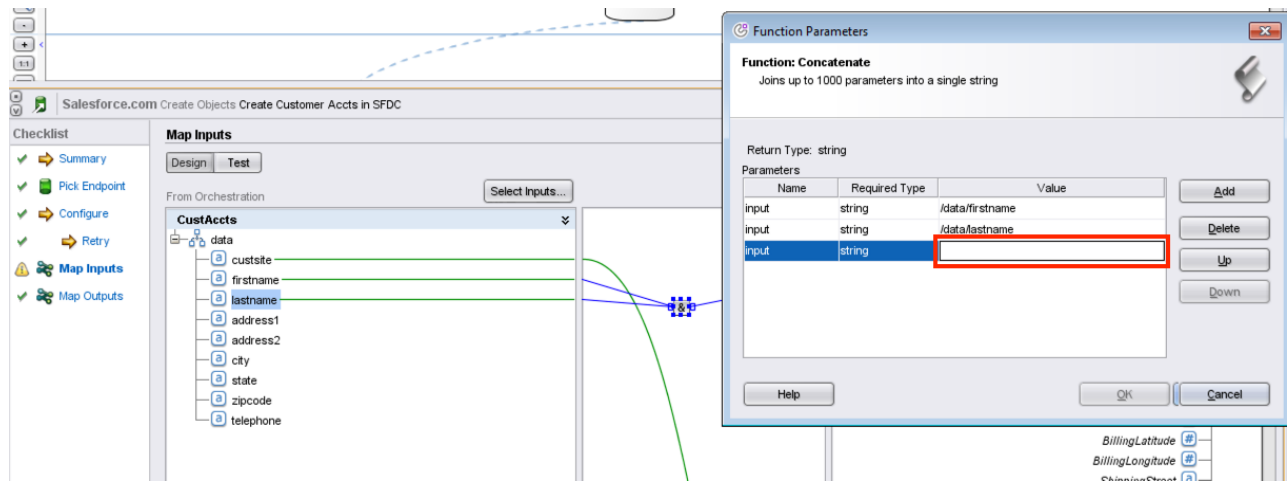


We will now separate the **firstname** and **lastname** with a space so that they appear correctly in our target system.

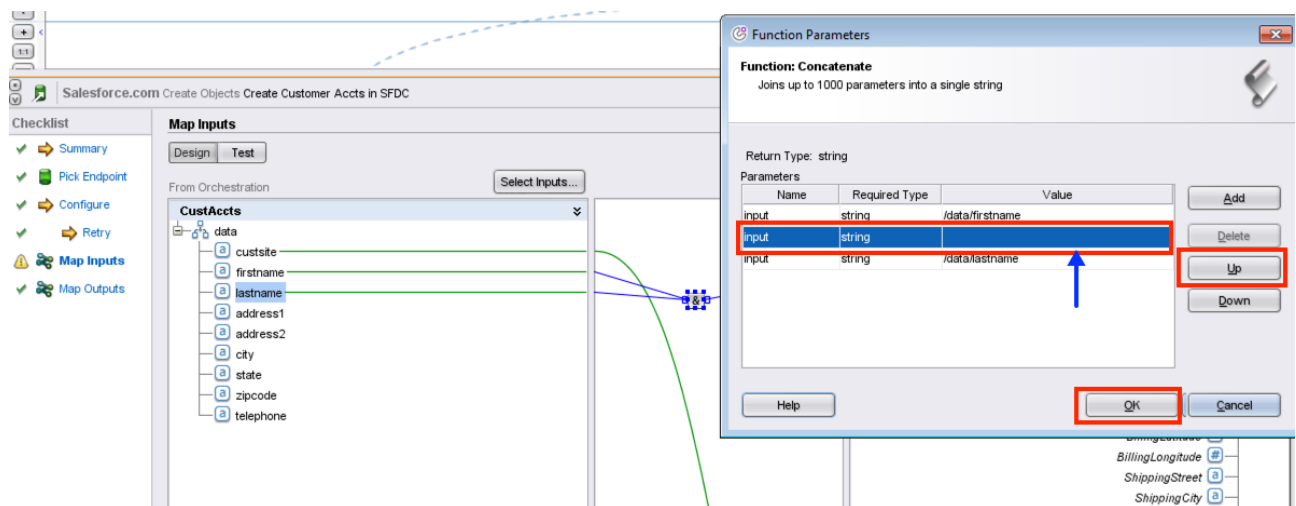
- __52. Click the **Add** button which will then show an additional **input** string field.



- __53. Double click in the blank area under the **Value** column for the new input string and then press the **space bar** once to add a space and click the **Enter** key.



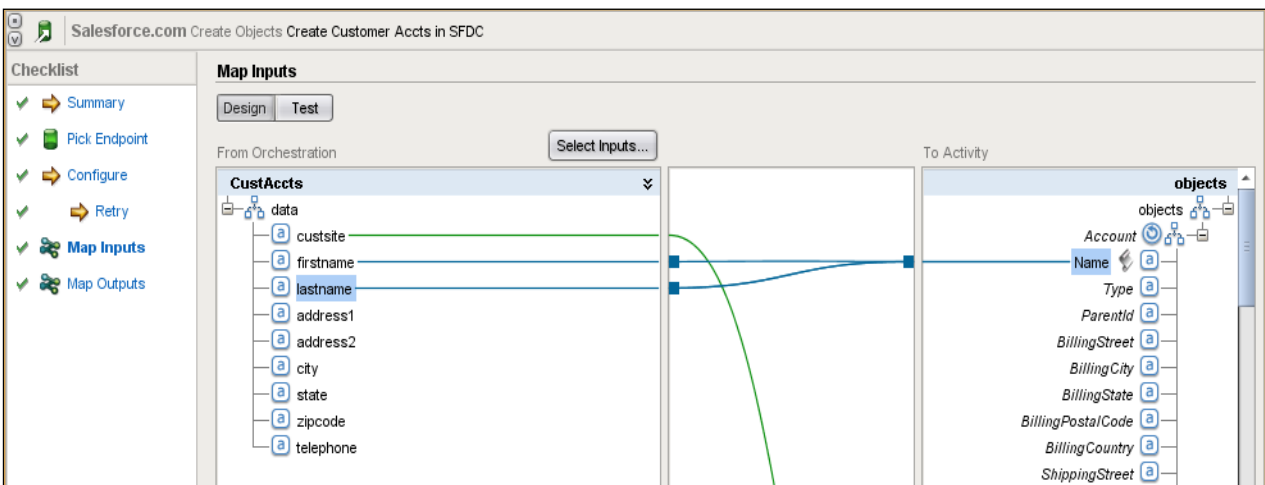
- __54. Click the **Up** button so that we can move the space between the **firstname** and **lastname** fields. Then click **OK**.



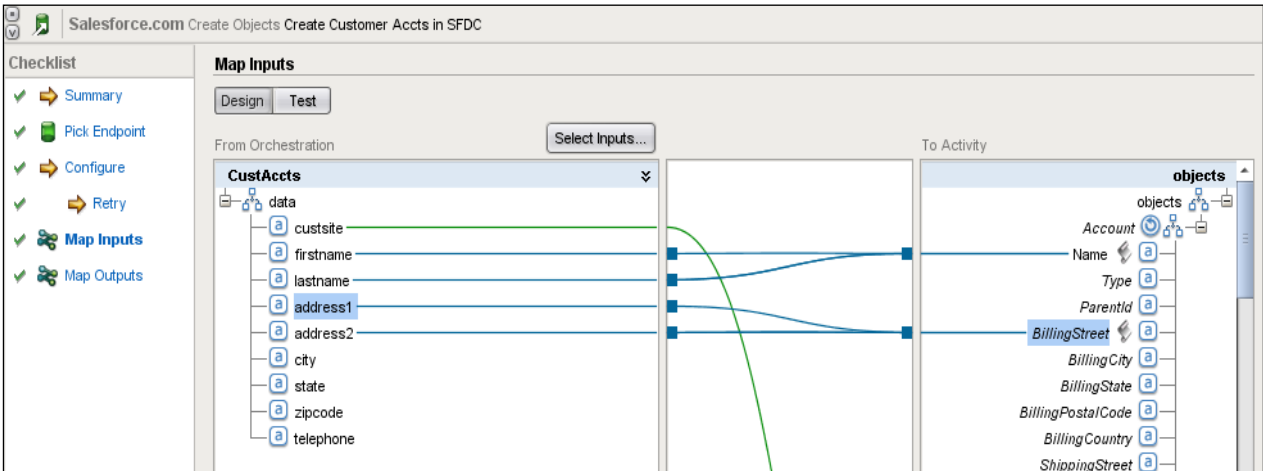
55. The Function Parameters dialog box will disappear. Now place your cursor anywhere in the area below the **Concatenate** icon and right click to apply the **Function Graph** to the activity. Click **Apply Function Graph**.



End Result:

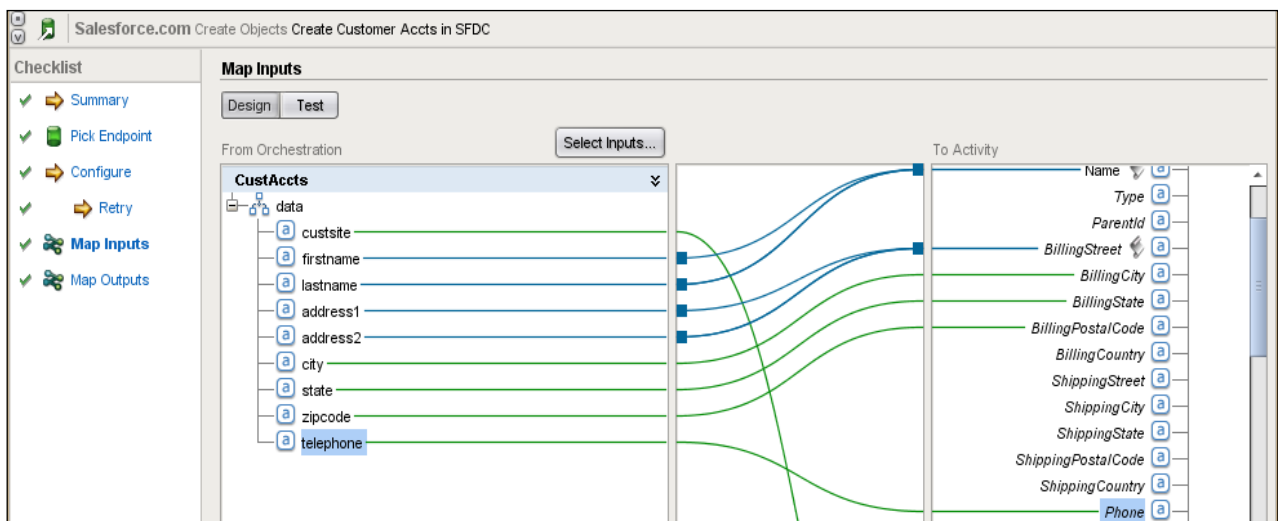


- ___56. **Repeat the same process** as above (steps 25-33) in order to concatenate the **address1** and **address2** fields located under the source **From Orchestration** column to the **Billing Street** field located under the target **To Activity** column. The end result should look like this:



- ___57. Now map the remaining fields under the source **From Orchestration** column to the corresponding columns below located under the target **To Activity** column.

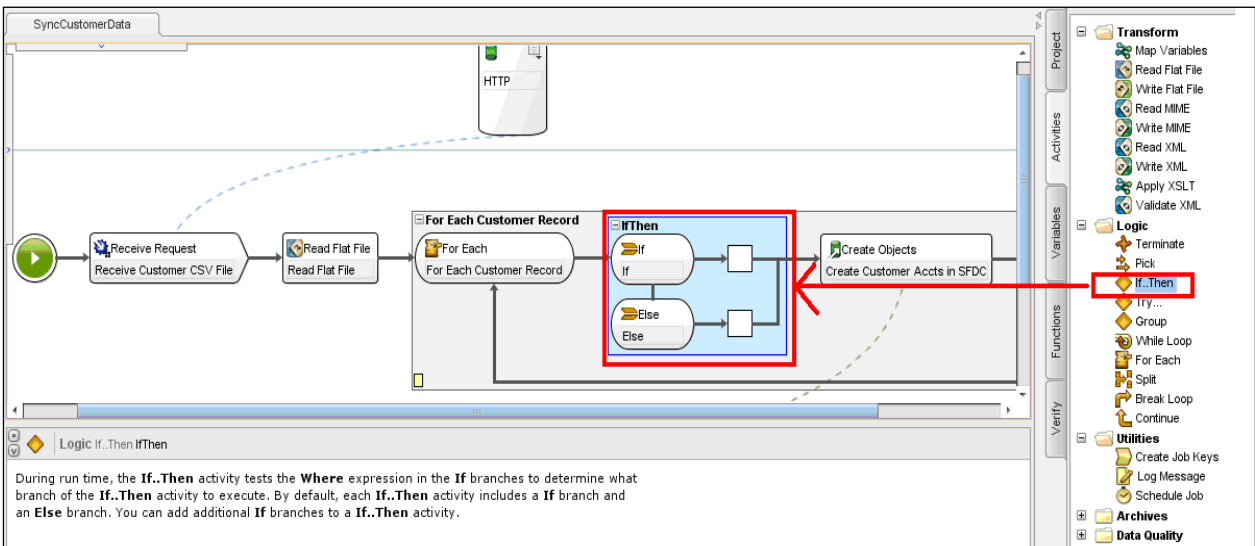
city → Billing City
 state → Billing State
 zipcode → Billing Postal Code
 telephone → Phone



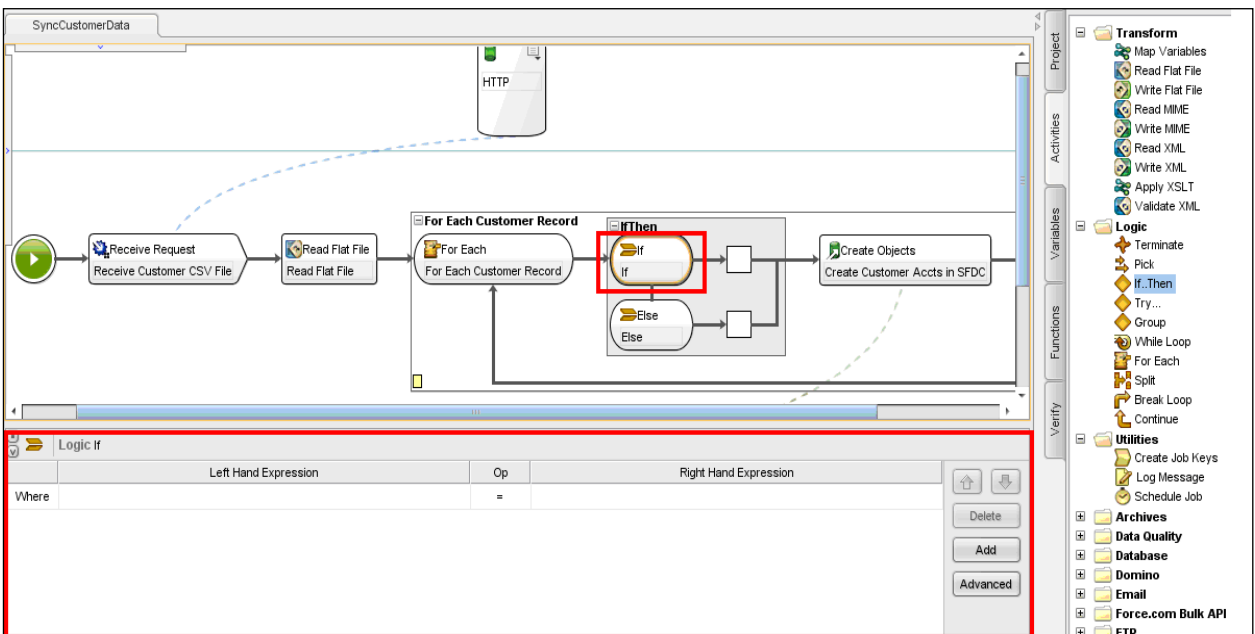
The configuration for the Salesforce.com Create Objects Activity is complete.

2.7.4 Now we will add some Business Logic to the orchestration.

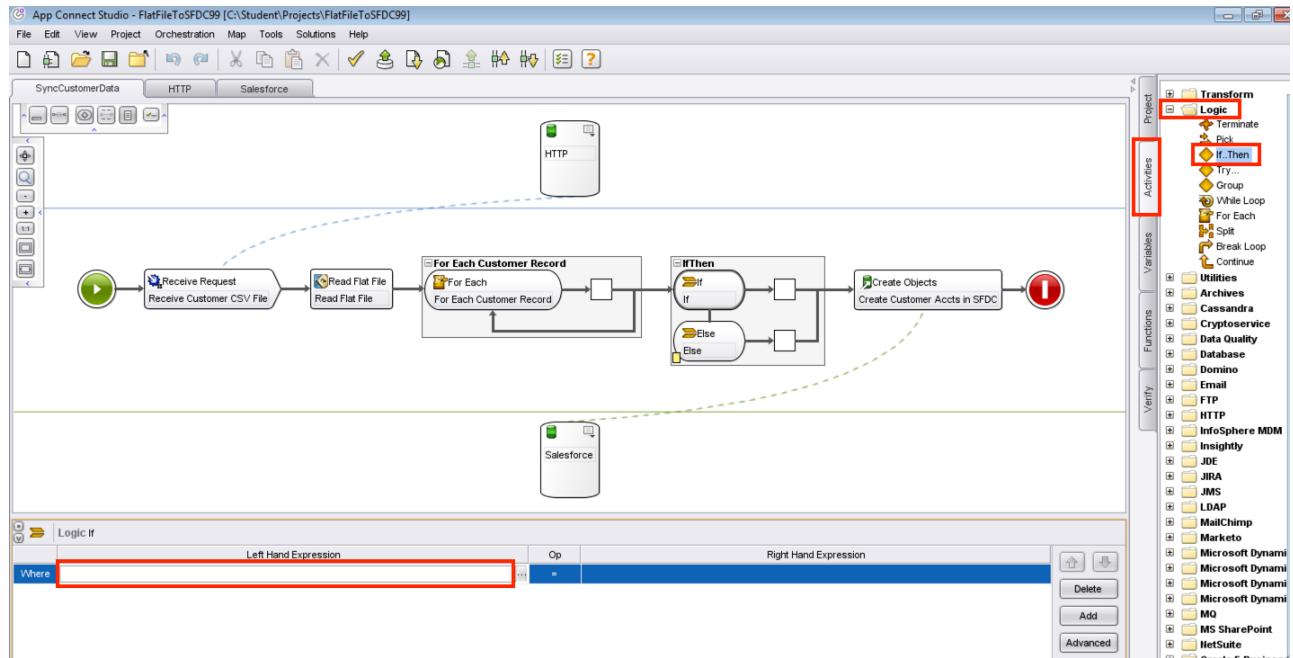
- __58. Drag the **If Then Else** Activity located on the **Activities** tab under the **Logic** folder to the area between the **For Each** activity and the Salesforce **Create Objects** activity.



- __59. Click the top **If** input box and you will notice that it opens the **If-Then** area below the orchestration workflow.



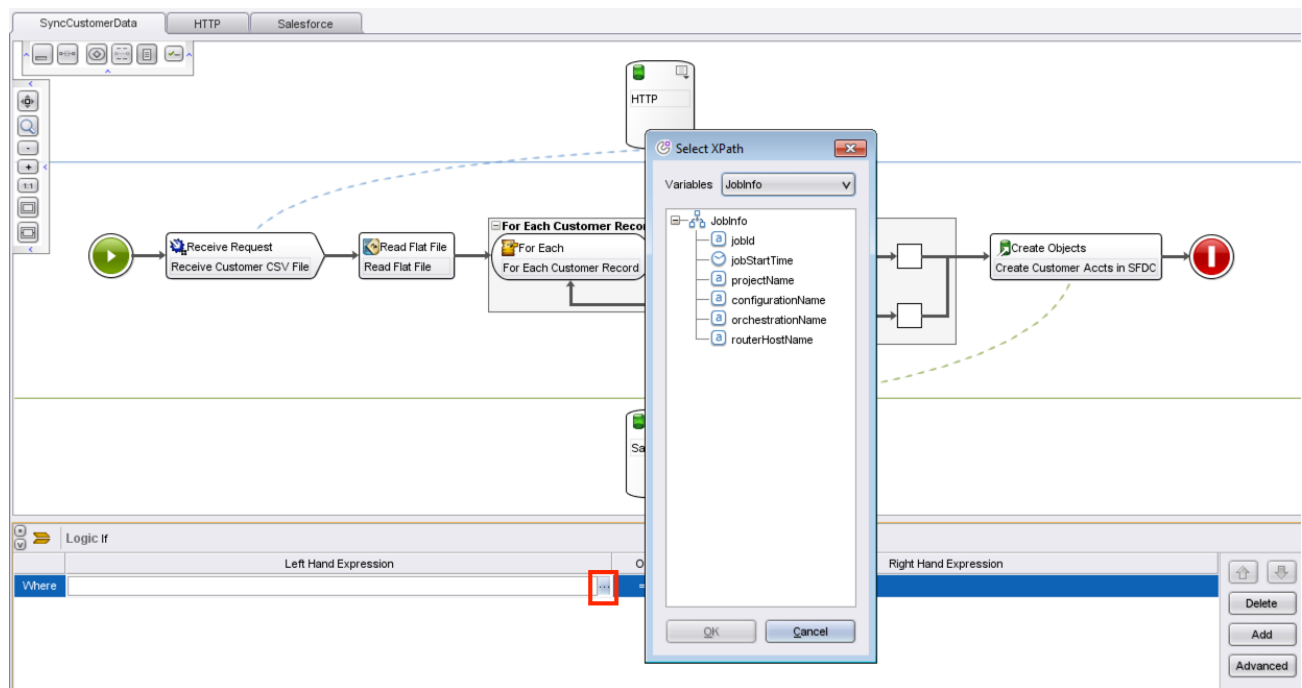
- __60. Place your cursor in the blank area next to the **Where** parameter and click to open up the dialog box under the **Left Hand Expression**.



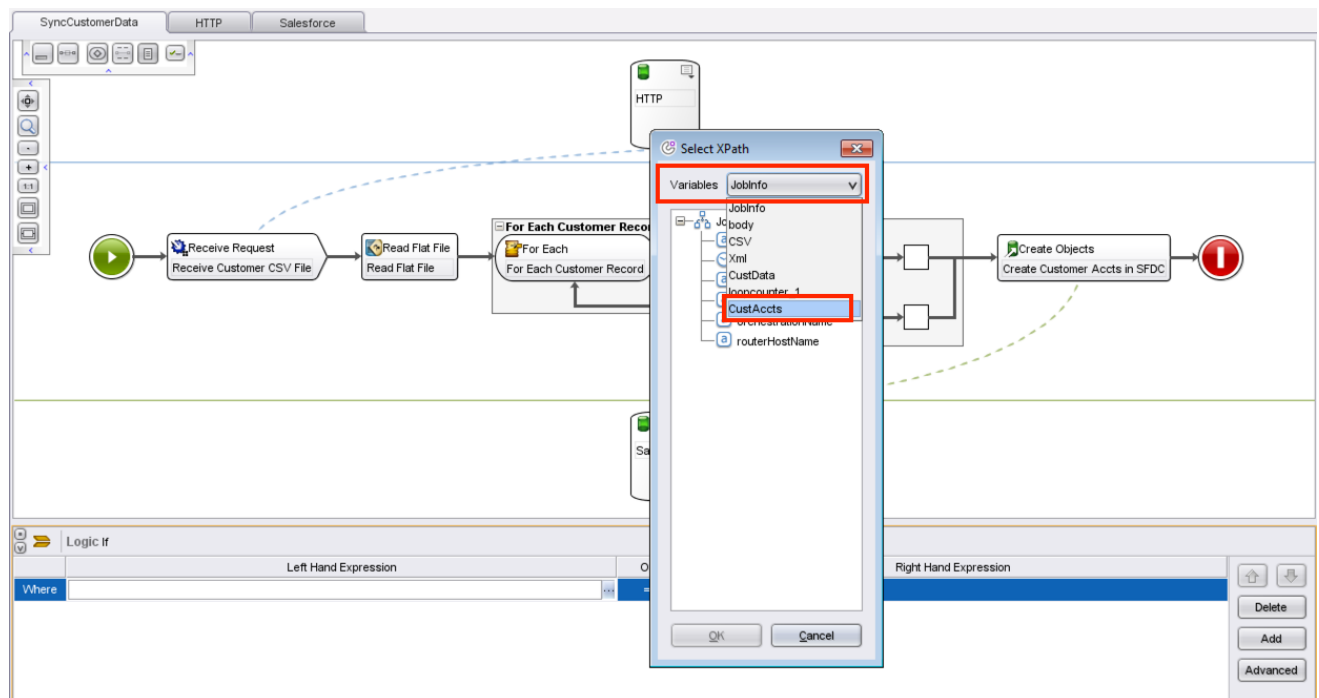
You can type in an expression or you can use the Logic Wizard...For this lab you will use the **Logic Wizard** to create the business logic expression:

If the Customer Site is not equal to the value East, then create the new record in Salesforce.com...If it is equal to the value East then you will disregard and log an error message to be viewed at a later time in the Lab.

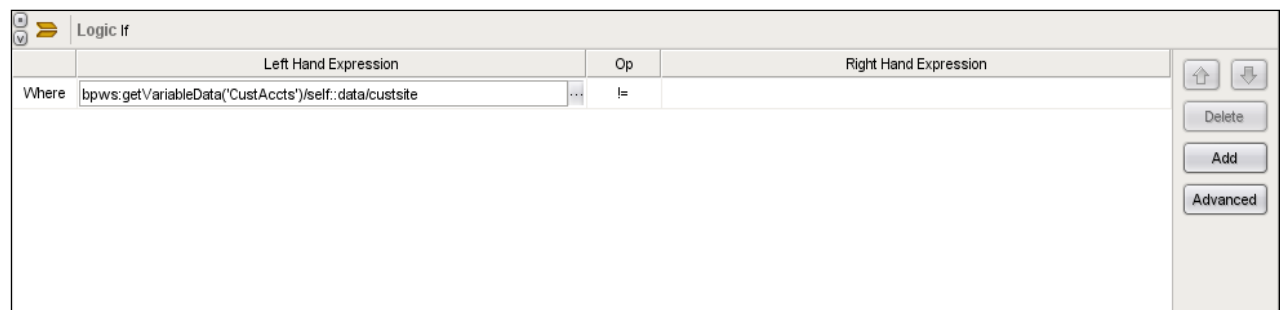
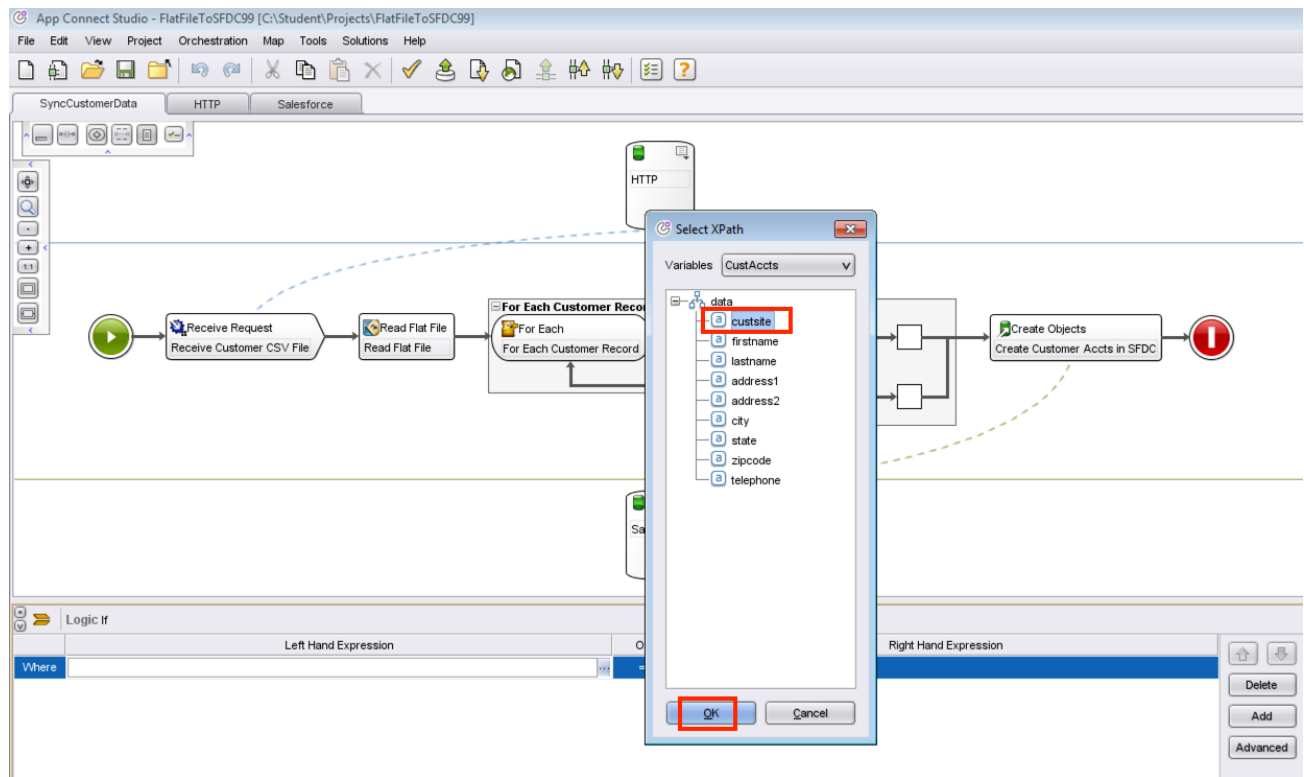
- __61. Click the ... box located next to the blank **Left Hand Expression** field to invoke the **Logic Wizard**.



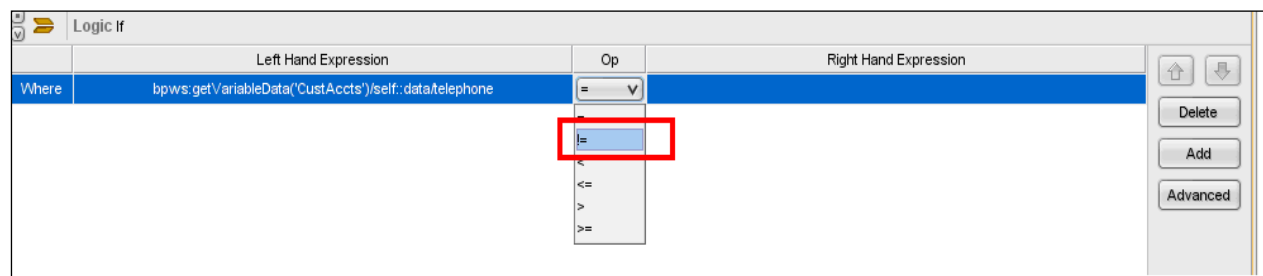
- __62. Click the **Variables** drop down box to select the variable **CustAccts** that will be used as input. Once you select **CustAccts**, you will be shown a list of the fields available for the business logic.



__63. Click the variable **custsite** and then **OK**.



__64. Click the **operator** field located under the **Op** column to select the “Not Equal” (!=) parameter.



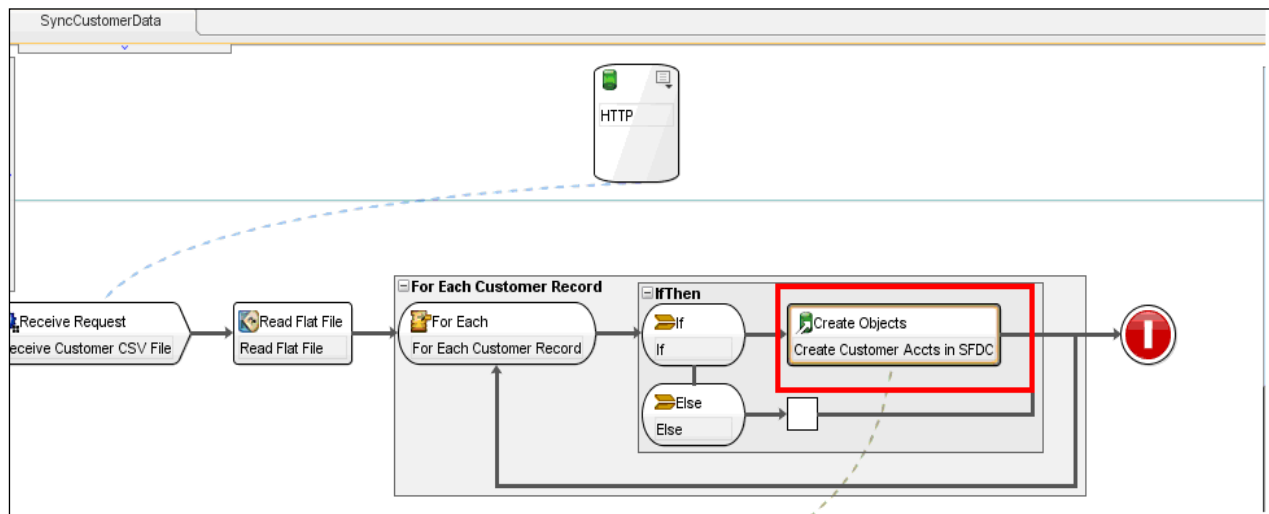
- __65. Now click the blank area under the **Right Hand Expression** column to add the value.

	Left Hand Expression	Op	Right Hand Expression
Where	bpws: getVariableData('CustAccts')/self::data/telephone	=	

- __66. Enter the value as typed here and press the **Enter** key: 'East' (Make sure to use quotes)

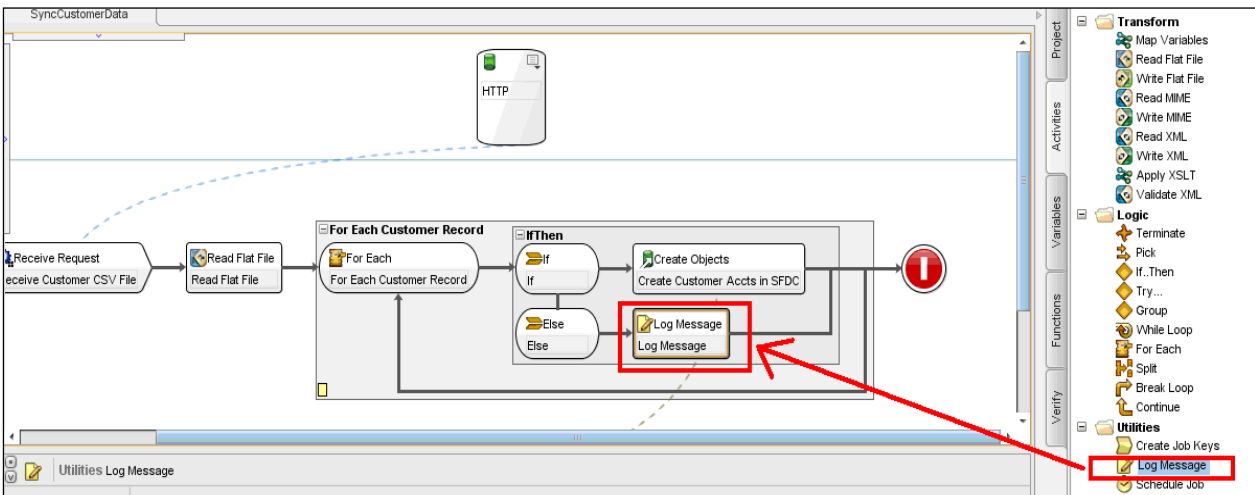
	Left Hand Expression	Op	Right Hand Expression
Where	bpws: getVariableData('CustAccts')/self::data/custsite	=	'East'

- __67. Next you will click the Salesforce.com **Create Objects** activity and drag it to the empty box located next to right of the **IF** box. This will allow any records that pass the business logic test to be created in Salesforce.com.

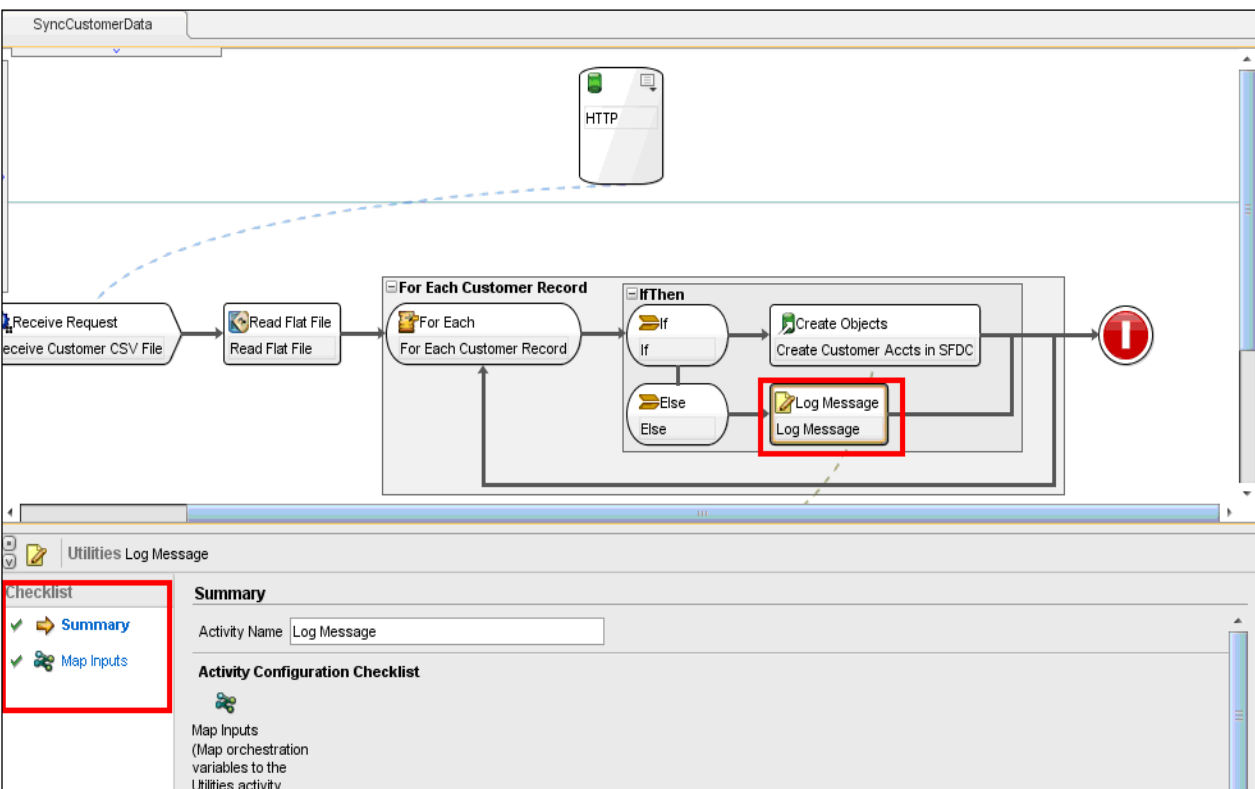


2.7.5 Next you will create an error message for those records that do not meet the Business Logic validation.

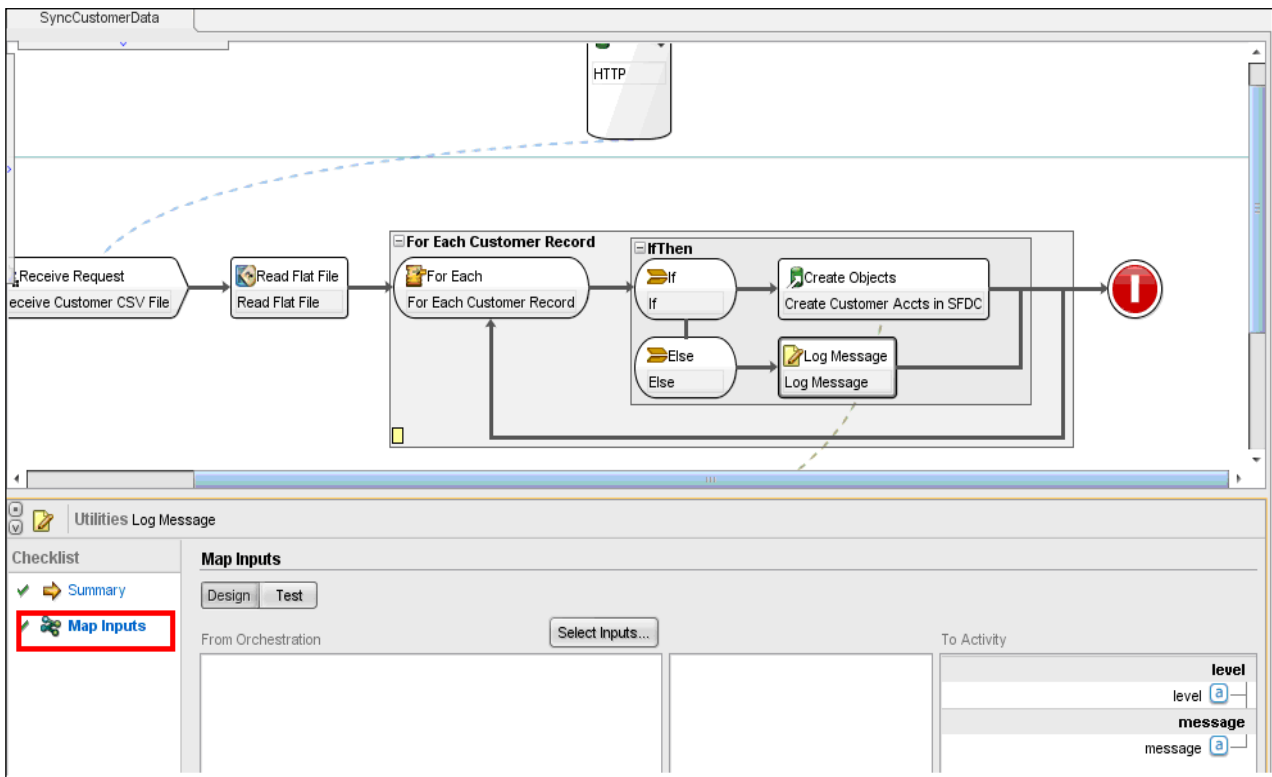
- __68. From the **Activities** tab **Utilities** folder select the **Log Message** activity and drag it to the blank box next to the **Else** box.



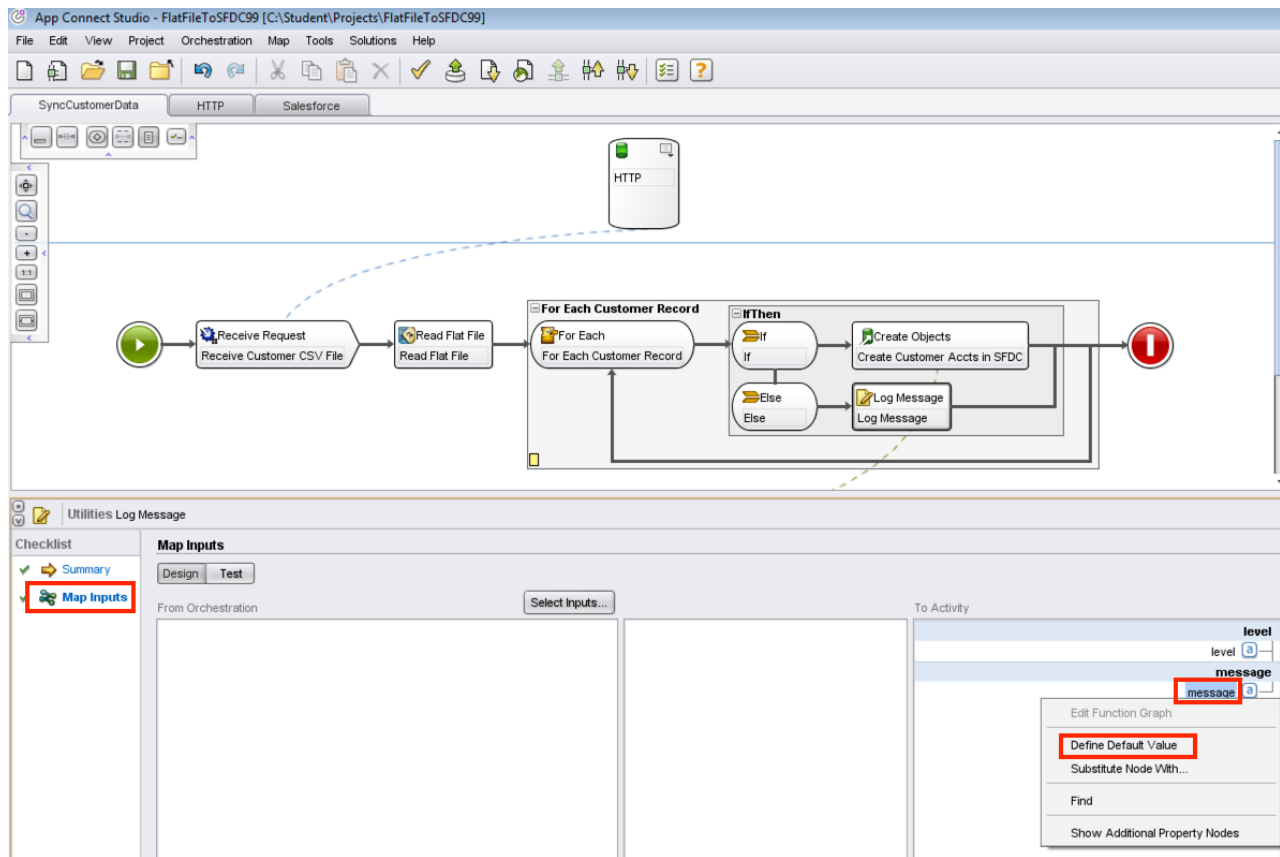
- __69. Click the **Log Message** activity to display the **Checklist**.



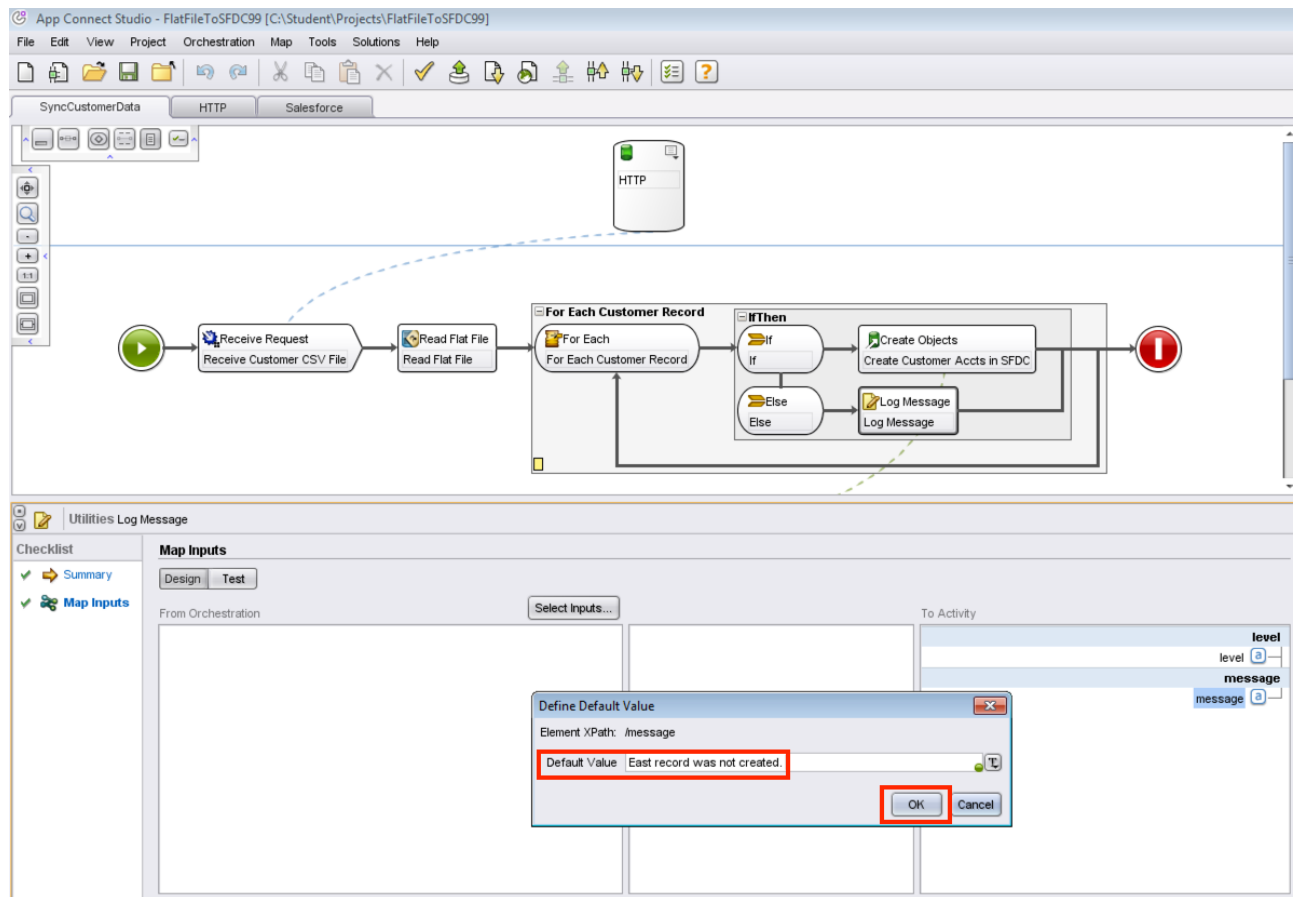
__70. Click the **Checklist** item **Map Inputs**.



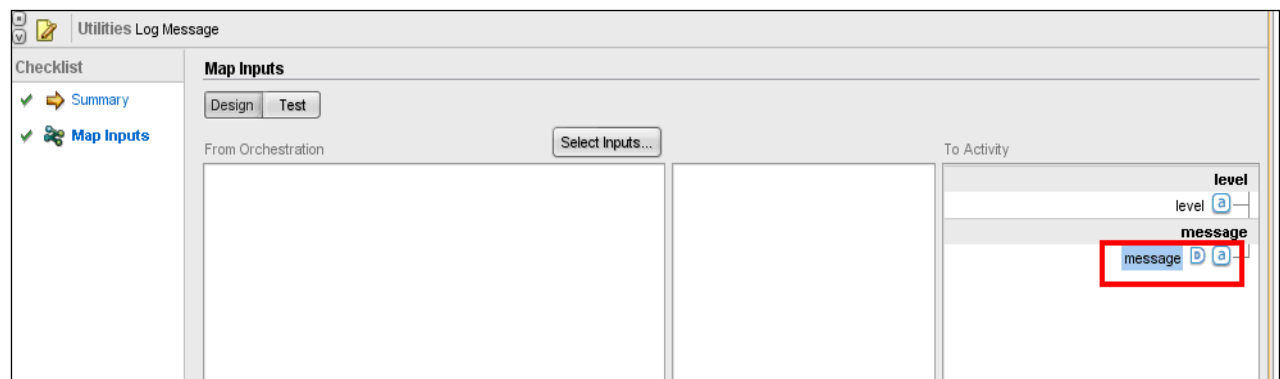
__71. Right click **message** field to assign a default message.



__72. Type in the following: **East Record was not created.** then press **OK.**



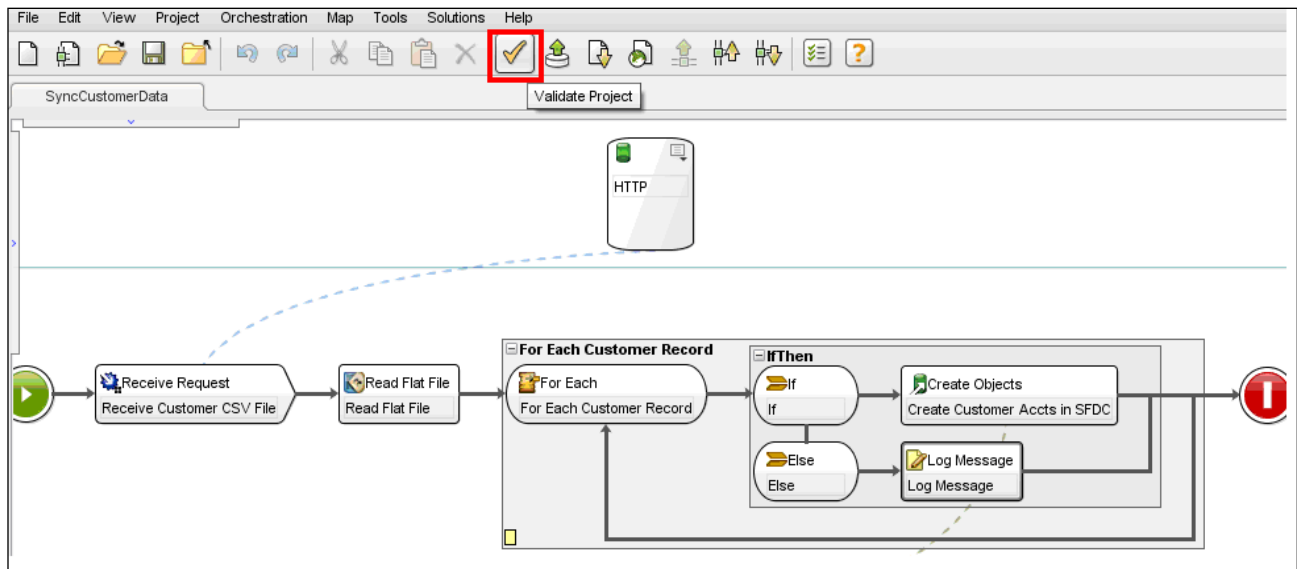
Notice that a **D** is now placed next to the message field indicating that a default value has been created.



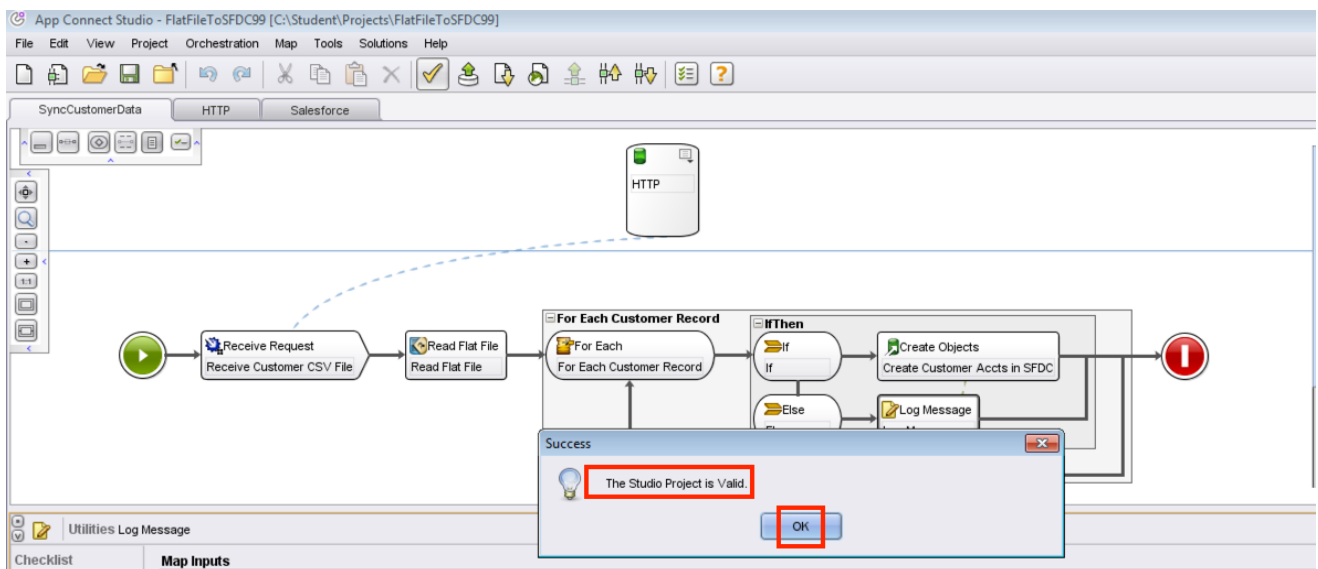
The IF Then Else Activity has been completed.

2.7.6 You are now ready to Validate your Orchestration to make sure all of your Checklist Items have been answered correctly.

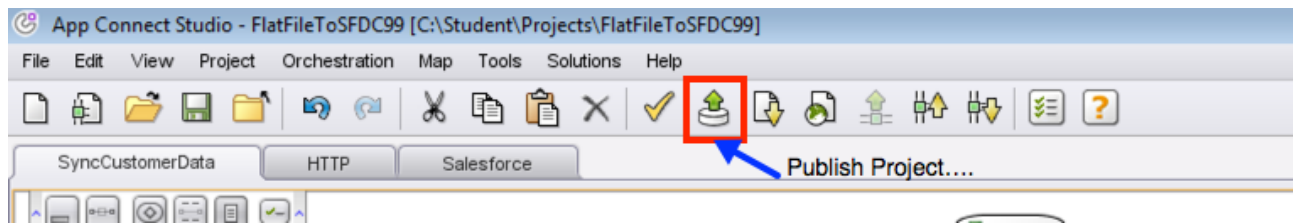
__73. Click the **Validate Project** icon located at the top of App Connect Studio.



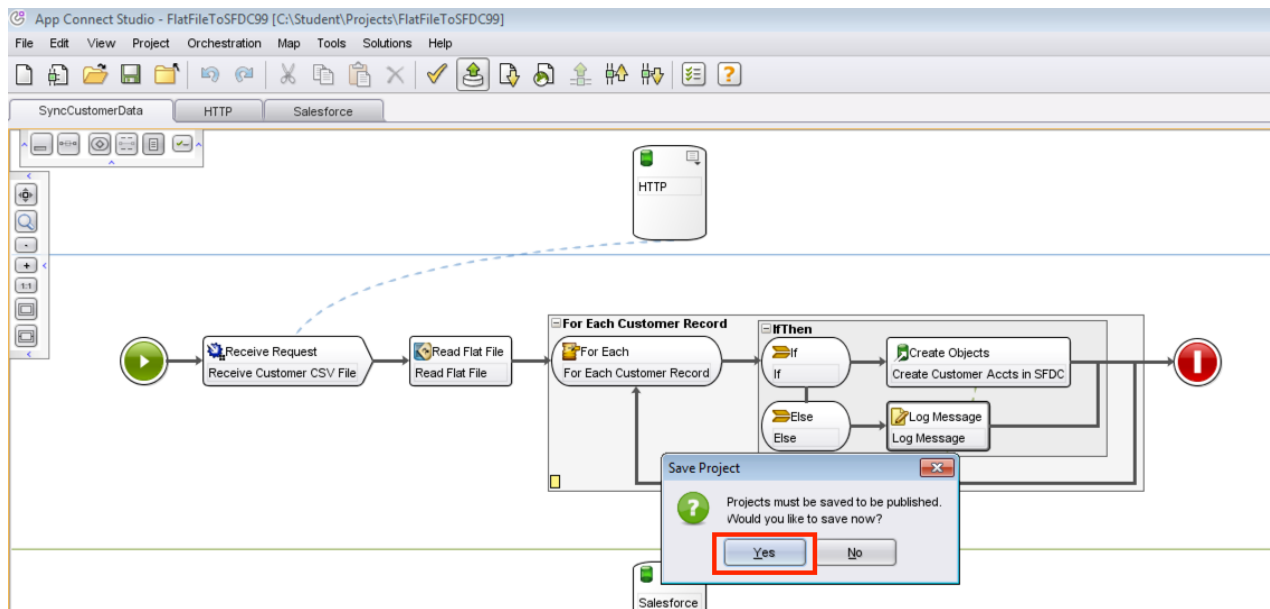
If the Orchestration validates correctly you should see a **Success** message box appear, then click **OK**.



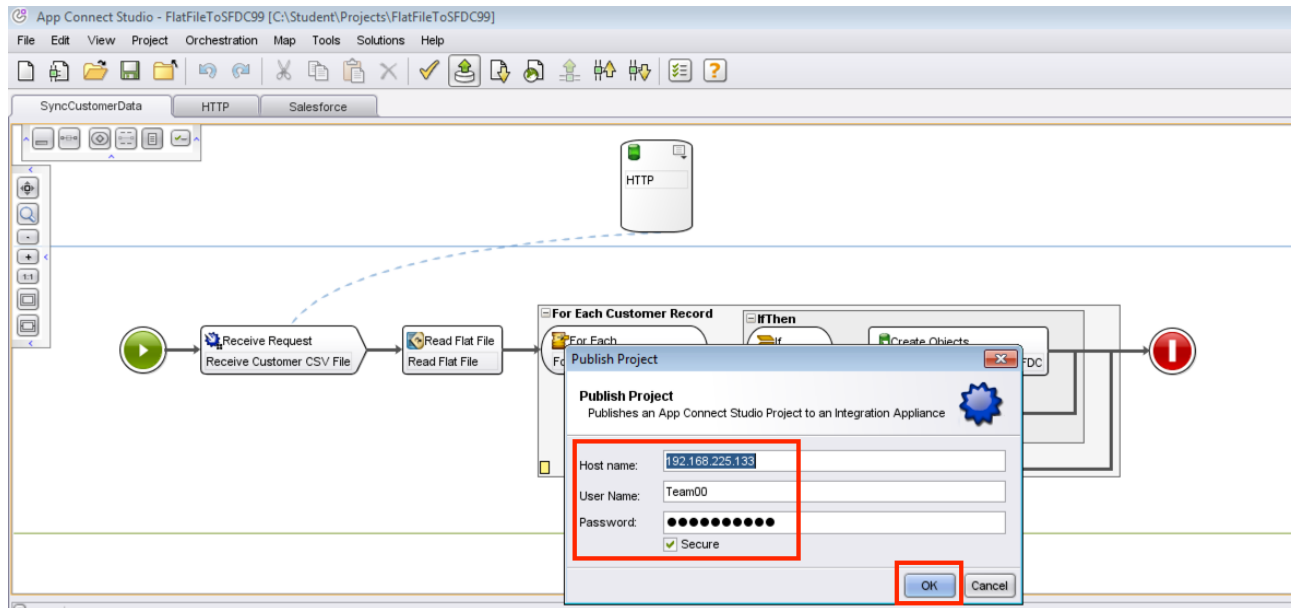
__74. In order to publish the project, click the **Publish** icon located in the tool bar.



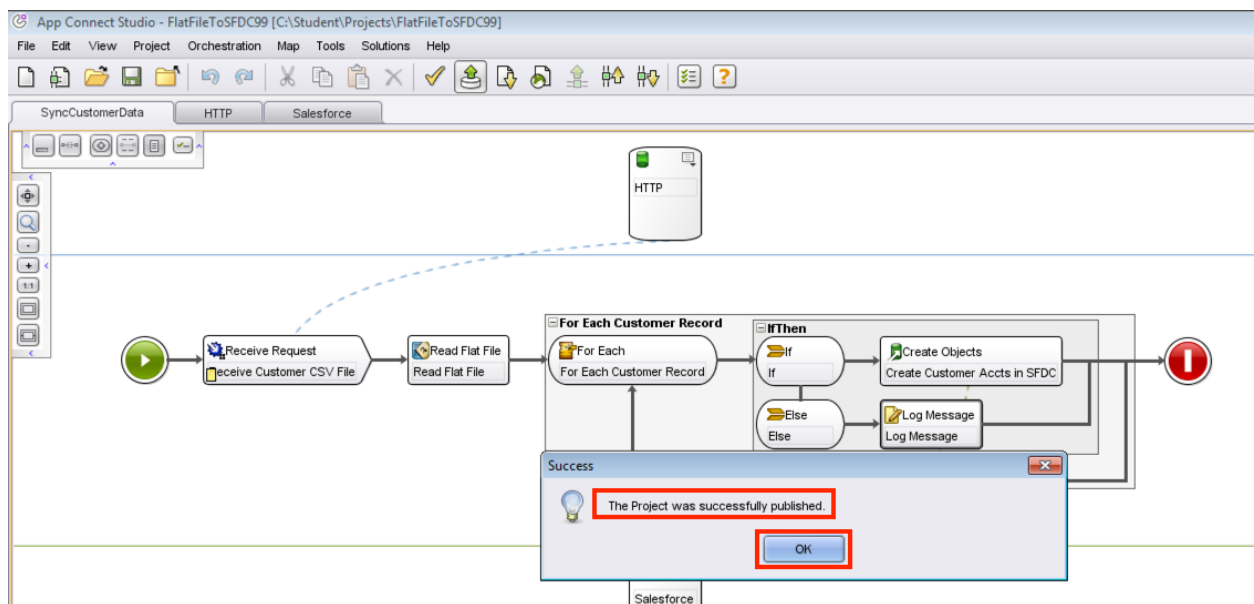
__75. Select **Yes** to publish the project.



- __76. Fill in the fields with the correct App Connect Professional Management IP for Host name, User Name and Password parameters and click **OK**.



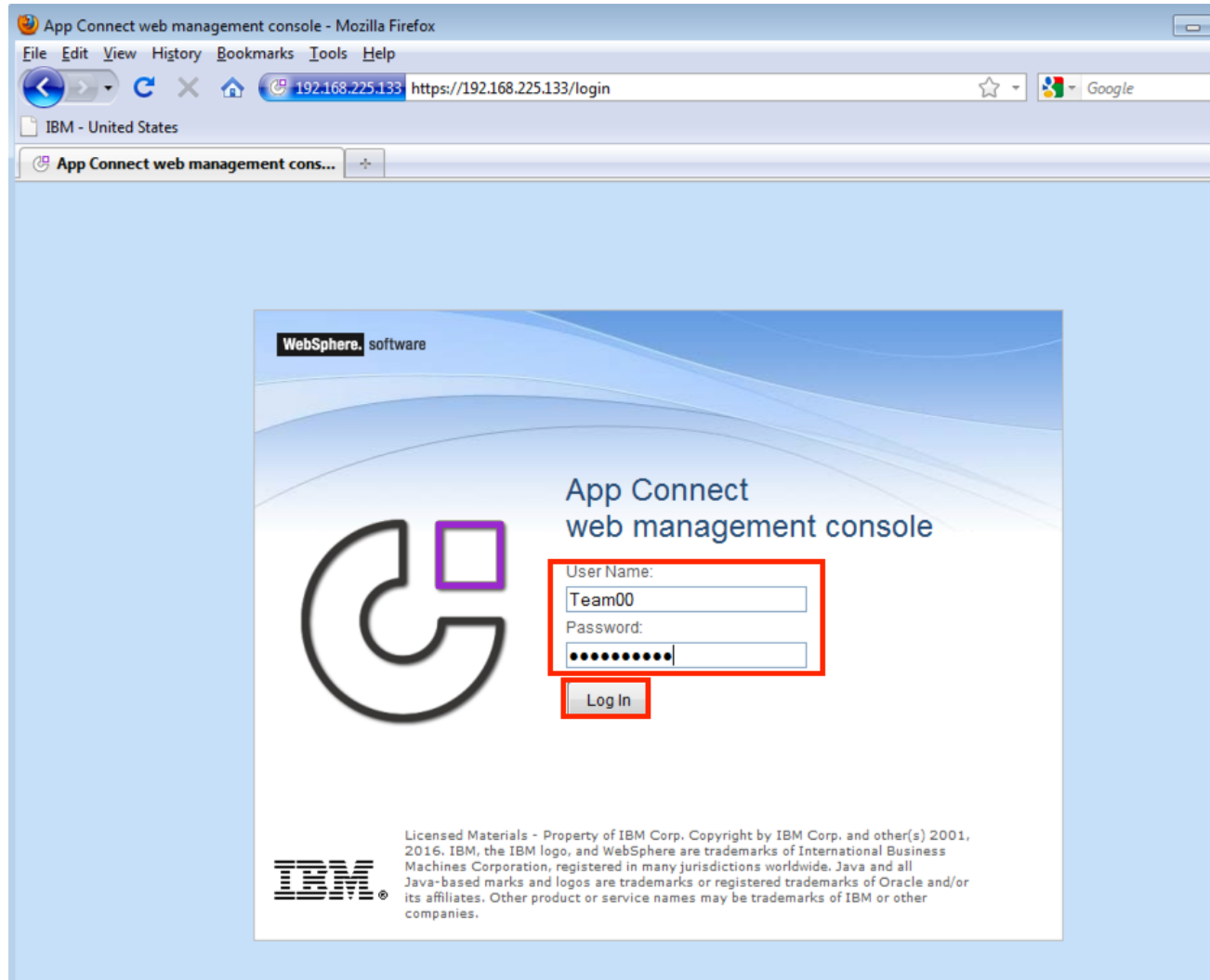
- __77. If the project publishes correctly, you should see the **Success** popup. Click **OK**.



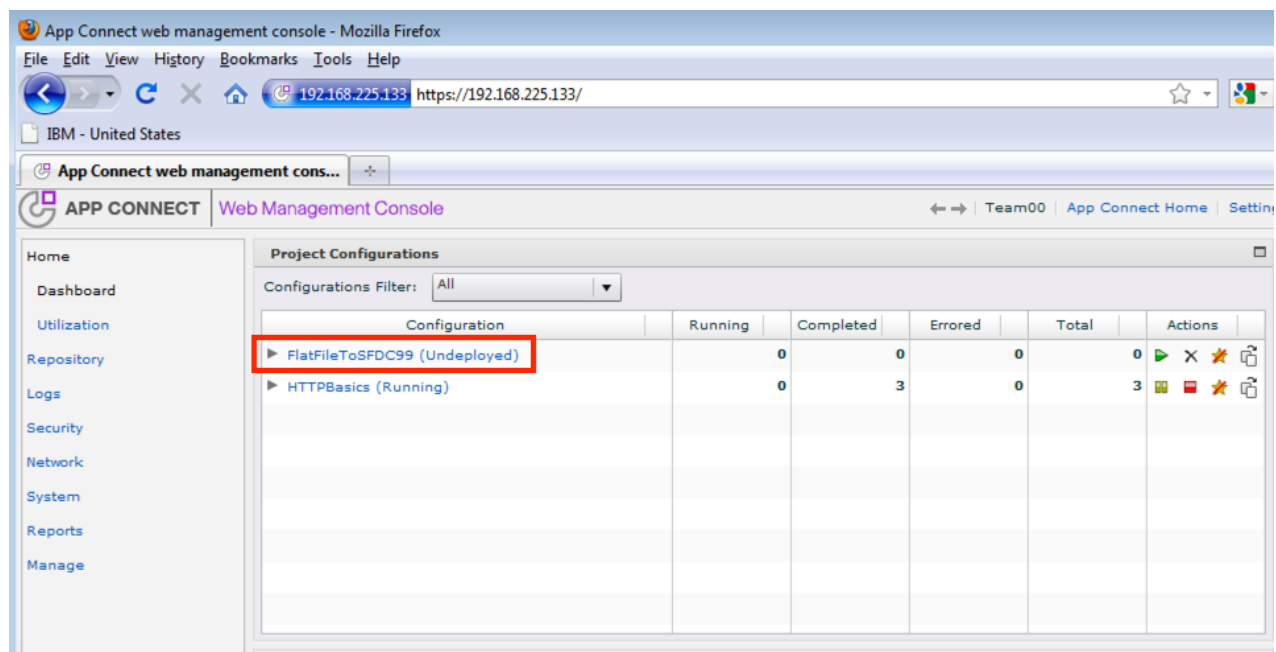
2.8 Web Management Console

2.8.1 Now it is time to Deploy the Project on the IBM App Connect Professional Web Management Console (WMC).

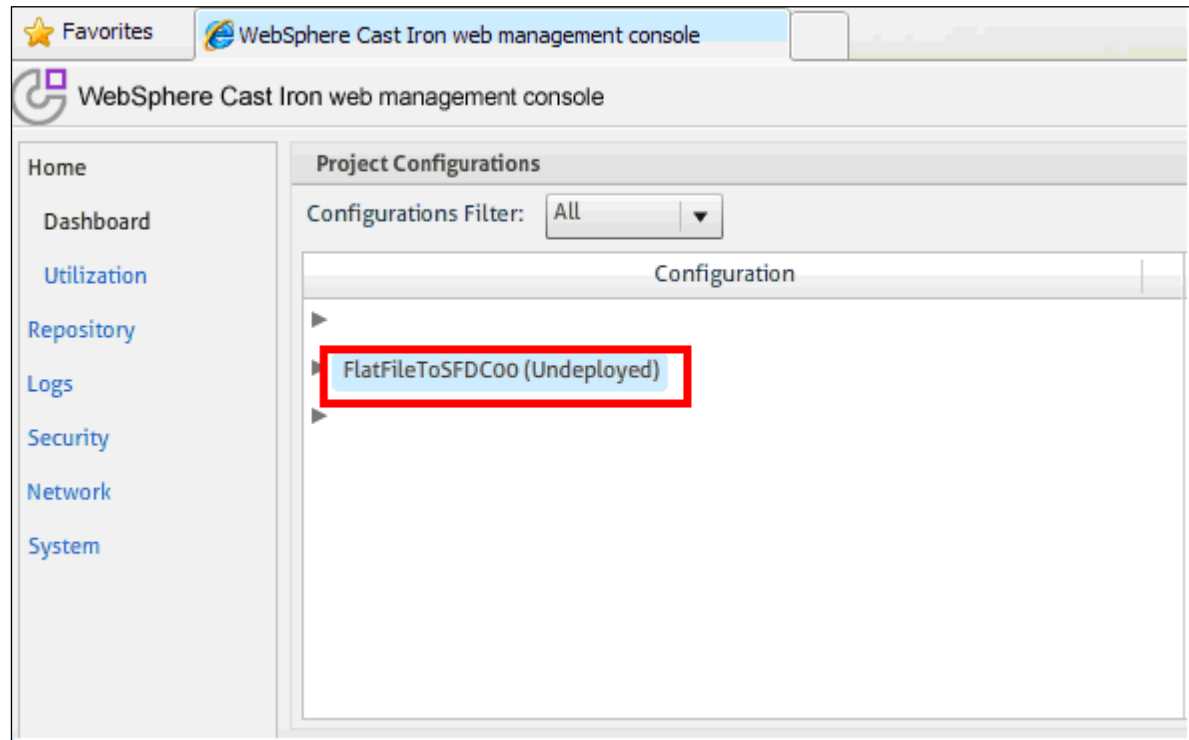
- ___1. Login in to the **WMC** using the URL and login credentials provided by the instructor. Click the **Log In** button.

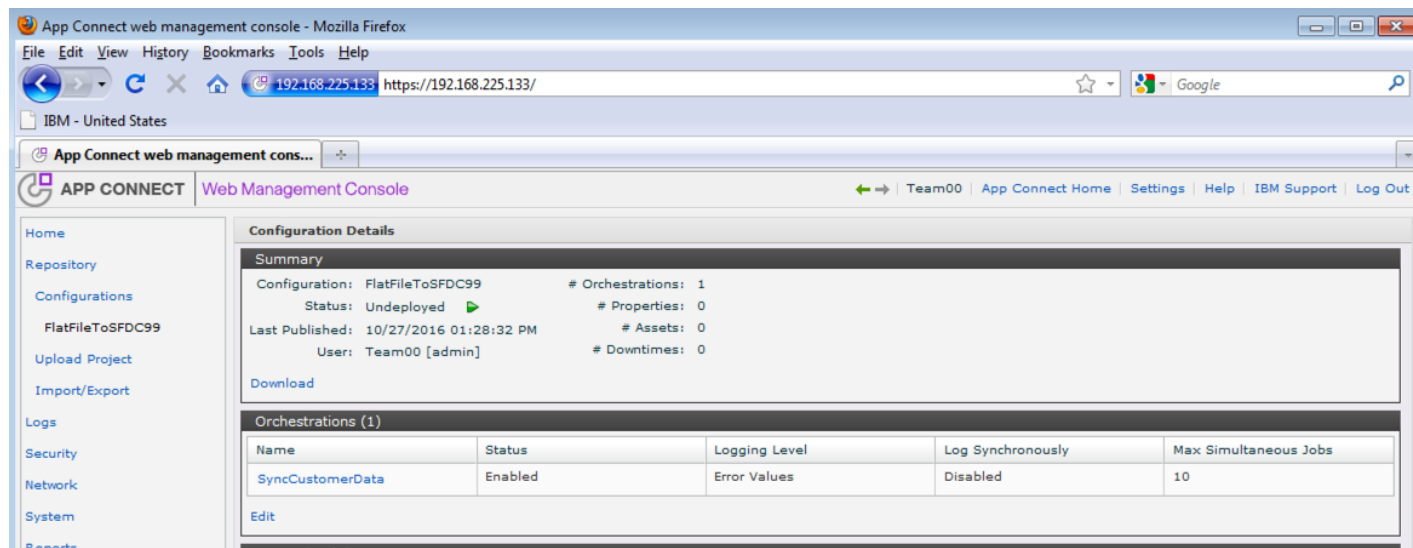


You should see your published project listed on the WMC dashboard.

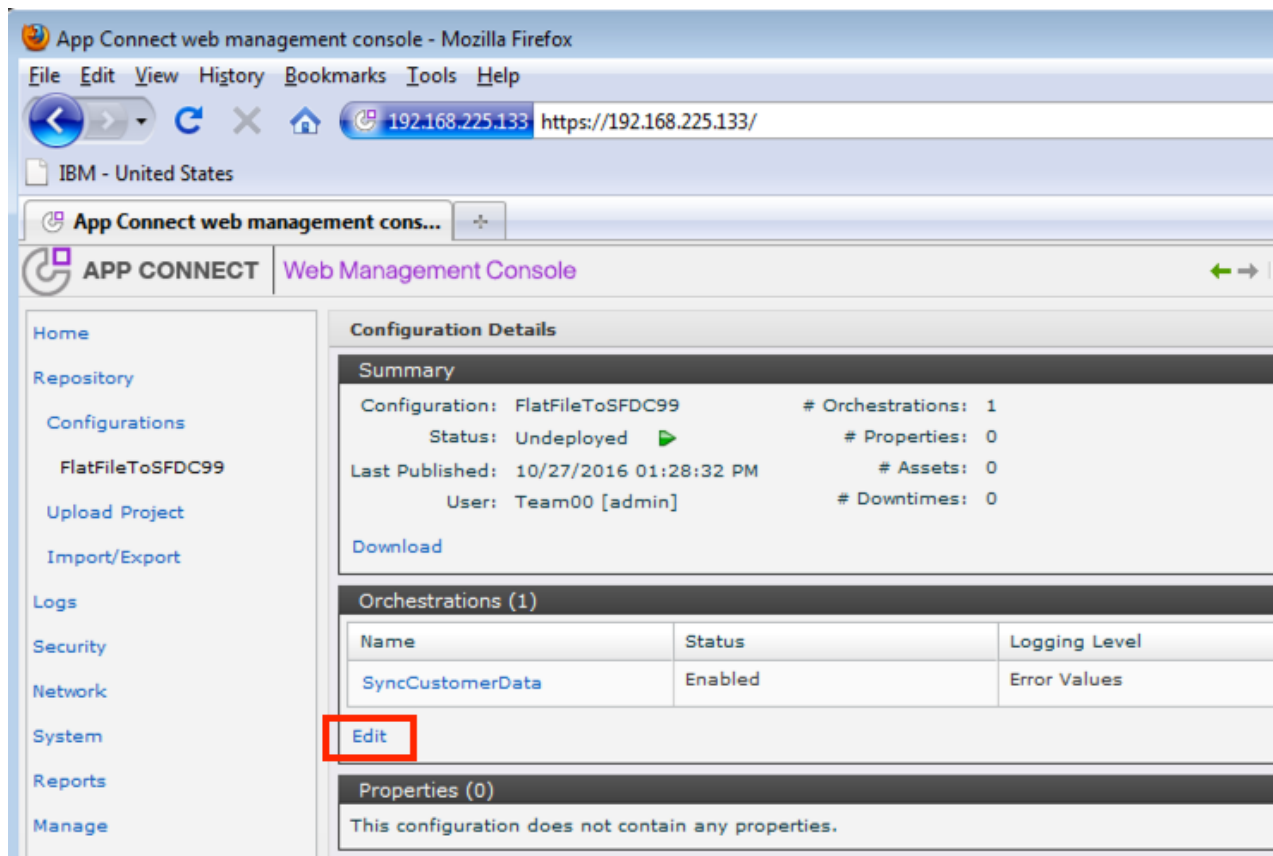


__2. Place the cursor on your project name and click to bring up the details.





3. Click the **Edit** button located under your Orchestration name **SyncCustomerData** to change the logging level.



- __4. Select **ALL** from the **Logging Level** drop down menu for your enabled Orchestration.

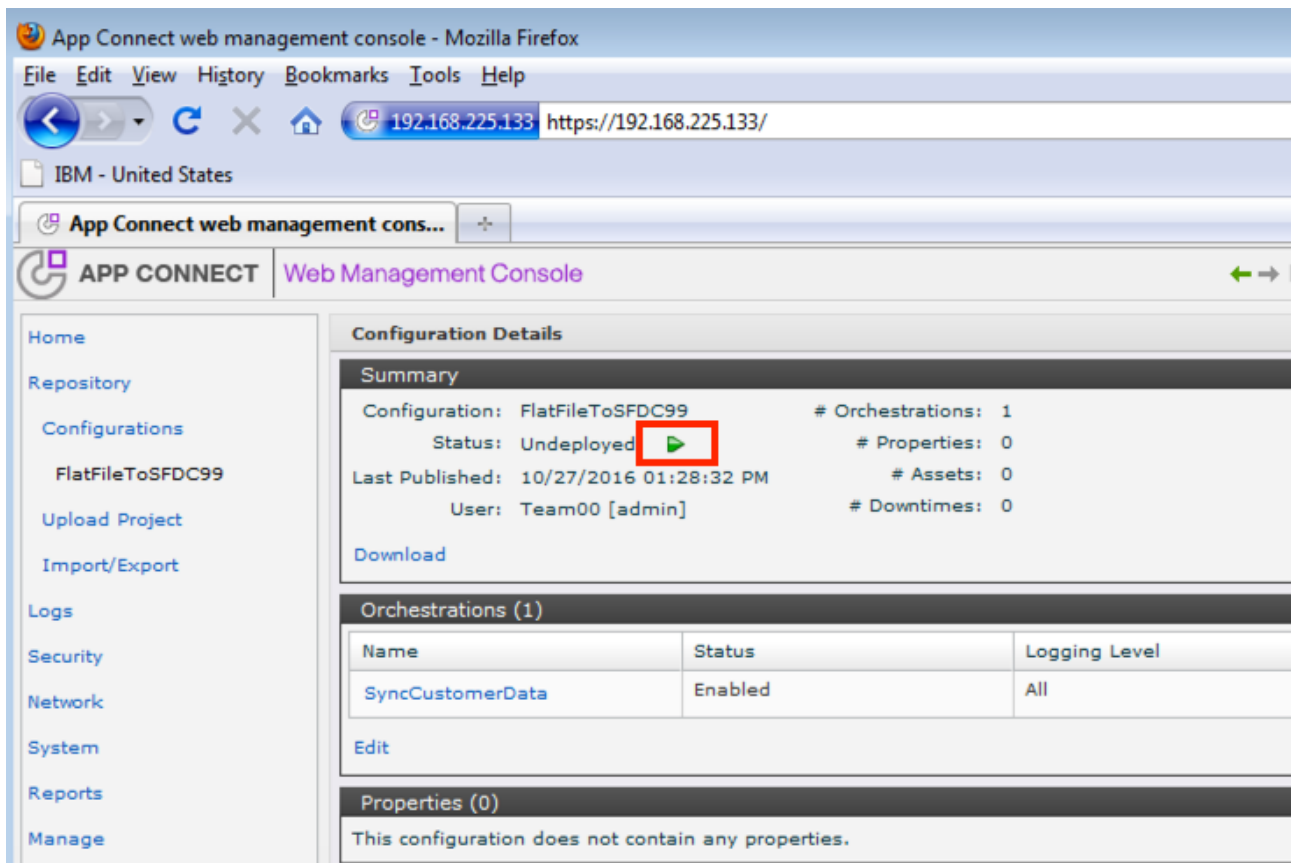
Edit Orchestration Settings				
Name	Enabled	Logging Level	Log Synchronously	Max Simultaneous Jobs
	☐	None	☐	☒ Unlimited
SyncCustomerData	☒	Error Values	☐	☐ Unlimited 10
		None		
		Initial Values		
		Initial and Error Values		
		Error Values		
		Inline		
		All		

- __5. Click the **Save** button located in the lower left hand corner of the screen to save the changes.

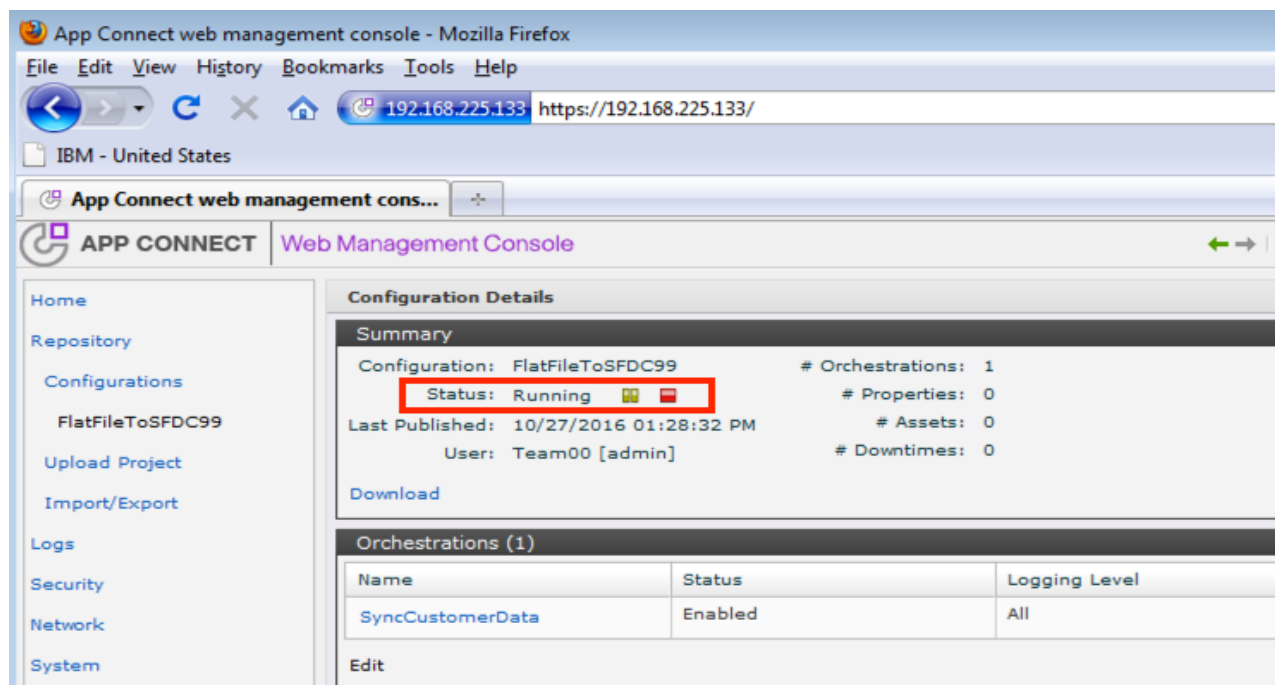
Edit Orchestration Settings				
Name	Enabled	Logging Level	Log Synchronously	Max Simultaneous Jobs
	☐	None	☐	☒ Unlimited
SyncCustomerData	☒	All	☐	☐ Unlimited 10

Save [Cancel](#) | [Help](#)

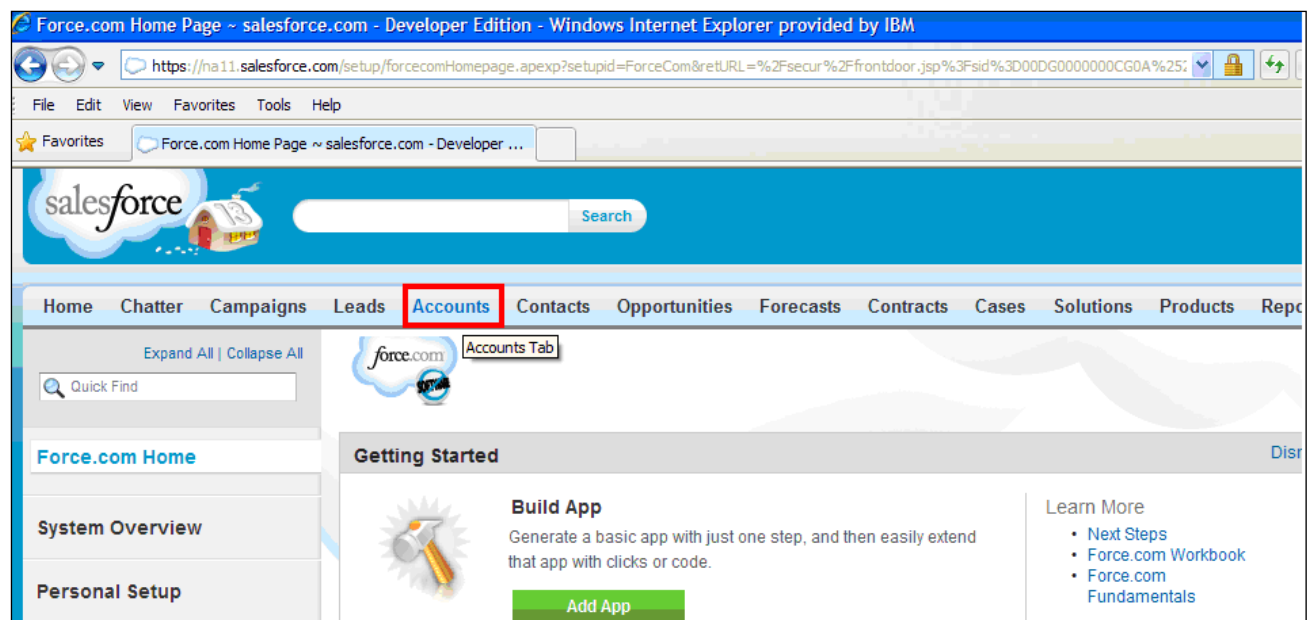
- __6. Click the green **Run Configuration** button to deploy the orchestration.



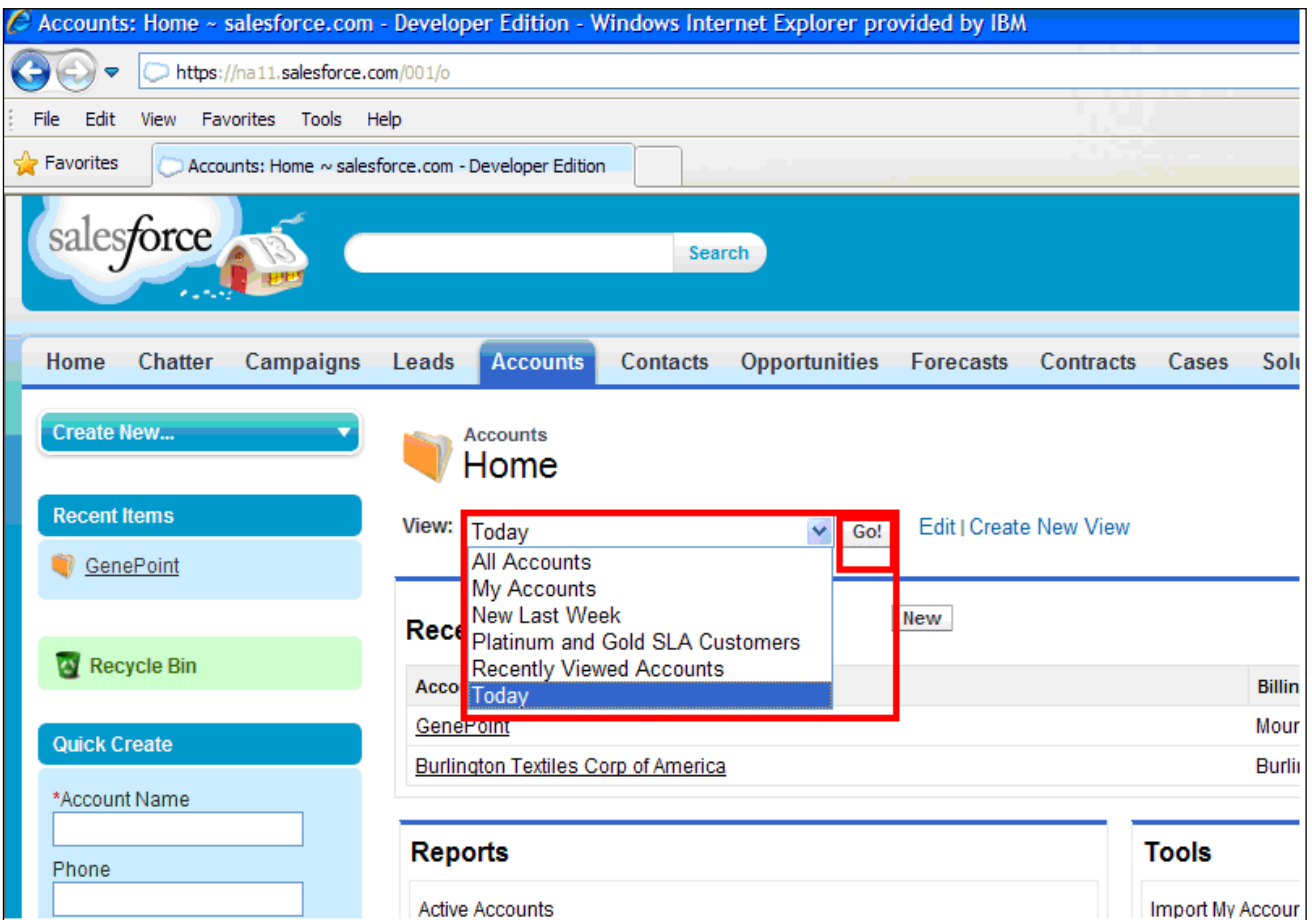
The orchestration will then show as being deployed and in a running state.



__7. Using a browser, login to your Salesforce.com account and click **Accounts.**



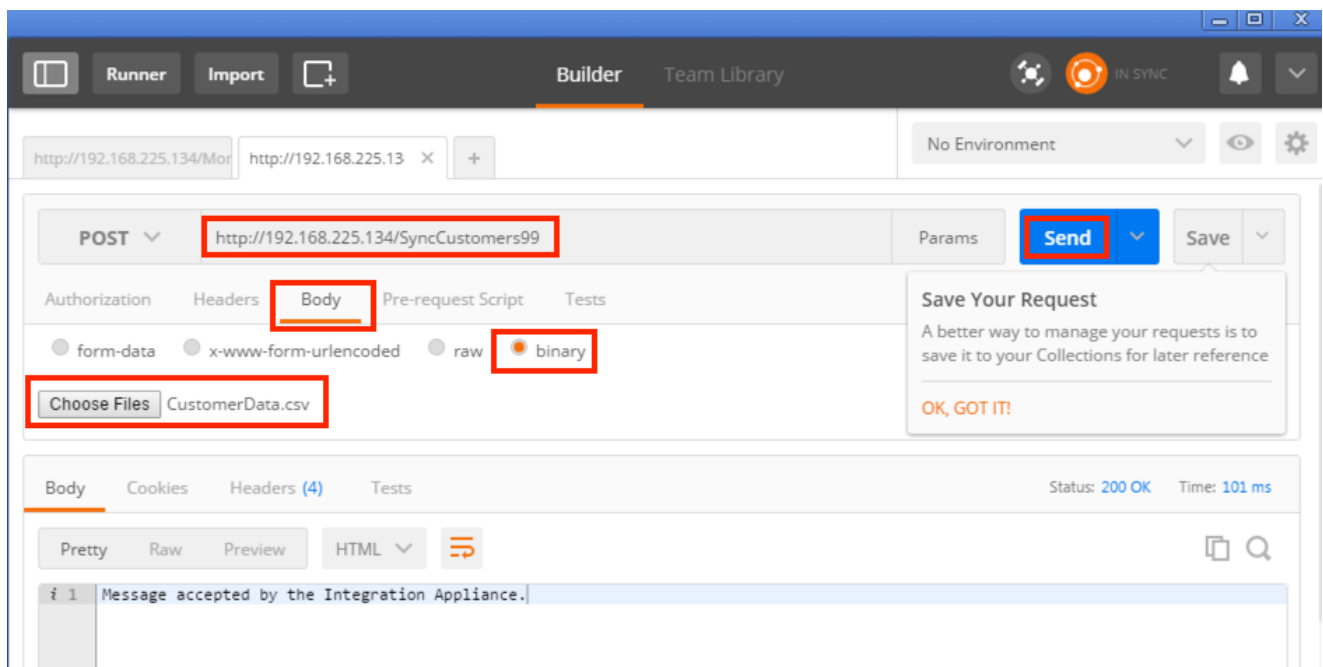
- __8. Select **Today** from the drop down menu and click **Go!**



Your list of accounts should not show any of the test records to be used, **Customer One**, **Customer Two**, **Customer Three** or **Customer Four** in the list.

Action	Account Name	Account Site	Billing State/Province	Phone
Edit Del	Burlington Textiles Cor...		NC	(336) 222-7000
Edit Del	Dickenson plc		KS	(785) 241-6200
Edit Del	Edge Communications		TX	(512) 757-6000
Edit Del	Express Logistics and...		OR	(503) 421-7800
Edit Del	GenePoint		CA	(650) 867-3450
Edit Del	Grand Hotels & Resort...		IL	(312) 596-1000
Edit Del	Pyramid Construction I...			(014) 427-4427
Edit Del	sForce		CA	(415) 901-7000
Edit Del	United Oil & Gas Corp.		NY	(212) 842-5500
Edit Del	United Oil & Gas, Sing...		Singapore	(650) 450-8810
Edit Del	United Oil & Gas, UK		UK	+44 191 4956203
Edit Del	University of Arizona		AZ	(520) 773-9050

- ___9. In order to test your Orchestration, open the **Postman** utility via the Desktop as in the previous lab.
- ___10. Enter the URL string containing the App Connect Professional DATA IP address provided by the instructor. Append the URL parameter **/SyncCustomersXX** that will be used to invoke the orchestration on the App Connect Professional appliance.
- ___11. Click the **ChooseFile** button and navigate to the **C:\Student\Lab Resources\Lab 2** folder to locate the **CustomerData.csv** document to post. Click **Open**. Then Click the send button to invoke the orchestration. If done right, you will get a message below indicating that the “Message was Accepted by the Integration Appliance”.



The CustomerData file contains four(4) records to be processed. Since the orchestration is screening the records as they are processed, only those records where the Custsite is not equal to 'East' will pass the validation. Only three (3) records will be created in your Salesforce.com instance.

Microsoft Excel - CustomerData.csv											
File Edit View Insert Format Tools Data Window Help											
A1 fx CustSite											
	A	B	C	D	E	F	G	H	I	J	K
1	CustSite	FirstName	LastName	Address1	Address2	City	State	Zipcode	Telephone		
2	Central	Customer	One	123	West Avenue	New Orleans	LA	70058	5045551212		
3	SouthEast	Customer	Two	3876	Norfolk Drive	Atlanta	GA	30328	7705551212		
4	West	Customer	Three	8873	Coast Highway	Mountain View	CA	90645	6505551212		
5	East	Customer	Four	1000	Broadway	New York	NY	10093	2125551212		
6											
7											
8											

At this point if your orchestration ran successfully, you should be able to go back to your Salesforce.com window and refresh your **Accounts Today** screen and you will see the three records that were created. Notice that the **Customer Four 'East'** record was not created.

The screenshot shows the Salesforce interface with the 'Accounts' tab selected. The 'Today' view is active, displaying a list of accounts. The table has columns for Action, Account Name, Account Site, Billing State/Province, and Phone. Three records are visible, highlighted with a red box:

Action	Account Name	Account Site	Billing State/Province	Phone
Edit Del +	Burlington Textiles Cor...		NC	(336) 222-7000
Edit Del +	Customer One	Central	LA	5045551212
Edit Del +	Customer Three	West	CA	6505551212
Edit Del +	Customer Two	SouthEast	GA	7705551212

This completes Lab #2

Lab 3 Single Database Table Operation into Salesforce.com

3.1 Overview

In this lab, you will read a DB2 database table record and create a new or update an existing Salesforce Account. Since an account originates in DB2, it is the “system of record”. The **Try** and **Log Message** activities are used to catch and log errors.

This exercise will work as a synchronization of records from DB2 into Salesforce.com.

3.2 Concepts

The version 3.0 Salesforce.com connector is built-in, so its usage is similar to that of the database connector. As with the database connector, Studio discovers the structure of connected objects. Logic activities can be used to interrogate input data and change processing paths depending on field values, or to capture and handle database errors.

3.3 Activities Used

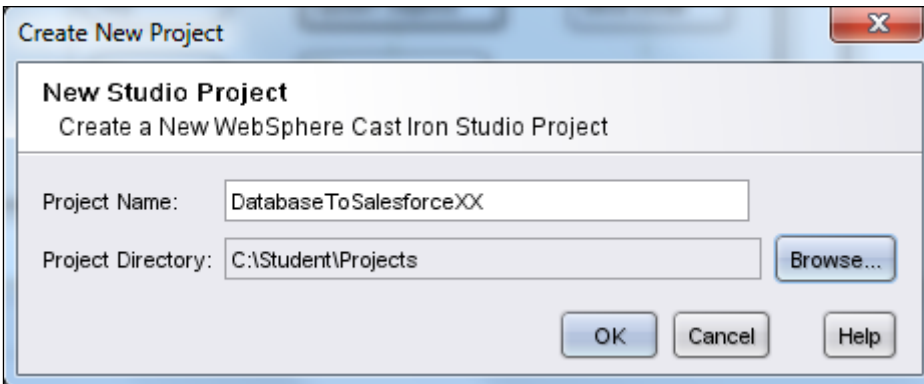
- Database Poll Table
- Logic For Each
- Salesforce.com Upsert Objects
- Email Send Email
- Logic Try
- Utilities Log Message

3.4 Document Keys

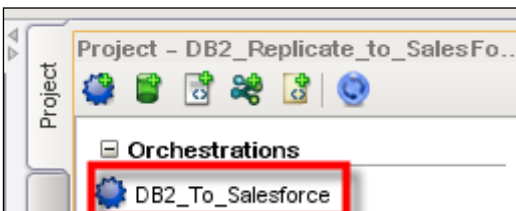
- Menu bar: Edit → Preferences
- Studio Artifact: **HTTP Receive Request**
- Entered Text: 'Hello Receive Request'
- Project Object Name: **SampleProject1**
- Context Menu Item: **Define Default Value**
- Activity Checklist Step: **Pick Endpoint**
- Orchestration Variable: **Account**
- Function Name: **Mathematical / Sum**

3.5 Load Database records into Salesforce

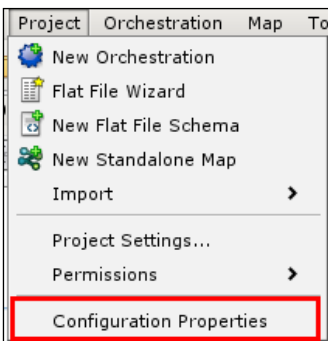
- __1. Create a new project [DatabaseToSalesforceXX](#). Where XX is your team number.



- __2. Define the Orchestration by renaming the default orchestration to [DB2_To_Salesforce](#). This can be done by Right Clicking the Orchestration and *select* → *Rename* or double clicking on the orchestration name.



- __3. Create configuration properties. This can be done by selecting *Project* → *Configuration Properties*



- __a. Add the following Configuration Properties (Refer to the '**Connection Parameters Spreadsheet**'):
- | | |
|----------------|---|
| 1. DB2DBName | SAMPLE |
| 2. DB2Server | XX.XX.XX.XX - Endpoint Server Address (DB2) |
| 3. DB2UserName | db2admin |
| 4. DB2UserPass | db2admin |
- *** (type = password) see highlighted in **RED**.

Configuration Properties

Creates configuration properties.

Name: DB2DBName Type: String Create

Name	Type	Value
------	------	-------

Delete

- __b. When all the parameters and values are added, select **OK**. Make sure you have added 'DB2UserPass' as the **Password** type.

Configuration Properties

Creates configuration properties.

Name: Type: String Create

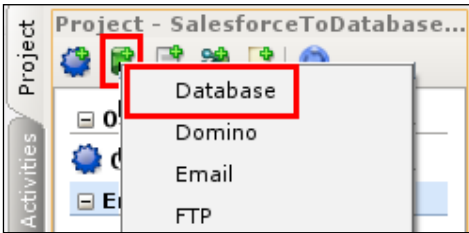
Name	Type	Value
DB2DBName	String	SAMPLE
DB2Server	String	192.168.225.128
DB2UserName	String	db2admin
DB2UserPass	Password	••••••••

Delete

Import

OK Cancel Help

__4. Create a Database DB2 endpoint.



- __a. On the right hand side of the screen, under the endpoint listing, Rename the endpoint by right clicking on the default endpoint name '**Database**' and rename it to '**DB2**'.
- __b. From the '**Database Type**' drop-down, select '**DB2 UDB**'.
- __c. Select the drop-down for the 4 fields corresponding to the configuration parameters entered in Step 3:

Under **Additional Parameters** set **PackageCollection** to string **NULLID**.

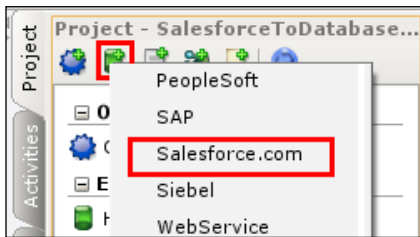
 A screenshot of the Database configuration window. The following fields are highlighted with red rectangles:

- Database Type:** A drop-down menu showing 'DB2 UDB'.
- Database Name:** A text field containing 'DB2DBNAME'.
- Network Location:** A section with 'Server' and 'Port' fields. The 'Server' field contains 'DB2SERVER' and the 'Port' field contains '50000'.
- Authentication:** A section with 'User Name' and 'Password' fields. The 'User Name' field contains 'DB2USERNAME' and the 'Password' field contains 'DB2USERPASS'.
- Additional Parameters:** A table with two columns: 'Parameter Name' and 'Parameter Value'. It contains two rows: 'MaxPooledStatements' with value '50', and 'PackageCollection' with value 'NULLID'.

- __d. Test connection on bottom of page.



- __5. Create a Salesforce.com endpoint (using information from 'Connection Parameters spreadsheet').



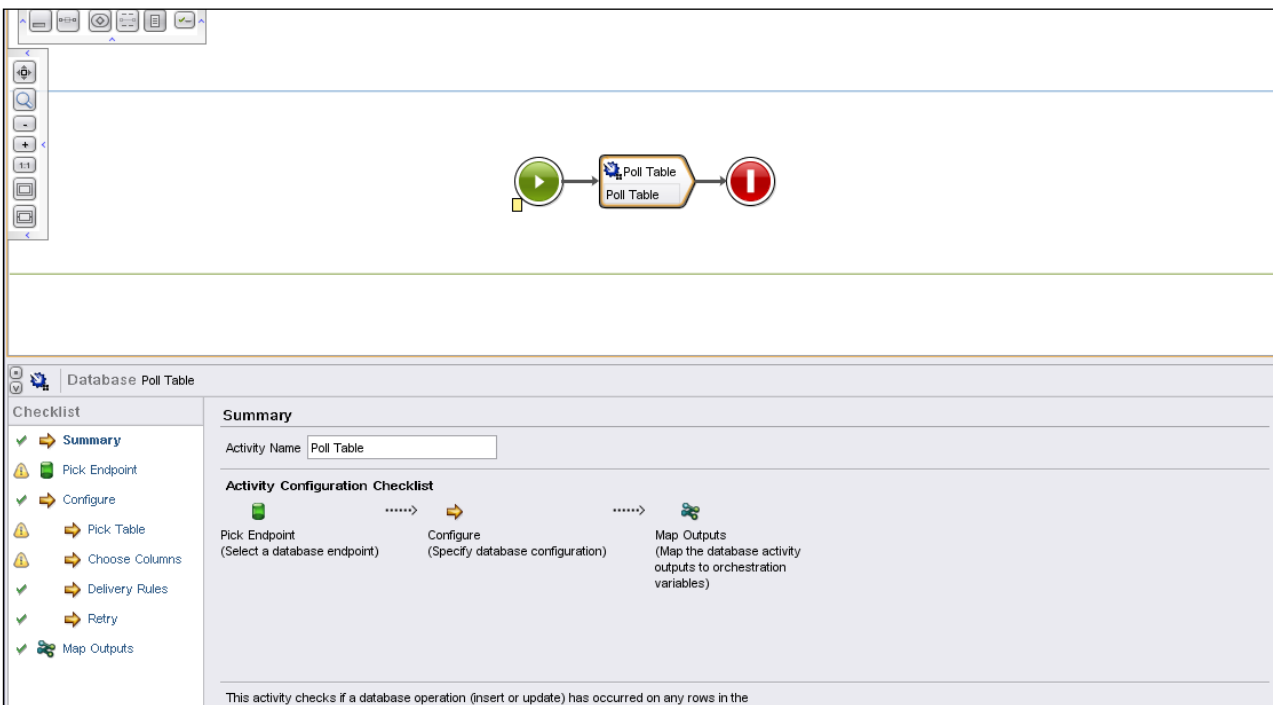
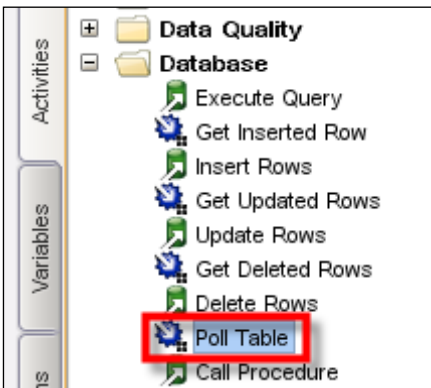
- __a. Enter your Salesforce username.
- __b. The Password field is constructed as: <password><security token>

A screenshot of the 'Salesforce.com Customer Login' dialog box. The 'User Name' field contains 'stan.getz@oldhome.org' and the 'Password' field is masked with dots. The 'Login Options' section shows 'Login normally' selected. The 'Login URL' field contains 'https://www.salesforce.com/services/Soap/u/18.0'.

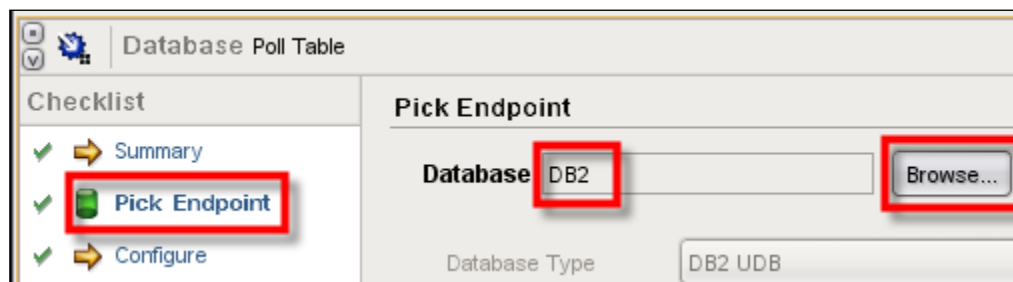
- __6. Test your connection.



- ___7. Add a **Database Poll Table** activity to the orchestration by dragging and dropping the activity on to the canvas, then click the activity and the check list will become active on the bottom part of the screen.

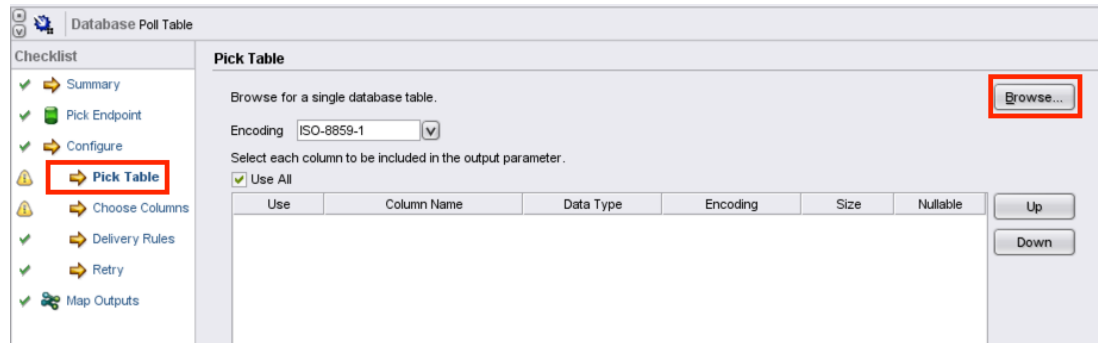


- ___a. Working your way down the checklist, start with **Pick Endpoint**: Select the **DB2** endpoint.

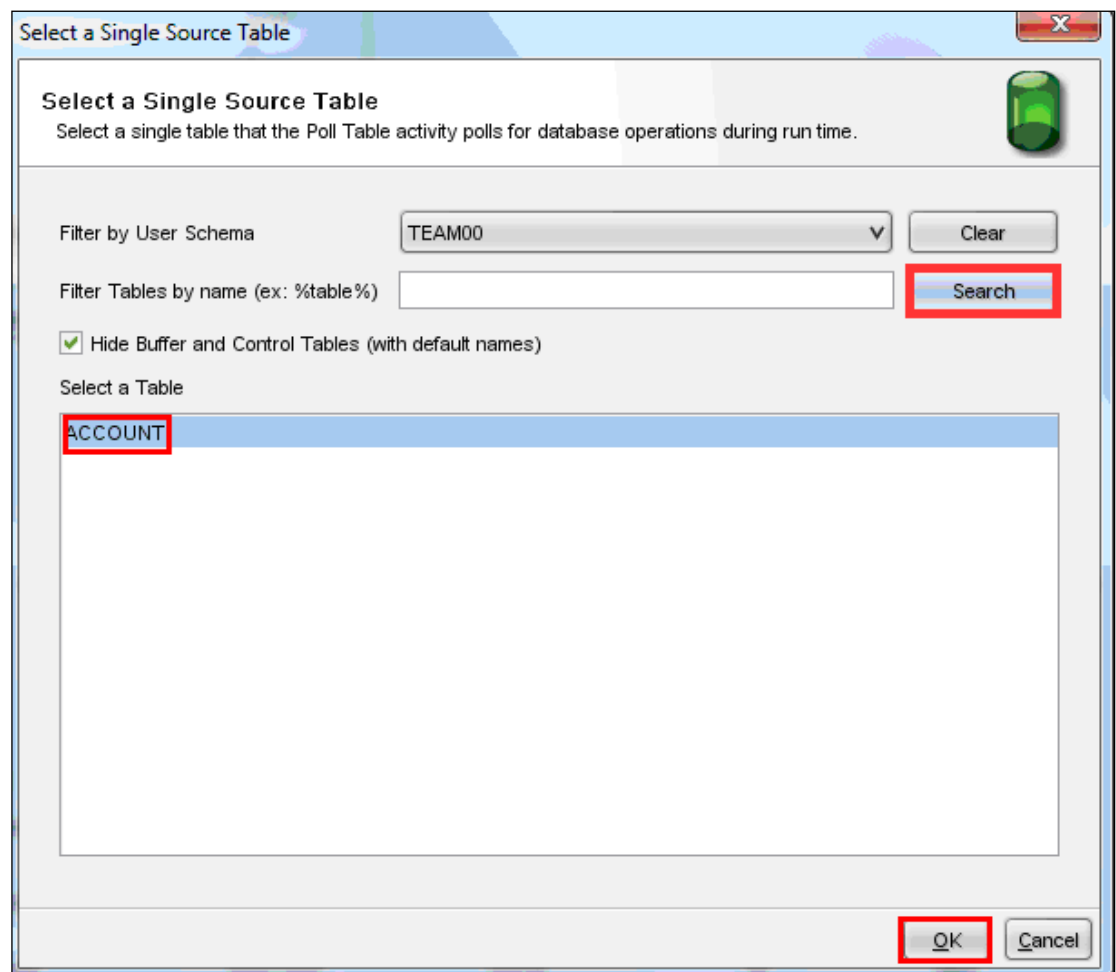


- ___b. **Pick Table**:

__i. Click Browse.



__ii. From the drop down, select the **Team00** schema. Click the **Search** button and the connector will populate the list of all the tables that are available for you. Once the requisite **Account** table is selected, click **OK**.



- __iii. Deselect '**Use All**' and select fields from the table below.

DB2 Activity Field
ACCOUNT_ID
ACCOUNT_NAME
ACCOUNT_TYPE
ACCOUNT_NUMBER
BILLING_STREET
BILLING_CITY
BILLING_STATE
BILLING_ZIP_CODE
BILLING_COUNTRY
PHONE
INDUSTRY
OWNERSHIP
ACCOUNT_RATING
CUSTOMER_PRIORITY
SLA
ACTIVE
NUMBER_OF_LOCATIONS
UPSELL_OPPORTUNITY
SLA_EXPIRATION_DATE
FLAG

- __c. **Choose Columns:** Ensure the **Sequence Column** and **Flag Column** properties are set up on the drop downs as per the screenshot below

Database Poll Table

Checklist

- ✓ Summary
- ✓ Pick Endpoint
- ✓ Configure
- ✓ Pick Table
- ✓ Choose Columns

Choose Columns

Select Sequence and Flag Columns

Sequence Column: ACCOUNT_ID

Flag Column: FLAG

__d. **Delivery Rules:** Ensure the polling parameters are indicated as such:

Database Poll Table

Checklist

- ✓ Summary
- ✓ Pick Endpoint
- ✓ Configure
- ✓ Pick Table
- ✓ Choose Columns
- ✓ **Delivery Rules**
- ✓ Retry
- ✓ Map Outputs

Delivery Rules

- 1) Poll for changes every seconds
- 2) Fetch up to rows per poll.
- 3) ☒ Batch rows into each message.
- 4) Deliver messages
- 5) ☒ Delete rows after they have been processed.

__8. Click **Map Outputs**. Click the **Copy** button to copy the rows returned from the DB to a variable. Rename the output variable from **rows** to **SFAccounts**. You are now finished with configured the Poll Table activity.

Database Poll Table

Checklist

- ✓ Summary
- ✓ Pick Endpoint
- ✓ Configure
- ✓ Pick Table
- ✓ Choose Columns
- ✓ Delivery Rules
- ✓ **Map Outputs**

Map Outputs

Design Test

From Activity

Copy

To Orchestration

Select Outputs...

SFAccounts

rows

row

ACCOUNT_ID

ACCOUNT_NAME

ACCOUNT_TYPE

ACCOUNT_NUMBER

BILLING_STREET

BILLING_CITY

BILLING_STATE

BILLING_ZIP_CODE

BILLING_COUNTRY

PHONE

INDUSTRY

OWNERSHIP

ACCOUNT_RATING

CUSTOMER_PRIORITY

SLA

ACTIVE

NUMBER_OF_LOCATIONS

UPSELL_OPPORTUNITY

SLA_EXPIRATION_DATE

FLAG

ACCOUNT_ID

ACCOUNT_NAME

ACCOUNT_TYPE

ACCOUNT_NUMBER

BILLING_STREET

BILLING_CITY

BILLING_STATE

BILLING_ZIP_CODE

BILLING_COUNTRY

PHONE

INDUSTRY

OWNERSHIP

ACCOUNT_RATING

CUSTOMER_PRIORITY

SLA

ACTIVE

NUMBER_OF_LOCATIONS

UPSELL_OPPORTUNITY

SLA_EXPIRATION_DATE

FLAG

- ___9. Drag and drop the **For Each** activity on to the canvas. This will tell the orchestration to iterate a loop that will go through each of the values returned from the database. Configure the activity as such:

Logic For Each For Each Account

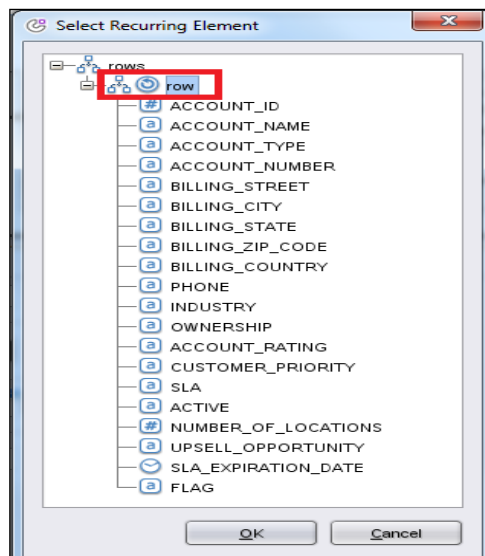
Activity Name: For Each Account

Variable that Contains the Recurring Element to Loop Through: SFAccounts

Recurring Element in Variable to Loop Through: /rows/row

Single Element Returned by the Loop: LoopAccount

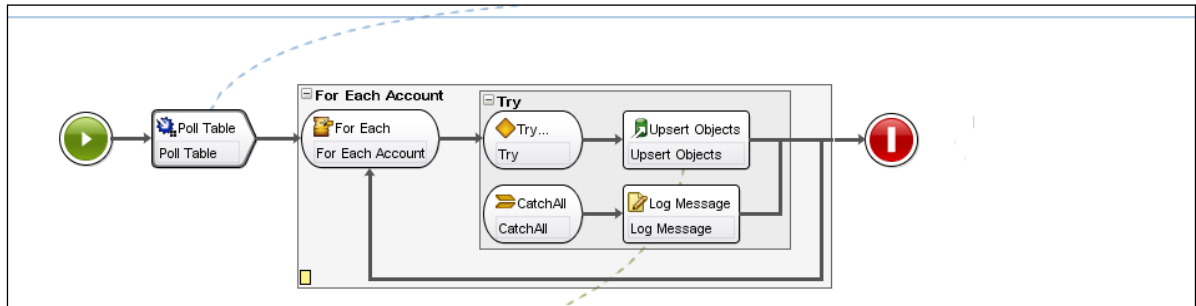
- ___a. **Variable Name (containing recurring element to loop through):** From the dropdown, select the variable name of the data set you want to iterate through. In this case, it is the variable **SFAccounts**.
- ___b. **Element Name:** Click the dialogue button on the right hand side to select the appropriate element on the data set. With the **For Each** function, this will always be a repeating variable that has the  symbol. In our case, it's going to be the **row** variable. Select **row** and then click 'Ok'.



- ___c. **Variable Name (containing single element contained by group):** This will be the variable that will be created that represents each individual occurrence of **row**. Name this variable **LoopAccount**.

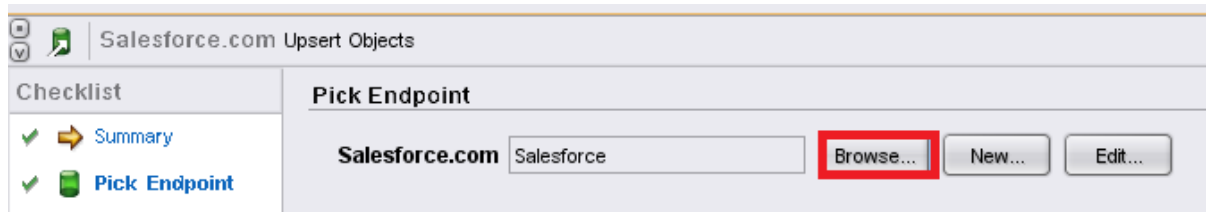
- __10. We will now create the process that will execute once for each row coming in from the input. All of the subsequent activities will be dropped into the **For Each** box. Drag and drop the following activities from the **Activities** tab as depicted in the picture below. We will configure each subsequent activity afterwards.

- __a. **Try..Catch**
- __b. **Salesforce.com -> Upsert Objects** (next to the Try..Catch Box)
- __c. **Log Message** (next to the **Catch All** Block)



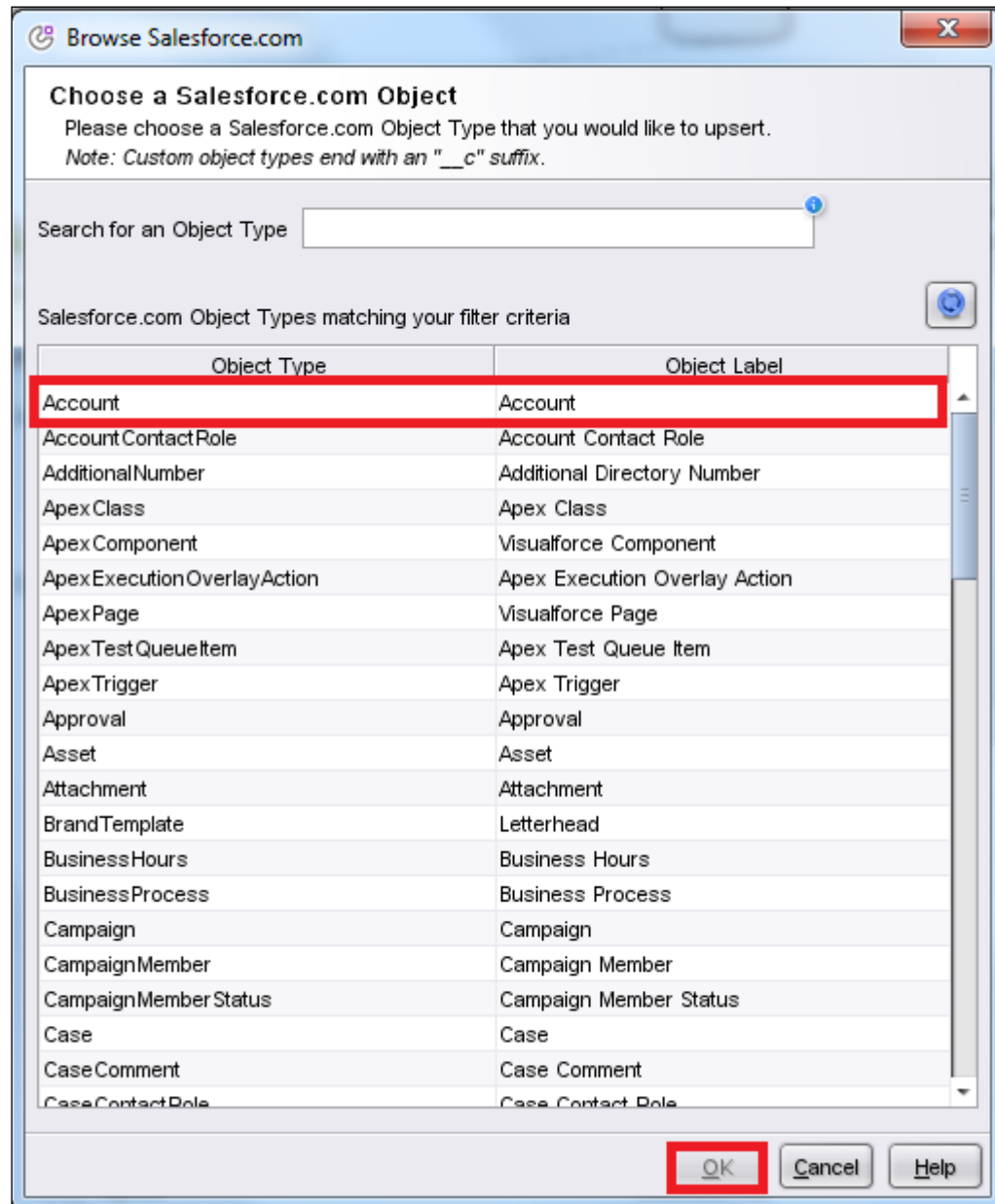
- __11. The next Activity to Configure is the Salesforce.com **Upsert Objects**. Click the activity on the orchestration canvas.

- __a. For the **Pick Endpoint** step in the checklist, click **Browse** and select the Salesforce endpoint.



__b. **Configure.**

__i. Click the **Browse** button to connect to Salesforce and select the appropriate object. For this lab, we will be using the **Account** object. Select **Account** and **OK**.



- __ii. Select the proper **External ID** to be used to drive the functionality around the Upsert. This custom field represents a key value that will tie the record in Salesforce with the data in your on-premise DB2 table. If the record exists, it will update the data, if it does not, it will insert a new record.



If you do not have a custom field already setup for the External ID, see Appendix B

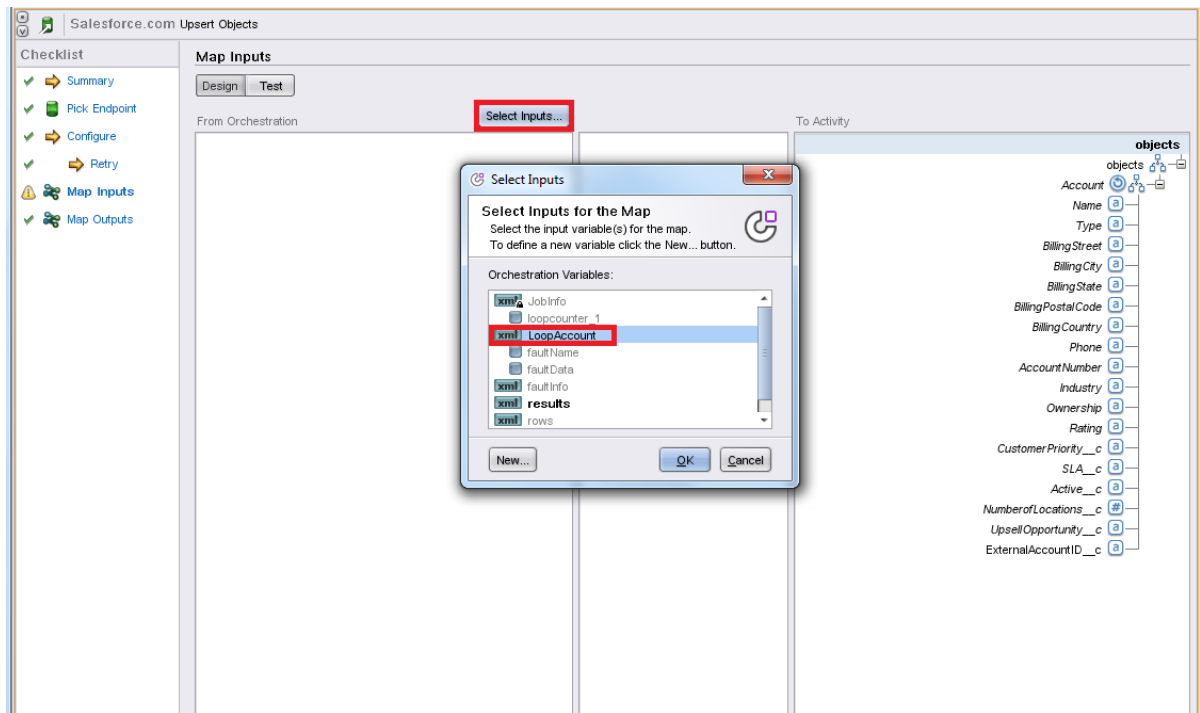
- __iii. Ensure the tick box at the top of the field select screen is not selected. This will deselect all fields. Proceed to the next step to select the fields to be used.

Name	Label	Type
☒ Name	Account Name	Name
☒ Type	Account Type	Picklist
☒ ParentId	Parent Account ID	Lookup(Account)(Account)
☒ BillingStreet	Billing Street	Text Area(255)
☒ BillingCity	Billing City	Text(40)
☒ BillingState	Billing State/Province	Text(20)
☒ BillingPostalCode	Billing Zip/Postal Code	Text(20)
☒ BillingCountry	Billing Country	Text(40)
☒ ShippingStreet	Shipping Street	Text Area(255)
☒ ShippingCity	Shipping City	Text(40)
☒ ShippingState	Shipping State/Province	Text(20)
☒ ShippingPostalCode	Shipping Zip/Postal Code	Text(20)
☒ ShippingCountry	Shipping Country	Text(40)
☒ Phone	Account Phone	Phone
☒ Fax	Account Fax	Phone
☒ AccountNumber	Account Number	Text(40)
☒ Website	Website	URL(255)
☒ Sic	SIC Code	Text(20)
☒ Industry	Industry	Picklist
☒ AnnualRevenue	Annual Revenue	Currency(18, 0)
☒ NumberOfEmployees	Employees	Number(8, 0)
☒ Ownership	Ownership	Picklist
☒ TickerSymbol	Ticker Symbol	Text(20)
☒ Description	Account Description	Long Text Area(32000)
☒ Rating	Account Rating	Picklist
☒ Site	Account Site	Text(80)
☒ OwnerId	Owner ID	Lookup(User)(User)

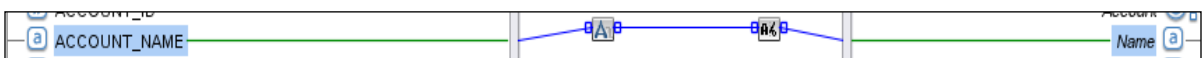
- ___iv. Select the following fields to be used in the **Account** object. Simply tick the check box in front of the field name to enable it.

Salesforce.com Fields
AccountNumber
Name
Type
BillingStreet
BillingCity
BillingState
BillingPostalCode
BillingCountry
Phone
Industry
Ownership
Rating
CustomerPriority__c
SLA__c
Active__c
NumberOfLocations__c
UpsellOpportunity__c
SLAExpirationDate__c
ExternalAccountID__c

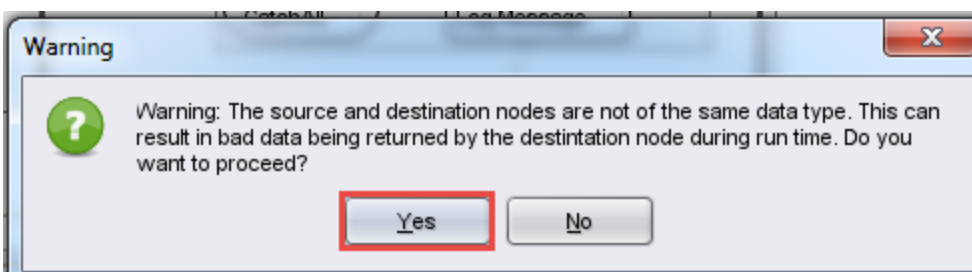
- ___c. **Map Inputs** Map the fields from the left side (input) to the right side (output). Use the mapping guide chart below to assist.



Use the functions tab to apply any transformation rules. Drag and drop the **Upper Case** and **Trim** functions onto the center pane and drag and drop the inputs on to the function. When properly mapped, account name looks as such (be sure to right click and apply function graph when done).

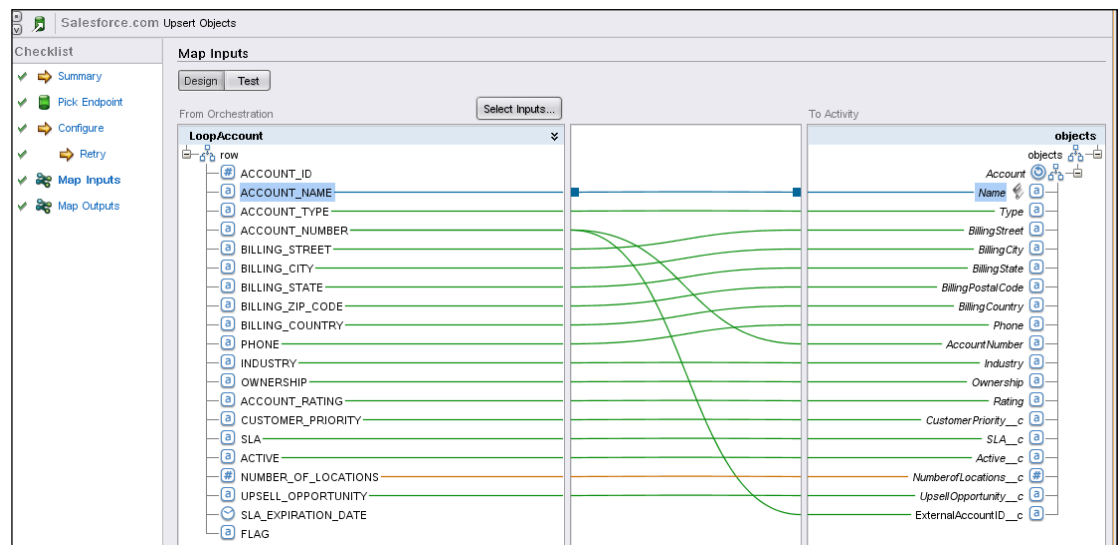


You may receive a warning regarding mismatched data types when mapping numeric fields. For this exercise you may disregard this warning.



A	B	C
Input Field	Output Field	Transformation Rules
ACCOUNT_ID	n/a	Not Mapped
ACCOUNT_NAME	Name	Uppercase, Trimmed
ACCOUNT_TYPE	Type	Direct Mapping, no transformation
ACCOUNT_NUMBER	AccountNumber and ExternalAccountID__c	Direct Mapping, no transformation
BILLING_STREET	BillingStreet	Direct Mapping, no transformation
BILLING_CITY	BillingCity	Direct Mapping, no transformation
BILLING_STATE	BillingState	Direct Mapping, no transformation
BILLING_ZIP_CODE	BillingPostalCode	Direct Mapping, no transformation
BILLING_COUNTRY	BillingCountry	Direct Mapping, no transformation
PHONE	Phone	Direct Mapping, no transformation
INDUSTRY	Industry	Direct Mapping, no transformation
OWNERSHIP	Ownership	Direct Mapping, no transformation
ACCOUNT_RATING	Rating	Direct Mapping, no transformation
CUSTOMER_PRIORITY	CustomerPriority__c	Direct Mapping, no transformation
SLA	SLA__c	Direct Mapping, no transformation
ACTIVE	Active__c	Direct Mapping, no transformation
NUMBER_OF_LOCATIONS	NumberOfLocations__c	Direct Mapping, no transformation
UPSELL OPPORTUNITY	UpsellOpportunity__c	Direct Mapping, no transformation
SLA_EXPIRATION_DATE	n/a	Not Mapped
FLAG	n/a	Not Mapped

__i. When complete, the mapping grid will look as such:

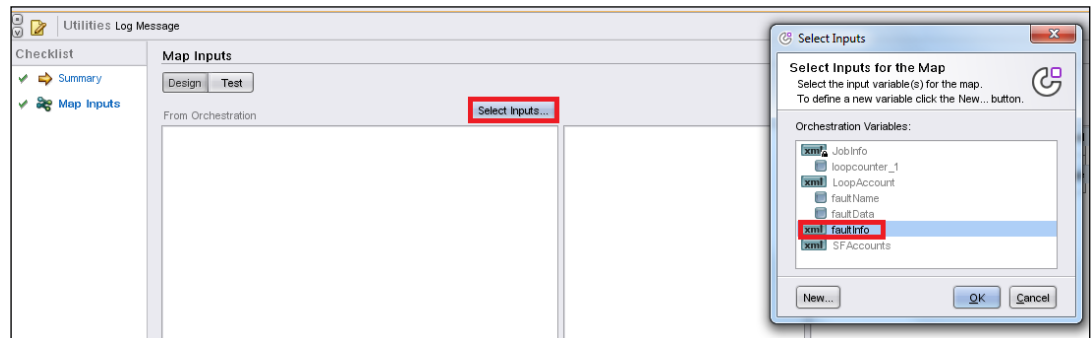


__12. Configure the **Log Message** activity. This Activity is in the **Catch All** block of the **Try..Catch** function. Whenever there is a failure to insert the record into Salesforce.com, the API will provide an error message. We will map the requisite information to populate the WMC with some customized messaging to assist with troubleshooting.

__13. Move on to the **Map Inputs** step

__a. **Map Inputs.**

- __i. By default, when a **Try..Catch** is placed onto an orchestration, it automatically creates a variable called **faultInfo**. This variable contains all the data about a given fault. Go ahead and add that variable to the screen via the **Select Inputs** button. Click **OK**.

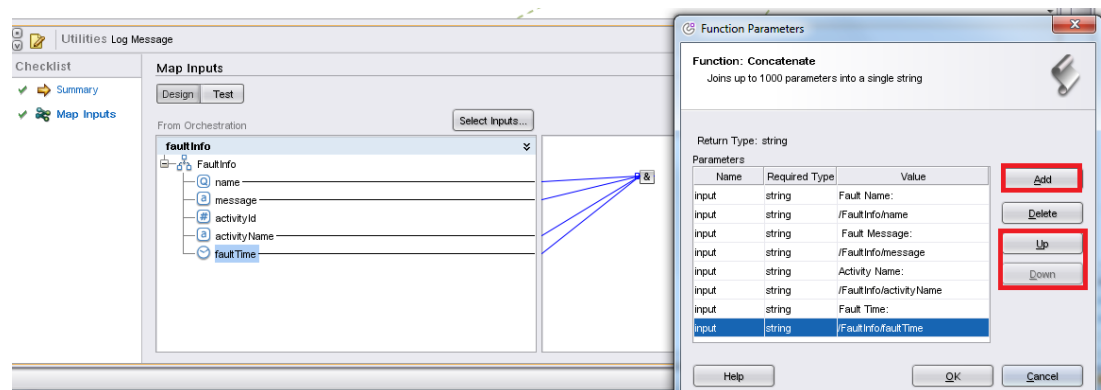


- __ii. Map the fields per the mapping guide below

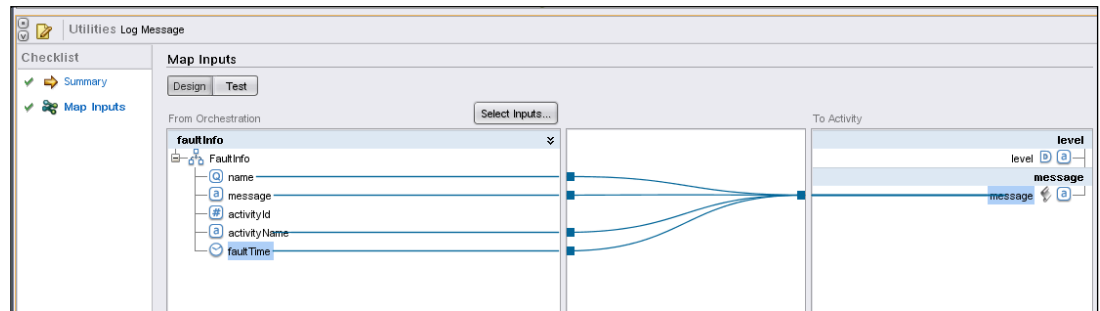
Input Field	Output Field	Transformation Rules
Fault Name	message	concatenated
Message	message	concatenated
activityId	message	not mapped
activityName	n/a	concatenated
faultTime	message	concatenated
	level	set to default "Warning"



Use literal values on the concatenate to separate the fields via the **Add** button on the concatenate function dialogue. Use the **Up** and **Down** buttons to adjust the order of the fields that are being concatenated.



- __iii. When complete, the activity should look as such:



- __14. Deploy the project to your appliance. This done via **File → Publish** or using the  icon from the top tool bar. Use the credentials for the previous lab.

- __15. Log into the Web Management Console and then find your project on the dashboard. Use the **Configurations Filter** to only show your projects if you like. To start the orchestration, click the green play button.

Configuration	Running	Completed	Errored	Total	Actions
DatabaseToSalesforceXX (Undeployed)	0	0	0	0	  

- __16. To run the orchestration, you need to insert some rows into the account table. You can do so by executing the following script from your VM.

- __a. Go to Start -> Command Prompt. From the command prompt type in **db2cmd**. This will start the DB2 command line interface.

- __b. Next change directories to **C:\Student\Lab Resources\Lab 3**. The screenshot below has the commands to do that
- __c. Execute the **InsertAccountRecords.bat** batch file. It will start the process to insert the records. You will see a prompt asking for your team id. All students will use the **00** team. Enter **00** and then click **Enter**.
- __d. The script will run and then insert the rows.

```

C:\DB2 CLP - DB2COPY1
Inserting Records for Team 00
Please Wait

    Database Connection Information

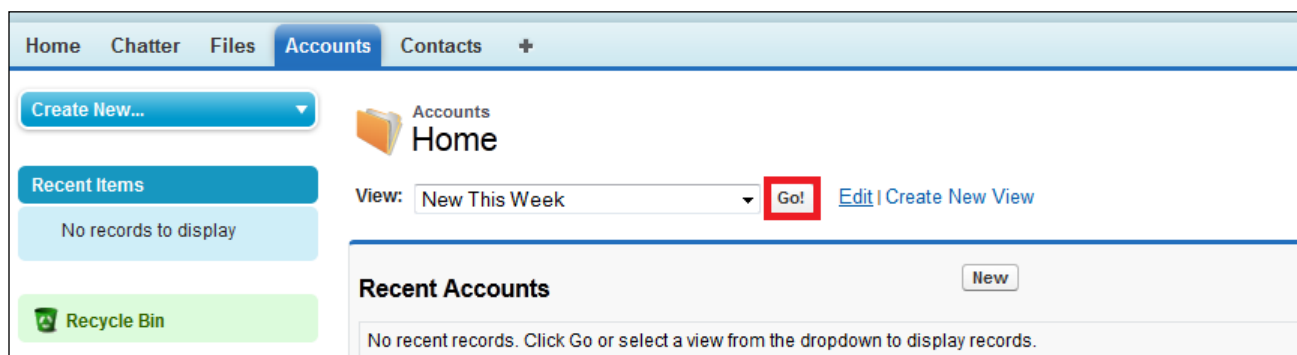
Database server           = DB2/NT 9.5.0
SQL authorization ID     = ADMINIST...
Local database alias     = SAMPLE

SQL0100W No row was found for FETCH, UPDATE or DELETE; or the result of a
query is an empty table. SQLSTATE=02000

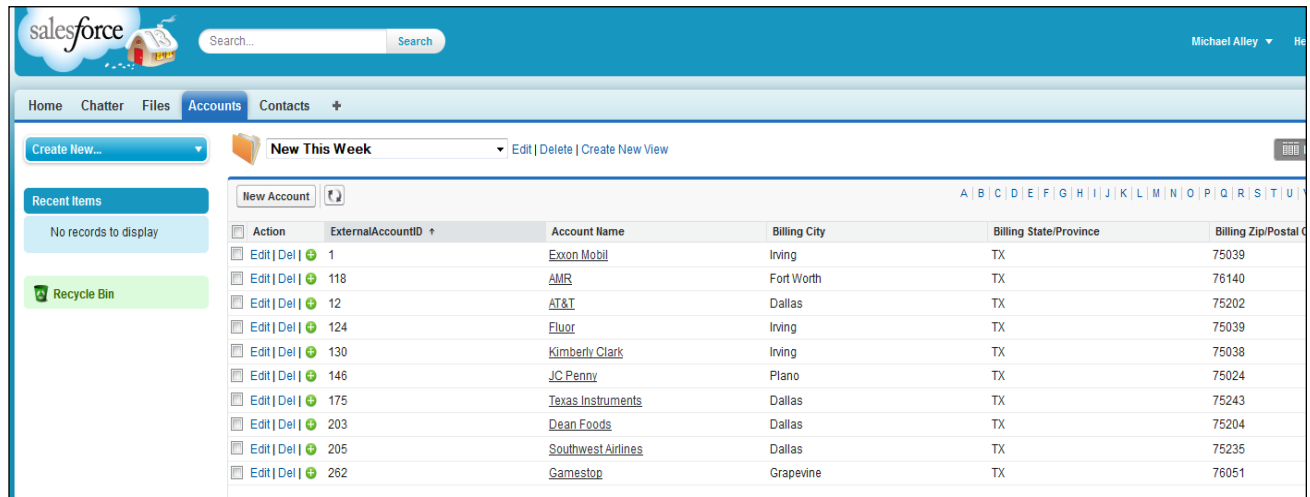
DB20000I The SQL command completed successfully.
DB20000I The SQL command completed successfully.
DB20000I The SQL command completed successfully.
DB20000I The SQL command completed successfully.
DB20000I The SQL command completed successfully.
DB20000I The SQL command completed successfully.
DB20000I The SQL command completed successfully.
DB20000I The SQL command completed successfully.
DB20000I The SQL command completed successfully.
DB20000I The SQL command completed successfully.
results of insert are found in the file DB2_Account_Records_Insert_Results.txt

```

- ___17. To see if everything ran properly, you will need to log into Salesforce.com and then go to the **Accounts** object. Click the new view called **New This Week**.



__18. If everything has run properly, you will see 10 new accounts.



The screenshot shows the Salesforce interface with the 'Accounts' tab selected. A table titled 'New This Week' displays 10 newly created accounts. The table has columns for Action, ExternalAccountID, Account Name, Billing City, Billing State/Province, and Billing Zip/Postal Code. Each row includes an 'Edit | Del' link and a green checkmark icon.

Action	ExternalAccountID	Account Name	Billing City	Billing State/Province	Billing Zip/Postal Code
Edit Del	1	Exxon Mobil	Irving	TX	75039
Edit Del	118	AMR	Fort Worth	TX	76140
Edit Del	12	AT&T	Dallas	TX	75202
Edit Del	124	Fluor	Irving	TX	75039
Edit Del	130	Kimberly Clark	Irving	TX	75038
Edit Del	146	JC Penny	Plano	TX	75024
Edit Del	175	Texas Instruments	Dallas	TX	75243
Edit Del	203	Dean Foods	Dallas	TX	75204
Edit Del	205	Southwest Airlines	Dallas	TX	75235
Edit Del	262	Gamestop	Grapevine	TX	76051

You can also log into the web management console to analyze the run results. If everything ran correctly, there will only be minimal details in the logs. You can enable more detailed logging by stopping the project, undeploying and setting the logging to all (see exercise #1 above, page 14 again for a sample).

This completes Lab #3

Lab 4 Load Salesforce.com Accounts to ESB via WMQ

4.1 Overview

This lab will demonstrate how App Connect Professional can be leveraged to extract information from a SaaS (Software as a Service) based application and format messages that can be passed along to an ESB backbone. ESB's expose their interfaces typically by two means: Web Services (either SOAP or REST) and/or Messaging Middleware, like IBM MQ which is what the focus of this lab will be.

4.2 Concepts

As seen in previous labs, App Connect Professional has a pre-configured Salesforce.com connector that provides a fast and simple means which to interact with the Force.com API with no code. We will be using the same concepts to extract Account data from Salesforce using the native Query language called SOQL (Salesforce Object Query Language – reference is here: http://www.salesforce.com/us/developer/docs/api/Content/sforce_api_calls_soql.htm). We will take this data and then serialize this into an XML message that will be dropped onto a Message Queue. At that point, it is assumed then the ESB will consume the message and route it to some other back end system.

4.3 Activities Used

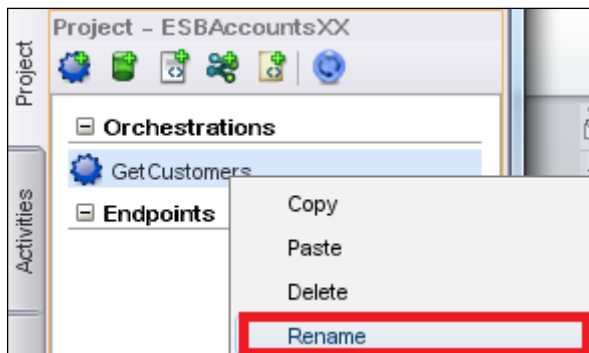
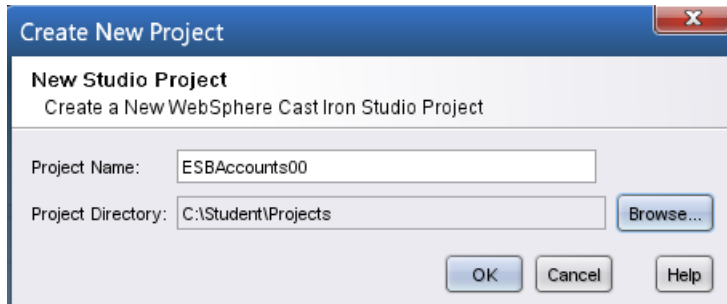
- HTTP Receive
- Salesforce.com Query Objects
- Map Variables
- Write XML
- IBM MQ Put Message

4.4 Document Keys

- Menu bar: Edit → Preferences
- Studio Artifact: **HTTP Receive Request**
- Entered Text: 'Hello Receive Request'
- Project Object Name: [SampleProject1](#)
- Context Menu Item: [Define Default Value](#)
- Activity Checklist Step: [Pick Endpoint](#)
- Orchestration Variable: [Account](#)
- Function Name: [Mathematical / Sum](#)

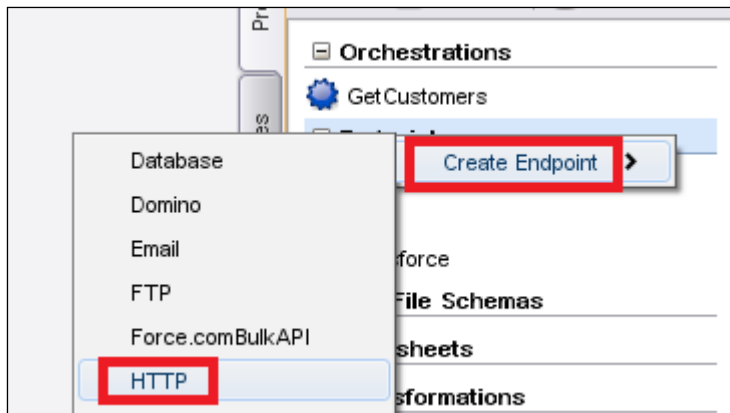
4.5 Load Salesforce.com Accounts to ESB via WMQ

- ___1. Create a new project **ESBAccountsXX**. Where XX is your Team name used in previous labs. Rename the orchestration to **GetCustomers**.

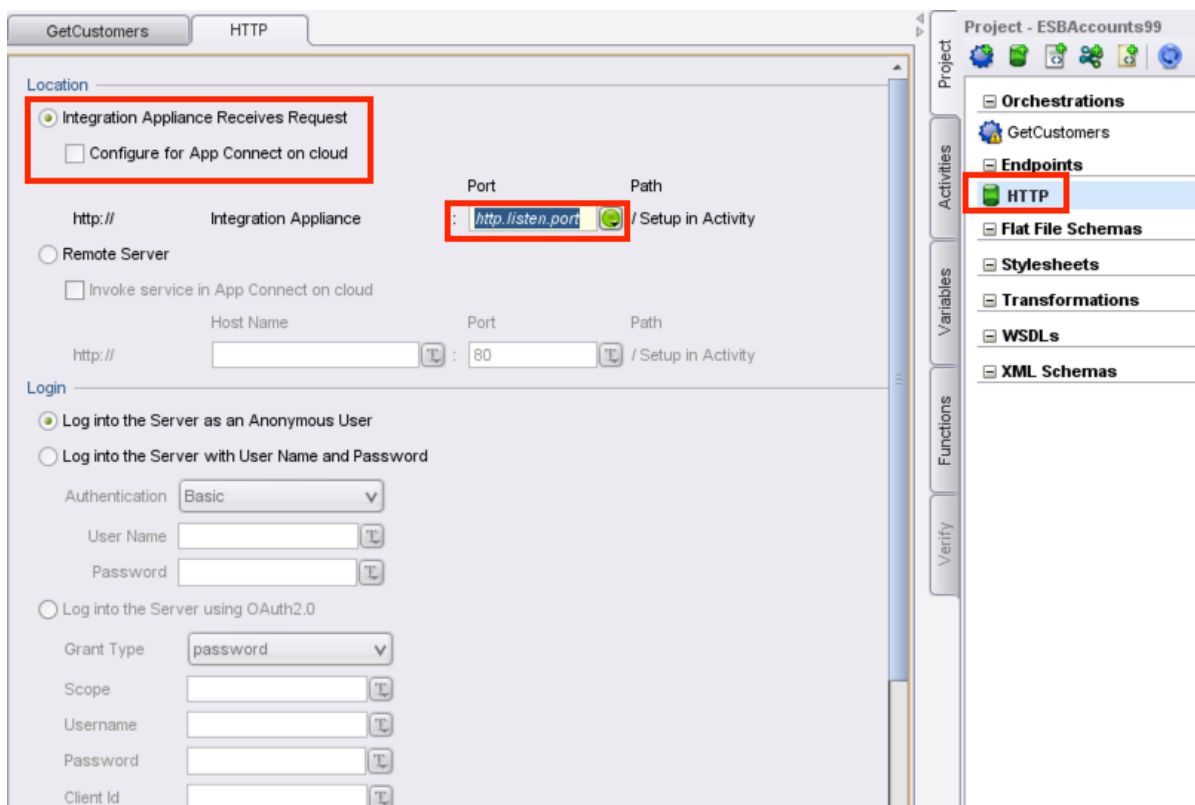


- ___2. Set up the configuration properties. This can be done by selecting *Project* → *Configuration Properties*. Create each of the configuration properties listed below:
 - ___a. **Salesforce.id** – This is your salesforce.com user id. Use the same id as in previous exercises
 - ___b. **Salesforce.pw** – create this as type password. Copy and paste in your password+token id.
 - ___c. **http.listen.port** – This will be the listening port for the HTTP endpoint of the orchestration. Set this to port **80**.
 - ___d. **QNAME** – This is the queue name to be used. The queue you will use is **STUDENT1.OUT**
 - ___e. **MQHOST** – This is the hostname of the WMQ server. Refer to the '**Connection Parameters Spreadsheet**' for the appropriate IP address or hostname.
 - ___f. **MQPORT** – This is hardcoded as '**1414**'
 - ___g. **QMGR** – This is hardcoded as **CASTIRON**
 - ___h. **ServerChannel** – This is hardcoded as **SVR.CHL**
 - ___i. **MQUser** – Hard Code this to **db2admin**
- ___3. Create the endpoints

- __a. Create a new **HTTP** Endpoint. Right click **Endpoints** and select **Create Endpoint**. This will be the HTTP Listener that will enable the orchestration to execute in real-time.



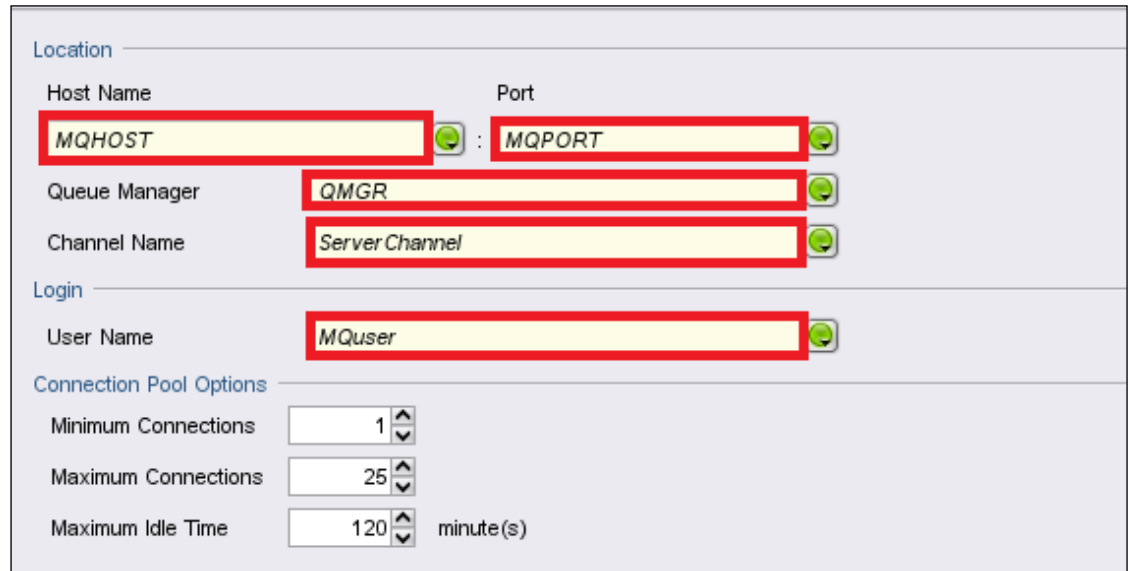
- __b. Configuration window should appear. If it doesn't, then double click the new HTTP endpoint created. Ensure the radio button for **Integration Appliance Receives Request** is selected, and the **Configure for IBM App Connect on cloud** checkbox is not selected. Set the HTTP Receive Port to the `http.listen.port` property name. Use the green button to the right to select the property.



__c. Create a new MQ Endpoint.

__i. Right click endpoints and select **Create Endpoint -> MQ**

__ii. Configuration window should appear. If it doesn't, double click the **MQ** endpoint to configure.



The screenshot shows the 'Create Endpoint -> MQ' configuration window. It has three main sections: 'Location', 'Login', and 'Connection Pool Options'.
 - In the 'Location' section, there are four fields: 'Host Name' (MQHOST), 'Port' (MQPORT), 'Queue Manager' (QMGR), and 'Channel Name' (ServerChannel). Each field has a green checkmark icon to its right.
 - In the 'Login' section, there is one field: 'User Name' (MQuser) with a green checkmark icon to its right.
 - In the 'Connection Pool Options' section, there are three fields: 'Minimum Connections' (1), 'Maximum Connections' (25), and 'Maximum Idle Time' (120 minute(s)). Each field has up and down arrow icons for adjustment.

__iii. Assign the following configuration properties as highlighted in the picture above:

- (1) Host Name = MQHOST
- (2) Port = MQPORT
- (3) Queue Manager = QMGR
- (4) Channel Name = ServerChannel
- (5) User Name = MQuser

__d. Create a new **Salesforce.com** endpoint.

__i. Right click endpoints and select **Create Endpoint -> Salesforce**.

__ii. Configuration window should appear. If it doesn't, double click the **Salesforce** endpoint to configure.

Salesforce.com Customer Login

User Name

Password

Login Options

☒ Login normally

☐ Login to Salesforce.com Sandbox

☐ Login to specified Partner WSDL Login URL

Login URL

Connection Timeout

Time out after second(s) when establishing a connection to the Endpoint.

Proxy

☐ Connect via a Proxy Server

Authentication Realm

Host Name

Port

User Name

Password

Test Connection

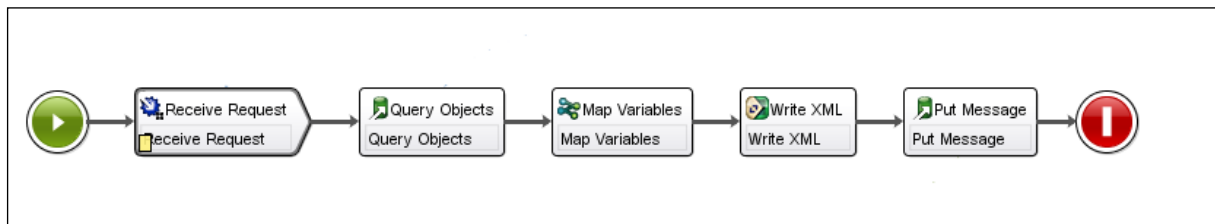
__i. Assign the following configuration properties as highlighted in the picture above:

(1) User Name = **salesforce.id**

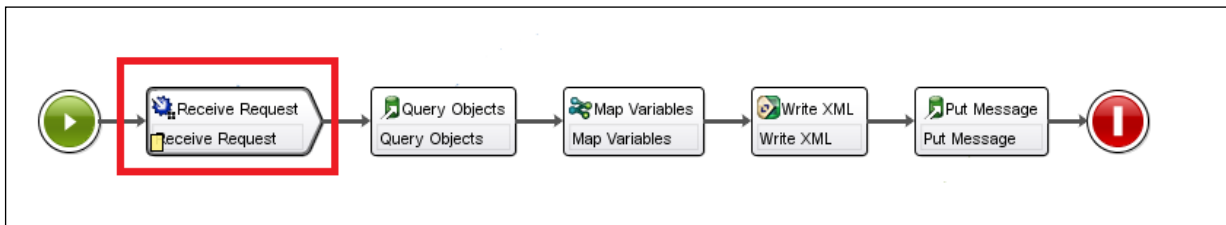
(2) Password = **salesforce.pw**

__ii. To test the connectivity to Salesforce, click **Test Connection**.

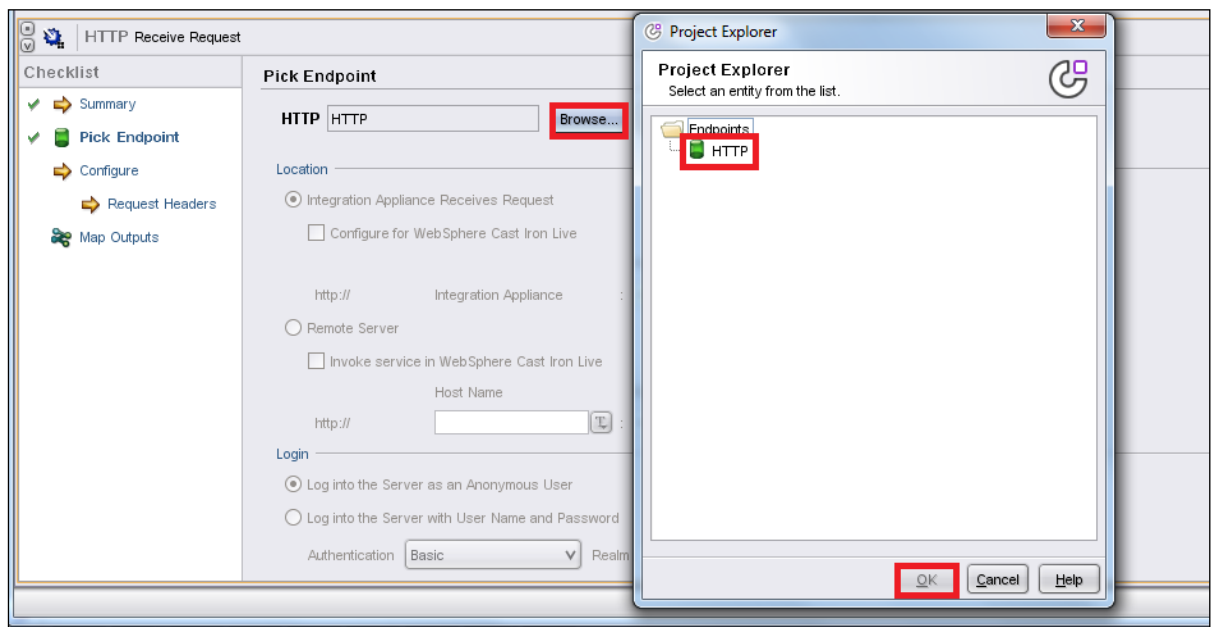
__4. Configure the Orchestration by dragging and dropping activities as per the screenshot below:



- __5. Here are the activities you will need, in the order you need them:
- __a. **HTTP -> Receive Request**
 - __b. **Salesforce -> Query Objects**
 - __c. **Transform -> Map Variables**
 - __d. **Transform -> Write XML**
 - __e. **MQ -> Put Message**
- __6. Configure the **HTTP Receive Request** activity by clicking on the activity in the orchestration.



- __a. **Pick Endpoint.** Select the **HTTP** endpoint via the **Browse** button. Click **OK** to select.



- __b. **Configure.** Change the URL to the hardcoded value of **CustomerExtract**.

Checklist

- ✓ ➔ Summary
- ✓ 📁 Pick Endpoint
- ➔ **Configure**
- ➔ Request Headers
- 🔗 Map Outputs

Configure

Listen at _____

URL (Path After Hostname) **CustomerExtract**

For example, "url/{path}?name1=value1&name2=value2".

Accept the following HTTP method _____

Method: **Any Method**

For Messages _____

Of Type ☒ Text ☐ Binary

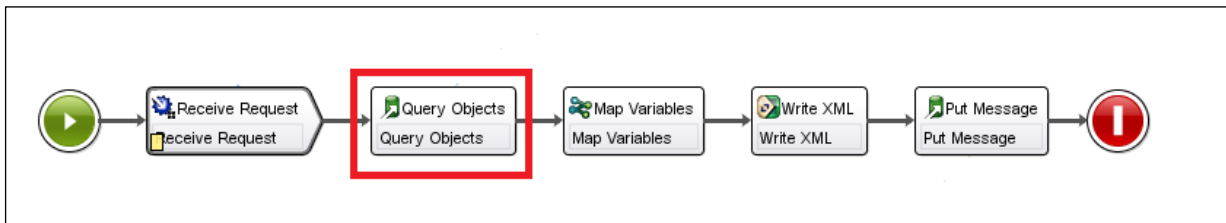
Options _____

☐ Requires a Reply

☐ Use Compression/Decompression

- __c. **Map Outputs.** No mapping required here, as the **HTTP Receive** is only set up as a mechanism to kick off the orchestration.

- __7. Configure the **Query Objects** activity by clicking on the activity in the orchestration.



- __a. **Pick Endpoint.** Select the **Salesforce** endpoint via the **Browse** button. Click **OK** to select.

Salesforce.com Query Objects

Checklist

- ✓ ➔ Summary
- ✓ 📁 **Pick Endpoint**

Pick Endpoint

Salesforce.com **Salesforce** **Browse...**

Salesforce.com Customer Login _____

User Name **salesforce.id**

Project Explorer

Select an entity from the list.

Endpoints

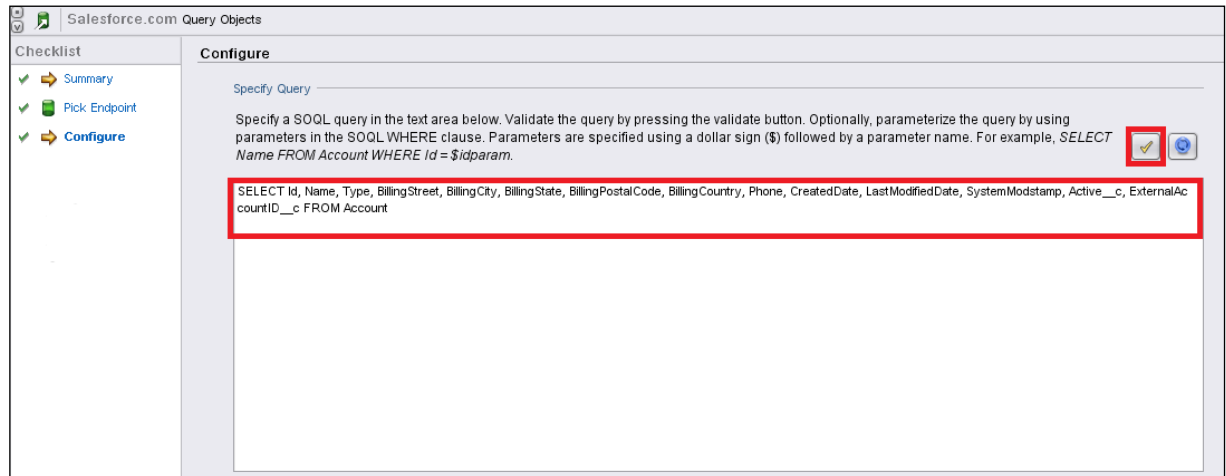
- 📁 Endpoints
- 📁 **Salesforce**

OK **Cancel** **Help**

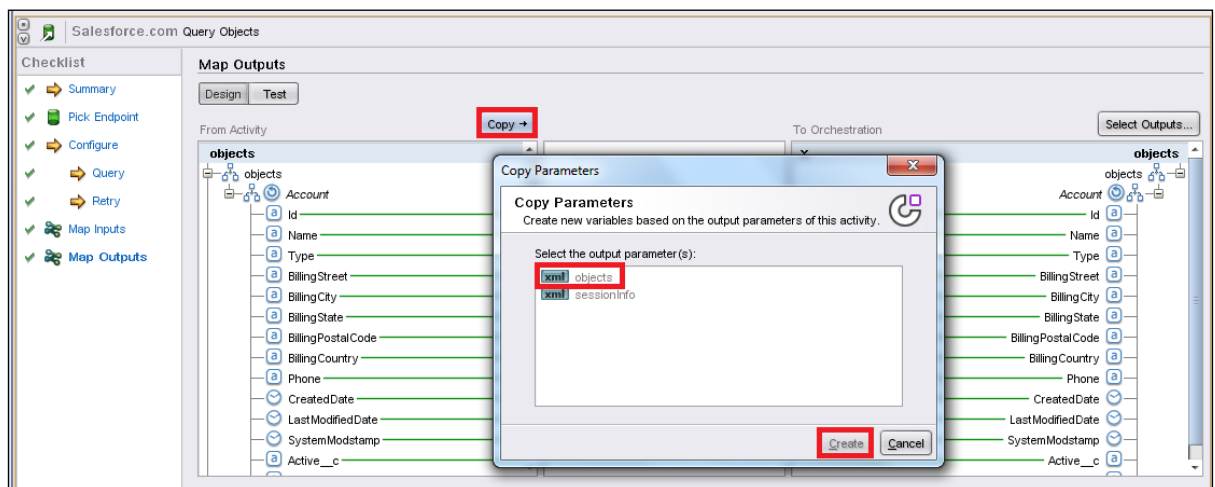
- ___b. **Configure.** Enter in the SOQL query in the white space. The SOQL Query to use for this lab is provided below as well as in this directory **c:\Student\Lab Resources\Lab 4\soql.txt**

Click the check mark button  to confirm syntax and objects that were entered in:

SELECT Id, Name, Type, BillingStreet, BillingCity, BillingState, BillingPostalCode, BillingCountry, Phone, CreatedDate, LastModifiedDate, SystemModstamp, Active__c, ExternalAccountID__c FROM Account

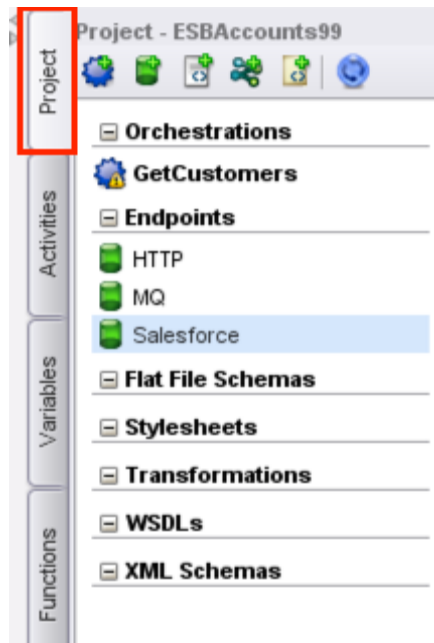


- ___c. **Query and Retry.** Skip these steps.
- ___d. **Map Inputs.** If our SOQL query had any input parameters to map into the query, we would handle that here. In this Lab, this step can be skipped.
- ___e. **Map Outputs.** Click the **Copy** button to initialize a new variable that will store all the data that is retrieved from the query. Be sure to select **objects** variable and then click **Create**.

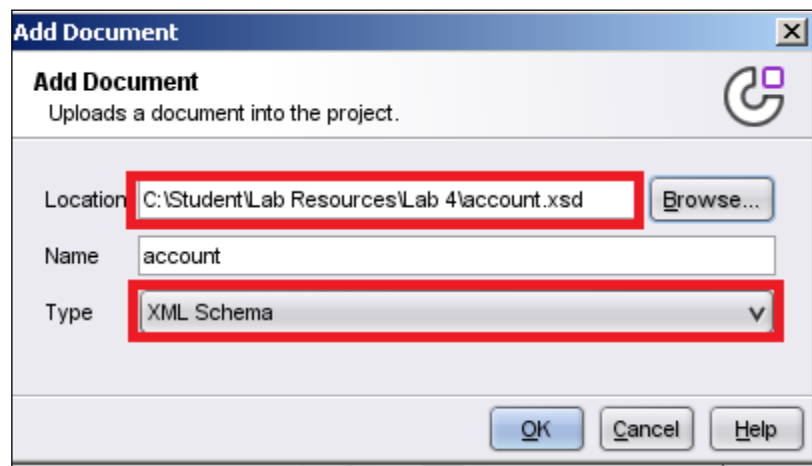


- ___8. Before we can continue with the next activity, we will need to add the XML schema for the data set we will be generating. To do so, follow these steps:

- ___a. Click the **Project** tab.



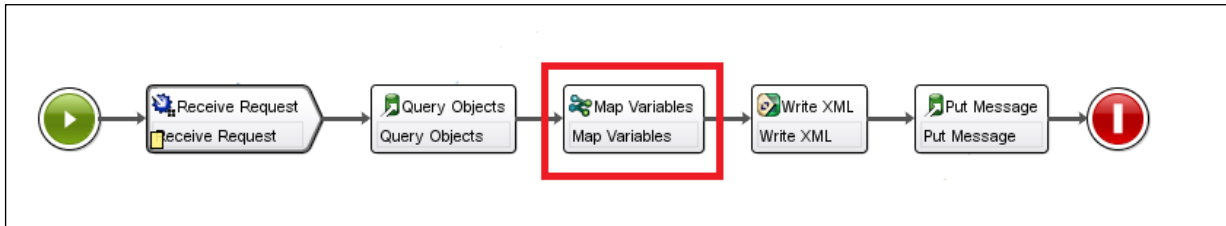
- __b. Right Click the **XML Schemas** heading and select **Add Document**. Browse to the **c:\Student\Lab Resources\Lab 4** directory and select the **account.xsd** file. Click **OK** to add the document to the project.



- __c. Once the schema has been added, you will see it under the schema tab.

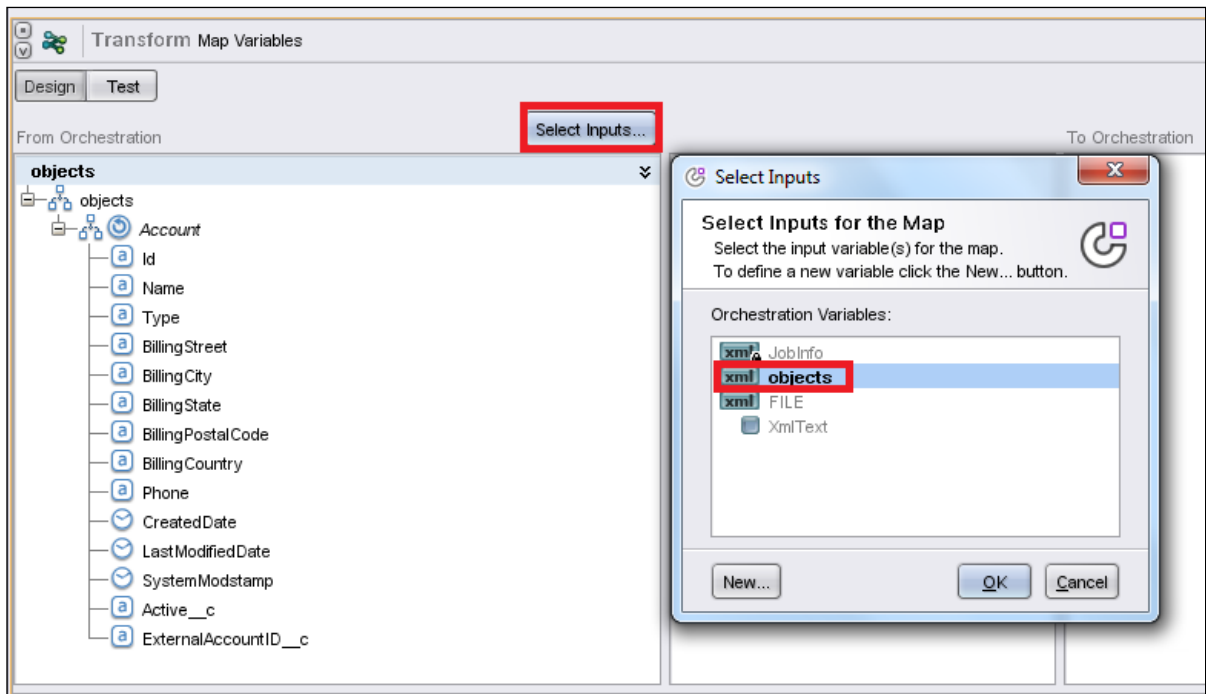


- __9. Configure the **Map Variables** activity by clicking on the activity in the orchestration.

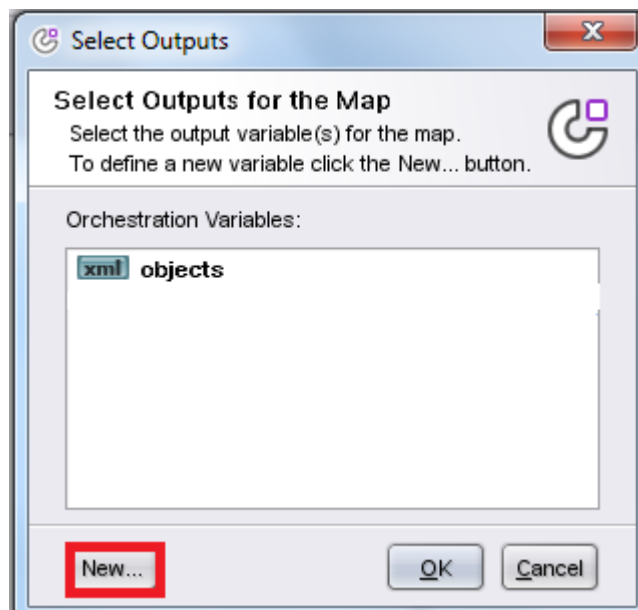


- __a. This activity has no checklist. Map Variables is an optional activity that you can use to segregate transformation from other activities for ease of maintenance. All of the mapping needed for this particular orchestration will be sequestered here, which will make the subsequent activities much quicker to configure.

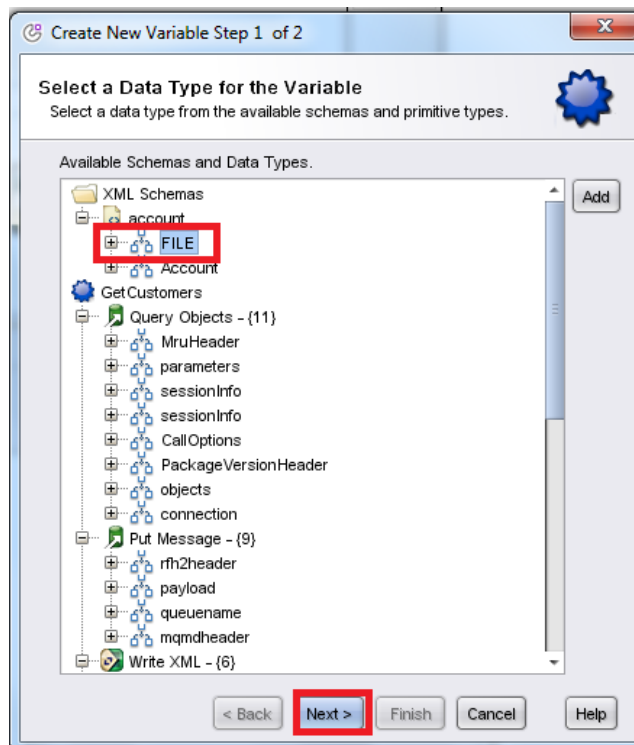
- __b. Click the **Select Inputs** button. Add the **objects** variable and click **OK**. This is the result of the Salesforce Query that will act as the input.



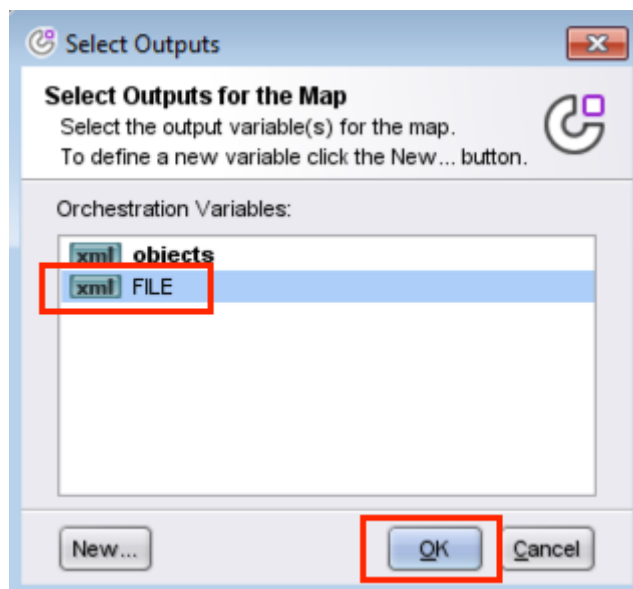
- __c. Click the **Select Outputs** button. Click the **New** button and then click **OK**. This will allow us to initialize a new variable that will represent the XML Schema for the output data we will be sending to the ESB.



__d. Select the **FILE** node and then click **Next**.



__e. Name the node **File** and then click **Finish**.



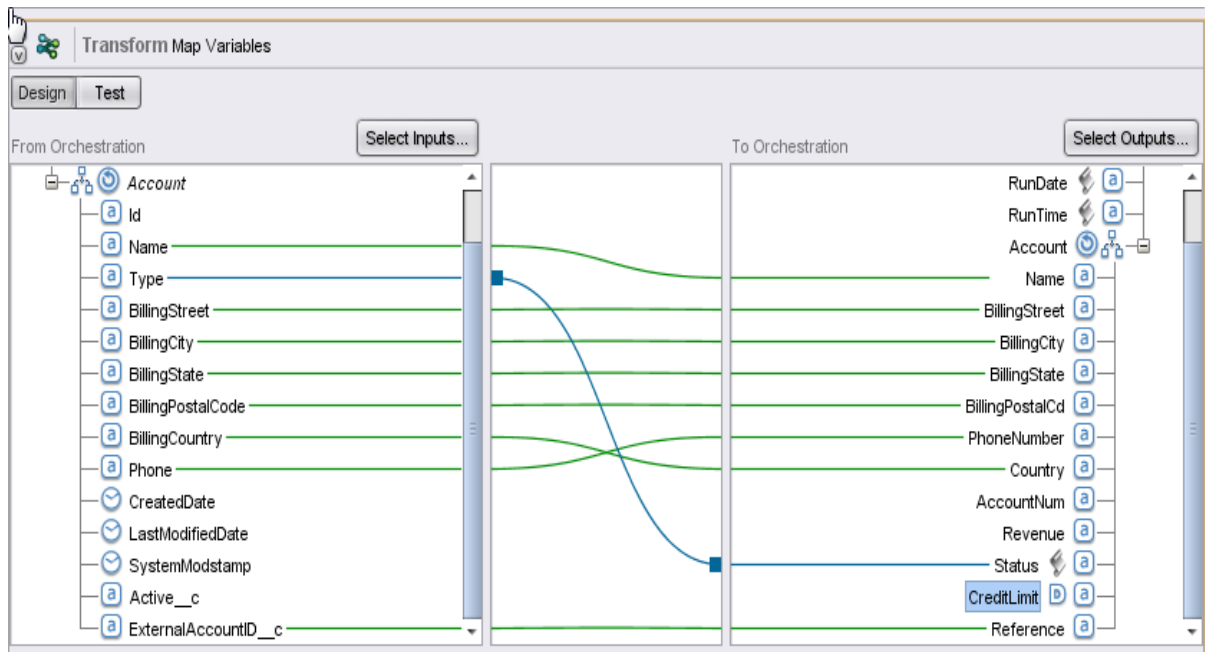
__f. Fields now can be mapped. Use the following mapping matrix as your guide. Hint:



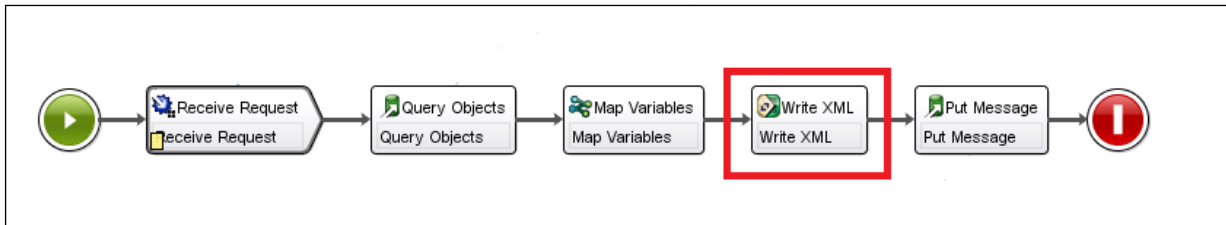
Use the **Substring After** function for the transformation of the “**Type**” field.

Input Field	Output Field	Transformation Rules
Id	n/a	Not Mapped
Name	Name	Direct mapping, no transformation
Type	Status	Parse (Substring) Input field to extract all data after "-"
BillingStreet	BillingStreet	Direct mapping, no transformation
BillingCity	BillingCity	Direct mapping, no transformation
BillingState	BillingState	Direct mapping, no transformation
BillingPostalCode	BillingPostalCd	Direct mapping, no transformation
BillingCountry	Country	Direct mapping, no transformation
Phone	PhoneNumber	Direct mapping, no transformation
CreatedDate	n/a	Not Mapped
LastModifiedDate	n/a	Not Mapped
SystemTimeStamp	n/a	Not Mapped
Active__c	n/a	Not Mapped
ExternalAccountID__c	Reference	Direct mapping, no transformation
	RunDate	CurrentDate
	RunTime	CurrentTime
	CreditLimit	Default Value of 10000
	Revenue	Not Mapped

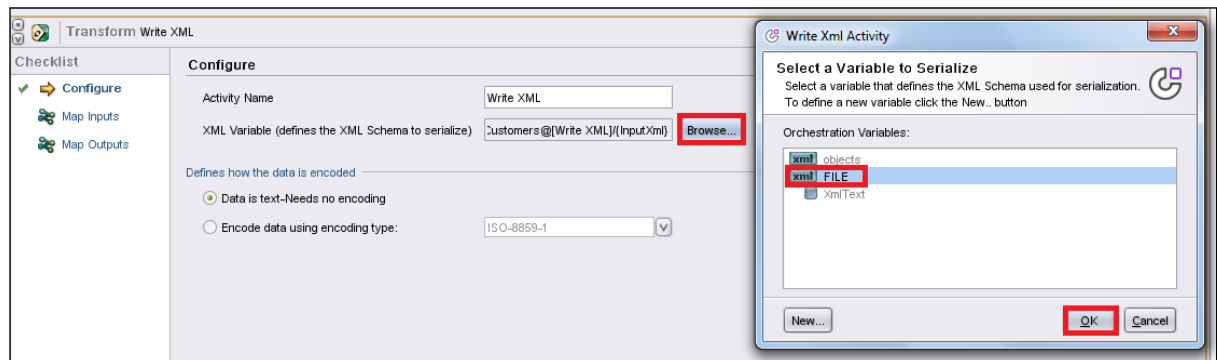
__g. When complete, the mapping should look as such:



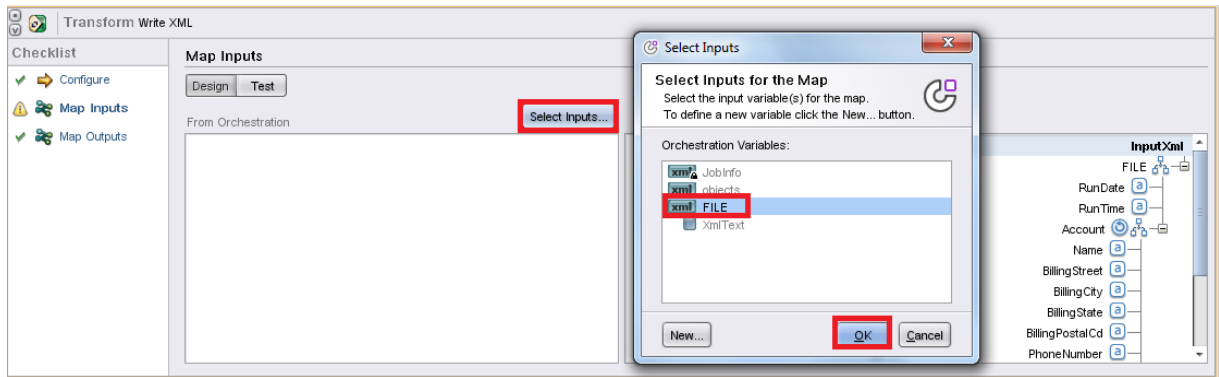
__10. Configure the **Write XML** activity by clicking on the activity in the orchestration.



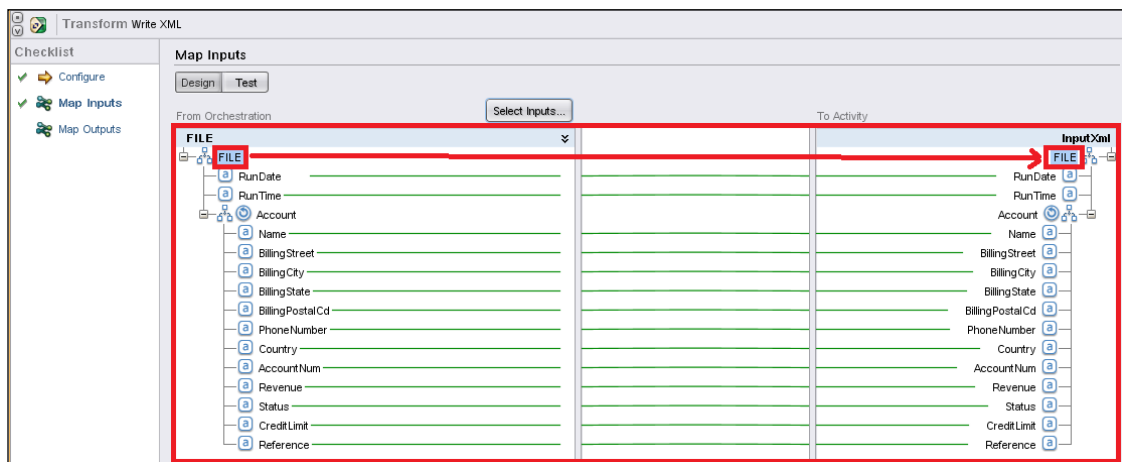
__a. **Configure.** Click the **Browse** button next to the **XML Variable** field to select the schema to use. Select the **File** variable as indicated below. Click **OK** to complete the task.



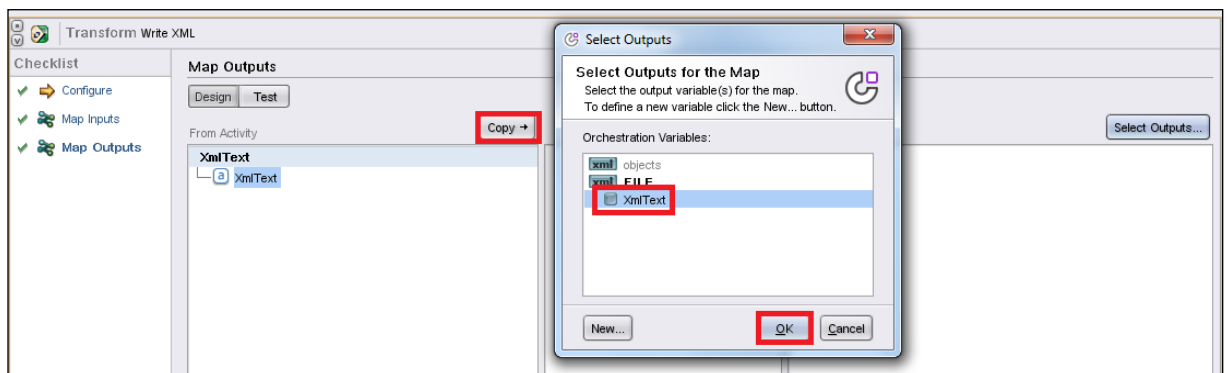
__b. **Map Inputs** Click **Select Inputs** and then browse for the **File** variable. Click **OK**.



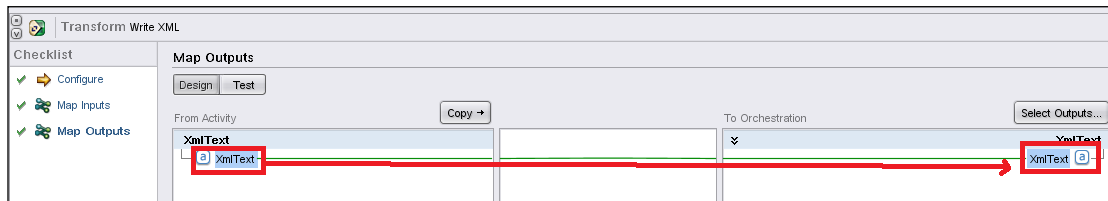
- ___c. Drag and drop the **File** node from the input side over to the **File** node on the output side. The fields will be automapped.



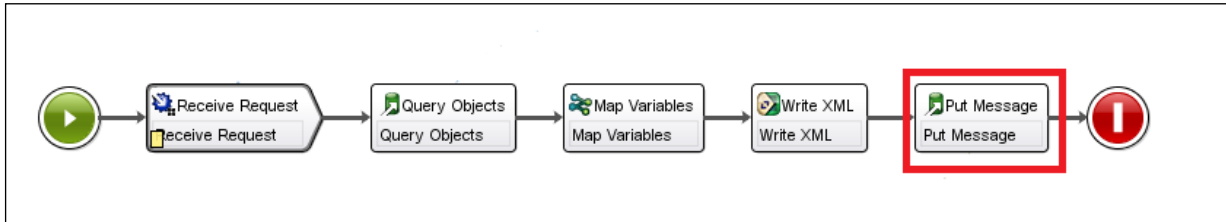
- ___d. **Map Outputs** Copy the variable **XmlText** by clicking on **Copy**, select the **XmlText** variable in the pop up box and then click **OK**. This will take the XML mapped in the previous step and serialize into a single “blob”, creating a payload in a single field we will be using in the next activity.



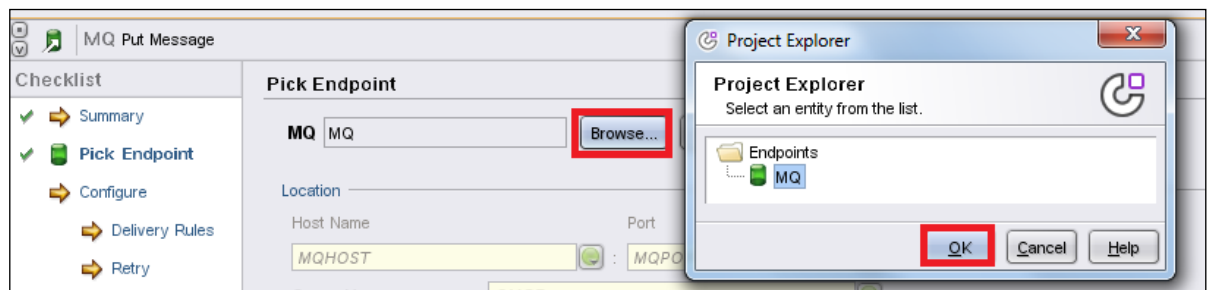
- __e. Drag and drop the **XmlText** field from the input over to the output.



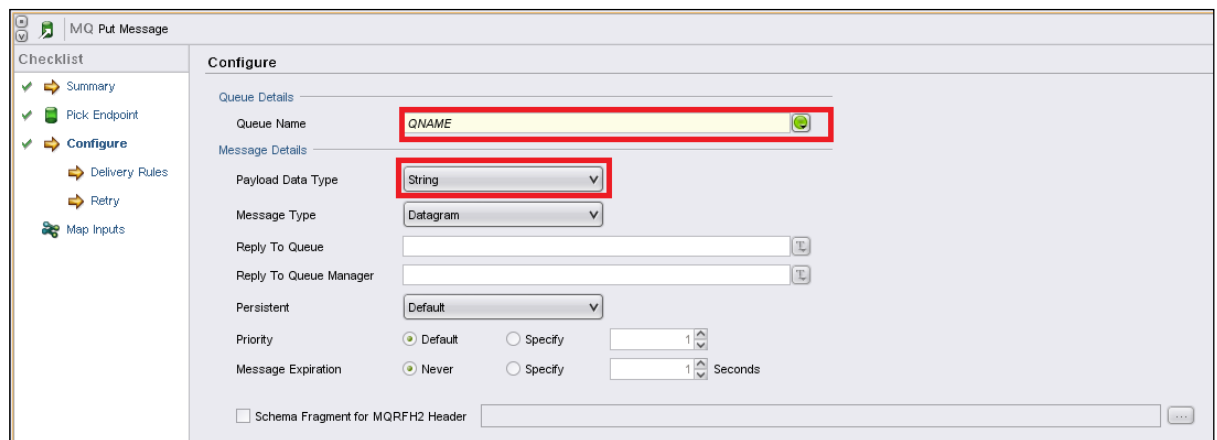
- __11. Configure the **Put Messages** activity by clicking on the activity in the orchestration.



- __a. **Pick Endpoint** Select the **MQ** endpoint via the **Browse** button. Click **OK** to complete.

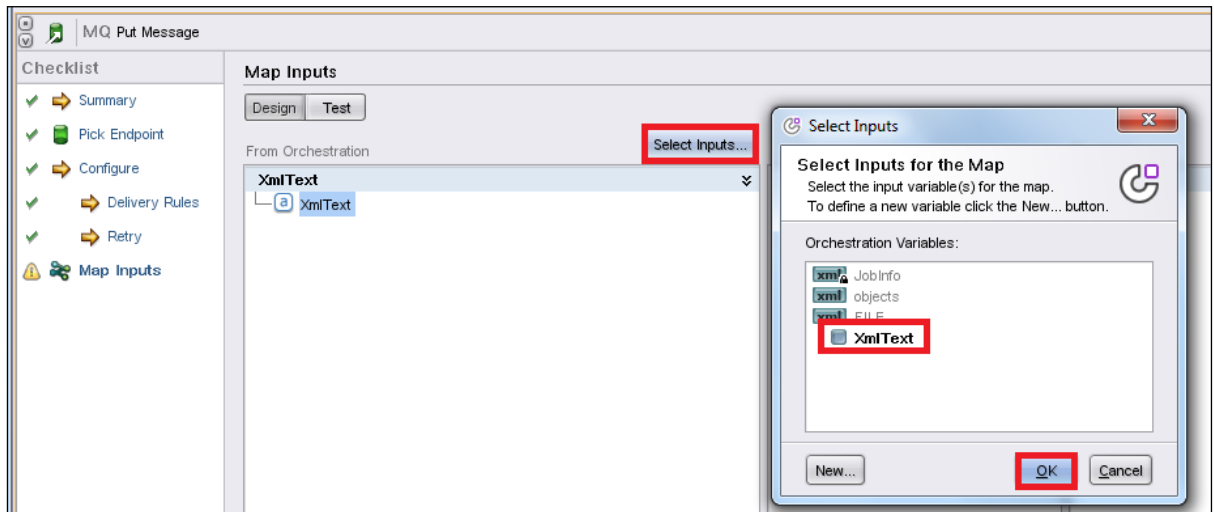


- __b. **Configure** - assign the QNAME configuration property variable to **Queue Name**.

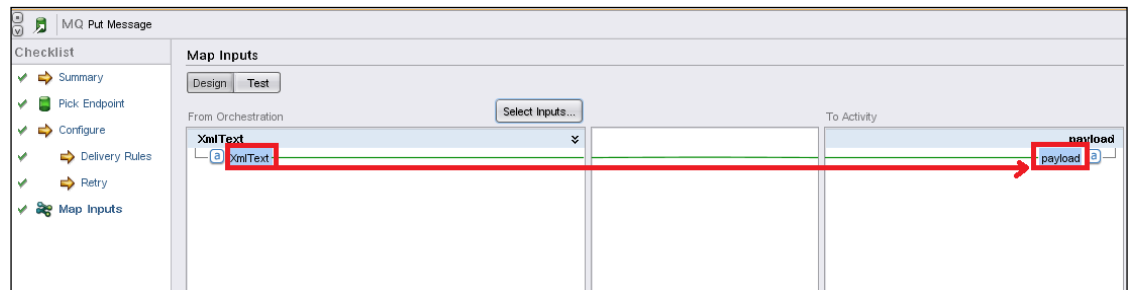


- __i. Ensure the **Payload Data Type** is set to **String**
- __c. **Delivery Rules** – no changes
- __d. **Retry** – no changes

- __e. **Map Inputs** add the **XmlText** variable to the left side via **Select Inputs**. Click **OK**.

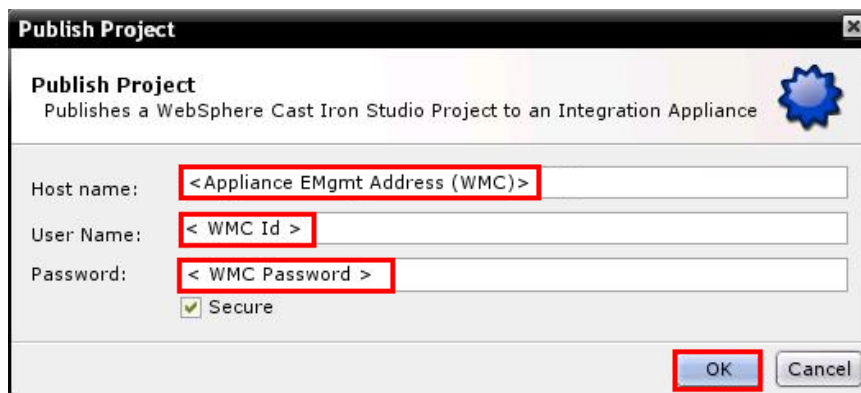


- __i. Map over the **XmlText** node from the left side to the **payload** node on the right hand side.

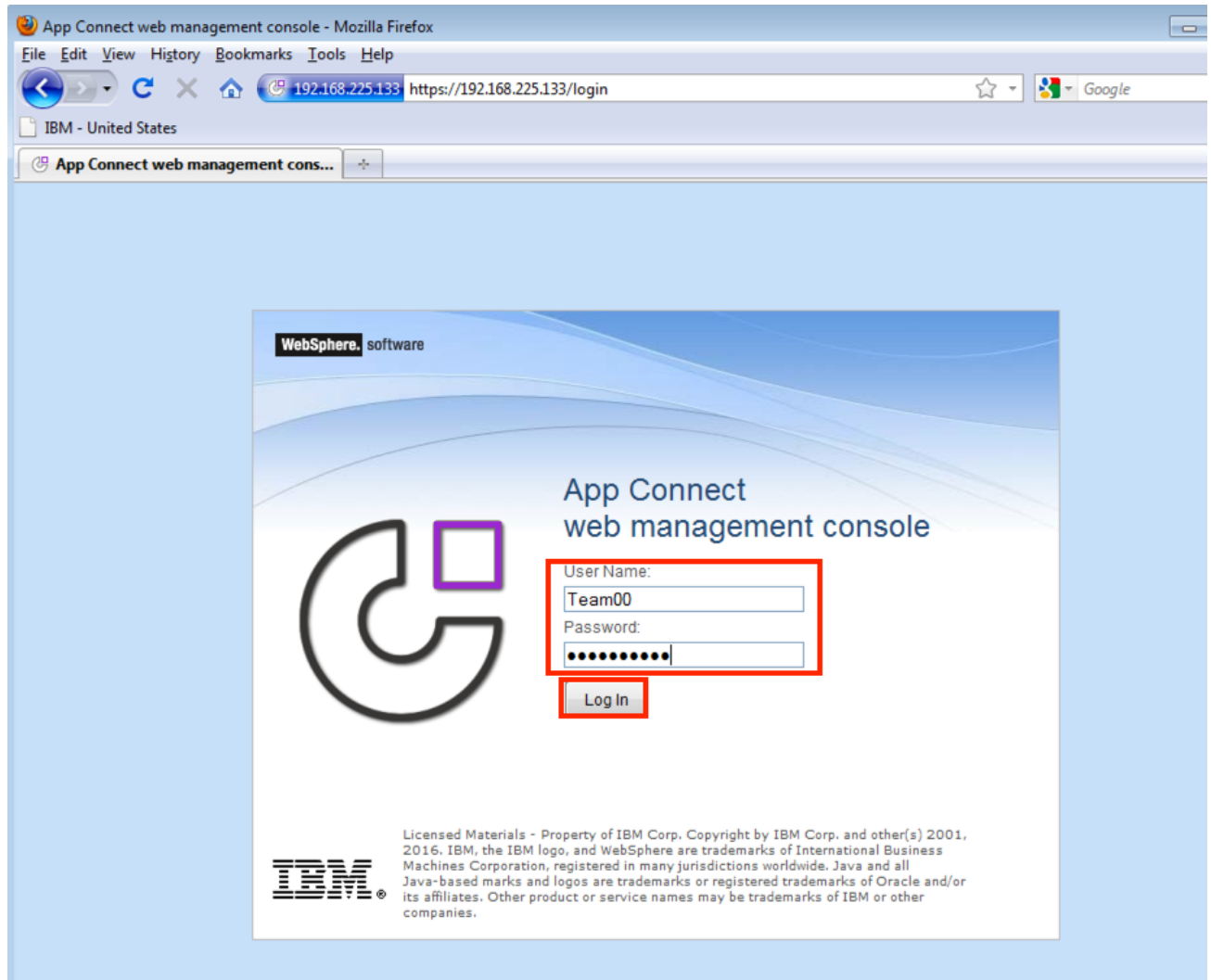


- __12. Deploy the Orchestration to your runtime Environment.

- __13. Deploy the project to your appliance. This done via **File** → **Publish** or using the  icon from the top tool bar. Use the credentials for the previous lab.



- ___14. Login in to the WMC using the URL and login credentials provided by the instructor. Click the **Log In** button.

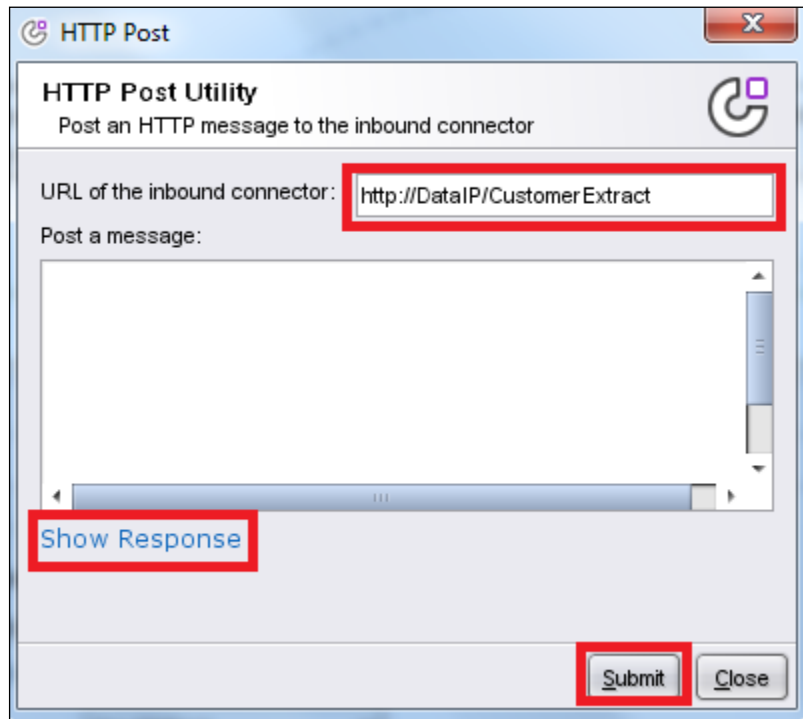


- ___15. Find your project on the list and double click the green arrow on the right hand side to enable it. Move on to the next section to run data through.

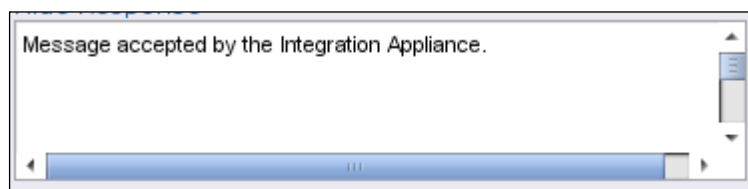


4.6 Run the Load Salesforce.com Accounts to ESB via WMQ Lab

1. In the App Connect Studio, use the HTTP Post utility to kick off the Orchestration. Click *Tools -> HTTP Post Utility*. Fill out the values on the form according to the value of the Data IP Port on your appliance. Refer to the '**Connection Parameters Spreadsheet**' for the appropriate information. Before clicking **Submit**, click the **Show Response** link to see the response from the Appliance after the request is submitted.



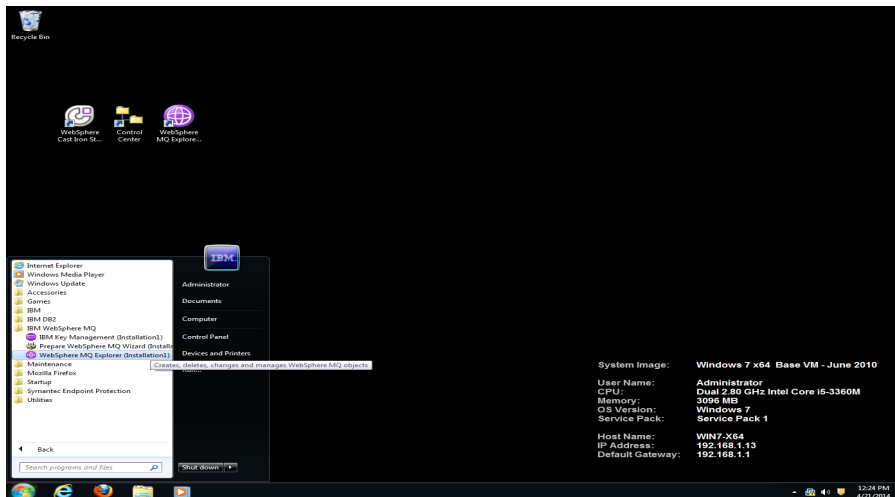
2. If it runs properly you will see the following message in the response section.



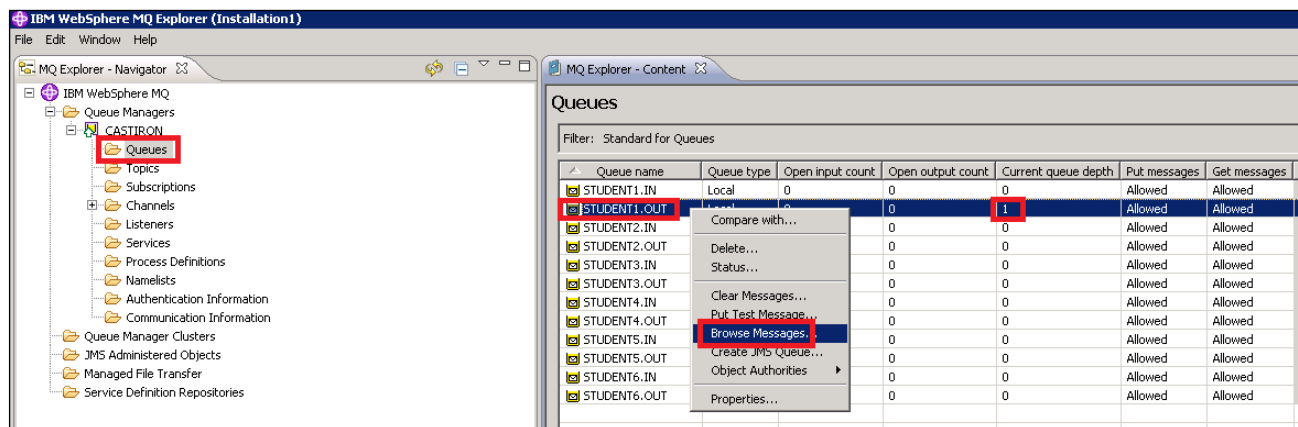
3. You can confirm the output is generated properly by firstly checking in the Web Management Console.

Orchestrations	Status
GetCustomers	Completed in 3.64s

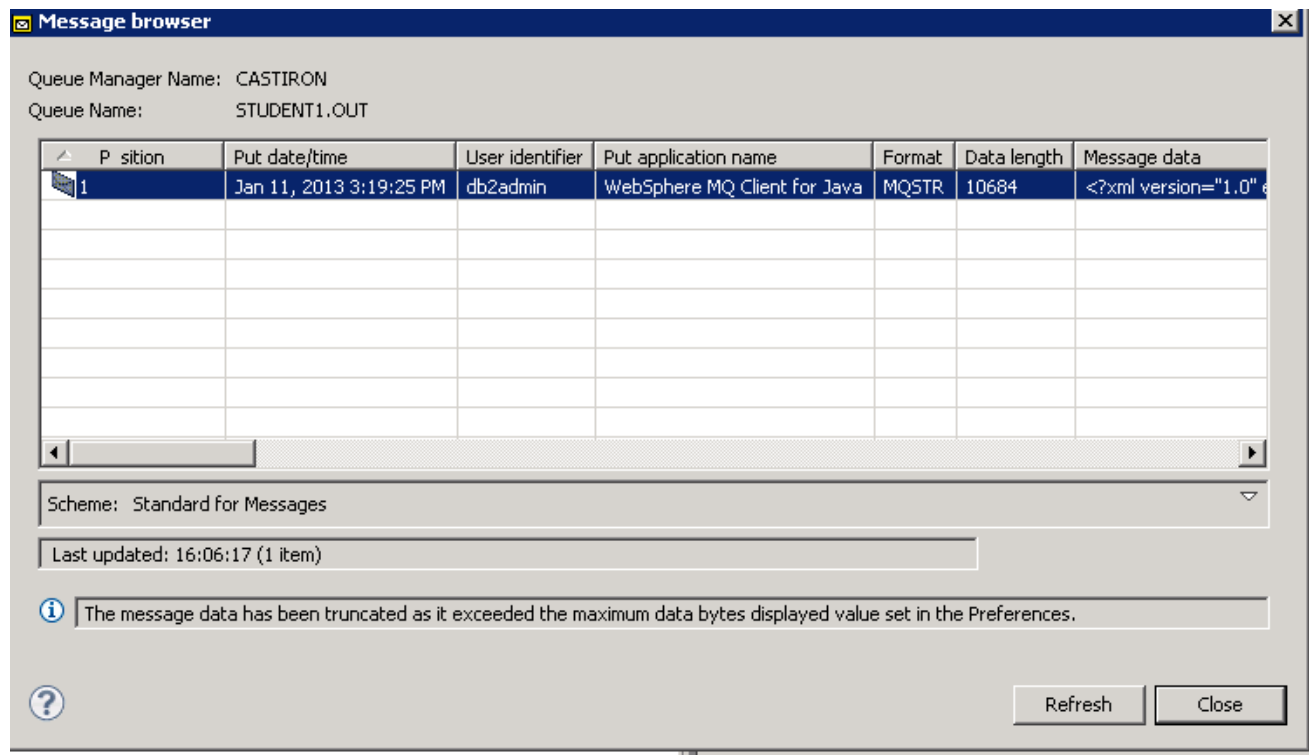
- __4. Optionally – you can check inside of IBM MQ to ensure the data was enqueued properly.
- __a. You can either click the IBM MQ Explorer on the Desktop or Go to *Start -> All Programs -> IBM MQ -> IBM MQ Explorer*



- __5. Inside of MQ Explorer, click **Queues**, then select the **STUDENT1.OUT** queue. The queue depth should be 1. Right click the **STUDENT1.OUT** queue and select **Browse Messages**.



__6. Approximate message size and data will be visible here.



This completes Lab #4

Lab 5 Invoke Web Service

5.1 Overview

In this lab, you will create a new project, WebServiceBasicsXX (where xx is your team number) that will have an orchestration that invokes a web service.

5.2 Concepts

Web services are described by WSDL files that are used to configure a consuming web service connector. Then, the **Web Services Invoke Service** activity may be used to invoke the web service and interpret its response. A web service operation is defined in terms of the XML documents that represent input for and output of the operation. Often, these documents are simple one-element documents where the element contains a serialized XML document.

5.3 Activities Used

- HTTP Receive Request
- HTTP Send Response
- Web Services Invoke Service
- Transform Read XML

5.4 Document Keys

Menu bar: *Edit → Preferences*

Studio Artifact: **HTTP Receive Request**

Entered Text: 'Hello Receive Request'

Project Object Name: *SampleProject1*

Context Menu Item: *Define Default Value*

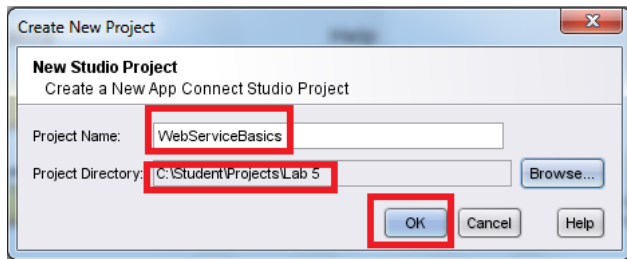
Activity Checklist Step: *Pick Endpoint*

Orchestration Variable: *Account*

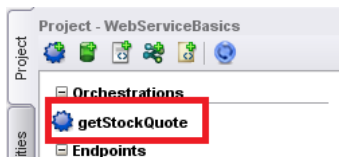
Function Name: *Mathematical / Sum*

5.5 Invoke WebService with Fixed Symbol

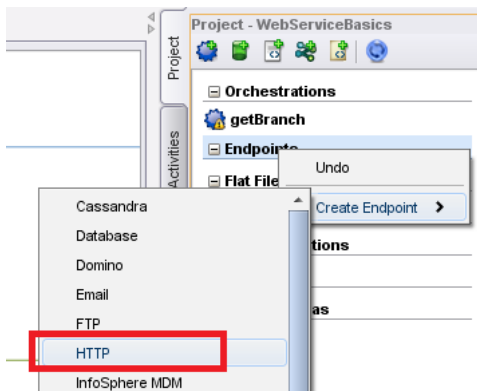
- __1. Create a new project **WebServiceBasics** and save it in C:\Student\Projects\Lab5.



- __2. Rename the orchestration to **getStockQuote**.

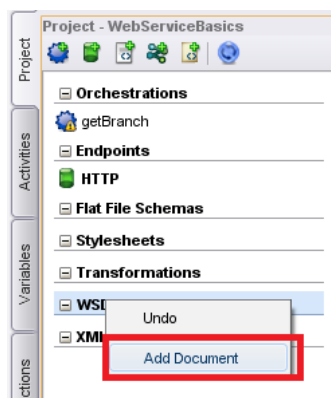


- __3. Create an inbound HTTP endpoint and take the defaults.

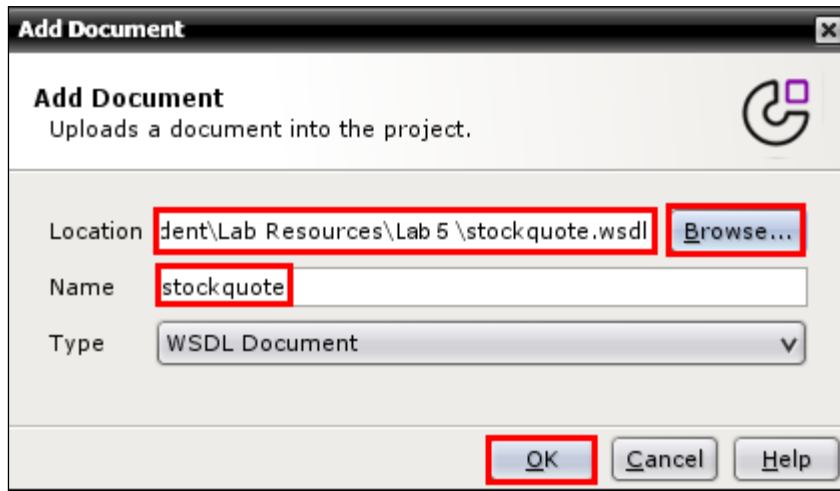


- __4. The WSDL that describes the service needs to be loaded into the project.

- __a. Bring up the context menu for the **WSDLs** section of the Project tab and select **Add Document**.

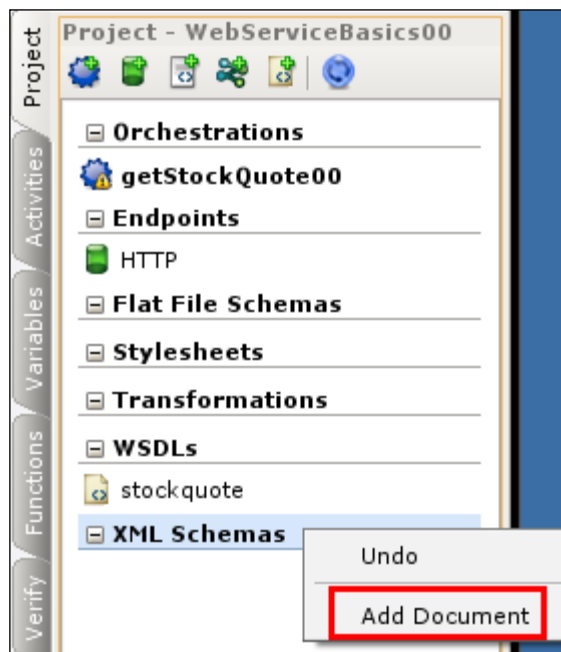


- __b. Browse for the 'stockquote.wsdl' file in: C:\Student\Lab Resources\Lab 5.

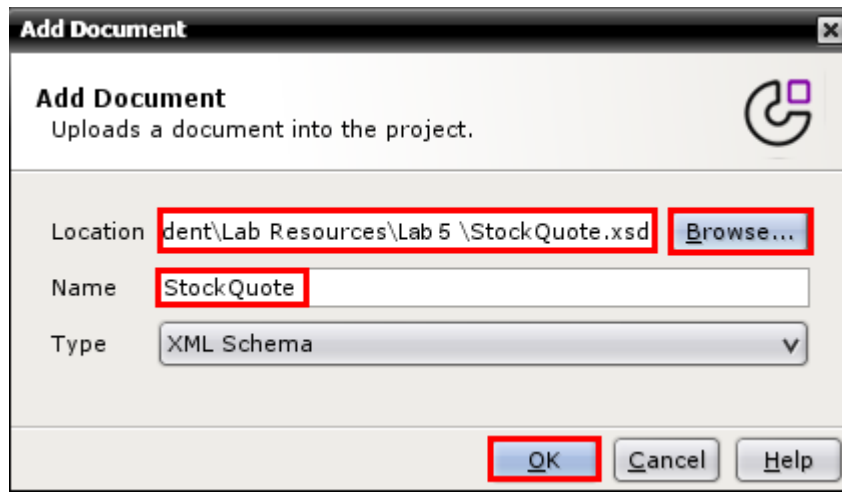


- __5. The schema describing the output also needs to be loaded.

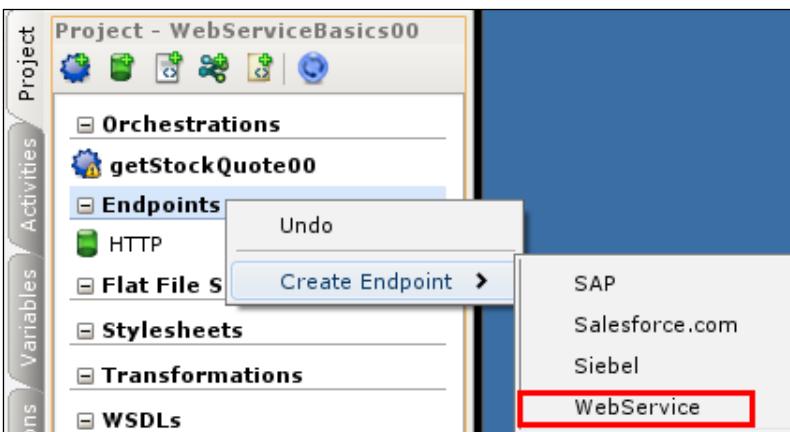
- __a. Bring up the context menu for the **XML Schemas** section of the **Project** tab and select **Add Document**.



- __b. Browse for the 'StockQuote.xsd' file in: C:\Student\Lab Resources\Lab 5.

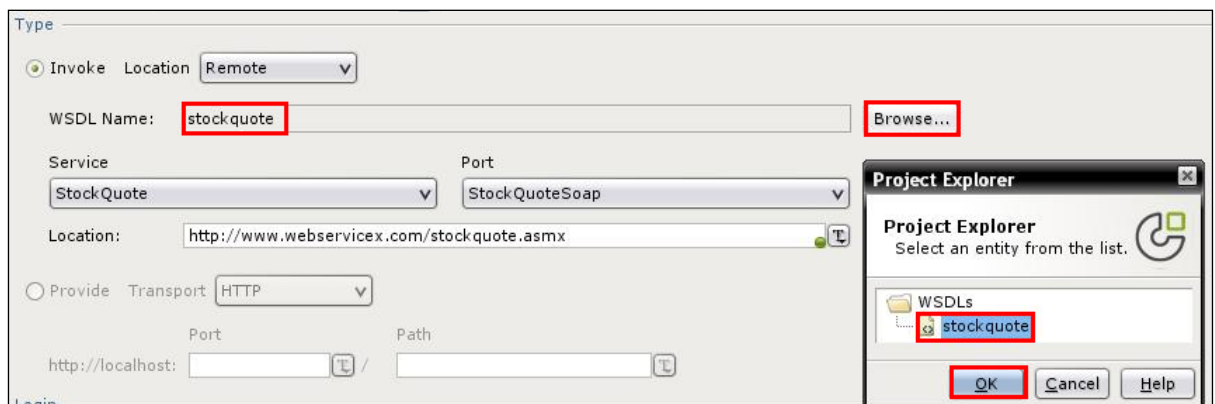


- __6. Create a WebService endpoint: WSCStockQuote. The service takes a securities symbol as input and returns data as an XML document.

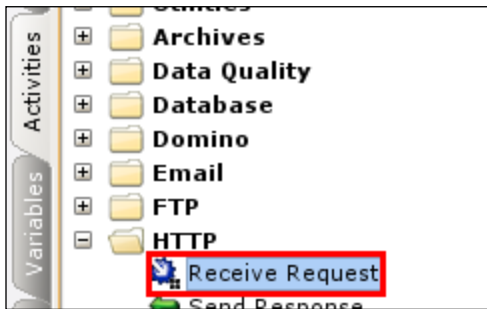


- __7. Browse for the WSDL file just loaded.

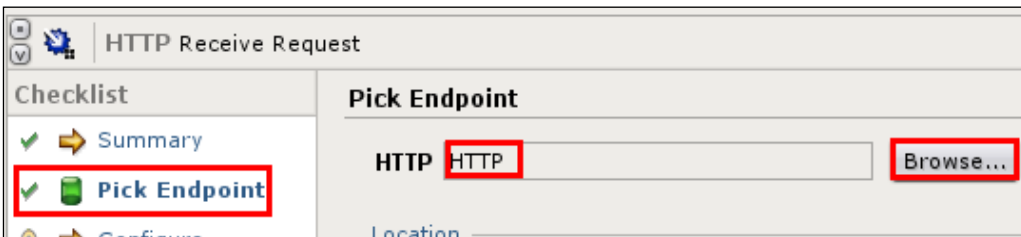
- __a. The Location, Port and Service fields will be generated from the WSDL.



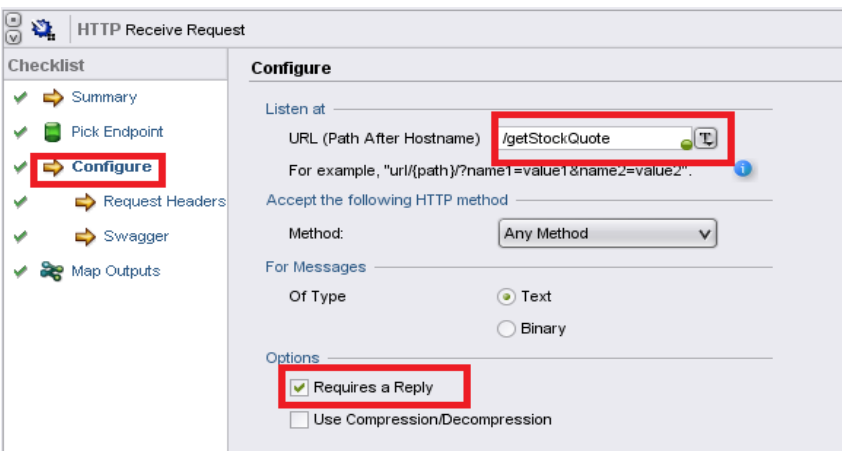
- __8. Add an HTTP **Receive Request** activity to the orchestration.



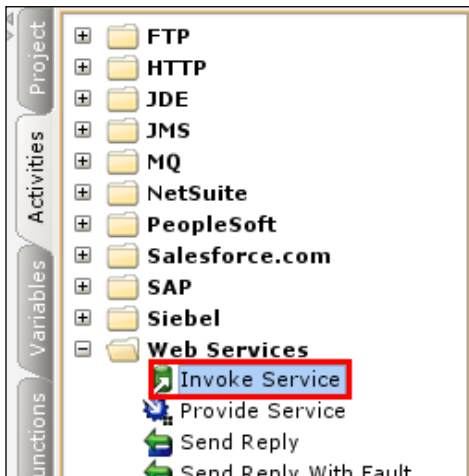
- __9. **Pick Endpoint**: Browse for the HTTP endpoint created in Step 2.



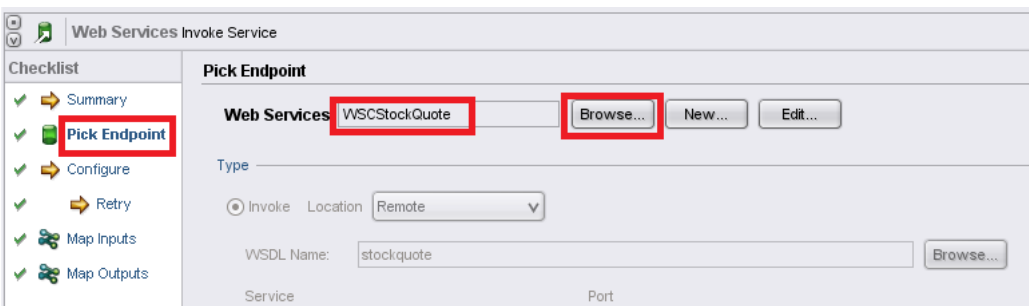
- __10. **Configure**: URL = "getStockQuote", and ensure that the Requires a Reply checkbox is selected.



__11. Add a Web Services **Invoke Service** activity to the orchestration.

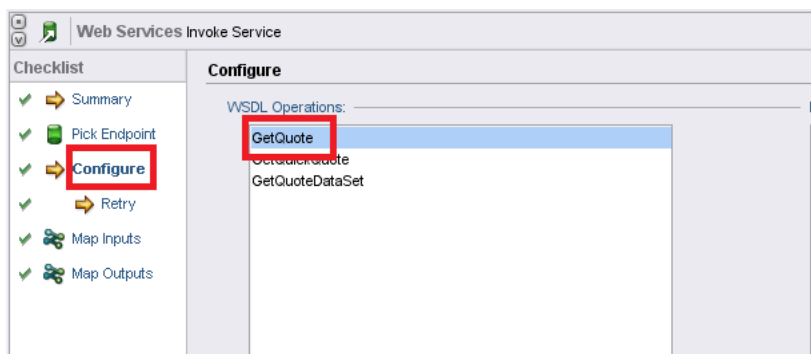


__12. **Pick Endpoint**: Select the Web Service Endpoint WSCStockQuote.



__13. **Configure**: Select the operation GetQuote in the Operations pane.

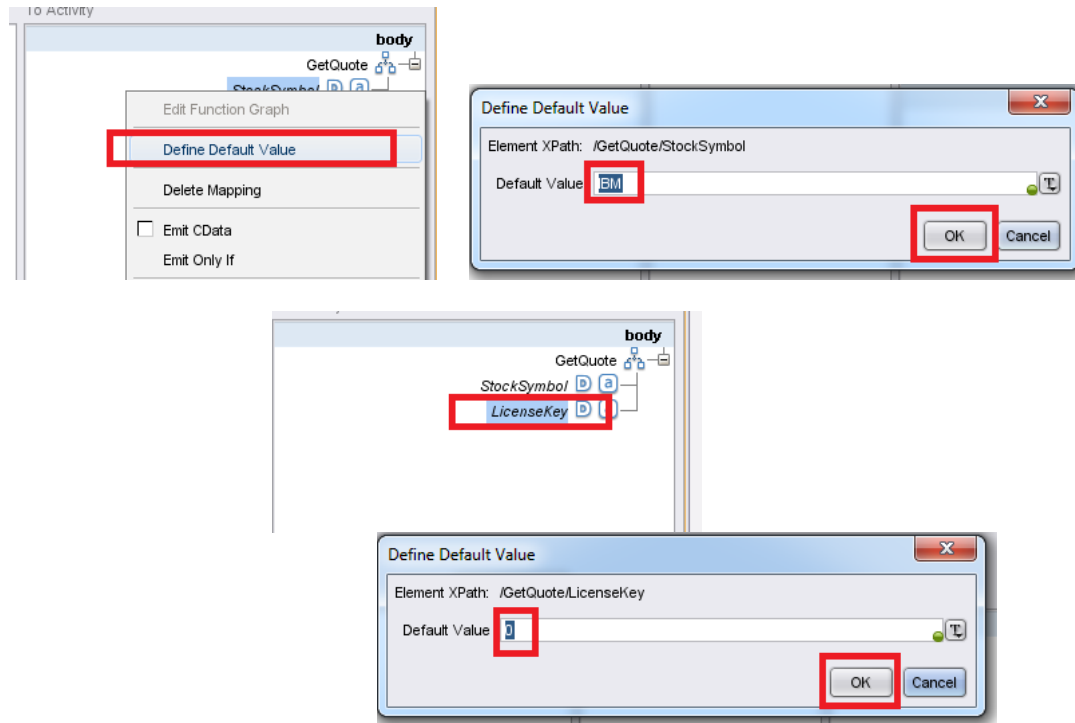
__a. There may be only one operation for a WSDL, but for more complex services all of the operations will be listed.



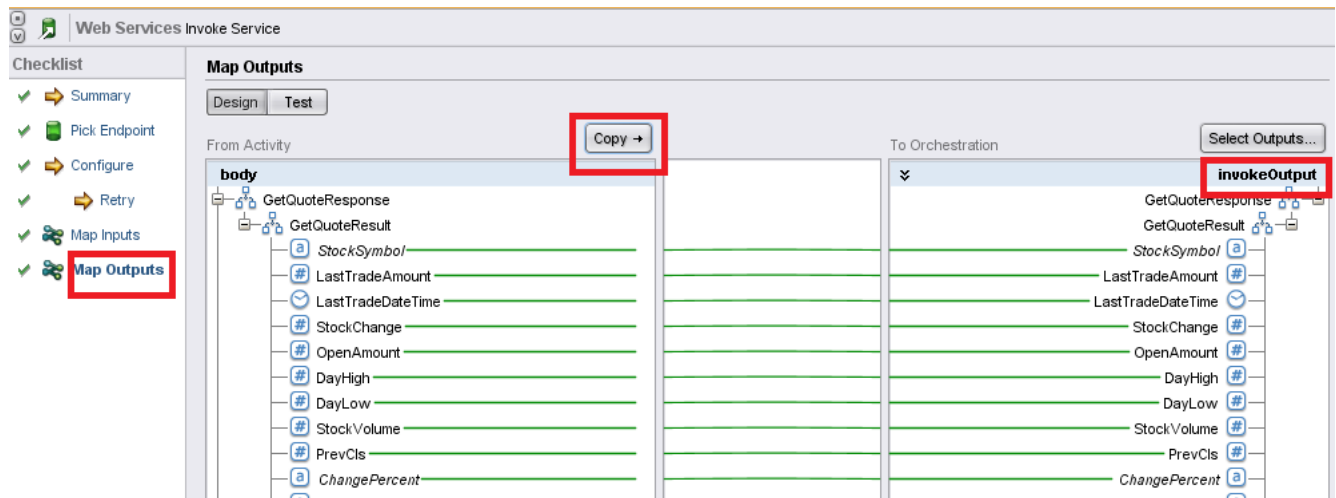
__b. When you select the operation, the next **Map** steps will reflect the operation's structure.

__14. **Map Inputs:** For now there will be no dynamic input.

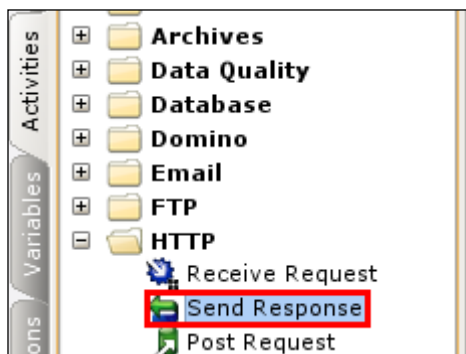
- a. Assign a default value to the 'StockSymbol' field, such as 'IBM' by right clicking on StockSymbol and selecting Define Default Value...Then click on "OK". Repeat the process and assign a Default Value for 'LicenseKey' which is '0'...Then click "OK".



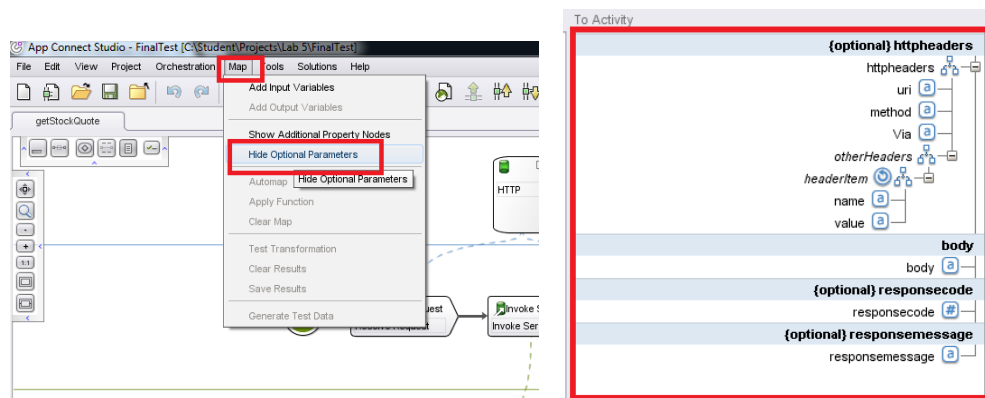
- ___15. **Map Outputs:** Copy only the activity's body to the orchestration and rename the variable `invokeOutput` :



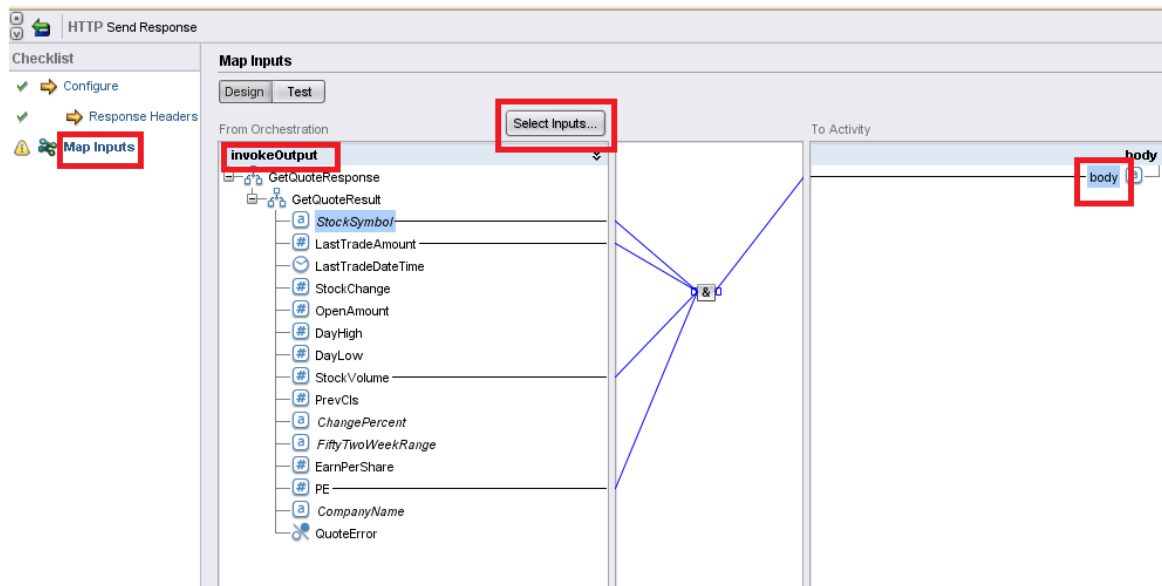
- ___16. Add an **HTTP Send Response** activity to the orchestration.



- ___17. Select Map/Hide Additional Parameters. This will display just the 'body' parameter under 'To Activity'.



- ___18. **Map Inputs:** Map the following input fields of the invokeOutput structure to build an output message to the body. Select invokeOutput from the Select Inputs button...Use the *Concatenate* function.

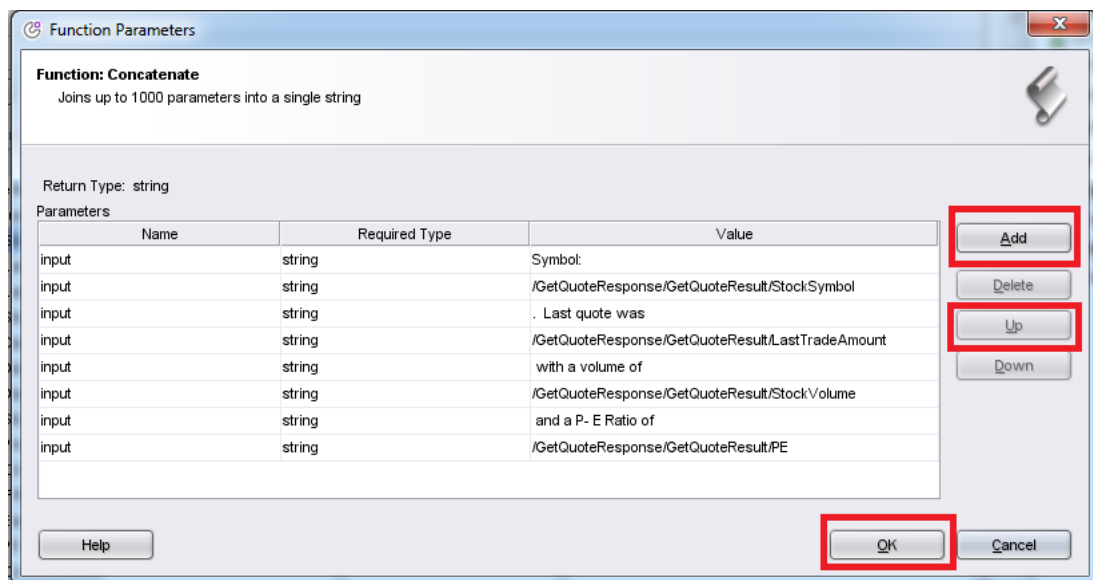


- __19. Double-click the **Concatenate** function and Add/move UP the following string entries to build a message like this:

Symbol: IBM. Last quote was 206.65 with a volume of 1673640 and a P-E Ratio of 14.18

- “Symbol: ”
- “. Last quote was “
- “with a volume of “
- “and a P- E Ratio of “

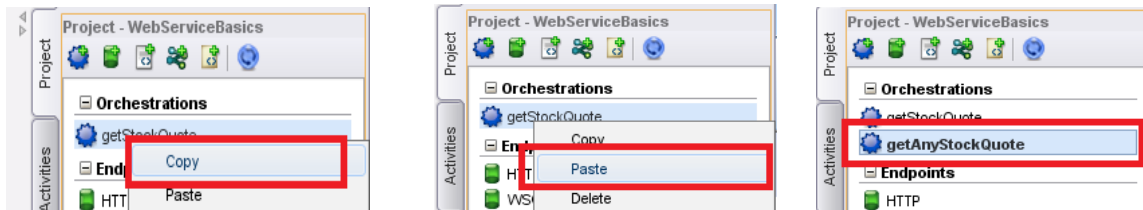
Note: Do not include the double quote characters in the **Value** entries.



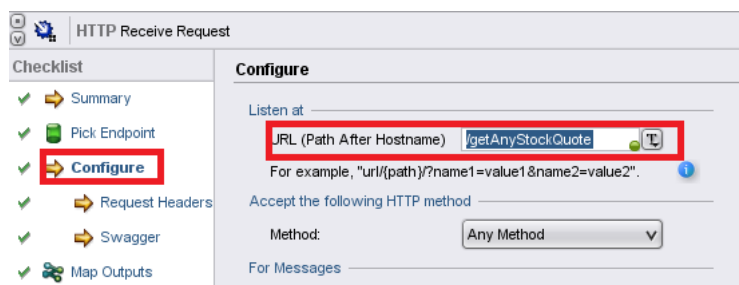
- __20. Right-click on the **Concatenate** function and **Apply Function Graph**.
- __21. Publish, deploy and test using the Studio Post Utility or Postman. Remember to POST using the dataport of your runtime environment and using the URI as defined in the HTTP Post activity.

5.6 Invoke WebService with Input

- __19. Use the previous WebService Lab as an example for extracting the ticker symbol from user input.
- __20. Copy/Paste the existing getStockQuote orchestration and create a new one, getAnyStockQuote.

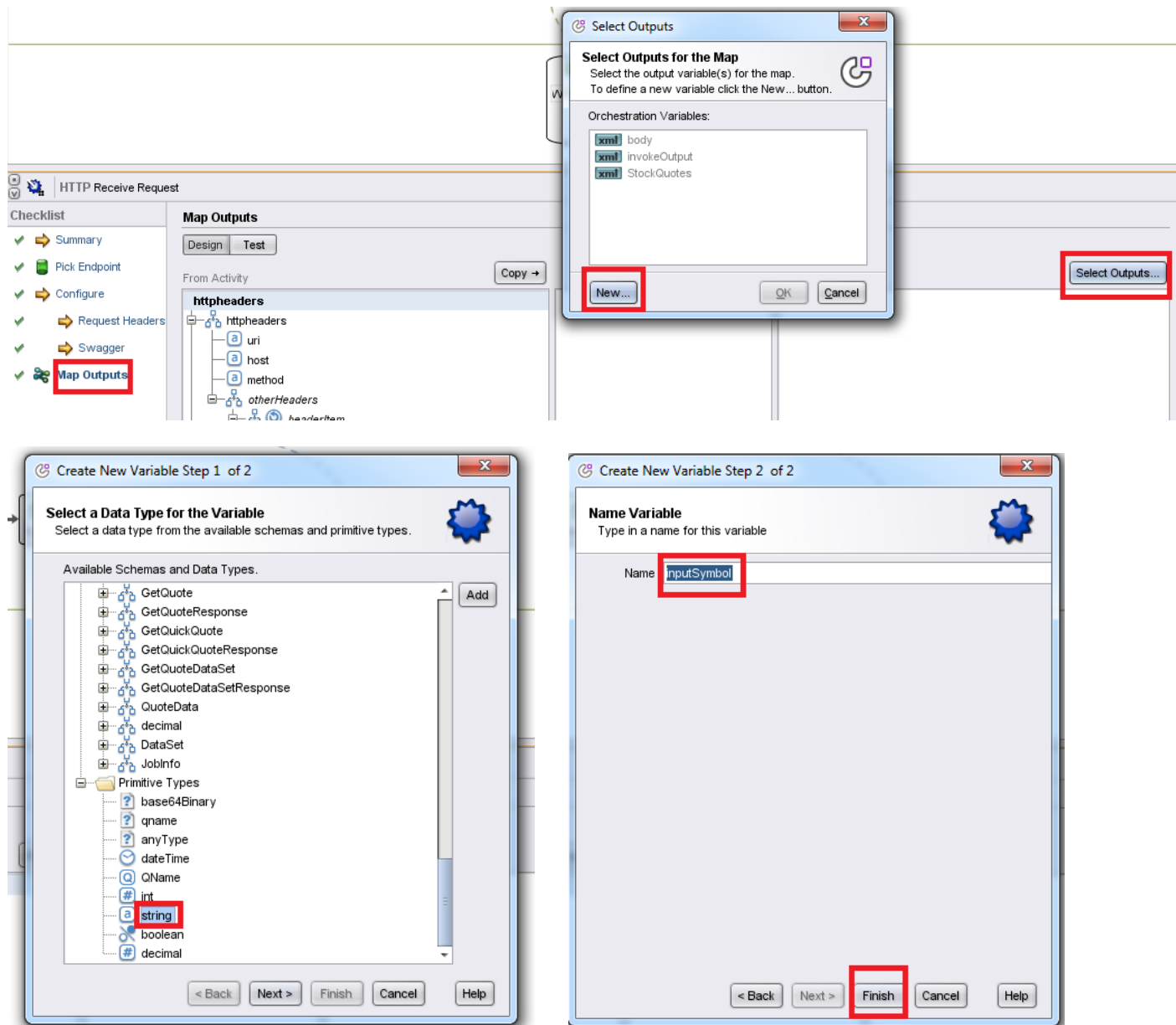


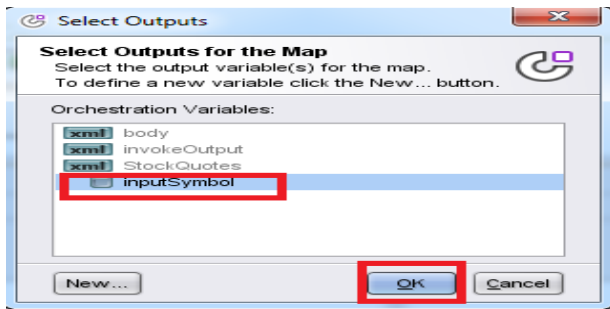
- __21. Change the URL for the **HTTP Receive** activity to `getAnyStockQuote`.



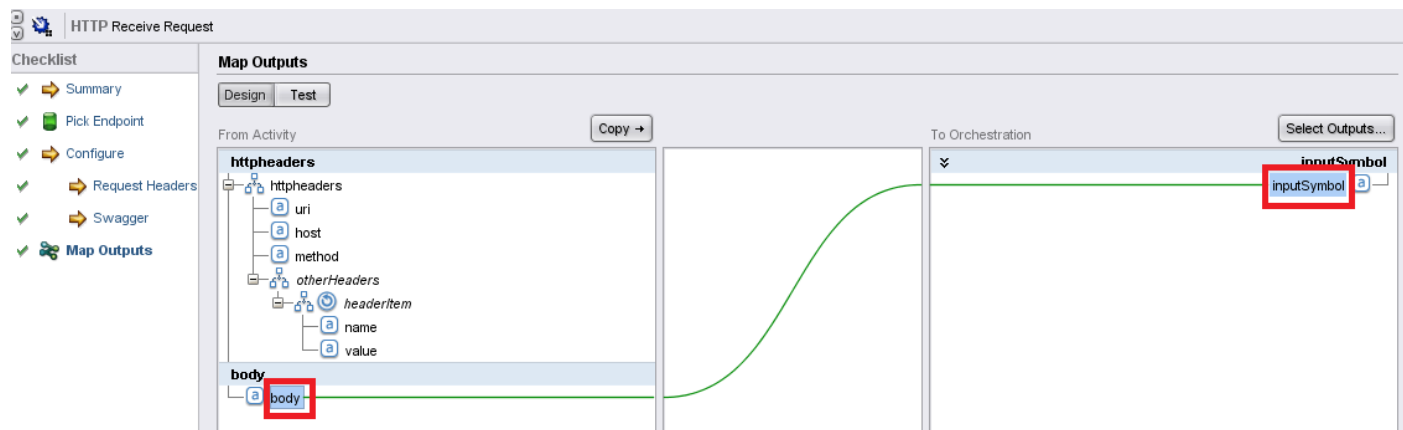
22. In the HTTP Receive activity, extract the symbol value from user input and store it in an orchestration variable `inputSymbol` defined as string.

Map Outputs: Select Outputs, New, string, `inputSymbol`, Finish then select `inputSymbol` & OK.

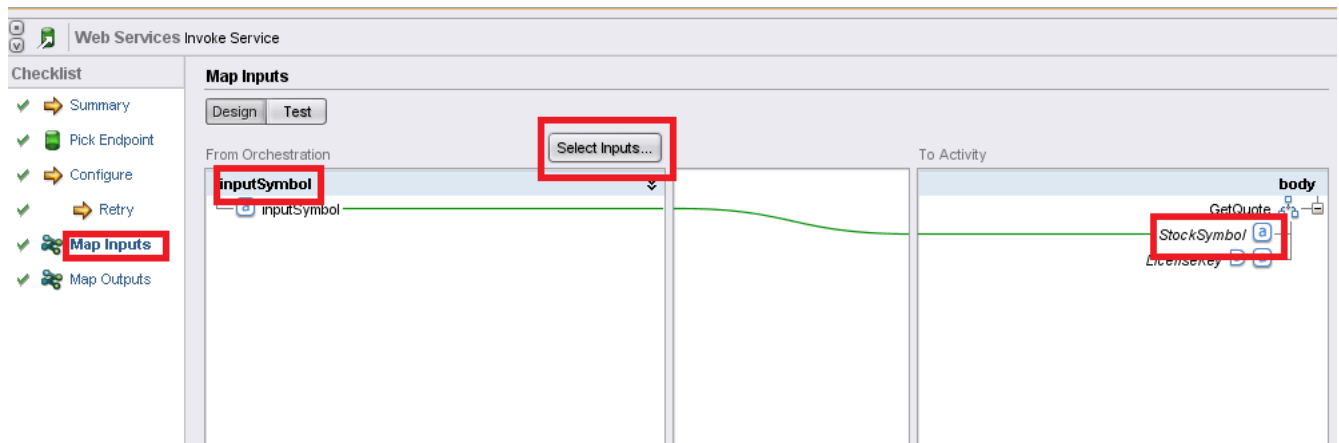




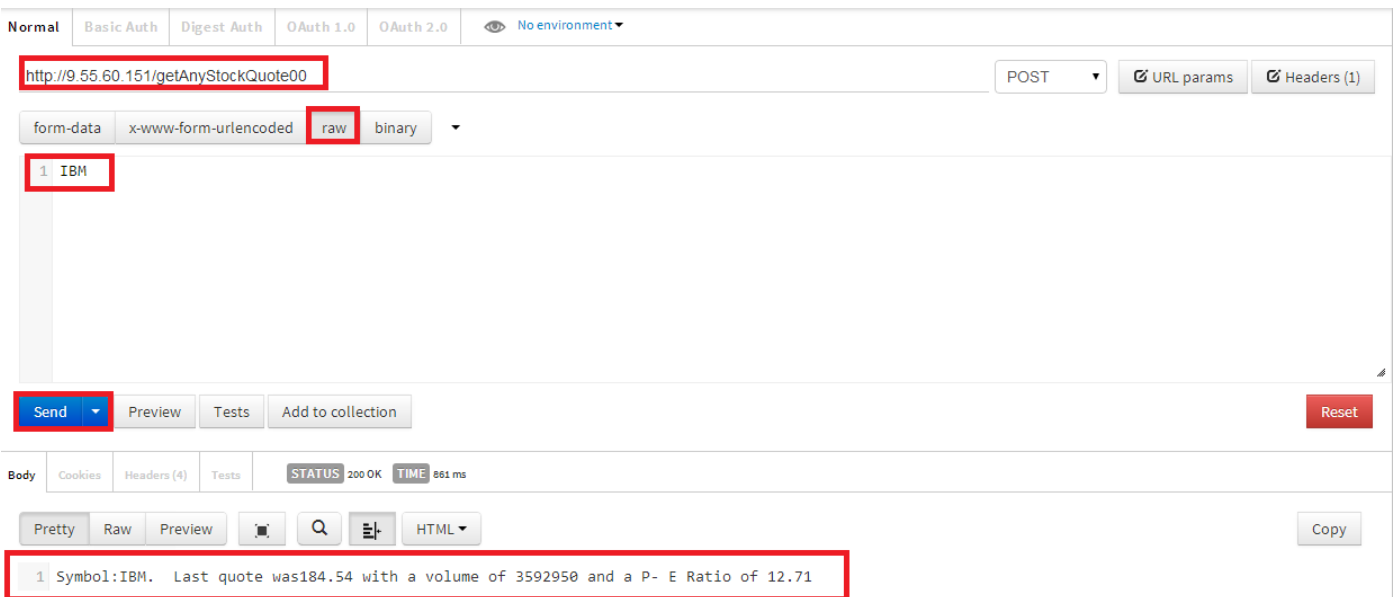
Map **body** to **inputSymbol**.



- __23. Go to the Invoke Service Activity/Map Inputs. Right click on StockSymbol and remove the default value of 'IBM'. Click on Select Inputs and choose 'inputSymbol'. Map 'inputSymbol' to 'StockSymbol'.



- __24. Save and Deploy the Project To your Appliance.
- __25. Test by opening the Postman utility in Chrome as in previous exercises. Use the Dataport of your appliance as the hostname. The resource URI is the /getAnyStockQuote you had set up in your HTTP Receive starter activity. Use the raw data setting and type in the Ticker Id of the stock you wish to look up. Click send when ready.



This completes Lab #5

Lab 6 Interacting with RESTful services and Salesforce

6.1 Overview

With the massive proliferation of mobile applications, JSON over REST is finding heavy use in the industry. This lab is built around illustrating IBM App Connect Professional's ability to expose and work with REST Based services as part of an infrastructure that integrates seamlessly with Software as a Service, in this case it is with the Force.com API exposed by Salesforce.com. A new project will be built that will itself be exposed as a RESTful service, able to be consumed as an incoming request with a parameter as part of the URI, call a back end REST Service and then take the output of that request, format it such that it can be then upserted into an object residing in Salesforce.com and provide a response to the original requestor.

6.2 Concepts

RESTful services are different from SOAP services in the regard that there is no underlying WSDL metadata that describes the actual data payload. REST is a preferred protocol to work with by developers because of its lightweight footprint and its ability to easily work with parameterized data as part of the URL. App Connect Professional makes it very easy to work with RESTful services that are now becoming part of many companies' cloud and mobile strategies.

6.3 Activities Used

- HTTP Receive Request
- HTTP Get Request
- HTTP Send Response
- IF/Then
- Read JSON
- Upsert Objects (Salesforce)

6.4 Document Keys

Menu bar: *Edit* → *Preferences*

Studio Artifact: **HTTP Receive Request**

Entered Text: 'Hello Receive Request'

Project Object Name: *SampleProject1*

Context Menu Item: *Define Default Value*

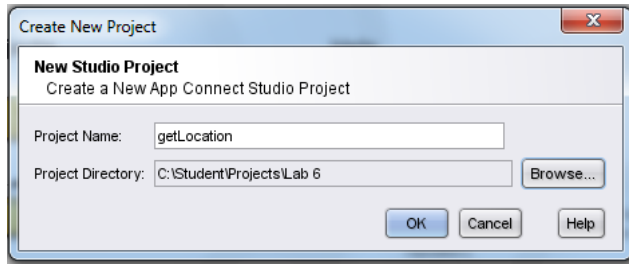
Activity Checklist Step: *Pick Endpoint*

Orchestration Variable: *Account*

Function Name: *Mathematical / Sum*

6.5 Expose an Orchestration as a RESTful service that invokes a second REST service and synchronizes data into Salesforce.com

- __1. Create a new Project. Call this project `getLocation`. Save this project to the following location – `c:\Student\Projects\Lab 6`



- __2. Rename the orchestration to `getAndSyncBranch`

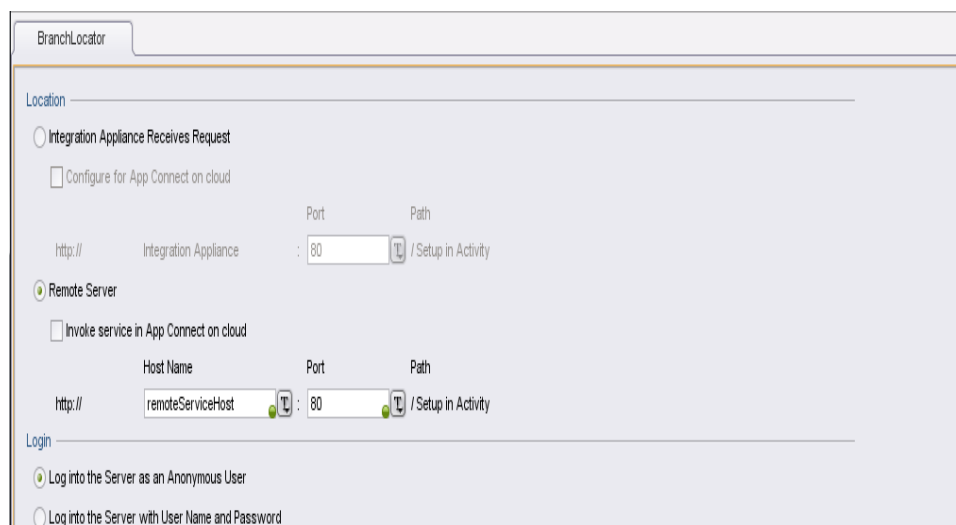


- __3. We will need to create three configuration properties.

- __a. `remoteServiceHost` - Hard code this value to 'banka.mybluemix.net'
- __b. `salesforce.user` - Set this to your salesforce.com user used in previous labs
- __c. `salesforce.pw` – Set this to your salesforce.com password+token id used in previous labs

- __4. This Project will contain three endpoints.

- __a. The first endpoint is an HTTP Endpoint. Call it "`BranchLocator`". Set this up as a "remote" server call. Set the hostname to the `remoteServiceHost` configuration property. Use the default port at Port 80. Ensure the Login section is set to the default "Log into Server as an Anonymous User".



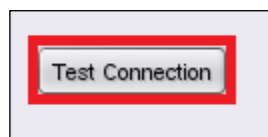
- __b. The second endpoint is the default HTTP Receive Endpoint. Leave the default name of “HTTP”.

The screenshot shows the 'HTTP' tab in a configuration window. Under the 'Location' section, the 'Integration Appliance Receives Request' radio button is selected. Below it, there are fields for 'Port' (set to 80) and 'Path' (set to /Setup in Activity). The 'Remote Server' radio button is also visible but not selected. Under the 'Login' section, the 'Log into the Server as an Anonymous User' radio button is selected.

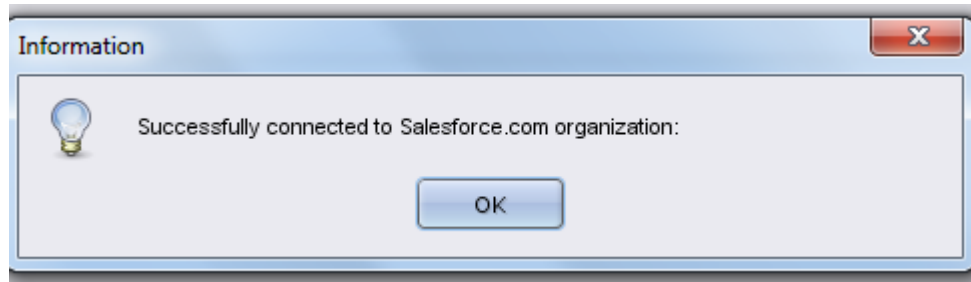
- __c. The 3rd Endpoint is Salesforce.com. Leave the default name of “Salesforce” for this endpoint. Use the **salesforce.user** and **salesforce.pw** configuration properties in the endpoint configuration as depicted below as done in previous exercises.

The screenshot shows the 'Salesforce' tab in a configuration window. Under the 'Salesforce.com Customer Login' section, the 'User Name' field is set to 'salesforce.user' and the 'Password' field is set to 'salesforce.pw'. Under the 'Login Options' section, the 'Login normally' radio button is selected. The 'Login URL' field is set to 'https://login.salesforce.com/services/Soap/u/33.0'. Under the 'Connection Timeout' section, the 'Time out after' field is set to 300 seconds. Under the 'Proxy' section, the 'Connect via a Proxy Server' checkbox is unchecked. At the bottom, there is a 'Test Connection' button.

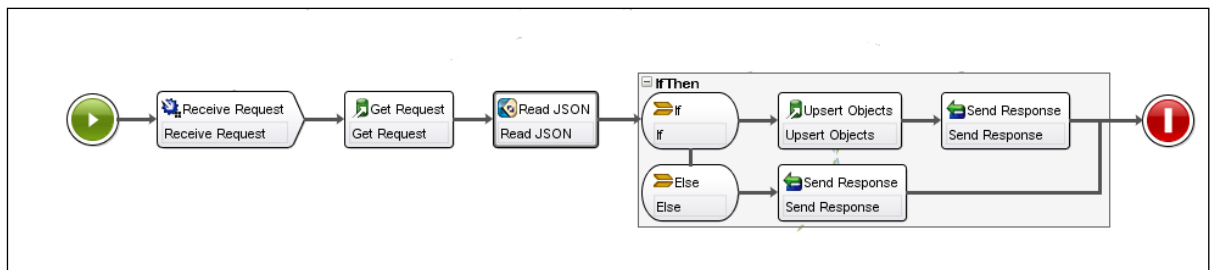
- __d. Test the Salesforce Endpoint by clicking the “Test Connection” button



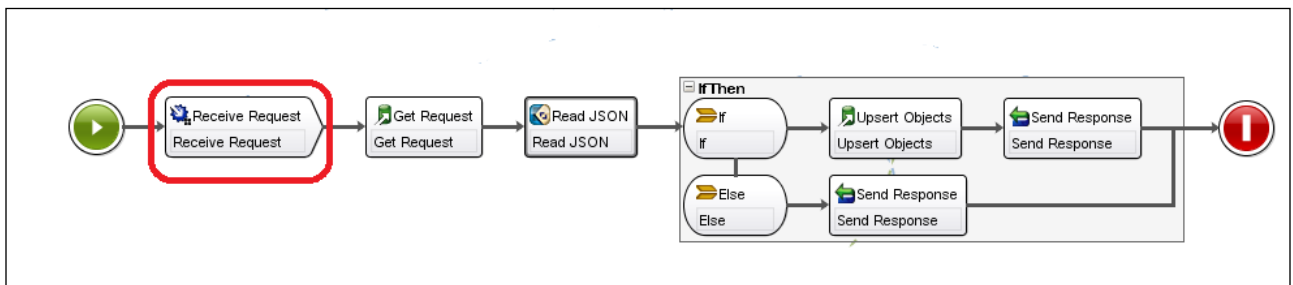
- __e. If successful, you will see the following response in Studio



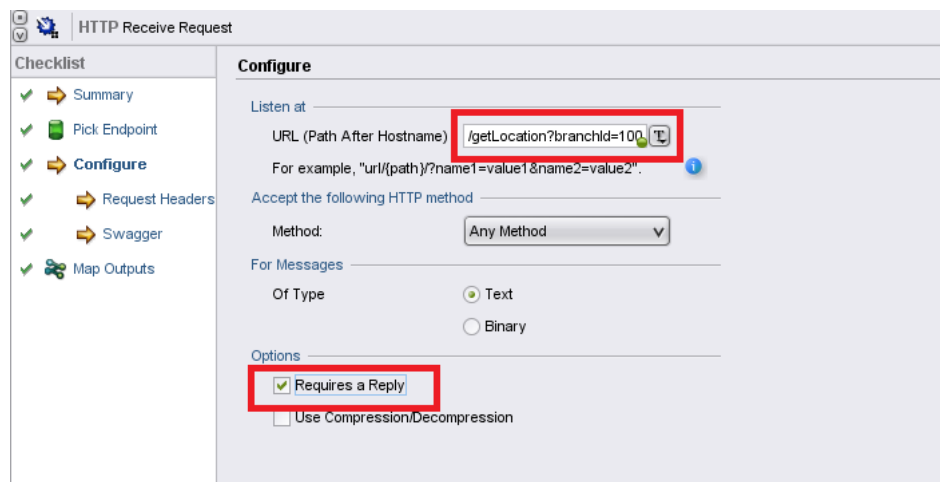
- __5. Drag and drop the following activities on to the getAndSyncBranch orchestration canvas in the following order:
- __a. **HTTP Receive Request**
 - __b. **HTTP Get Request**
 - __c. **Read JSON**
 - __d. **IF/Then Block.**
 - __e. Within the **If/Then** Block – for the top path drag in a Salesforce **Upsert Objects** and then a **HTTP Send Response**
 - __f. On the bottom (else) path of the **If/Then** – drag in a second **HTTP Send Response**.
 - __g. Confirm your getAndSyncBranch orchestration per the screen shot below



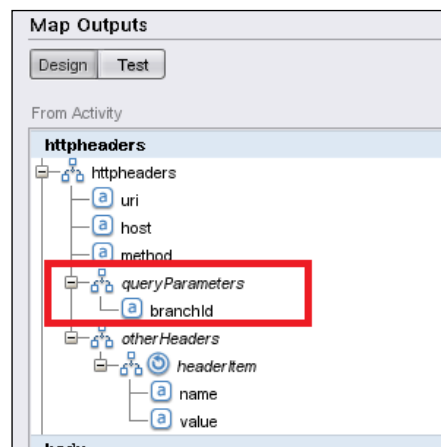
- __6. Configure the Activities within the Orchestration
- __a. Start with the **HTTP Receive Request** activity. Click on the activity to start the configuration



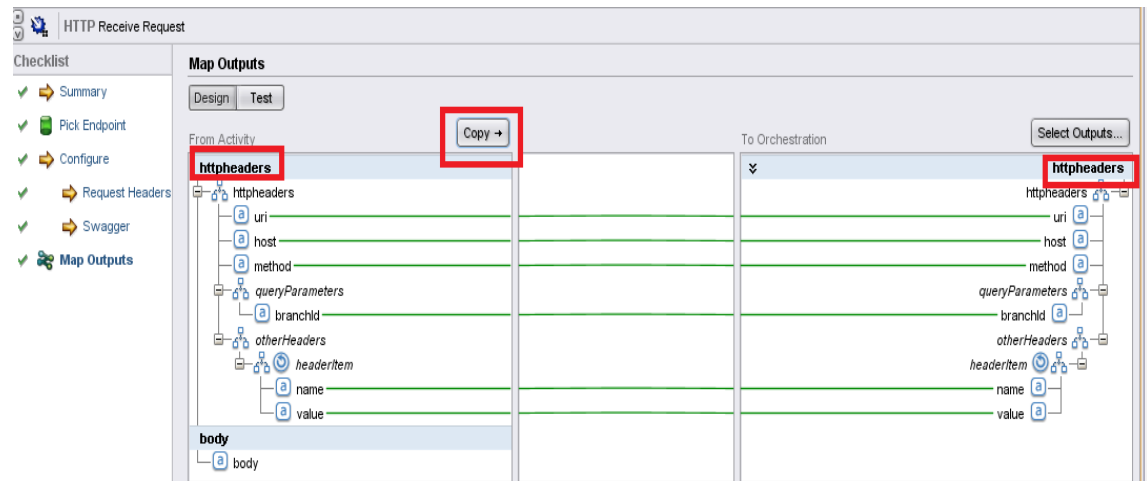
- __b. Complete **HTTP Receive Request** activity checklist.
- __i. Under the “**Pick Endpoint**” step of the checklist, select the HTTP endpoint. This represents the CI Runtime environment as the hosting endpoint for the Orchestration.
 - __ii. On the “**Configure**” Step of the checklist. Configure the step as follows. Use the screenshot below step iv below as your guide.
 - __iii. Set the URL to this value: `/getLocation?branchId=100`. This does not mean we are hardcoding the branchId value in the URL, rather we are creating a variable called “branchId” that will allow the invoker of this orchestration to pass in variable data as part of the URL that we can use in the Orchestration further downstream.
 - __iv. Ensure the “Requires a Reply” checkbox is checked.



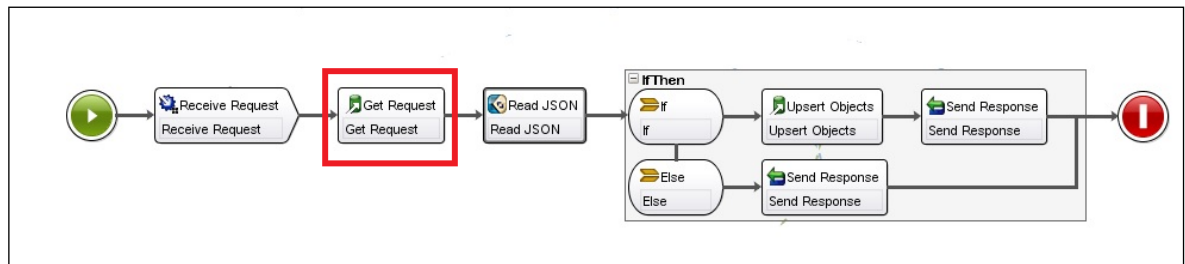
- __v. Go to the **Map Outputs** step. You will note that the branchId variable is set up under the queryParameters object. This allows us to easily capture and map the query parameters passed to the incoming request.



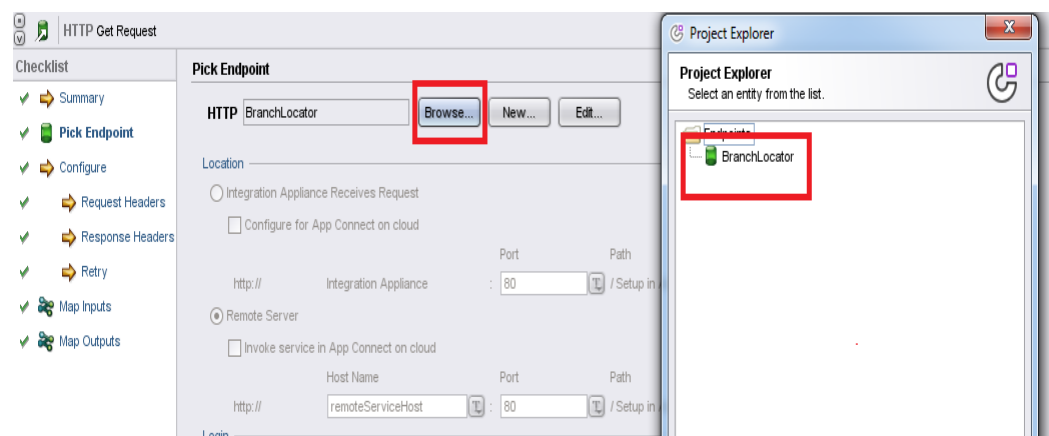
- __vi. You can now map the entire contents of the httpheaders variable over by using the Copy button.



- __c. Complete **HTTP Get** activity checklist. Click on the **HTTP Get** activity and proceed to the checklist



- __i. On the [Pick Endpoint](#) step, select the BranchLocator HTTP endpoint from the list by clicking Browse



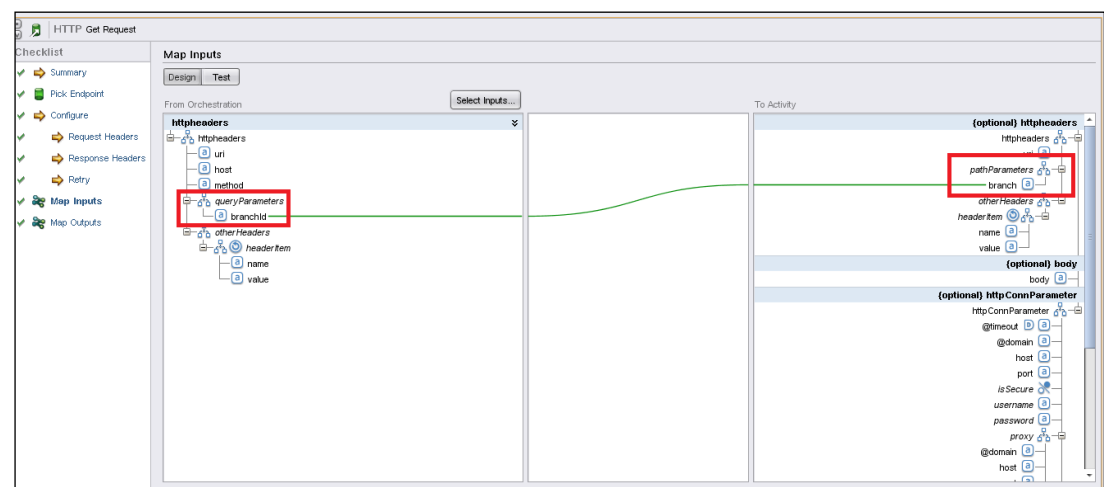
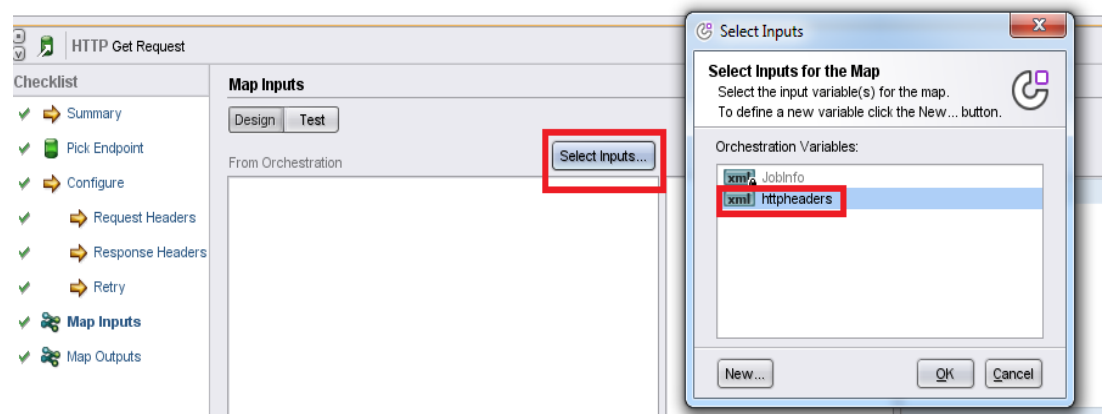
- __ii. On the [Configure](#) step, use the following as the URL for the HTTP Get:
`/branches/v1/{branch}/location`



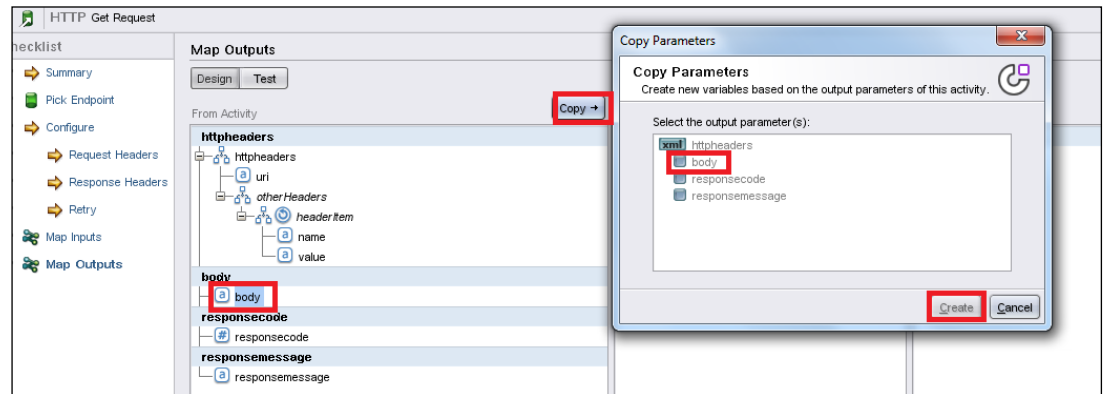
The {} notation in the URL represents a variable component of the URL of the HTTP Get. This will allow us to be able to map in a parameter. See the Infocenter for more information about this:

http://pic.dhe.ibm.com/infocenter/wci/v7r0m0/topic/com.ibm.wci.doc/http_get_request_activity.html

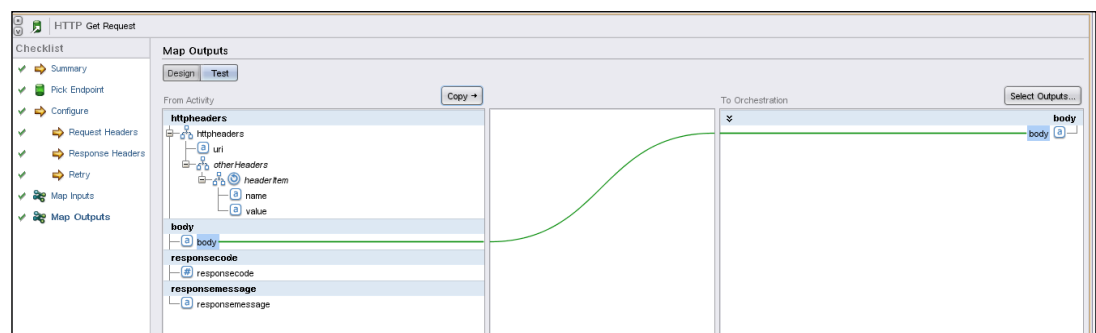
- ___iii. Accept the defaults for the Request/Response Headers as well as the Retry steps of the checklist.
- ___iv. On the Map Inputs step, on the “From Orchestration” on the left hand side, first click on Select Inputs and choose “httpheaders”. Then drag and drop the branchId variable from the queryParameters to the branch variable under pathParameters on the “To Activity”.



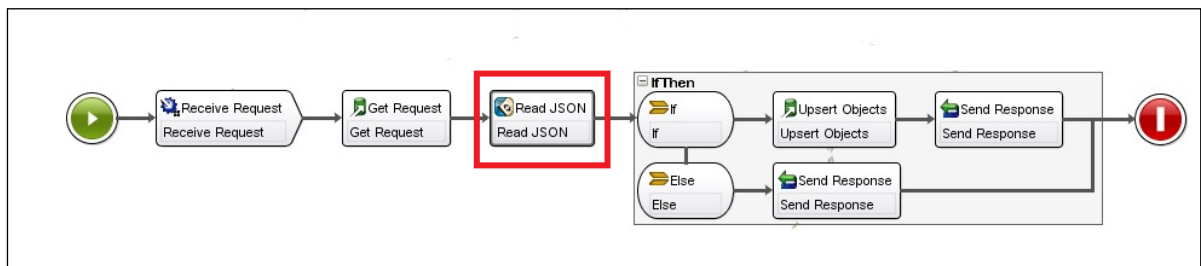
- ___v. On the **Map Outputs** step, copy the “body” object to a variable, by selecting the “Copy” button, then on the “Copy Parameters” pop-up window, select “body” and click “Create”



__vi. When complete, your map outputs step will look like the following:

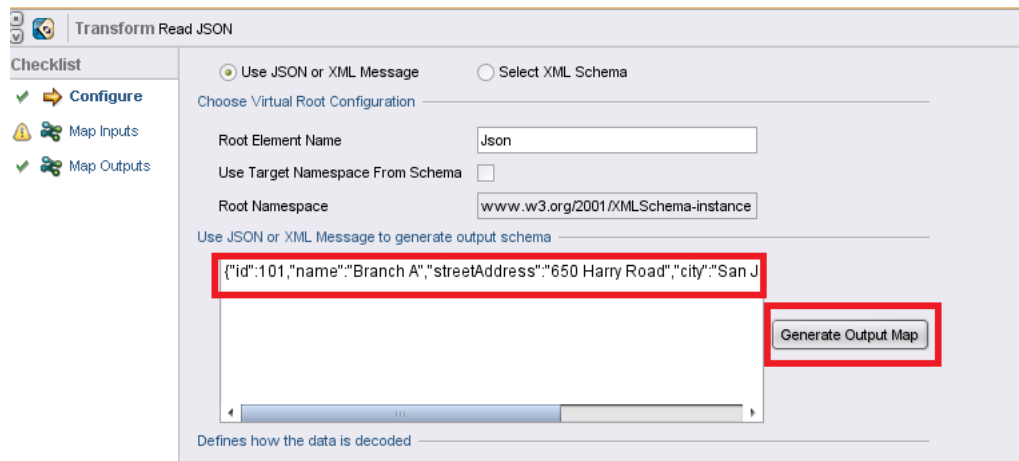


__d. Complete the **Read JSON** activity. Click on the activity and then start with the checklist.



__i. Start with the **Configure** step in the checklist. Copy and paste the following Sample JSON into the lower whitespace of the activity. This data is to be used to reflect the output from the BranchLocator service. Then click the Generate Output Map button.

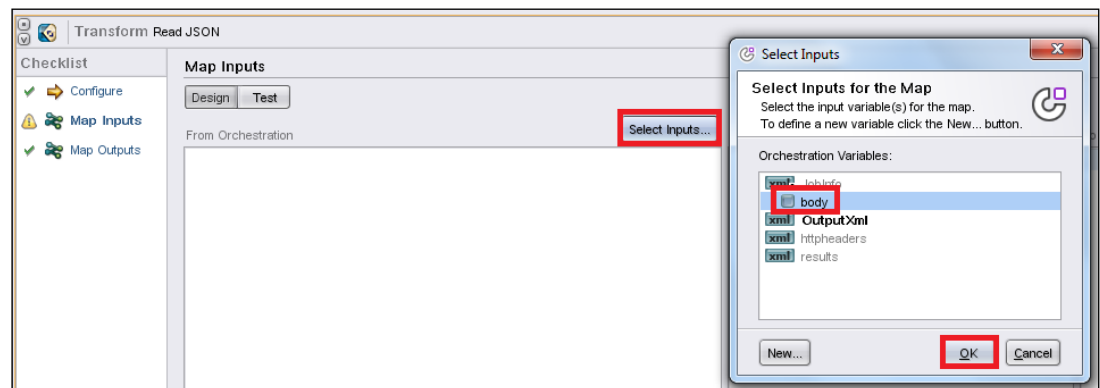
```
{
  "id": "101",
  "name": "Branch A",
  "streetAddress": "650 Harry Road",
  "city": "San Jose",
  "state": "CA",
  "zipCode": "95120-6099"
}
```



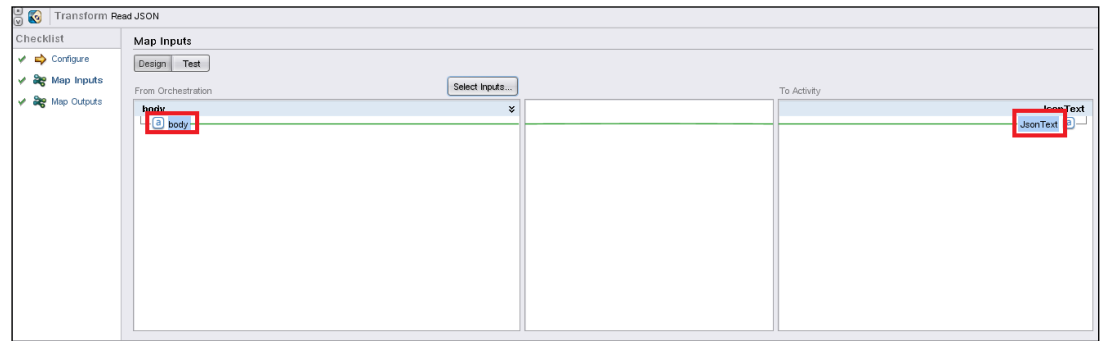
Alternatively, you could get the JSON data sample by running the service manually in a browser by going to this URL:

<http://banka.mybluemix.net/branches/v1/101/location>

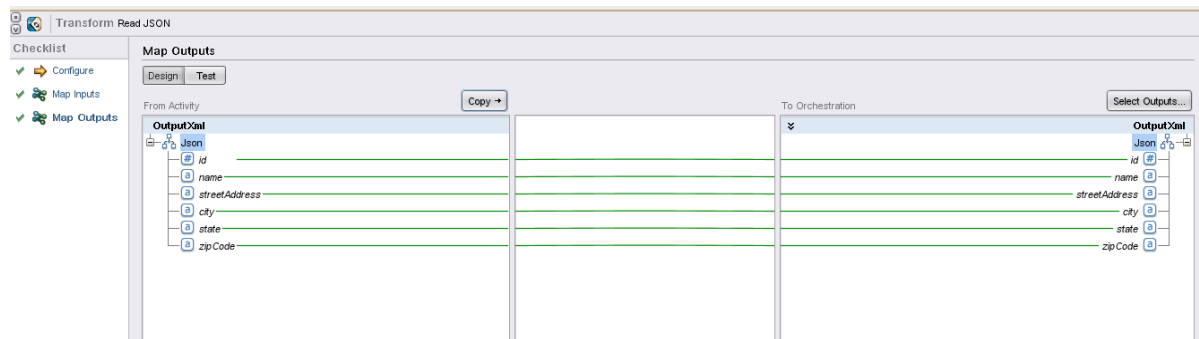
- __ii. Moving to the **Map Inputs** step. Add the body variable by clicking on the Select Inputs button, highlight the body variable, then click Ok.



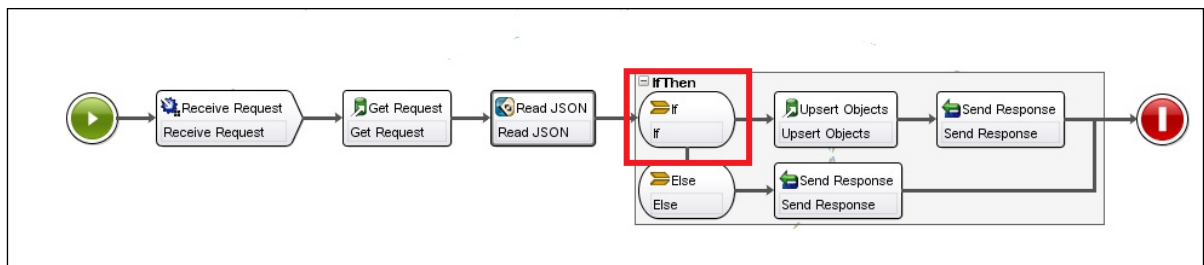
- __iii. Drag and drop the body variable on the “From Orchestration” (left) side over to the JsonText variable on the “To Activity” (right) side.



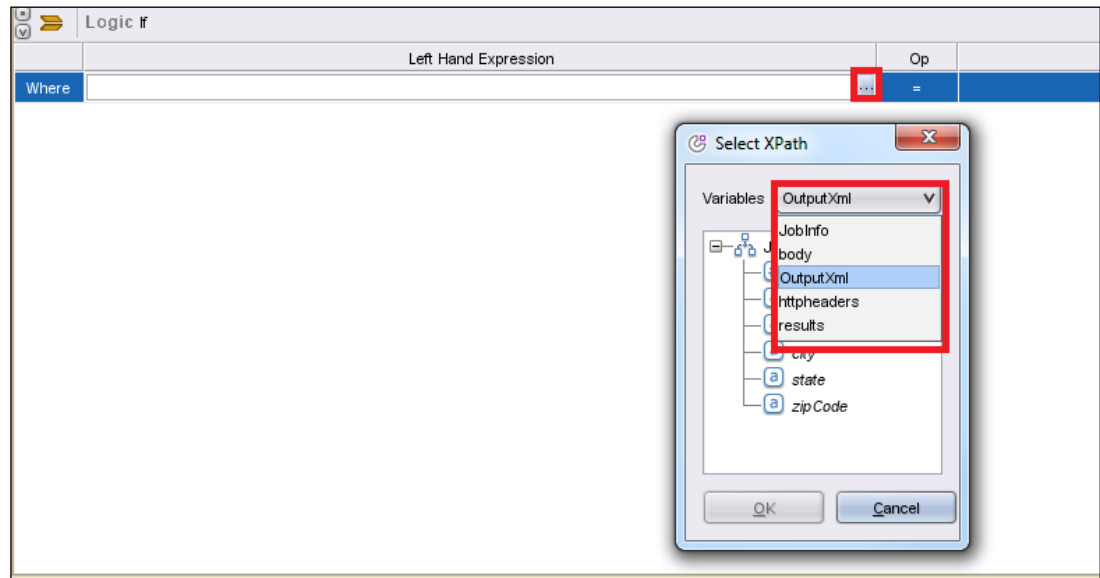
- __iv. Click on the Map Outputs step. Click the “Copy” Button and then select the OutputXML variable and then click Create. This will map the variable that represents the parsed JSON data.



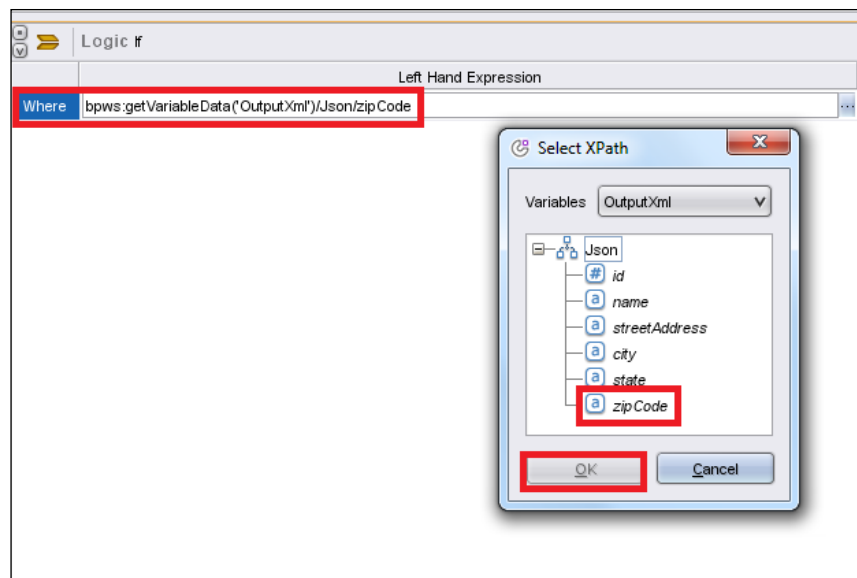
- __e. Configure the **IfThen** Activity. Click on the activity and then proceed to configure.



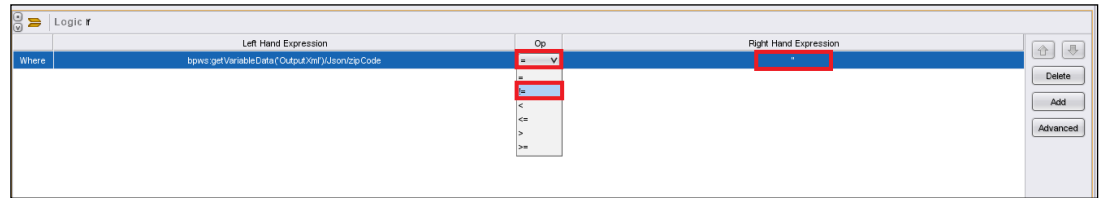
- ___i. Just the right of the Left Hand Expression whitespace, click on the box with the three dots on it. This will then bring up the Select XPath window where you can select the variable you wish to evaluate. Select the OutputXML variable.



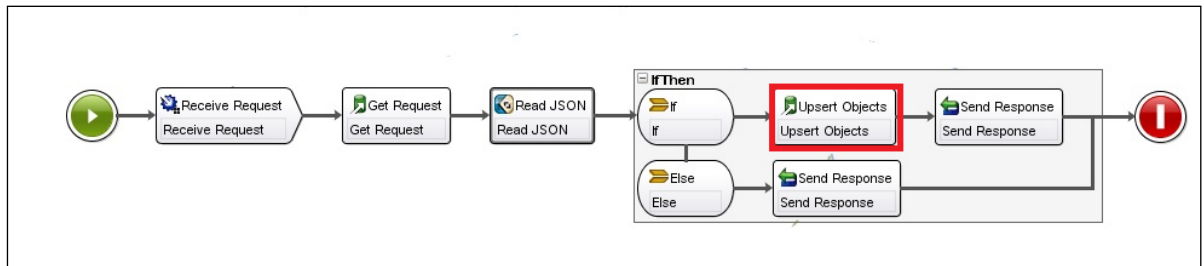
- ___ii. Select the ZipCode field and then click OK. This will automatically populate the Left Hand Expression with the requisite xpath.



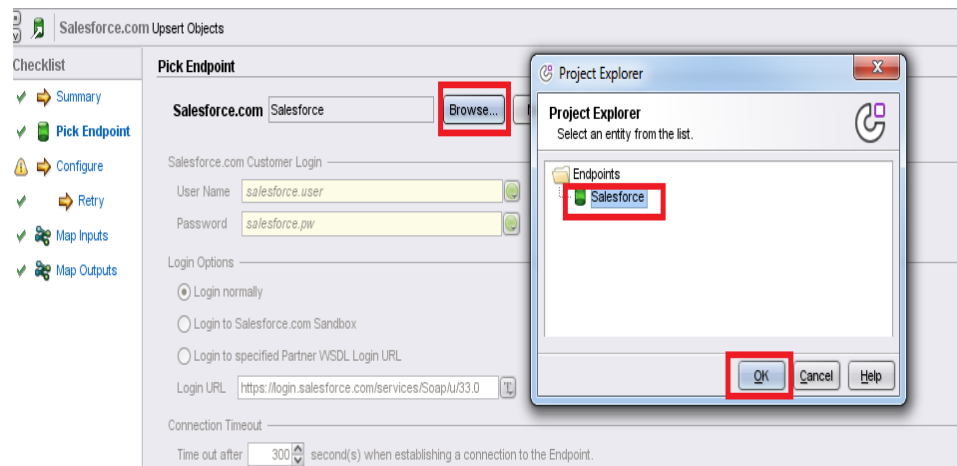
- ___iii. Set up your Expression next where you will be evaluating the ZipCode field to be Present or Not Equal to None. This is done by selecting the Not Equals operation (!=) from the Op drop down. Next type in two single quotes (") into the Right Hand Expression.



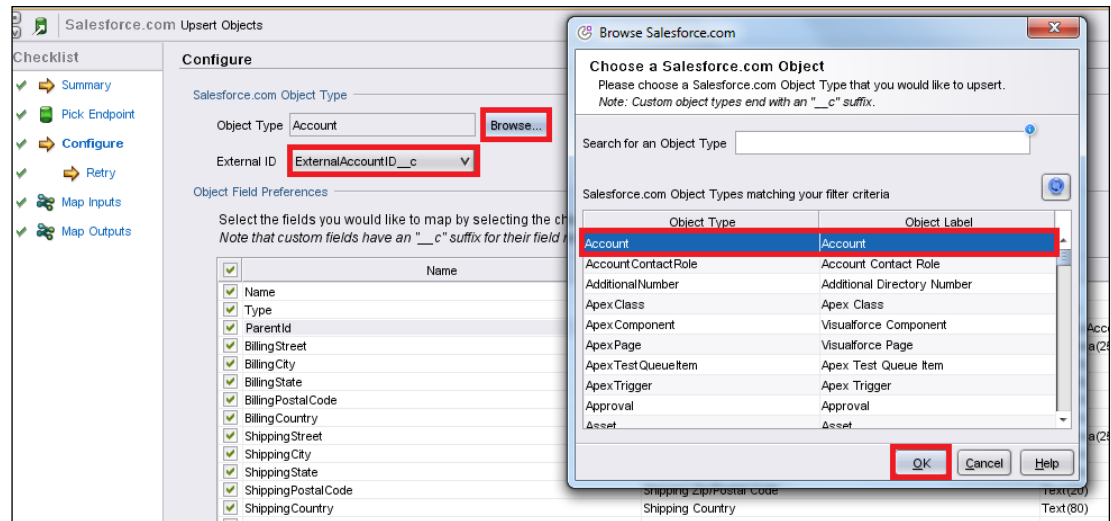
- ___f. Configure the **Upsert Objects** Activity. Click on the activity and then proceed to configure.



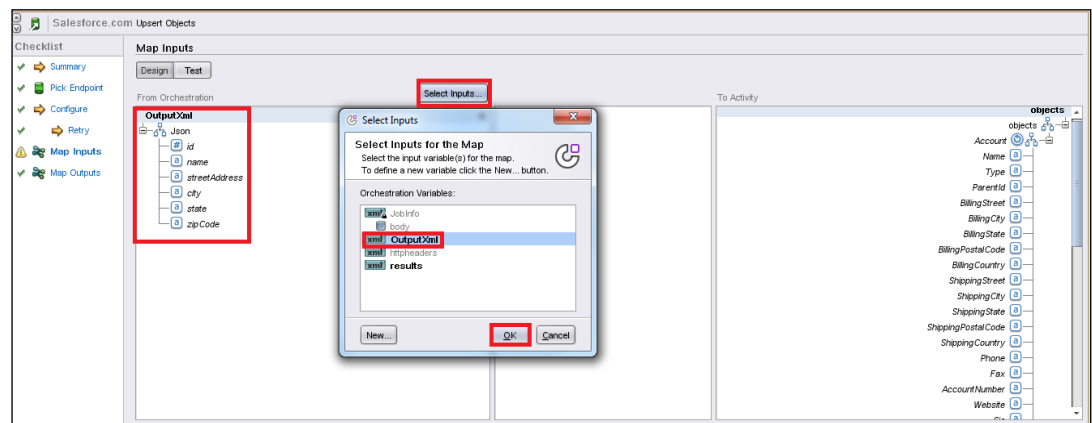
- ___i. Configure the **Pick Endpoint** step by clicking the Browse button and selecting the Salesforce endpoint from the list. Click Ok to complete the step.



- ___i. Go to the **Configure** step in the checklist and then click the Browse button. This will allow Studio to connect to Salesforce to get a list of your objects. Select the Account object as indicated below and then click Ok. This will bring back the metadata for your Account, including the fields and their respective properties. Next ensure that the External ID is selected to the ExternalAccountID__c custom field.



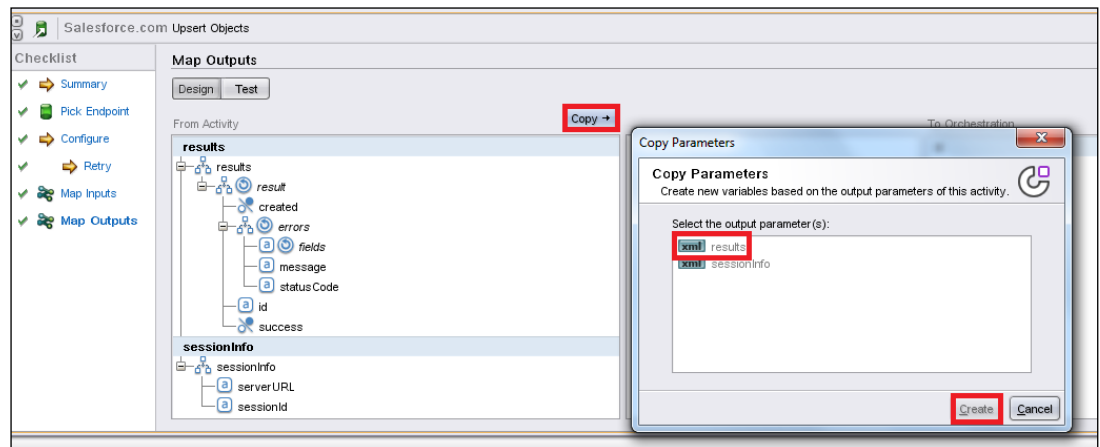
- ___ii. Go to the **Map Inputs** Step. Click on the Select Inputs button and then select the OutputXML variable from the list. Click OK. The OutputXML then is now the input data to be mapped into Salesforce.



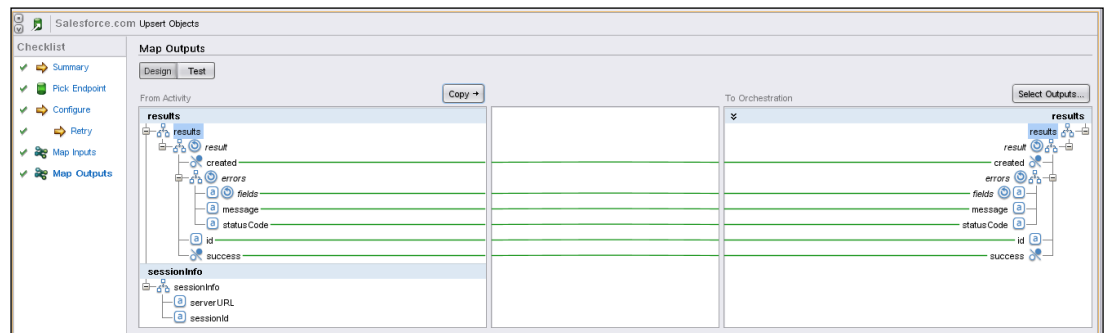
- ___iii. We can now start to map fields from the input. Use the following Mapping Guide below

Input Field	Output Field	Transformation Rules
Id	ExternalAccountID__c	Direct mapping, no transformation
Name	Name	Direct mapping, no transformation
streetAddress	BillingStreet	Direct mapping, no transformation
City	BillingCity	Direct mapping, no transformation
State	BillingState	Direct mapping, no transformation
ZipCode	BillingPostalCode	Direct mapping, no transformation

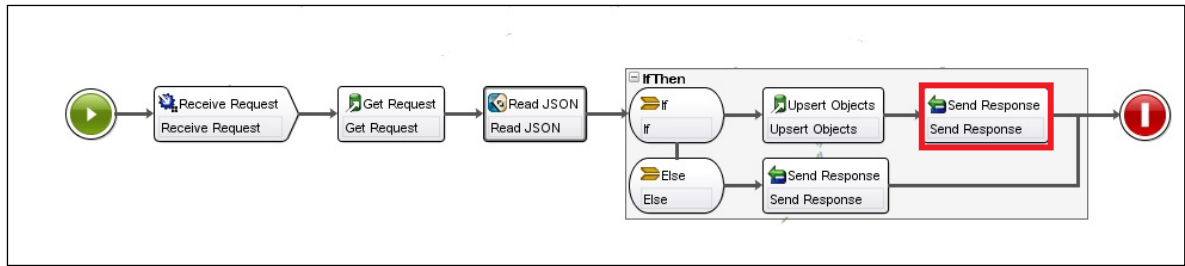
- ___iv. Go to the [Map Outputs](#) step. Here we will map the result set returned from Salesforce.com after the upsert, and then proceed to map the output to a variable. Click on Copy, then in the Copy Parameters pop up, select the results variable, and click Create.



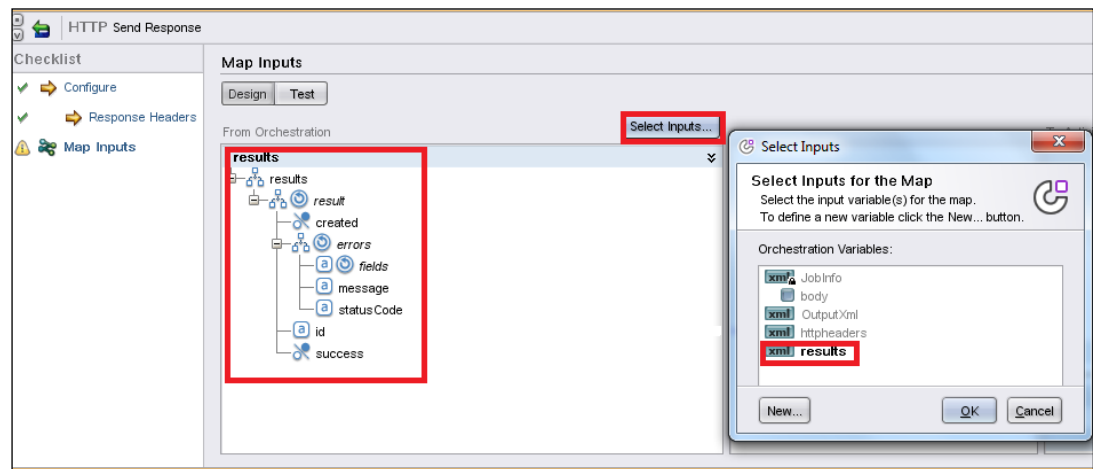
- ___v. Once it is complete, your [Map Outputs](#) step should look as such:



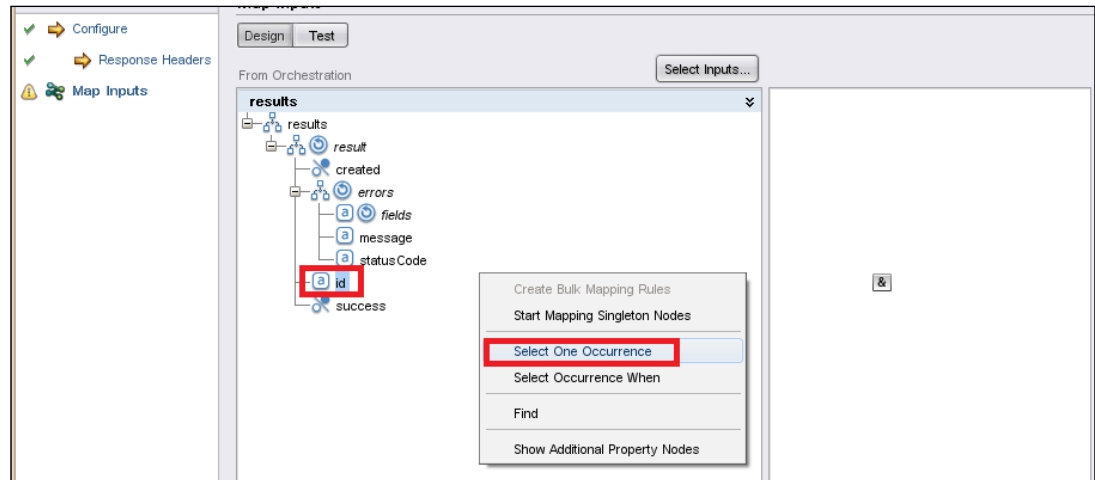
- __g. Configure the **Send Response** Activity for the top branch of the **IfThen**. Click on the activity and then proceed to configure.



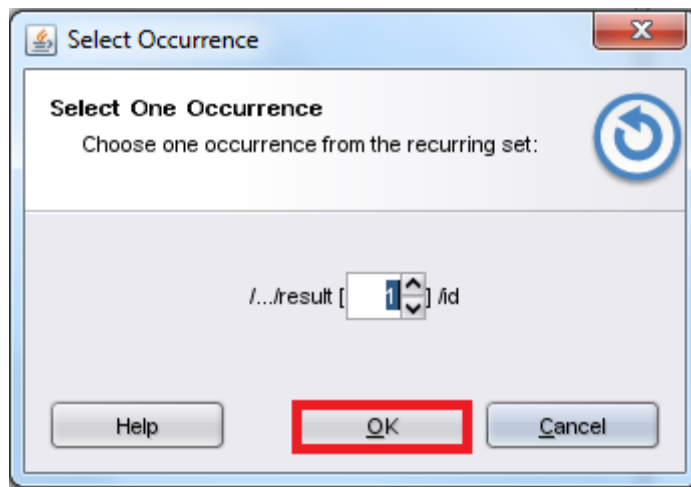
- __i. In the **Map Inputs** Step, click the **Select Inputs** button and in the pop-up window, select the results variable. Click OK and the results variable with its elements will appear on the left hand side of the window.



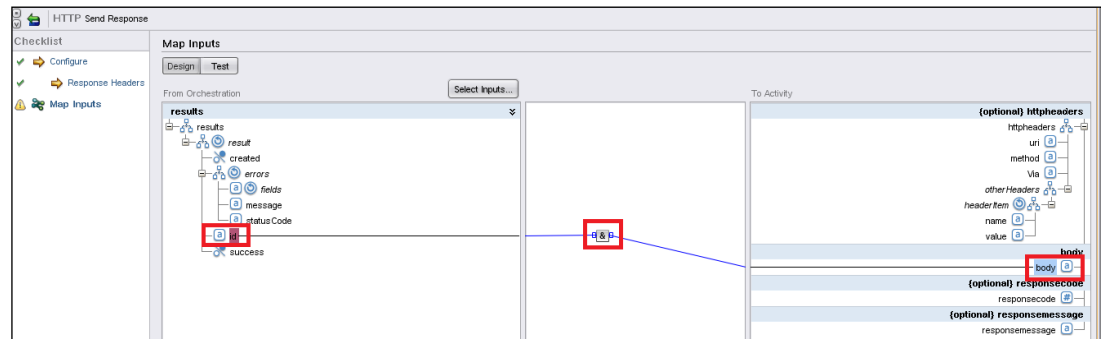
- __ii. Next, click on the Functions tab on the right on side of the workspace. Drag and drop the *Concatenate* function from String Functions folder on to the middle mapping pane. In order to map over the id value from the results variable, we will need to select a specific occurrence of that variable since the result object is a repeating structure. To do so, right click on the id variable and click the “Select One Occurrence” option.



- __iii. When the next pop-up comes up. Click the OK button to finish the selection



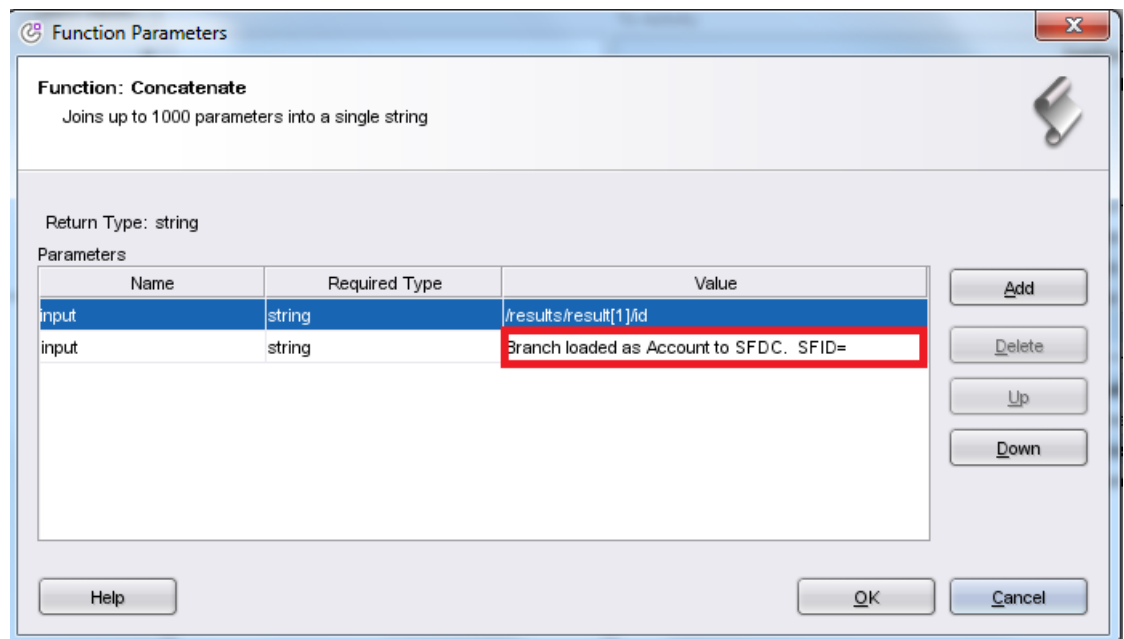
- ___iv. Once that is complete you can then drag and drop the id field into the *concatenate* function, and then click and drag the concatenate to the body field on the output side.



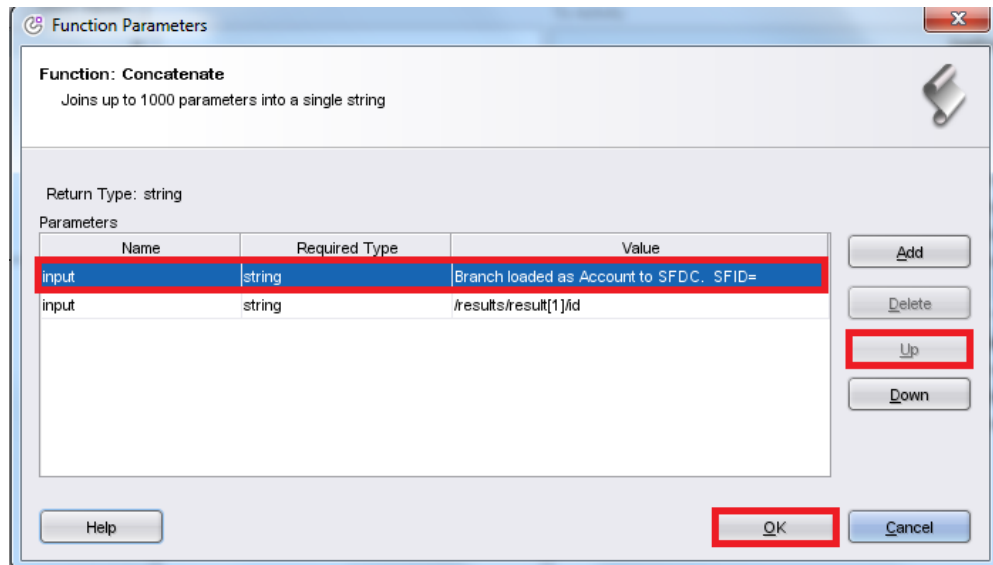
- ___v. Next, double click on the concatenate function that was just placed on the middle mapping plane in the step above to configure it. You will note the id field will be in the concatenate list, as well as an empty field that will be concatenated together along with the id. Type in the following literal value into the field

Branch loaded as Account to SFDC. SFID=

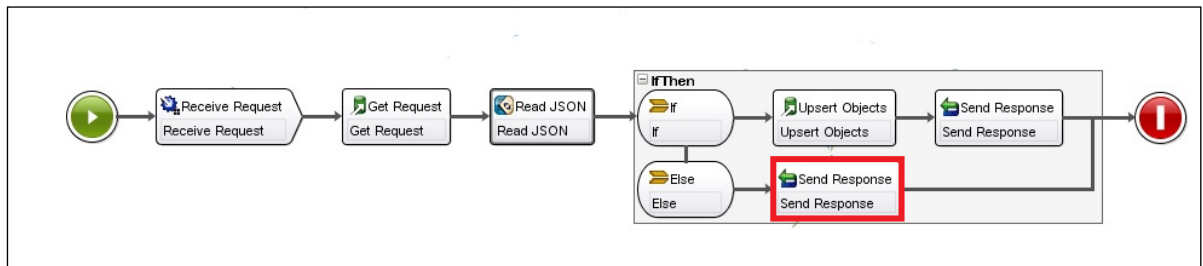
When complete, the function should look as such:



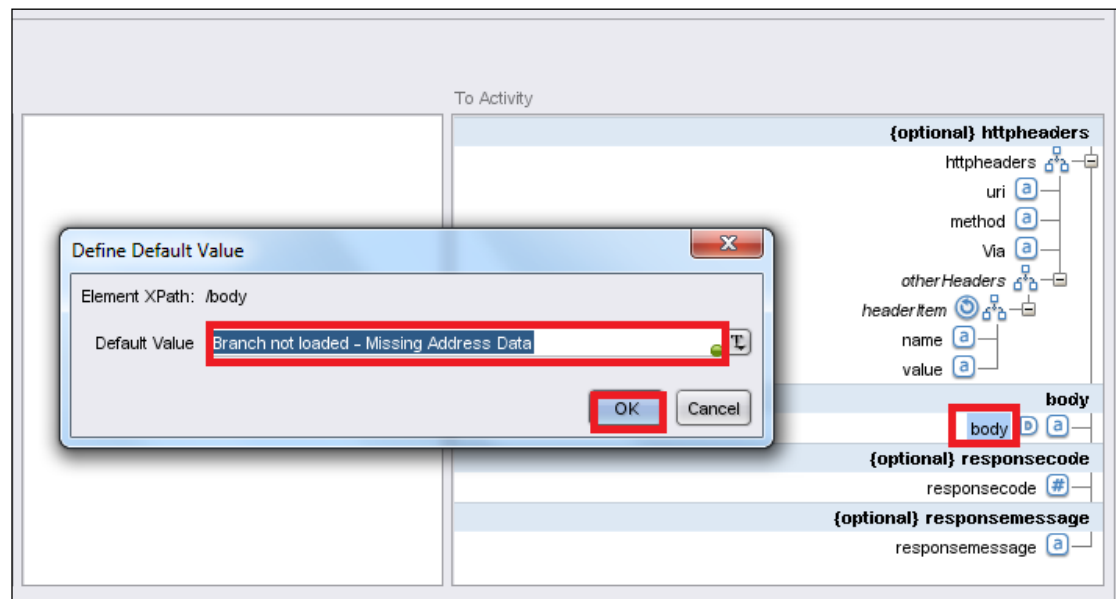
- __vi. To reorder the field listing in the *concatenate* function, we will have to select the field we just entered, and then move it up to the top by clicking the “Up” button such that it is situated ahead of the id field. Once that is done, click “Ok”.



- __h. Configure the **Send Response** Activity for the bottom branch of the **IfThen**. This represents our response if the data retrieved from the BranchLocator service is missing the Zip Code and therefore is not valid. Click on the activity and then proceed to configure.



- __i. Configure the [Map inputs](#) step by providing a literal value for the body of this HTTP reply to reflect that the Branch information was not loaded. Use the literal value indicated in the screen shot. Click Ok when complete.



- __g. This completes the orchestration configuration. Deploy this project to your runtime environment as per previous exercises.
- __h. Run the Orchestration by directing a browser windows or HTTP Post utility to this URL:
- `http://<dataport of appliance>/getLocation?branchId=101`

- ___i. The output of the orchestration will look as such in the browser. Also is a screenshot of the output inside of Salesforce, in the Account object. Feel free to run additional tests. Valid values for this service are branchId's of 101, 102 and 103.

The screenshot shows a Salesforce interface for an Account record named 'Branch B'. The URL in the browser is `dataport/getLocation?branchId=102`. The page header indicates 'Branch loaded as Account to SFDC. SFID= 0016000000yqEF0AAU'. The account owner is Michael Alley. The account name is 'Branch B'. The external account ID is 102. The billing address is 11501 Burnet Road. The shipping address is also 11501 Burnet Road. The account is currently offline.

Account Detail	
Account Owner	Michael Alley (Change)
Account Name	Branch B (View Hierarchy)
Parent Account	
Account Number	
Account Site	
Type	
Industry	
Annual Revenue	
External Account ID	102
UpdateFlag	☐
Balance	
Customer Birthday	
CurrencyCd	
Billing Address	11501 Burnet Road
Shipping Address	

This completes Lab #6

Appendix A - Creating a Salesforce.com Developer Account

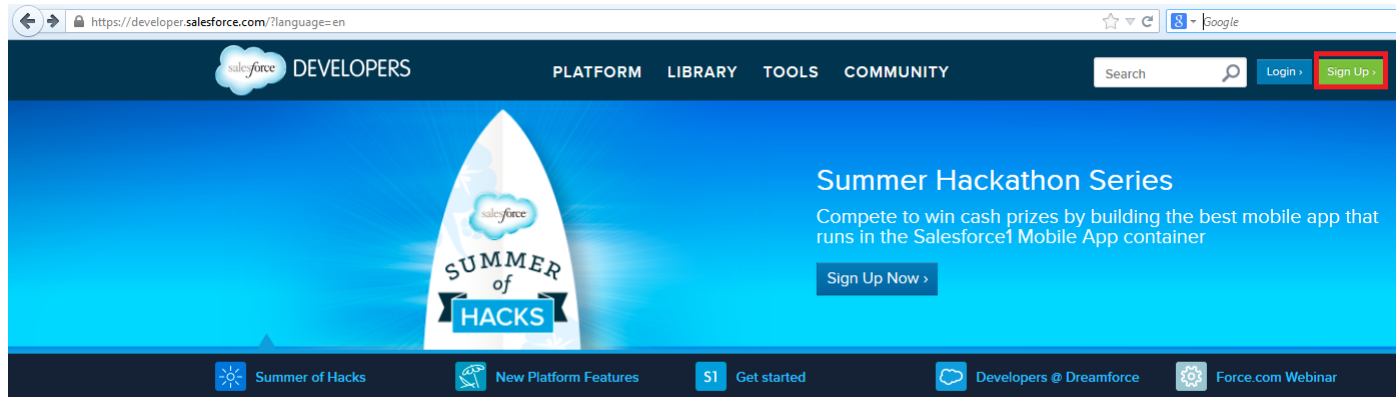
In order to complete the labs you will need a valid Salesforce.com username and password as you will be accessing a Salesforce.com account. As connectivity is provided through the Salesforce.com api you will also need to know the Salesforce.com security token. Do this first before starting the labs.

The following section will take you through Setting up an account, Activating the access to your Salesforce account and obtaining the security token. You should do this from within the Proof of Technology virtual machine and not from the native PC.

If you already have a developer account and know this information then you will still need to run through section to activate the laptop to access your Salesforce account.

Setting up an account

- __1. Using a web browser go to the URL: <http://developer.force.com/> and click 'Sign up'.



- __2. Fill out the form details and submit. You will need a valid email address and you will need to be able to access the email.
- __3. Sign into your email account.
- __4. You will have received an email from Salesforce.com. Access the one with the subject 'Salesforce.com login confirmation' and follow the link to confirm the account setup.

__5. Enter a password of your choice.

You now have access to the developer account. Verify the sample data using the following steps.

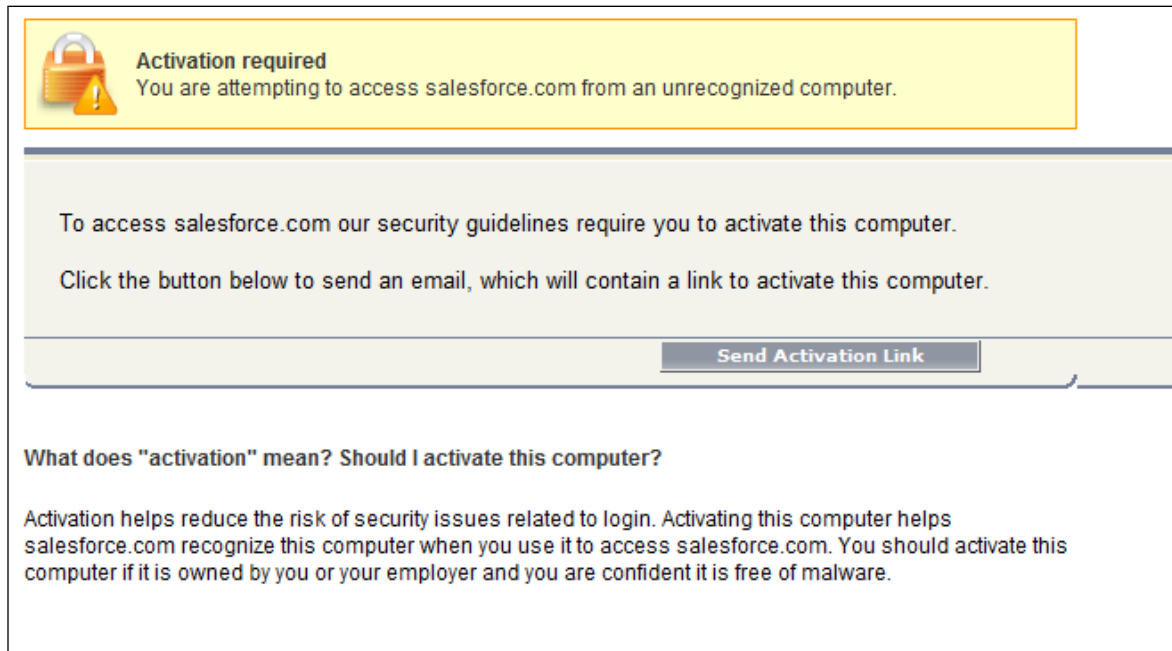
__6. Click the **Accounts** tab. In the 'View' dropdown, select **'All Accounts'** and click the **'Go'** button. You will now be shown the list of sample accounts. You can look at the detail for these accounts by clicking on an account name.

Leads Accounts Contacts Opportunities Forecasts Contracts Cases Solutions Products Reports Dashboards +						
All Accounts Edit Delete Create New View List Feed						
New Account A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Oth						
Action	Account Name ↑	Account Site	Billing State/Province	Phone	Type	Account Owner Alias
Edit Del New	Burlington Textiles Corp of Am...		NC	(336) 222-7000	Customer - Direct	IDeve
Edit Del New	Dickenson plc		KS	(785) 241-6200	Customer - Channel	IDeve
Edit Del New	Edge Communications		TX	(512) 757-6000	Customer - Direct	IDeve
Edit Del New	Express Logistics and Transport		OR	(503) 421-7800	Customer - Channel	IDeve
Edit Del New	GenePoint		CA	(650) 867-3450	Customer - Channel	IDeve
Edit Del New	Grand Hotels & Resorts Ltd		IL	(312) 596-1000	Customer - Direct	IDeve
Edit Del New	Pyramid Construction Inc.			(014) 427-4427	Customer - Channel	IDeve
Edit Del New	sForce		CA	(415) 901-7000		IDeve
Edit Del New	United Oil & Gas Corp.		NY	(212) 842-5500	Customer - Direct	IDeve
Edit Del New	United Oil & Gas, Singapore		Singapore	(650) 450-8810	Customer - Direct	IDeve
Edit Del New	United Oil & Gas, UK		UK	+44 191 4956203	Customer - Direct	IDeve
Edit Del New	University of Arizona		AZ	(520) 773-9050	Customer - Direct	IDeve

You now need to retrieve your security token

Activating Salesforce.com for use on the laptop

- __7. **Login to Salesforce.com.** You will see the message requiring activation. If you do not see this message you do not have to activate the laptop.

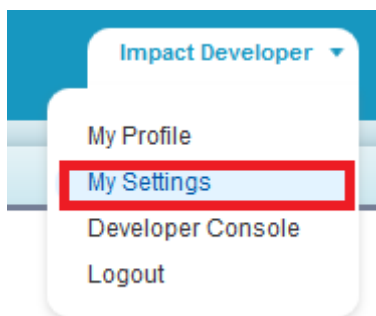


- __8. Click '**Send Activation Link**'. This will send an email to the email address used for your Salesforce.com account.
- __9. Access your email and follow the link provided in that email. You will now be able to access your Salesforce.com account from this machine.

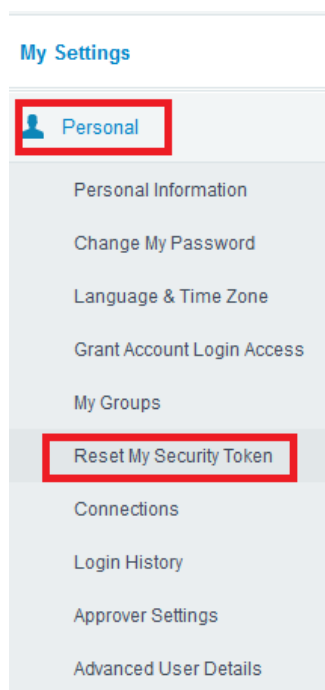
Set and Retrieve the Security Token for Existing Salesforce.com Accounts

If you have an existing Salesforce.com account you will need to activate the laptop to access your Salesforce.com account prior to completing this section.

1. Log in to Salesforce.com.
2. Click your name at the top right of the screen and select 'My Settings':



3. Expand the 'Personal' menu located on the left side of the screen and select 'Reset My Security Token'.



4. Click the button 'Reset Security Token'.

Reset My Security Token

[Help for this Page](#)

Clicking the button below invalidates your existing token. After resetting your token, you will have to use the new token in all API applications.

When accessing salesforce.com from outside of your company's trusted networks, you must add a security token to your password to log in to the API or a desktop client such as Connect for Outlook, Connect Offline, Connect for Office, Connect for Lotus Notes, or the Data Loader.



Your security token is tied to your password and subject to any password policies your administrators have configured. Whenever your password is reset, your security token is also reset.

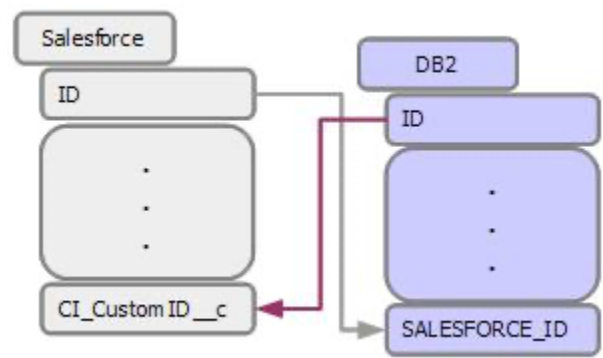
For security reasons, your security token is delivered to the email address associated with your account. To reset and send your security token, click the button below.

Reset Security Token

An email will be sent to your email address. Please access this email and make a note of the security token. You will need this in order to set up the connection from App Connect Professional to your Salesforce.com account. Note that also every time you change your password, the security token will be changed as well. It is important to note that when you copy and paste the token into the endpoint configuration that you trim out any leading or trailing spaces as that might cause authentication errors.

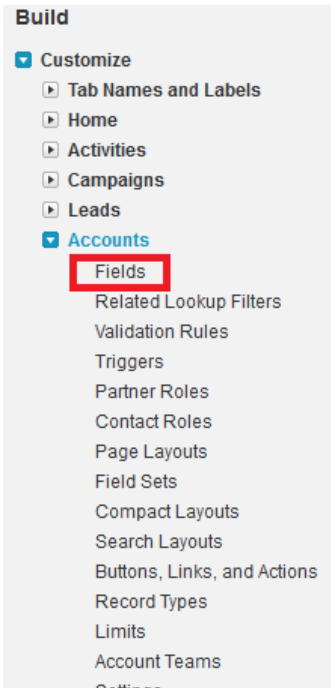
Appendix B - Updating the Salesforce Account Object

When coordinating the Salesforce Account with a legacy Account object in a database, it is customary for each object to define a field that contains the id value for the opposite environment.



Use these steps to add an External ID field to your Salesforce Account.

- 1. **Log on to your Salesforce Developer account** from <http://developer.force.com>.
- 2. Go to **Setup** → **Customize (under Build)** → **Accounts** → **Fields**



- 3. **Click the New button** in the 'Account Custom Fields & Relationships' section at the bottom.

Account Custom Fields & Relationships				New	Field Dependencies
Action	Field Label	API Name	Data Type		
Edit Del Replace	Active	Active__c	Picklist		
Edit Del Replace	Customer Priority	CustomerPriority__c	Picklist		

- __4. On the first page select **'Text'** as the data type and click **Next**
- __5. Enter **'ExternalAccountID'** for a **field label and name**. The name that's shown in Studio is the API name, which is generated by appending '___c' to the Field Name: 'ExternalAccountID___c'. The appending of the "___c" is done automatically.
- __6. Enter **20** for the **field length**, and **check** the **'External ID'** box

Step 2. Enter the details

Field Label

Please enter the maximum length for a text field below.

Length

Field Name

Description

Help Text

Required ☐ Always require a value in this field in order to save a record

Unique ☐ Do not allow duplicate values

☐ Treat "ABC" and "abc" as duplicate values (case insensitive)

☐ Treat "ABC" and "abc" as different values (case sensitive)

External ID ☒ Set this field as the unique record identifier from an external system

Default Value [Show Formula Editor](#)

Use formula syntax; e.g., Text in double quotes: "hello", Number: 25, Percent as decimal: 0.10, Date

- __7. Click **Next** and take the defaults to the end and you should see the new field listed as shown below:

Account Custom Fields & Relationships				New	Field Dependencies
Action	Field Label	API Name	Data Type		
Edit Del Replace	<u>Active</u>	Active___c	Picklist		
Edit Del Replace	<u>Customer Priority</u>	CustomerPriority___c	Picklist		
Edit Del	<u>ExternalAccountID</u>	ExternalAccountID___c	Text(20) (External ID)		
Edit Del	<u>Number of Locations</u>	NumberofLocations___c	Number(3, 0)		
Edit Del Replace	<u>SLA</u>	SLA___c	Picklist		
Edit Del	<u>SLA Expiration Date</u>	SLAExpirationDate___c	Date		
Edit Del	<u>SLA Serial Number</u>	SLASerialNumber___c	Text(10)		
Edit Del Replace	<u>Upsell Opportunity</u>	UpsellOpportunity___c	Picklist		

__8. Opening the field object will show the API field name

Account Custom Field

ExternalAccountID

[Back to Account Fields](#)

[Validation Rules \[0\]](#)

Custom Field Definition Detail

Edit

Set Field-Level Security

View Field Accessibility

Field Information

Field Label	ExternalAccountID	Object Name	Account
Field Name	ExternalAccountID	Data Type	Text
API Name	ExternalAccountID__c		
Description			

Now the ExternalAccountID__c field will be listed as an External ID field in the **Salesforce Upsert Objects** activity.

Appendix C - Notices

This information was developed for products and services offered in the U.S.A.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing
IBM Corporation
North Castle Drive
Armonk, NY 10504-1785
U.S.A.

For license inquiries regarding double-byte (DBCS) information, contact the IBM Intellectual Property Department in your country or send inquiries, in writing, to:

IBM World Trade Asia Corporation
Licensing
2-31 Roppongi 3-chome, Minato-ku
Tokyo 106-0032, Japan

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

Any references in this information to non-IBM Web sites are provided for convenience only and do not in any manner serve as an endorsement of those Web sites. The materials at those Web sites are not part of the materials for this IBM product and use of those Web sites is at your own risk.

IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Any performance data contained herein was determined in a controlled environment. Therefore, the results obtained in other operating environments may vary significantly. Some measurements may have been made on development-level systems and there is no guarantee that these measurements will be the same

on generally available systems. Furthermore, some measurements may have been estimated through extrapolation. Actual results may vary. Users of this document should verify the applicable data for their specific environment.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

All statements regarding IBM's future direction and intent are subject to change or withdrawal without notice, and represent goals and objectives only.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to the names and addresses used by an actual business enterprise is entirely coincidental. All references to fictitious companies or individuals are used for illustration purposes only.

COPYRIGHT LICENSE:

This information contains sample application programs in source language, which illustrate programming techniques on various operating platforms. You may copy, modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the application programming interface for the operating platform for which the sample programs are written. These examples have not been thoroughly tested under all conditions. IBM, therefore, cannot guarantee or imply reliability, serviceability, or function of these programs.

__7. Trademarks and copyrights

The following terms are trademarks of International Business Machines Corporation in the United States, other countries, or both:

IBM	AIX	CICS	ClearCase	ClearQuest	Cloudscape
Cube Views	DB2	developerWorks	DRDA	IMS	IMS/ESA
Informix	Lotus	Lotus Workflow	MQSeries	OmniFind	
Rational	Redbooks	Red Brick	RequisitePro	System i	
<i>System z</i>	<i>Tivoli</i>	<i>IBM</i>	<i>Workplace</i>	<i>System p</i>	

Adobe, Acrobat, Portable Document Format (PDF), and PostScript are either registered trademarks or trademarks of Adobe Systems Incorporated in the United States, other countries, or both.

Cell Broadband Engine is a trademark of Sony Computer Entertainment, Inc. in the United States, other countries, or both and is used under license therefrom.

Java and all Java-based trademarks and logos are trademarks of Sun Microsystems, Inc. in the United States, other countries, or both. See Java Guidelines

Microsoft, Windows, Windows NT, and the Windows logo are registered trademarks of Microsoft Corporation in the United States, other countries, or both.

Intel, Intel logo, Intel Inside, Intel Inside logo, Intel Centrino, Intel Centrino logo, Celeron, Intel Xeon, Intel SpeedStep, Itanium, and Pentium are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the United States and other countries.

UNIX is a registered trademark of The Open Group in the United States and other countries.

Linux is a registered trademark of Linus Torvalds in the United States, other countries, or both.

ITIL is a registered trademark and a registered community trademark of the Office of Government Commerce, and is registered in the U.S. Patent and Trademark Office.

IT Infrastructure Library is a registered trademark of the Central Computer and Telecommunications Agency which is now part of the Office of Government Commerce.

Other company, product and service names may be trademarks or service marks of others.

NOTES

NOTES



© Copyright IBM Corporation 2011.

The information contained in these materials is provided for informational purposes only, and is provided AS IS without warranty of any kind, express or implied. IBM shall not be responsible for any damages arising out of the use of, or otherwise related to, these materials. Nothing contained in these materials is intended to, nor shall have the effect of, creating any warranties or representations from IBM or its suppliers or licensors, or altering the terms and conditions of the applicable license agreement governing the use of IBM software. References in these materials to IBM products, programs, or services do not imply that they will be available in all countries in which IBM operates. This information is based on current IBM product plans and strategy, which are subject to change by IBM without notice. Product release dates and/or capabilities referenced in these materials may change at any time at IBM's sole discretion based on market opportunities or other factors, and are not intended to be a commitment to future product or feature availability in any way.

IBM, the IBM logo and ibm.com are trademarks or registered trademarks of International Business Machines Corporation in the United States, other countries, or both. If these and other IBM trademarked terms are marked on their first occurrence in this information with a trademark symbol (® or ™), these symbols indicate U.S. registered or common law trademarks owned by IBM at the time this information was published. Such trademarks may also be registered or common law trademarks in other countries. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at ibm.com/legal/copytrade.shtml

Other company, product and service names may be trademarks or service marks of others.



Please Recycle

